- Linked Lists
  - How do you reverse a singly linked list? How do you reverse a doubly linked list? Write a C program to do the same. ***Updated!***
  - Given only a pointer to a node to be deleted in a singly linked list, how do you delete it? ***Updated!***
  - How do you sort a linked list? Write a C program to sort a linked list. ***Updated!***
  - How to declare a structure of a linked list?
  - Write a C program to implement a Generic Linked List.
  - How do you reverse a linked list without using any C pointers? ***Updated!***
  - How would you detect a loop in a linked list? Write a C program to detect a loop in a linked list.
  - How do you find the middle of a linked list? Write a C program to return the middle of a linked list
  - If you are using C language to implement the heterogeneous linked list, what pointer type will you use?
  - How to compare two linked lists? Write a C program to compare two linked lists.
  - How to create a copy of a linked list? Write a C program to create a copy of a linked list.
  - Write a C program to free the nodes of a linked list ***Updated!***
  - Can we do a Binary search on a linked list?
  - Write a C program to return the nth node from the end of a linked list. ***New!***
  - How would you find out if one of the pointers in a linked list is corrupted or not? ***New!***
  - Write a C program to insert nodes into a linked list in a sorted fashion ***New!***
  - Write a C program to remove duplicates from a sorted linked list ***New!***
  - How to read a singly linked list backwards? ***New!***
  - How can I search for data in a linked list? ***New!***
- Write your own ....
  - Write your own C program to implement the atoi() function ***Updated!***

- Implement the memmove() function. What is the difference between the memmove() and memcpy() function? ***Updated!***
- Write C code to implement the strstr() (search for a substring) function.
- Write your own printf() function in C
- Implement the strcpy() function. ***Updated!***
- Implement the strcmp(str1, str2) function. ***Updated!***
- Implement the substr() function in C.
- Write your own copy() function
- Write C programs to implement the toupper() and the isupper() functions ***Updated!***
- Write a C program to implement your own strdup() function.
- Write a C program to implement the strlen() function
- Write your own strcat() function ***New!***

Programs

- Write a C program to swap two variables without using a temporary variable
- What is the 8 queens problem? Write a C program to solve it.
- Write a C program to print a square matrix helically.
- Write a C program to reverse a string ***Updated!***
- Write a C program to reverse the words in a sentence in place. ***Updated!***
- Write a C program generate permutations.
- Write a C program for calculating the factorial of a number
- Write a C program to calculate pow(x,n)?
- Write a C program which does wildcard pattern matching algorithm
- How do you calculate the maximum subarray of a list of numbers?
- How to generate fibonacci numbers? How to find out if a given number is a fibonacci number or not? Write C programs to do both.
- Solve the Rat In A Maze problem using backtracking.
- What Little-Endian and Big-Endian? How can I determine whether a machine's byte order is big-endian or little endian? How can we convert from one to another?
- Write C code to solve the Tower of Hanoi problem.
- Write C code to return a string from a function ***Updated!***
- Write a C program which produces its own source code as its output
- Write a C progam to convert from decimal to any base (binary, hex, oct etc...)
- Write C code to check if an integer is a power of 2 or not in a single line?
- Write a C program to find the GCD of two numbers. ***Updated!***
- Finding a duplicated integer problem
- Write code to remove duplicates in a sorted array. ***Updated!***
- Find the maximum of three integers using the ternary operator.
- How do you initialize a pointer inside a function?
- Write C code to dynamically allocate one, two and three dimensional arrays (using malloc()) ***New!***
- How would you find the size of structure without using sizeof()?
- Write a C program to multiply two matrices.
- Write a C program to check for palindromes.

- Write a C program to convert a decimal number into a binary number.
- Write C code to implement the Binary Search algorithm.
- Wite code to evaluate a polynomial.
- Write code to add two polynomials
- Write a program to add two long positive numbers (each represented by linked lists).

- How do you compare floating point numbers? ***Updated!***
- What's a good way to implement complex numbers in C?
- How can I display a percentage-done indication on the screen?
- Write a program to check if a given year is a leap year or not?
- Is there something we can do in C but not in C++?
- How to swap the two nibbles in a byte ?
- How to scan a string till we hit a new line using scanf()?
- Write pseudocode to compare versions (like 115.10.1 vs 115.11.5).
- How do you get the line numbers in C?
- How to fast multiply a number by 7?
- Write a simple piece of code to split a string at equal intervals
- Is there a way to multiply matrices in lesser than o(n^3) time complexity?
- How do you find out if a machine is 32 bit or 64 bit?
- Write a program to have the output go two places at once (to the screen and to a file also)
- Write code to round numbers
- How can we sum the digits of a given number in single statement?
- Given two strings A and B, how would you find out if the characters in B were a subset of the characters in A?
- Write a program to merge two arrays in sorted order, so that if an integer is in both the arrays, it gets added into the final array only once. *
- Write a program to check if the stack grows up or down
- How to add two numbers without using the plus operator?
- How to generate prime numbers? How to generate the next prime after a given prime?

- Write a program to print numbers from 1 to 100 without using loops!
- Write your own trim() or squeeze() function to remove the spaces from a string. ***New!***

- Write your own random number generator function in C.*
- Write your own sqrt() function in C*

Trees

- Write a C program to find the depth or height of a tree.
- Write a C program to determine the number of elements (or size) in a tree.
- Write a C program to delete a tree (i.e, free up its nodes)
- Write C code to determine if two trees are identical
- Write a C program to find the mininum value in a binary search tree.
- Write a C program to compute the maximum depth in a tree?

- Write a C program to create a mirror copy of a tree (left nodes become right and right nodes become left)!
- Write C code to return a pointer to the nth node of an inorder traversal of a BST. ***New!***
- Write C code to implement the preorder(), inorder() and postorder() traversals. Whats their time complexities? ***Updated!***
- Write a C program to create a copy of a tree
- Write C code to check if a given binary tree is a binary search tree or not?
- Write C code to implement level order traversal of a tree. ***New!***
- Write a C program to delete a node from a Binary Search Tree?
- Write C code to search for a value in a binary search tree (BST).
- Write C code to count the number of leaves in a tree
- Write C code for iterative preorder, inorder and postorder tree traversals ***New!***
- Can you construct a tree using postorder and preorder traversal?
- Construct a tree given its inorder and preorder traversal strings. Similarly construct a tree given its inorder and post order traversal strings. ***Updated!***
- Find the closest ancestor of two nodes in a tree. ***New!***
- Given an expression tree, evaluate the expression and obtain a paranthesized form of the expression.
- How do you convert a tree into an array?
- What is an AVL tree? ***Updated!***
- How many different trees can be constructed using n nodes?
- A full N-ary tree has M non-leaf nodes, how many leaf nodes does it have?
- Implement Breadth First Search (BFS) and Depth First Search (DFS) ***Updated!***
- Write pseudocode to add a new node to a Binary Search Tree (BST) ***Updated!***
- What is a threaded binary tree?

Bit Fiddling

- Write a C program to count bits set in an integer? ***Updated!***
- What purpose do the bitwise and, or, xor and the shift operators serve?
- How to reverse the bits in an interger?
- Check if the 20th bit of a 32 bit integer is on or off?
- How to reverse the odd bits of an integer?
- How would you count the number of bits set in a floating point number?*

Sorting Techniques

- What is heap sort?
- What is the difference between Merge Sort and Quick sort?
- Give pseudocode for the mergesort algorithm
- Implement the bubble sort algorithm. How can it be improved? Write the code for selection sort, quick sort, insertion sort.
- How can I sort things that are too large to bring into memory? ***New!***

C Pointers

- What does *p++ do? Does it increment p or the value pointed by p?
- What is a NULL pointer? How is it different from an unitialized pointer? How is a NULL pointer defined?
- What is a null pointer assignment error?
- Does an array always get converted to a pointer? What is the difference between arr and &arr? How does one declare a pointer to an entire array?
- Is the cast to malloc() required at all?
- What does malloc() , calloc(), realloc(), free() do? What are the common problems with malloc()? Is there a way to find out how much memory a pointer was allocated?
- What's the difference between const char *p, char * const p and const char * const p? ***Updated!***
- What is a void pointer? Why can't we perform arithmetic on a void * pointer?
- What do Segmentation fault, access violation, core dump and Bus error mean?
- What is the difference between an array of pointers and a pointer to an array?
- What is a memory leak?
- What are brk() and sbrk() used for? How are they different from malloc()?
- What is a dangling pointer? What are reference counters with respect to pointers?
- What do pointers contain?
- Is *(*(p+i)+j) is equivalent to p[i][j]? Is num[i] == i[num] == *(num + i) == *(i + num)?
- What operations are valid on pointers? When does one get the Illegal use of pointer in function error?
- What are near, far and huge pointers? ***New!***
- What is the difference between malloc() and calloc()? ***New!***
- Why is sizeof() an operator and not a function? ***New!***
- What is an opaque pointer?
- What are the common causes of pointer bugs?

C Functions

- How to declare a pointer to a function?
- Does extern in a function declaration mean anything?
- How can I return multiple values from a function?
- Does C support function overloading?
- What is the purpose of a function prototype?
- What are inline functions?
- How to declare an array of N pointers to functions returning pointers to functions returning pointers to characters?
- Can we declare a function that can return a pointer to a function of the same type?
- How can I write a function that takes a variable number of arguments? What are the limitations with this? What is vprintf()?
- With respect to function parameter passing, what is the difference between call-by-value and call-by-reference? Which method does C use? ***Updated!***
- If I have the name of a function in the form of a string, how can I invoke that function? ***New!***
- What does the error, invalid redeclaration of a function mean?

- How can I pass the variable argument list passed to one function to another function.
- How do I pass a variable number of function pointers to a variable argument (va_arg) function?
- Will C allow passing more or less arguments than required to a function.

C Statements

- Whats short-circuiting in C expressions?
- Whats wrong with the expression a[i]=i++; ? Whats a sequence point? ***Updated!***
- Does the ?: (ternary operator) return a lvalue? How can I assign a value to the output of the ternary operator?
- Is 5[array] the same as array[5]?
- What are #pragmas?
- What is the difference between if(0 == x) and if(x == 0)?
- Should we use goto or not?
- Is ++i really faster than i = i + 1?
- What do lvalue and rvalue mean?
- What does the term cast refer to? Why is it used?
- What is the difference between a statement and a block?
- Can comments be nested in C?
- What is type checking?
- Why can't you nest structure definitions?
- What is a forward reference?
- What is the difference between the & and && operators and the | and || operators?
- Is C case sensitive (ie: does C differentiate between upper and lower case letters)?
- Can goto be used to jump across functions?
- Whats wrong with #define myptr int *?
- What purpose do #if, #else, #elif, #endif, #ifdef, #ifndef serve?
- Can we use variables inside a switch statement? Can we use floating point numbers? Can we use expressions?
- What is more efficient? A switch() or an if() else()?
- What is the difference between a deep copy and a shallow copy?
- What is operator precedence?*

C Arrays

- How to write functions which accept two-dimensional arrays when the width is not known before hand?
- Is char a[3] = "abc"; legal? What does it mean?
- If a is an array, is a++ valid?
- How can we find out the length of an array dynamically in C?

C Variables

- What is the difference between the declaration and the definition of a variable?

- Do Global variables start out as zero?
- Does C have boolean variable type?
- Where may variables be defined in C?
- To what does the term storage class refer? What are auto, static, extern, volatile, const classes?
- What does the typedef keyword do?
- What is the difference between constants defined through #define and the constant keyword?
- What are Trigraph characters?
- How are floating point numbers stored? Whats the IEEE format? ***Updated!***
- When should the register modifier be used? ***New!***
- When should a type cast be used? ***New!***

- C Structures

- Can structures be assigned to variables and passed to and from functions?
- Can we directly compare two structures using the == operator? ***Updated!***
- Can we pass constant values to functions which accept structure arguments?
- How does one use fread() and fwrite()? Can we read/write structures to/from files?
- Why do structures get padded? Why does sizeof() return a larger size?
- Can we determine the offset of a field within a structure and directly access that element?
- What are bit fields in structures? ***Updated!***
- What is a union? Where does one use unions? What are the limitations of unions? ***New!***

- C Macros

- How should we write a multi-statement macro?
- How can I write a macro which takes a variable number of arguments?
- What is the token pasting operator and stringizing operator in C?
- Define a macro called SQR which squares a number. ***Updated!***

- C Headers

- What should go in header files? How to prevent a header file being included twice? Whats wrong with including more headers?
- Is there a limit on the number of characters in the name of a header file?
- Is it acceptable to declare/define a variable in a C header? ***New!***

- C File operations

- How do stat(), fstat(), vstat() work? How to check whether a file exists?
- How can I insert or delete a line (or record) in the middle of a file?
- How can I recover the file name using its file descriptor?
- How can I delete a file? How do I copy files? How can I read a directory in a C

program?
- Whats the use of fopen(), fclose(), fprintf(), getc(), putc(), getw(), putw(), fscanf(), feof (), ftell(), fseek(), rewind(), fread(), fwrite(), fgets(), fputs(), freopen(), fflush(), ungetc()?

- How to check if a file is a binary file or an ascii file? ***New!***

C Declarations and Definitions

- What is the difference between char *a and char a[]? ***Updated!***
- How can I declare an array with only one element and still access elements beyond the first element (in a valid fashion)?
- What is the difference between enumeration variables and the preprocessor #defines?

C Functions - built-in

- Whats the difference between gets() and fgets()? Whats the correct way to use fgets() when reading a file?
- How can I have a variable field width with printf?
- How can I specify a variable width in a scanf() format string?
- How can I convert numbers to strings (the opposite of atoi)?
- Why should one use strncpy() and not strcpy()? What are the problems with strncpy()?

- How does the function strtok() work?
- Why do we get the floating point formats not linked error?
- Why do some people put void cast before each call to printf()?
- What is assert() and when would I use it?
- What do memcpy(), memchr(), memcmp(), memset(), strdup(), strncat(), strcmp(), strncmp(), strcpy(), strncpy(), strlen(), strchr(), strchr(), strpbrk(), strspn(), strcspn(), strtok() do? ***Updated!***
- What does alloca() do?
- Can you compare two strings like string1==string2? Why do we need strcmp()?
- What does printf() return? ***New!***
- What do setjmp() and longjump() functions do? ***New!***

C Functions - The main function

- Whats the prototype of main()? Can main() return a structure?
- Is exit(status) equivalent to returning the same status from main()?
- Can main() be called recursively?
- How to print the arguments recieved by main()?

OS Concepts

- What do the system calls fork(), vfork(), exec(), wait(), waitpid() do? Whats a Zombie process? Whats the difference between fork() and vfork()?
- How does freopen() work? *
- What are threads? What is a lightweight process? What is a heavyweight process?

How different is a thread from a process? *
- How are signals handled? *
- What is a deadlock? *
- What are semaphores? *
- What is meant by context switching in an OS?*
- What is Belady's anomaly?
- What is thrashing?*
- What are short-, long- and medium-term scheduling?
- What are turnaround time and response time?
- What is the Translation Lookaside Buffer (TLB)?
- What is cycle stealing?
- What is a reentrant program?
- When is a system in safe state?
- What is busy waiting?
- What is pages replacement? What are local and global page replacements?*
- What is meant by latency, transfer and seek time with respect to disk I/O?
- What are monitors? How are they different from semaphores?*
- In the context of memory management, what are placement and replacement algorithms?
- What is paging? What are demand- and pre-paging?*
- What is mounting?
- What do you mean by dispatch latency?
- What is multi-processing? What is multi-tasking? What is multi-threading? What is multi-programming?*
- What is compaction?
- What is memory-mapped I/O? How is it different frim I/O mapped I/O?
- List out some reasons for process termination.

General Concepts

- What is the difference between statically linked libraries and dynamically linked libraries (dll)? ***Updated!***
- What are the most common causes of bugs in C?
- What is hashing?
- What do you mean by Predefined?
- What is data structure?
- What are the time complexities of some famous algorithms? ***Updated!***
- What is row major and column major form of storage in matrices?
- Explain the BigOh notation. ***Updated!***
- Give the most important types of algorithms.
- What is marshalling and demarshalling? ***New!***
- What is the difference between the stack and the heap? Where are the different types of variables of a program stored in memory? ***New!***
- Describe the memory map of a C program.
- What is infix, prefix, postfix? How can you convert from one representation to another?

    - How do you evaluate these expressions?
    - How can we detect and prevent integer overflow and underflow?*

- Compiling and Linking
    - How to list all the predefined identifiers?
    - How the compiler make difference between C and C++?
    - What are the general steps in compilation?
    - What are the different types of linkages? ***New!***
    - What do you mean by scope and duration? ***New!***
    - What are makefiles? Why are they used?*

- HTTP
    - What are the basics of http? What is the difference be http and https? *

- ProC
    - How to work in ProC?

- Miscellaneous
    - Gotcha! programs ***New!***

- SQL/PL-SQL/Database
    - Most frequently asked SQL/PL-SQL/Database questions *

- Unix/Linux
    - Most frequently asked unix questions *

- Quantitative Aptitude
    - Most frequently asked Quantitative aptitude questions*

- Graphs
    - Most frequently asked questions on Graphs*

- OOPs/OOAD
    - Most frequently asked OOPs and OOAD questions*

- Computer Networks

- Most frequently asked questions on Computer Networks*

Client Server Technologies

- Most frequently asked Client - Server questions*

Scripting basics

- Introduction to perl ***New!***

CRACK THE INTERVIEW
Last updated: October 26, 2005

WWW.CRACKTHEINTERVIEW.COM

Most IT interviews revolve around the same set of questions and similar topics. Quite a few sites on the Internet do provide answers to these most frequently asked interview questions, but none of them provide good, dependable answers to these interview questions nor do they have a clean, clutter free interface.

Have you been looking around for answers to questions like How to reverse a linked list?, How to count the number of bits set in an integer?, How to write your own substr(), atoi(), printf(), memmove() function?, How to get the depth of a tree?, How to code the 8 queens problem using backtracing?, What is an AVL tree?, How to generate all the permutations of a set of numbers?, How to construct a tree given its inorder and preorder representation?, How to print a matrix helically?, How to copy bytes if the source and destination overlap?, How to detect if a linked list has a cycle in it?

If yes, then you are really lucky to have come here. This site provides answers to all these most frequently asked interview questions and also answers to these questions!. Most of the interviews conducted by leading IT companies hover around the questions.

As of now, this site concentrates mainly on C and the Datastructures in C. There is lots and lots more to come. If you find any questions or its answer to be wrong, feel free to drop in a mail to me. I have a link at the end of each question which you can use. I will try my best to see that the FAQ is kept uptodate and accurate. Note that this site does not host interview question papers.

I wish you All The Best in your efforts. Remember, if the interviewer can steal questions from the net, so can you :). Afterall, it should be a fair match, isn't it?

So what are you waiting for?, Lets get STARTED! .

Google linked list Search

300+ SURE SHOT QUESTIONS!


40,000+ HITS!

Google linked list Search

FreeFind search linked list Search

Linked Lists
(1 / 301)

How do you reverse a singly linked list? How do you reverse a doubly linked list? Write a C program to do the same.

This is THE most frequently asked interview question. The most!.

Singly linked lists

Here are a few C programs to reverse a singly linked list.

Method1 (Iterative)

```
#include <stdio.h>

// Variables
typedef struct node
{
   int value;
   struct node *next;
}mynode;


// Globals (not required, though).
mynode *head, *tail, *temp;


// Functions
void add(int value);
void iterative_reverse();
void print_list();


// The main() function
int main()
{
    head=(mynode *)0;

    // Construct the linked list.
    add(1);
    add(2);
    add(3);

    //Print it
    print_list();

    // Reverse it.
    iterative_reverse();

    //Print it again
```

```
    print_list();

    return(0);
}


// The reverse function
void iterative_reverse()
{
     mynode *p, *q, *r;

     if(head == (mynode *)0)
     {
        return;
     }


     p       = head;
     q       = p->next;
     p->next = (mynode *)0;


     while (q != (mynode *)0)
     {
        r       = q->next;
        q->next = p;
        p       = q;
        q       = r;
     }

     head = p;
}


// Function to add new nodes to the linked list
void add(int value)
{
   temp = (mynode *) malloc(sizeof(struct node));
   temp->next=(mynode *)0;
   temp->value=value;

   if(head==(mynode *)0)
   {
      head=temp;
      tail=temp;
   }
   else
   {
     tail->next=temp;
     tail=temp;
   }
}


// Function to print the linked list.
void print_list()
{
  printf("\n\n");
  for(temp=head; temp!=(mynode *)0; temp=temp->next)
  {
     printf("[%d]->",(temp->value));
  }
  printf("[NULL]\n\n");
}
```

Method2 (Recursive, without using any temporary variable)

```
#include <stdio.h>

// Variables
typedef struct node
{
```

```
   int value;
   struct node *next;
}mynode;

// Globals.
mynode *head, *tail, *temp;


// Functions
void add(int value);
mynode* reverse_recurse(mynode *root);
void print_list();


// The main() function
int main()
{
    head=(mynode *)0;

    // Construct the linked list.
    add(1);
    add(2);
    add(3);

    //Print it
    print_list();

    // Reverse it.
    if(head != (mynode *)0)
    {
      temp = reverse_recurse(head);
      temp->next = (mynode *)0;
    }

    //Print it again
    print_list();

    return(0);
}


// Reverse the linked list recursively
//

// This function uses the power of the stack to make this
// *magical* assignment
//
// node->next->next=node;
//
// :)

mynode* reverse_recurse(mynode *root)
{
  if(root->next!=(mynode *)0)
  {
      reverse_recurse(root->next);
      root->next->next=root;
      return(root);
  }
  else
  {
      head=root;
  }
}



// Function to add new nodes to the linked list.
void add(int value)
{
     temp = (mynode *) malloc(sizeof(struct node));
     temp->next=(mynode *)0;
     temp->value=value;

     if(head==(mynode *)0)
     {
        head=temp;
        tail=temp;
     }
```

```
     else
     {
       tail->next=temp;
       tail=temp;
     }
}

// Function to print the linked list.
void print_list()
{
    printf("\n\n");
    for(temp=head; temp!=(mynode *)0; temp=temp->next)
    {
       printf("[%d]->",(temp->value));
    }
    printf("[NULL]\n\n");
}
```

Method3 (Recursive, but without ANY global variables. Slightly messy!)

```
#include <stdio.h>

// Variables
typedef struct node
{
   int value;
   struct node *next;
}mynode;


// Functions
void add(mynode **head, mynode **tail, int value);
mynode* reverse_recurse(mynode *current, mynode *next);
void print_list(mynode *);


int main()
{
    mynode *head, *tail;
    head=(mynode *)0;

    // Construct the linked list.
    add(&head, &tail, 1);
    add(&head, &tail, 2);
    add(&head, &tail, 3);

    //Print it
    print_list(head);

    // Reverse it.
    head = reverse_recurse(head, (mynode *)0);

    //Print it again
    print_list(head);

    getch();
    return(0);
}


// Reverse the linked list recursively
mynode* reverse_recurse(mynode *current, mynode *next)
{
  mynode *ret;

  if(current==(mynode *)0)
  {
    return((mynode *)0);
  }
```

```
  ret = (mynode *)0;
  if (current->next != (mynode *)0)
  {
     ret = reverse_recurse(current->next, current);
  }
  else
  {
     ret = current;
  }

  current->next = next;
  return ret;
}


// Function to add new nodes to the linked list.
// Takes pointers to pointers to maintain the
// *actual* head and tail pointers (which are local to main()).

void add(mynode **head, mynode **tail, int value)
{
     mynode *temp1, *temp2;

     temp1 = (mynode *) malloc(sizeof(struct node));
     temp1->next=(mynode *)0;
     temp1->value=value;

     if(*head==(mynode *)0)
     {
        *head=temp1;
        *tail=temp1;
     }
     else
     {
        for(temp2 = *head; temp2->next!= (mynode *)0; temp2=temp2->next);
        temp2->next = temp1;
        *tail=temp1;
     }
}


// Function to print the linked list.
void print_list(mynode *head)
{
    mynode *temp;
    printf("\n\n");
    for(temp=head; temp!=(mynode *)0; temp=temp->next)
    {
       printf("[%d]->",(temp->value));
    }
    printf("[NULL]\n\n");
}
```

Doubly linked lists

This is really easy, just keep swapping the prev and next pointers and at the end swap the head and the tail:)

```
#include<stdio.h>
#include<ctype.h>

typedef struct node
{
  int value;
  struct node *next;
  struct node *prev;
}mynode ;

mynode *head, *tail;
void add_node(int value);
void print_list();
void reverse();
```

```
int main()
{


 head=NULL;
 tail=NULL;

 add_node(1);
 add_node(2);
 add_node(3);
 add_node(4);
 add_node(5);

 print_list();

 reverse();

 print_list();

 return(1);

}


void add_node(int value)
{
  mynode *temp, *cur;
  temp = (mynode *)malloc(sizeof(mynode));
  temp->next=NULL;
  temp->prev=NULL;

  if(head == NULL)
  {

    printf("\nAdding a head pointer\n");
    head=temp;
    tail=temp;
    temp->value=value;

  }
  else
  {



   for(cur=head;cur->next!=NULL;cur=cur->next);
   cur->next=temp;
   temp->prev=cur;
   temp->value=value;
   tail=temp;

  }

}


void print_list()
{
  mynode *temp;

  printf("\n--------------------------------\n");
  for(temp=head;temp!=NULL;temp=temp->next)
  {
    printf("\n[%d]\n",temp->value);
  }

}



void reverse()
{
  mynode *cur, *temp, *save_next;
  if(head==tail)return;
  if(head==NULL || tail==NULL)return;
  for(cur=head;cur!=NULL;)
  {
    printf("\ncur->value : [%d]\n",cur->value);
    temp=cur->next;
```

```
    save_next=cur->next;
    cur->next=cur->prev;
    cur->prev=temp;
    cur=save_next;
  }

  temp=head;
  head=tail;
  tail=temp;
}
```

Having shown all these different methods, if someone can mail me a really, really good practical application of reversing a linked list (singly or doubly linked list), I would be really thankful to them. I have not found one good application of this. All I see is an urge to understand how well the candidate handles the pointer manipulation.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

cracktheinterview.com

Get your interviewer on his knees :-)

The time now is Fri Dec 30, 2005 5:52 pm
cracktheinterview.com Forum Index

View unanswered posts

Forum	Topics	Posts	Last Post
Questions and Discussions			
Discuss Existing Questions Use this forum to discuss existing questions on the website	0	0	No Posts
New Questions Discuss new questions here...	18	51	Fri Dec 30, 2005 10:17 am qadthebad
Errors and typos Please use this forum to discuss and errors or typos on the website..	0	0	No Posts
Suggestions and comments..			
Guest Book This is my Guest Book!	1	4	Thu Dec 08, 2005 5:49 pm mth
Polls			
Currently running Polls Take some online polls to help improve the website!	0	0	No Posts

Mark all forums read

All times are GMT

Who is Online

Our users have posted a total of **55** articles
We have **95** registered users
The newest registered user is **bindu_boda**

In total there is **1** user online :: 0 Registered, 0 Hidden and 1 Guest [Administrator] [Moderator]
Most users ever online was **7** on Tue Nov 15, 2005 6:24 am
Registered Users: None

This data is based on users active over the past five minutes

Log in

Username: Password: Log me on automatically each visit Log in

New posts No new posts Forum is locked

Powered by phpBB 2.0.8 © 2001, 2002 phpBB Group

Guest Book

Moderators: None

Users browsing this forum: None

newtopic cracktheinterview.com Forum Index -> Guest Book

Mark all topics read

Topics	Replies	Author	Views	Last Post
A TUTORIAL ON POINTERS AND ARRAYS IN C	3	vastare	727	Thu Dec 08, 2005 5:49 pm mth

Display topics from previous: All Topics Go

newtopic cracktheinterview.com Forum Index -> Guest Book

All times are GMT

Page 1 of 1

Jump to: Select a forum Go

New posts | No new posts | Announcement

New posts [Popular] | No new posts [Popular] | Sticky

New posts [Locked] | No new posts [Locked]

You **cannot** post new topics in this forum
You **cannot** reply to topics in this forum
You **cannot** edit your posts in this forum
You **cannot** delete your posts in this forum
You **cannot** vote in polls in this forum

Powered by phpBB 2.0.8 © 2001, 2002 phpBB Group

Currently running Polls

Moderators: None

Users browsing this forum: None

newtopic cracktheinterview.com Forum Index -> Currently running Polls

Mark all topics read

Topics	Replies	Author	Views	Last Post
There are no posts in this forum. Click on the **Post New Topic** link on this page to post one.				

Display topics from previous: All Topics Go

newtopic cracktheinterview.com Forum Index -> Currently running Polls

All times are GMT

Jump to: Select a forum Go

New posts
New posts [Popular]
New posts [Locked]

No new posts
No new posts [Popular]
No new posts [Locked]

Announcement
Sticky

You **cannot** post new topics in this forum
You **cannot** reply to topics in this forum
You **cannot** edit your posts in this forum
You **cannot** delete your posts in this forum
You **cannot** vote in polls in this forum

Powered by phpBB 2.0.8 © 2001, 2002 phpBB Group

New Questions

Moderators: None

Users browsing this forum: None

newtopic cracktheinterview.com Forum Index -> New Questions

Mark all topics read

Topics	Replies	Author	Views	Last Post
how to implement sizeof	8	bsenthil	947	Fri Dec 30, 2005 10:17 am qadthebad
How do you sort an array of integers with no duplicates	0	rarish911	45	Wed Dec 28, 2005 7:20 am rarish911
weird Q	3	bsenthil	696	Mon Dec 26, 2005 8:33 am qadthebad
sigma linked list	0	kool	95	Sat Dec 24, 2005 8:18 pm kool
Java Questions	0	amitxlnc	145	Wed Dec 21, 2005 7:41 am amitxlnc
What is initialization and why is it done in c++?	0	Taz	196	Mon Dec 19, 2005 2:57 am Taz
code snippet -2	1	arunajay	471	Thu Dec 08, 2005 6:14 pm Shukra
code snippets -3	1	arunajay	469	Thu Dec 08, 2005 6:05 pm Shukra
a code snippet	8	arunajay	1107	Thu Dec 08, 2005 5:48 pm Shukra
dout	1	vastare	726	Fri Nov 25, 2005 3:14 am administrator
datastructures	1	neha1312	996	Thu Nov 24, 2005 8:25 am vastare
print out all the nodes in a given level in a tree	1	Gypsy	757	Wed Nov 23, 2005 3:09 am administrator
manage a heap using an existing array	0	Gypsy	675	Wed Nov 23, 2005 1:55 am Gypsy
Only Huge Pointer with example	1	sam	849	Mon Nov 21, 2005 8:41 am Yash
Difference between struct and union	1	goldenfish	941	Mon Nov 21, 2005 8:11 am Shukra
Endianess	2	girishps76	1111	Thu Nov 17, 2005 2:26 am administrator
howto arrange a matrix members in ascending/descending order	3	chitra	1553	Tue Nov 15, 2005 6:23 am sandhya529
wat s the difference	2	arunajay	1226	Sat Nov 12, 2005 11:22 am Shukra

Display topics from previous: All Topics Go

Jump to: Select a forum Go

 New posts
 No new posts
 Announcement
 New posts [Popular]
 No new posts [Popular]
 Sticky
 New posts [Locked]
 No new posts [Locked]

You **cannot** post new topics in this forum
You **cannot** reply to topics in this forum
You **cannot** edit your posts in this forum
You **cannot** delete your posts in this forum
You **cannot** vote in polls in this forum

Powered by phpBB 2.0.8 © 2001, 2002 phpBB Group

Linked Lists
(2 / 301)

Given only a pointer to a node to be deleted in a singly linked list, how do you delete it?

This is a very good interview question

The solution to this is to copy the data from the next node into this node and delete the next node!. Ofcourse this wont work if the node to be deleted is the last node. Mark it as dummy in that case. If you have a Circular linked list, then this might be all the more interesting. Try writing your own C program to solve this problem. Having a doubly linked list is always better.

Linked Lists
(3 / 301)

How do you sort a linked list? Write a C program to sort a linked list.

This is a very popular interview question, which most people go wrong. The ideal solution to this problem is to keep the linked list sorted as you build it. Another question on this website teaches you how to insert elements into a linked list in the sorted order. This really saves a lot of time which would have been required to sort it.

However, you need to Get That Job....

Method1 (Usual method)

The general idea is to decide upon a sorting algorithm (say bubble sort). Then, one needs to come up with different scenarios to swap two nodes in the linked list when they are not in the required order. The different scenarios would be something like

```
1. When the nodes being compared are not adjacent and one of them is the first node.
2. When the nodes being compared are not adjacent and none of them is the first node
3. When the nodes being compared are adjacent and one of them is the first node.
4. When the nodes being compared are adjacent and none of them is the first node.
```

One example bubble sort for a linked list goes like this (working C code!)....

```
#include<stdio.h>
#include<ctype.h>

typedef struct node
{
  int value;
  struct node *next;
  struct node *prev;
}mynode ;

void add_node(struct node **head, int *count, int value);
void print_list(char *listName, struct node *head);
mynode *bubbleSort(mynode *head, int count);


int main()
{
 mynode *head;
 int count = 0;

 head = (struct node *)NULL;
```

```
 add_node(&head, &count, 100);
 add_node(&head, &count, 3);
 add_node(&head, &count, 90);
 add_node(&head, &count, 7);
 add_node(&head, &count, 9);

 print_list("myList(BEFORE)", head);
 head = bubbleSort(head, count);
 print_list("myList(AFTER) ", head);


 getch();
 return(0);

}


mynode *bubbleSort(mynode *head, int count)
{
  int i, j;
  mynode *p0, *p1, *p2, *p3;

  for(i = 1; i < count; i++)
  {
    p0 = (struct node *)NULL;
    p1 = head;
    p2 = head->next;
    p3 = p2->next;

    for(j = 1; j <= (count - i); j++)
    {
       if(p1->value > p2->value)
       {
           // Adjust the pointers...
           p1->next = p3;
           p2->next = p1;
           if(p0)p0->next=p2;


           // Set the head pointer if it was changed...
           if(head == p1)head=p2;


           // Progress the pointers
           p0 = p2;
           p2 = p1->next;
           p3 = p3->next!=(struct node *)NULL?p3->next: (struct node *)NULL;

       }
       else
       {
          // Nothing to swap, just progress the pointers...
          p0 = p1;
          p1 = p2;
          p2 = p3;
          p3 = p3->next!=(struct node *)NULL?p3->next: (struct node *)NULL;
       }
    }
  }

  return(head);
}


void add_node(struct node **head, int *count, int value)
{
  mynode *temp, *cur;
  temp = (mynode *)malloc(sizeof(mynode));
  temp->next=NULL;
  temp->prev=NULL;

  if(*head == NULL)
  {
     *head=temp;
     temp->value=value;
  }
  else
  {
   for(cur=*head;cur->next!=NULL;cur=cur->next);
```

```
   cur->next=temp;
   temp->prev=cur;
   temp->value=value;
  }

  *count = *count + 1;
}


void print_list(char *listName, struct node *head)
{
  mynode *temp;

  printf("\n[%s] -> ", listName);
  for(temp=head;temp!=NULL;temp=temp->next)
  {
    printf("[%d]->",temp->value);
  }

  printf("NULL\n");

}
```

As you can see, the code becomes quite messy because of the pointer logic. Thats why I have not elaborated too much on the code, nor on variations such as sorting a doubly linked list. You have to do it yourself once to understand it.

Method2 (Divide and Conquer using Merge Sort)

Here is some cool working C code...

```
#include<stdio.h>
#include<ctype.h>

typedef struct node
{
  int value;
  struct node *next;
  struct node *prev;
}mynode ;



void add_node(struct node **head, int value);
void print_list(char *listName, struct node *head);
void mergeSort(struct node** headRef);
struct node *merge2SortedLLs(struct node *head1, struct node *head2);
void splitLLInto2(struct node* source, struct node** frontRef, struct node** backRef);


// The main function..
int main()
{
   mynode *head;
   head = (struct node *)NULL;

   add_node(&head, 1);
   add_node(&head, 10);
   add_node(&head, 5);
   add_node(&head, 70);
   add_node(&head, 9);
   add_node(&head, -99);
   add_node(&head, 0);


   print_list("myList", head);
   mergeSort(&head);
   print_list("myList", head);

   getch();
   return(0);
}
```

```
// This is a recursive mergeSort function...
void mergeSort(struct node** headRef)
{
  struct node* head = *headRef;
  struct node* a;
  struct node* b;

  // Base case -- length 0 or 1
  if ((head == NULL) || (head->next == NULL))
  {
    return;
  }

  // Split head into 'a' and 'b' sublists
  splitLLInto2(head, &a, &b);

  // Recursively sort the sublists
  mergeSort(&a);
  mergeSort(&b);

  // Merge the two sorted lists together
  *headRef = merge2SortedLLs(a, b);
}
```

```
// This is an iterative function that joins two already sorted
// Linked lists...
struct node *merge2SortedLLs(struct node *head1, struct node *head2)
{
   struct node *a, *b, *c, *newHead, *temp;

   a = head1;
   b = head2;
   c       = (struct node *)NULL;
   newHead = (struct node*)NULL;


   if(a==NULL)return(b);
   else if(b==NULL)return(a);

   while(a!=NULL && b!=NULL)
   {
     if(a->value < b->value)
     {
       //printf("\na->value < b->value\n");
       if(c==NULL)
       {
         c  = a;
       }
       else
       {
         c->next = a;
         c = c->next;
       }
       a  = a->next;
     }
     else if(a->value > b->value)
     {
       //printf("\na->value > b->value\n");
       if(c==NULL)
       {
         c  = b;
       }
       else
       {
         c->next = b;
         c = c->next;
       }
       b  = b->next;
     }
     else
     {
```

```
      // Both are equal.
      // Arbitrarily chose to add one of them and make
      // sure you skip both!

      if(c == NULL)
      {
        c  = a;
      }
      else
      {
        c->next = a;
        c = c->next;
      }

      a  = a->next;
      b  = b->next;
    }

    // Make sure the new head is set...
    if(newHead == NULL)
     newHead = c;

  }

  if(a==NULL && b==NULL)
    return(newHead);

  if(a==NULL)
    c->next = b;
  else if(b==NULL)
    c->next = a;

  return(newHead);

}


// Uses the fast/slow pointer strategy
//
// This efficient code splits a linked list into two using
// the same technique as the one used to find the
// middle of a linked list!

void splitLLInto2(struct node* source, struct node** frontRef, struct node** backRef)
{
  struct node* fast;
  struct node* slow;

  if (source==NULL || source->next==NULL)
  {
    // length < 2 cases
    *frontRef = source;
    *backRef = NULL;
  }
  else
  {
    slow = source;
    fast = source->next;
    // Advance 'fast' two nodes, and advance 'slow' one node
    while (fast != NULL)
    {
       fast = fast->next;
       if (fast != NULL)
       {
          slow = slow->next;
          fast = fast->next;
       }
    }
    // 'slow' is before the midpoint in the list, so split it in two
    // at that point.
    *frontRef = source;
    *backRef = slow->next;
    slow->next = NULL;
  }
}
```

```
void add_node(struct node **head, int value)
{
  mynode *temp, *cur;
  temp = (mynode *)malloc(sizeof(mynode));
  temp->next=NULL;
  temp->prev=NULL;

  if(*head == NULL)
  {
     *head=temp;
     temp->value=value;
  }
  else
  {
   for(cur=*head;cur->next!=NULL;cur=cur->next);
   cur->next=temp;
   temp->prev=cur;
   temp->value=value;
  }
}


void print_list(char *listName, struct node *head)
{
  mynode *temp;

  printf("\n[%s] -> ", listName);
  for(temp=head;temp!=NULL;temp=temp->next)
  {
    printf("[%d]->",temp->value);
  }

  printf("NULL\n");

}
```

The code to merge two already sorted sub-linked lists into a sorted linked list could be either iterative or recursive. You already saw the iterative version above. Here is a recursive version of the same...

Recursive solution to merge two already sorted linked lists into a single linked list

```
struct node* sortedMergeRecursive(struct node* a, struct node* b)
{
  struct node* result = NULL;

  if (a==NULL) return(b);
  else if (b==NULL) return(a);

  // Pick either a or b, and recur
  if (a->data <= b->data)
  {
     result = a;
     result->next = sortedMergeRecursive(a->next, b);
  }
  else
  {
     result = b;
     result->next = sortedMergeRecursive(a, b->next);
  }

  return(result);
}
```

Also, see how the splitLLInto2() function uses the same technique used to find the middle of a linked list to split a linked list into two without having to keep a count of the number of nodes in the linkes list!

Here is another solution (not that great, though) to split a linked list into two. It used the count of the number of nodes to decide where to split

```
void splitLLInto2(struct node* source,struct node** frontRef, struct node** backRef)
{
```

```
  int len = Length(source); //Get the length of the original LL..
  int i;
  struct node* current = source;

  if (len < 2)
  {
    *frontRef = source;
    *backRef = NULL;
  }
  else
  {
     int hopCount = (len-1)/2;

     for (i = 0; i<hopCount; i++)
     {
        current = current->next;
     }

     // Now cut at current
    *frontRef = source;
    *backRef = current->next;
    current->next = NULL;
  }
}
```

Using recursive stack space proportional to the length of a list is not recommended. However, the recursion in this case is ok ? it uses stack space which is proportional to the log of the length of the list. For a 1000 node list, the recursion will only go about 10 deep. For a 2000 node list, it will go 11 deep. If you think about it, you can see that doubling the size of the list only increases the depth by 1.

Cool, eh?

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to declare a structure of a linked list?

The right way of declaring a structure for a linked list in a C program is

```
struct node {
  int value;
  struct node *next;
};
typedef struct node *mynode;
```

Note that the following are not correct

```
typedef struct {
  int value;
  mynode next;
} *mynode;
```

The typedef is not defined at the point where the "next" field is declared.

```
struct node {
  int value;
  struct node next;
};
typedef struct node mynode;
```

You can only have pointer to structures, not the structure itself as its recursive!

Write a C program to implement a Generic Linked List.

Here is a C program which implements a generic linked list. This is also one of the very popular interview questions thrown around. The crux of the solution is to use the void C pointer to make it generic. Also notice how we use function pointers to pass the address of different functions to print the different generic data.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct list {
    void *data;
    struct list *next;
} List;

struct check {
    int i;
    char c;
    double d;
} chk[] = { { 1, 'a', 1.1 },
          { 2, 'b', 2.2 },
          { 3, 'c', 3.3 } };

void insert(List **, void *, unsigned int);
void print(List *, void (*)(void *));
void printstr(void *);
void printint(void *);
void printchar(void *);
void printcomp(void *);

List *list1, *list2, *list3, *list4;

int main(void)
{
    char c[]    = { 'a', 'b', 'c', 'd' };
    int i[]     = { 1, 2, 3, 4 };
    char *str[] = { "hello1", "hello2", "hello3", "hello4" };

    list1 = list2 = list3 = list4 = NULL;

    insert(&list1, &c[0], sizeof(char));
    insert(&list1, &c[1], sizeof(char));
    insert(&list1, &c[2], sizeof(char));
    insert(&list1, &c[3], sizeof(char));

    insert(&list2, &i[0], sizeof(int));
    insert(&list2, &i[1], sizeof(int));
    insert(&list2, &i[2], sizeof(int));
    insert(&list2, &i[3], sizeof(int));
```

```
    insert(&list3, str[0], strlen(str[0])+1);
    insert(&list3, str[1], strlen(str[0])+1);
    insert(&list3, str[2], strlen(str[0])+1);
    insert(&list3, str[3], strlen(str[0])+1);

    insert(&list4, &chk[0], sizeof chk[0]);
    insert(&list4, &chk[1], sizeof chk[1]);
    insert(&list4, &chk[2], sizeof chk[2]);

    printf("Printing characters:");
    print(list1, printchar);
    printf(" : done\n\n");

    printf("Printing integers:");
    print(list2, printint);
    printf(" : done\n\n");

    printf("Printing strings:");
    print(list3, printstr);
    printf(" : done\n\n");

    printf("Printing composite:");
    print(list4, printcomp);
    printf(" : done\n");

    return 0;
}

void insert(List **p, void *data, unsigned int n)
{
    List *temp;
    int i;

    /* Error check is ignored */
    temp = malloc(sizeof(List));
    temp->data = malloc(n);
    for (i = 0; i < n; i++)
        *(char *)(temp->data + i) = *(char *)(data + i);
    temp->next = *p;
    *p = temp;
}

void print(List *p, void (*f)(void *))
{
    while (p)
    {
        (*f)(p->data);
        p = p->next;
    }
}

void printstr(void *str)
{
    printf(" \"%s\"", (char *)str);
}

void printint(void *n)
{
    printf(" %d", *(int *)n);
}

void printchar(void *c)
{
    printf(" %c", *(char *)c);
}

void printcomp(void *comp)
{
    struct check temp = *(struct check *)comp;
    printf(" '%d:%c:%f", temp.i, temp.c, temp.d);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(6 / 301)

How do you reverse a linked list without using any C pointers?

One way is to reverse the data in the nodes without changing the pointers themselves. One can also create a new linked list which is the reverse of the original linked list. A simple C program can do that for you. Please note that you would still use the "next" pointer fields to traverse through the linked list (So in effect, you are using the pointers, but you are not changing them when reversing the linked list).

Linked Lists
(7 / 301)

How would you detect a loop in a linked list? Write a C program to detect a loop in a linked list.

This is also one of the classic interview questions

There are multiple answers to this problem. Here are a few C programs to attack this problem.

Brute force method

Have a double loop, where you check the node pointed to by the outer loop, with every node of the inner loop.

```
typedef struct node
{
  void *data;
  struct node *next;
}mynode;


mynode * find_loop(NODE * head)
{
  mynode *current = head;

  while(current->next != NULL)
  {
    mynode *temp = head;
    while(temp->next != NULL && temp != current)
    {
      if(current->next == temp)
      {
        printf("\nFound a loop.");
        return current;
      }
      temp = temp->next;
    }
    current = current->next;
  }
  return NULL;
}
```

Visited flag

Have a visited flag in each node of the linked list. Flag it as visited when you reach the node. When you reach a node and the flag is already flagged as visited, then you know there is a loop in the linked list.

Fastest method

Have 2 pointers to start of the linked list. Increment one pointer by 1 node and the other by 2 nodes. If there's a loop, the 2nd pointer will meet the 1st pointer somewhere. If it does, then you know there's one.

Here is some code

```
p=head;
q=head->next;

while(p!=NULL && q!=NULL)
{
  if(p==q)
  {
    //Loop detected!
    exit(0);
  }
  p=p->next;
  q=(q->next)?(q->next->next):q->next;
}

// No loop.
```

The next question is how do you delete/eliminate the loop in a linked list once you detect it? I leave it up to you to do that!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(8 / 301)

How do you find the middle of a linked list? Write a C program to return the middle of a linked list

Another popular interview question

Here are a few C program snippets to give you an idea of the possible solutions.

Method1 (Uses one slow pointer and one fast pointer)

```
#include<stdio.h>
#include<ctype.h>

typedef struct node
{
  int value;
  struct node *next;
  struct node *prev;
}mynode ;


void add_node(struct node **head, int value);
void print_list(char *listName, struct node *head);
void getTheMiddle(mynode *head);

// The main function..
int main()
{
   mynode *head;
   head = (struct node *)NULL;

   add_node(&head, 1);
   add_node(&head, 10);
   add_node(&head, 5);
   add_node(&head, 70);
   add_node(&head, 9);
   add_node(&head, -99);
   add_node(&head, 0);
   add_node(&head, 555);
   add_node(&head, 55);

   print_list("myList", head);
   getTheMiddle(head);

   getch();
   return(0);
}
```

```
// This function uses one slow and one fast
// pointer to get to the middle of the LL.
//
// The slow pointer is advanced only by one node
// and the fast pointer is advanced by two nodes!

void getTheMiddle(mynode *head)
{
  mynode *p = head;
  mynode *q = head;

  if(q!=NULL)
  {
       while((q->next)!=NULL && (q->next->next)!=NULL)
       {
          p=(p!=(mynode *)NULL?p->next:(mynode *)NULL);
          q=(q!=(mynode *)NULL?q->next:(mynode *)NULL);
          q=(q!=(mynode *)NULL?q->next:(mynode *)NULL);
       }
       printf("The middle element is [%d]",p->value);
  }
}



// Function to add a node
void add_node(struct node **head, int value)
{
  mynode *temp, *cur;
  temp = (mynode *)malloc(sizeof(mynode));
  temp->next=NULL;
  temp->prev=NULL;

  if(*head == NULL)
  {
     *head=temp;
     temp->value=value;
  }
  else
  {
   for(cur=*head;cur->next!=NULL;cur=cur->next);
   cur->next=temp;
   temp->prev=cur;
   temp->value=value;
  }
}



// Function to print the linked list...
void print_list(char *listName, struct node *head)
{
  mynode *temp;

  printf("\n[%s] -> ", listName);
  for(temp=head;temp!=NULL;temp=temp->next)
  {
    printf("[%d]->",temp->value);
  }

  printf("NULL\n");

}
```

Here p moves one step, where as q moves two steps, when q reaches end, p will be at the middle of the linked list.

Method2(Uses a counter)

```
#include<stdio.h>
#include<ctype.h>

typedef struct node
{
```

```
  int value;
  struct node *next;
  struct node *prev;
}mynode ;



void add_node(struct node **head, int value);
void print_list(char *listName, struct node *head);
mynode *getTheMiddle(mynode *head);

// The main function..
int main()
{
   mynode *head, *middle;
   head = (struct node *)NULL;

   add_node(&head, 1);
   add_node(&head, 10);
   add_node(&head, 5);
   add_node(&head, 70);
   add_node(&head, 9);
   add_node(&head, -99);
   add_node(&head, 0);
   add_node(&head, 555);
   add_node(&head, 55);

   print_list("myList", head);
   middle = getTheMiddle(head);
   printf("\nMiddle node -> [%d]\n\n", middle->value);

   getch();
   return(0);
}


// Function to get to the middle of the LL
mynode *getTheMiddle(mynode *head)
{
  mynode *middle = (mynode *)NULL;
  int i;

  for(i=1; head!=(mynode *)NULL; head=head->next,i++)
  {
     if(i==1)
        middle=head;
     else if ((i%2)==1)
        middle=middle->next;
   }

   return middle;
}


// Function to add a new node to the LL
void add_node(struct node **head, int value)
{
  mynode *temp, *cur;
  temp = (mynode *)malloc(sizeof(mynode));
  temp->next=NULL;
  temp->prev=NULL;

  if(*head == NULL)
  {
     *head=temp;
     temp->value=value;
  }
  else
  {
   for(cur=*head;cur->next!=NULL;cur=cur->next);
   cur->next=temp;
   temp->prev=cur;
   temp->value=value;
  }
}


// Function to print the LL
void print_list(char *listName, struct node *head)
```

```
{
  mynode *temp;

  printf("\n[%s] -> ", listName);
  for(temp=head;temp!=NULL;temp=temp->next)
  {
    printf("[%d]->",temp->value);
  }

  printf("NULL\n");

}
```

In a similar way, we can find the 1/3 th node of linked list by changing (i%2==1) to (i%3==1) and in the same way we can find nth node of list by changing (i%2==1) to (i%n==1) but make sure ur (n<=i).

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(9 / 301) *

If you are using C language to implement the heterogeneous linked list, what pointer type will you use?

The heterogeneous linked list contains different data types in its nodes and we need a link, pointer to connect them. It is not possible to use ordinary pointers for this. So we go for void pointer. Void pointer is capable of storing pointer to any type as it is a generic pointer type.

Check out the C program to implement a Generic linked list in the same FAQ.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(10 / 301)

How to compare two linked lists? Write a C program to compare two linked lists.

Here is a simple C program to accomplish the same.

```
int compare_linked_lists(struct node *q, struct node *r)
{
    static int flag;

    if((q==NULL ) && (r==NULL))
    {
         flag=1;
    }
    else
    {
        if(q==NULL || r==NULL)
        {
            flag=0;
        }
        if(q->data!=r->data)
        {
            flag=0;
        }
        else
        {
           compare_linked_lists(q->link,r->link);
        }
    }
    return(flag);
}
```

Another way is to do it on similar lines as `strcmp()` compares two strings, character by character (here each node is like a character).

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(11 / 301)

How to create a copy of a linked list? Write a C program to create a copy of a linked list.

Check out this C program which creates an exact copy of a linked list.

```
copy_linked_lists(struct node *q, struct node **s)
{
    if(q!=NULL)
    {
        *s=malloc(sizeof(struct node));
        (*s)->data=q->data;
        (*s)->link=NULL;
        copy_linked_list(q->link, &((*s)->link));
    }
}
```

Linked Lists
(12 / 301)

Write a C program to free the nodes of a linked list

Before looking at the answer, try writing a simple C program (with a for loop) to do this. Quite a few people get this wrong.

This is the wrong way to do it

```
struct list *listptr, *nextptr;
for(listptr = head; listptr != NULL; listptr = listptr->next)
{
  free(listptr);
}
```

If you are thinking why the above piece of code is wrong, note that once you free the listptr node, you cannot do something like listptr = listptr->next!. Since listptr is already freed, using it to get listptr->next is illegal and can cause unpredictable results!

This is the right way to do it

```
struct list *listptr, *nextptr;
for(listptr = head; listptr != NULL; listptr = nextptr)
{
  nextptr = listptr->next;
  free(listptr);
}
head = NULL;
```

After doing this, make sure you also set the head pointer to NULL!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Can we do a Binary search on a linked list?

Great C datastructure question!

The answer is ofcourse, you can write a C program to do this. But, the question is, do you really think it will be as efficient as a C program which does a binary search on an array?

Think hard, real hard.

Do you know what exactly makes the binary search on an array so fast and efficient? Its the ability to access any element in the array in constant time. This is what makes it so fast. You can get to the middle of the array just by saying array[middle]!. Now, can you do the same with a linked list? The answer is No. You will have to write your own, possibly inefficient algorithm to get the value of the middle node of a linked list. In a linked list, you loosse the ability to get the value of any node in a constant time.

One solution to the inefficiency of getting the middle of the linked list during a binary search is to have the first node contain one additional pointer that points to the node in the middle. Decide at the first node if you need to check the first or the second half of the linked list. Continue doing that with each half-list.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(14 / 301)

Write a C program to return the nth node from the end of a linked list.

Here is a solution which is often called as the solution that uses frames.

Suppose one needs to get to the 6th node from the end in this LL. First, just keep on incrementing the first pointer (ptr1) till the number of increments cross n (which is 6 in this case)

```
STEP 1    :   1(ptr1,ptr2) -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10

STEP 2    :   1(ptr2) -> 2 -> 3 -> 4 -> 5 -> 6(ptr1) -> 7 -> 8 -> 9 -> 10
```

Now, start the second pointer (ptr2) and keep on incrementing it till the first pointer (ptr1) reaches the end of the LL.

```
STEP 3    :   1 -> 2 -> 3 -> 4(ptr2) -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 (ptr1)
```

So here you have!, the 6th node from the end pointed to by ptr2!

Here is some C code..

```
struct node
{
  int data;
  struct node *next;
}mynode;


mynode * nthNode(mynode *head, int n /*pass 0 for last node*/)
{
  mynode *ptr1,*ptr2;
  int count;

  if(!head)
  {
    return(NULL);
  }

  ptr1  = head;
  ptr2  = head;
  count = 0;

  while(count < n)
  {
     count++;
     if((ptr1=ptr1->next)==NULL)
     {
```

```
        //Length of the linked list less than n. Error.
        return(NULL);
     }
  }

  while((ptr1=ptr1->next)!=NULL)
  {
    ptr2=ptr2->next;
  }

  return(ptr2);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(15 / 301)

How would you find out if one of the pointers in a linked list is corrupted or not?

This is a really good interview question. The reason is that linked lists are used in a wide variety of scenarios and being able to detect and correct pointer corruptions might be a very valuable tool. For example, data blocks associated with files in a file system are usually stored as linked lists. Each data block points to the next data block. A single corrupt pointer can cause the entire file to be lost!

- Discover and fix bugs when they corrupt the linked list and not when effect becomes visible in some other part of the program. Perform frequent consistency checks (to see if the linked list is indeed holding the data that you inserted into it).
- It is good programming practice to set the pointer value to NULL immediately after freeing the memory pointed at by the pointer. This will help in debugging, because it will tell you that the object was freed somewhere beforehand. Keep track of how many objects are pointing to a object using reference counts if required.
- Use a good debugger to see how the datastructures are getting corrupted and trace down the problem. Debuggers like ddd on linux and memory profilers like Purify, Electric fence are good starting points. These tools should help you track down heap corruption issues easily.
- Avoid global variables when traversing and manipulating linked lists. Imagine what would happen if a function which is only supposed to traverse a linked list using a global head pointer accidently sets the head pointer to NULL!.
- Its a good idea to check the addNode() and the deleteNode() routines and test them for all types of scenarios. This should include tests for inserting/deleting nodes at the front/middle/end of the linked list, working with an empty linked list, running out of memory when using malloc() when allocating memory for new nodes, writing through NULL pointers, writing more data into the node fields then they can hold (resulting in corrupting the (probably adjacent) "prev" and "next" pointer fields), make sure bug fixes and enhancements to the linked list code are reviewed and well tested (a lot of bugs come from quick and dirty bug fixing), log and handle all possible errors (this will help you a lot while debugging), add multiple levels of logging so that you can dig through the logs. The list is endless...
- Each node can have an extra field associated with it. This field indicates the number of nodes after this node in the linked list. This extra field needs to be kept up-to-date when we inserte or delete nodes in the linked list (It might become slightly complicated when insertion or deletion happens not at end, but anywhere in the linked list). Then, if for any node, p->field > 0 and p->next == NULL, it surely points to a pointer corruption.
- You could also keep the count of the total number of nodes in a linked list and use it to check if the list is indeed having those many nodes or not.

The problem in detecting such pointer corruptions in C is that its only the programmer who knows that the pointer is corrupted. The program has no way of knowing that something is wrong. So the best way to fix these errors is check your logic and test your code to the maximum possible extent. I am not aware of ways in C to recover the lost nodes of a corrupted linked list. C does not track pointers so there is no good way to know if an arbitrary pointer has been corrupted or not. The platform may have a library service that checks if a pointer points to valid memory (for instance on Win32 there is a IsBadReadPtr, IsBadWritePtr API.) If you detect a cycle in the link list, it's definitely bad. If it's a doubly linked list you can verify, pNode->Next->Prev == pNode.

I have a hunch that interviewers who ask this question are probably hinting at something called Smart Pointers in C++. Smart pointers are particularly useful in the face of exceptions as they ensure proper destruction of dynamically allocated objects. They can also be used to keep track of dynamically allocated objects shared by multiple owners. This topic is out of scope here, but you can find lots of material on the Internet for Smart Pointers.

If you have better answers to this question, let me know!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(16 / 301)

Write a C program to insert nodes into a linked list in a sorted fashion

The solution is to iterate down the list looking for the correct place to insert the new node. That could be the end of the list, or a point just before a node which is larger than the new node.

Note that we assume the memory for the new node has already been allocated and a pointer to that memory is being passed to this function.

```
// Special case code for the head end
void linkedListInsertSorted(struct node** headReference, struct node* newNode)
{
  // Special case for the head end
  if (*headReference == NULL || (*headReference)->data >= newNode->data)
  {
     newNode->next = *headReference;
     *headReference = newNode;
  }
  else
  {
     // Locate the node before which the insertion is to happen!
     struct node* current = *headReference;
     while (current->next!=NULL && current->next->data < newNode->data)
     {
        current = current->next;
     }
     newNode->next = current->next;
     current->next = newNode;
   }
}
```

Linked Lists
(17 / 301)

Write a C program to remove duplicates from a sorted linked list

As the linked list is sorted, we can start from the beginning of the list and compare adjacent nodes. When adjacent nodes are the same, remove the second one. There's a tricky case where the node after the next node needs to be noted before the deletion.

```
// Remove duplicates from a sorted list
void RemoveDuplicates(struct node* head)
{
  struct node* current = head;
  if (current == NULL) return; // do nothing if the list is empty

  // Compare current node with next node
  while(current->next!=NULL)
  {
      if (current->data == current->next->data)
      {
         struct node* nextNext = current->next->next;
         free(current->next);
         current->next = nextNext;
      }
      else
      {
         current = current->next; // only advance if no deletion
      }
   }
}
```

Linked Lists
(18 / 301)

How to read a singly linked list backwards?

Use Recursion.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Linked Lists
(19 / 301) *

How can I search for data in a linked list?

The only way to search a linked list is with a linear search, because the only way a linked list?s members can be accessed is sequentially. Sometimes it is quicker to take the data from a linked list and store it in a different data structure so that searches can be more efficient.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(20 / 301)

Write your own C program to implement the atoi() function

The prototype of the atoi() function is ...

```
int atoi(const char *string);
```

Here is a C program which explains a different way of coding the atoi() function in the C language.

```
#include<stdio.h>

int myatoi(const char *string);

int main(int argc, char* argv[])
{
  printf("\n%d\n", myatoi("1998"));
  getch();
  return(0);
}

int myatoi(const char *string)
{
    int i;
    i=0;
    while(*string)
    {
        i=(i<<3) + (i<<1) + (*string - '0');
        string++;

        // Dont increment i!

    }
    return(i);
}
```

Try working it out with a small string like "1998", you will find out it does work!.

Ofcourse, there is also the trivial method ....

```
"1998" == 8 + (10 * 9) + (100 * 9) + (1 * 1000) = 1998
```

This can be done either by going from right to left or left to right in the string

One solution is given below

```
int myatoi(const char* string)
{
  int value = 0;

  if (string)
  {
    while (*string && (*string <= '9' && *string >= '0'))
    {
      value = (value * 10) + (*string - '0');
      string++;
    }
  }
  return value;
}
```

Note that these functions have no error handling incorporated in them (what happens if someone passes non-numeric data (say "1A998"), or negative numeric strings (say "-1998")). I leave it up to you to add these cases. The essense is to understand the core logic first.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(21 / 301)

Implement the memmove() function. What is the difference between the memmove() and memcpy () function?

One more most frequently asked interview question!.

memmove() offers guaranteed behavior if the source and destination arguments overlap. memcpy() makes no such guarantee, and may therefore be more efficient to implement. It's always safer to use memmove().

Note that the prototype of memmove() is ...

```
void *memmove(void *dest, const void *src, size_t count);
```

Here is an implementation..

```
#include <stdio.h>
#include <string.h>

void *mymemmove(void *dest, const void *src, size_t count);

int main(int argc, char* argv[])
{
  char *p1, *p2;
  char *p3, *p4;
  int  size;

  printf("\n--------------------------------\n");

  /* ----------------------------------------
   *
   * CASE 1 : From (SRC) < To (DEST)
   *
   *     +--+--------------------+--+
   *     |  |                    |  |
   *     +--+--------------------+--+
   *     ^  ^
   *     |  |
   *   From To
   *
   * ---------------------------------------- */

  p1 = (char *) malloc(12);
  memset(p1,12,'\0');
  size=10;

  strcpy(p1,"ABCDEFGHI");
```

```
  p2 = p1 + 2;

  printf("\n--------------------------------\n");
  printf("\nFrom (before) = [%s]",p1);
  printf("\nTo (before)   = [%s]",p2);

  mymemmove(p2,p1,size);

  printf("\n\nFrom (after) = [%s]",p1);
  printf("\nTo (after)   = [%s]",p2);

  printf("\n--------------------------------\n");



  /* ---------------------------------------
   *
   * CASE 2 : From (SRC) > To (DEST)
   *
   *     +--+---------------------+--+
   *     |  |                     |  |
   *     +--+---------------------+--+
   *     ^  ^
   *     |  |
   *    To From
   *
   * --------------------------------------- */


  p3 = (char *) malloc(12);
  memset(p3,12,'\0');
  p4 = p3 + 2;

  strcpy(p4, "ABCDEFGHI");

  printf("\nFrom (before) = [%s]",p4);
  printf("\nTo (before)   = [%s]",p3);

  mymemmove(p3, p4, size);

  printf("\n\nFrom (after) = [%s]",p4);
  printf("\nTo (after)   = [%s]",p3);

  printf("\n--------------------------------\n");


  /* ---------------------------------------
   *
   * CASE 3 : No overlap
   *
   * --------------------------------------- */

  p1 = (char *) malloc(30);
  memset(p1,30,'\0');
  size=10;

  strcpy(p1,"ABCDEFGHI");

  p2 = p1 + 15;

  printf("\n--------------------------------\n");
  printf("\nFrom (before) = [%s]",p1);
  printf("\nTo (before)   = [%s]",p2);

  mymemmove(p2,p1,size);

  printf("\n\nFrom (after) = [%s]",p1);
  printf("\nTo (after)   = [%s]",p2);

  printf("\n--------------------------------\n");

  printf("\n\n");

  return 0;
}



void *mymemmove(void *to, const void *from, size_t size)
{
    unsigned char *p1;
```

```
    const unsigned char *p2;

    p1 = (unsigned char *) to;
    p2 = (const unsigned char *) from;

    p2 = p2 + size;

    // Check if there is an overlap or not.
    while (p2 != from && --p2 != to);


    if (p2 != from)
    {
        // Overlap detected!

        p2  = (const unsigned char *) from;
        p2  = p2 + size;
        p1  = p1 + size;

        while (size-- != 0)
        {
            *--p1 = *--p2;
        }
    }
    else
    {
        // No overlap OR they overlap as CASE 2 above.
        // memcopy() would have done this directly.

        while (size-- != 0)
        {
            *p1++ = *p2++;
        }
    }

    return(to);
}
```

And here is the output

```
--------------------------------

From (before) = [ABCDEFGHI]
To   (before) = [CDEFGHI]


From (after) = [ABABCDEFGHI]
To   (after) = [ABCDEFGHI]

--------------------------------

From (before) = [ABCDEFGHI]
To   (before) = [α └ABCDEFGHI]


From (after) = [CDEFGHI]
To   (after) = [ABCDEFGHI]

--------------------------------

From (before) = [ABCDEFGHI]
To   (before) = [FEδ•&:F]


From (after) = [ABCDEFGHI]
To (after)   = [ABCDEFGHI]

--------------------------------
```

So then, whats the difference between the implementation of memmove() and memcpy(). Its just that memcpy() will not care if the memories overlap and will either copy from left to right or right to left without checking which method to used depending on the type of the overlap. Also note that the C code proves that the results are the same irrespective of the Endian-ness of the machine.

* *

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(22 / 301)

Write C code to implement the strstr() (search for a substring) function.

This is also one of the most frequently asked interview questions. Its asked almost 99% of the times. Here are a few C programs to implement your own strstr() function.

There are a number of ways to find a string inside another string. Its important to be aware of these algorithms than to memorize them. Some of the fastest algorithms are quite tough to understand!.

Method1

The first method is the classic Brute force method. The Brute Force algorithm checks, at all positions in the text between 0 and (n-m), if an occurrence of the pattern starts at that position or not. Then, after each successfull or unsuccessful attempt, it shifts the pattern exactly one position to the right. The time complexity of this searching phase is O(mn). The expected number of text character comparisons is 2n.

Here 'n' is the size of the string in which the substring of size 'm' is being searched for.

Here is some code (which works!)

```
#include<stdio.h>

void BruteForce(char *x /* pattern */,
                int m   /* length of the pattern */,
                char *y /* actual string being searched */,
                int n   /* length of this string */)
{
   int i, j;
   printf("\nstring    : [%s]"
          "\nlength    : [%d]"
          "\npattern   : [%s]"
          "\nlength    : [%d]\n\n", y,n,x,m);


   /* Searching */
   for (j = 0; j <= (n - m); ++j)
   {
      for (i = 0; i < m && x[i] == y[i + j]; ++i);
        if (i >= m) {printf("\nMatch found at\n\n->[%d]\n->[%s]\n",j,y+j);}
   }
}


int main()
{
  char *string  = "hereroheroero";
  char *pattern = "hero";

  BF(pattern,strlen(pattern),string,strlen(string));
  printf("\n\n");
```

```
  return(0);
}
```

This is how the comparison happens visually

```
hereroheroero
   !
hero

hereroheroero
 !
 hero

hereroheroero
  !
  hero

hereroheroero
   !
   hero

hereroheroero
    !
    hero

hereroheroero
     !
     hero

hereroheroero
      |||| ----> Match!
      hero

hereroheroero
       !
       hero

hereroheroero
        !
        hero

hereroheroero
         !
         hero
```

Method2

The second method is called the Rabin-Karp method.

Instead of checking at each position of the text if the pattern occurs or not, it is better to check first if the contents of the current string "window" looks like the pattern or not. In order to check the resemblance between these two patterns, a hashing function is used. Hashing a string involves computing a numerical value from the value of its characters using a hash function.

The Rabin-Karp method uses the rule that if two strings are equal, their hash values must also be equal. Note that the converse of this statement is not always true, but a good hash function tries to reduce the number of such hash collisions. Rabin-Karp computes hash value of the pattern, and then goes through the string computing hash values of all of its substrings and checking if the pattern's hash value is equal to the substring hash value, and advancing by 1 character every time. If the two hash values are the same, then the algorithm verifies if the two string really are equal, rather than this being a fluke of the hashing scheme. It uses regular string comparison for this final check. Rabin-Karp is an algorithm of choice for multiple pattern search. If we want to find any of a large number, say k, fixed length patterns in a text, a variant Rabin-Karp that uses a hash table to check whether the hash of a given string belongs to a set of hash values of patterns we are looking for. Other algorithms can search for a single pattern in time order O(n), hence they will search for k patterns in time order O (n*k). The variant Rabin-Karp will still work in time order O(n) in the best and average case because a hash table allows to check whether or not substring hash equals any of the pattern hashes in time order of O(1).

Here is some code (not working though!)

```
#include<stdio.h>

hashing_function()
{
  // A hashing function to compute the hash values of the strings.
  ....
}

void KarpRabinR(char *x, int m, char *y, int n)
{
   int hx, hy, i, j;

   printf("\nstring    : [%s]"
          "\nlength    : [%d]"
          "\npattern   : [%s]"
          "\nlength    : [%d]\n\n", y,n,x,m);


   /* Preprocessing  phase */
   Do preprocessing here..

   /* Searching */
   j = 0;
   while (j <= n-m)
   {
      if (hx == hy && memcmp(x, y + j, m) == 0)
      {
         // Hashes match and so do the actual strings!
         printf("\nMatch found at : [%d]\n",j);
      }

      hy = hashing_function(y[j], y[j + m], hy);
      ++j;
   }
}


int main()
{

  char *string="hereroheroero";
  char *pattern="hero";

  KarpRabin(pattern,strlen(pattern),string,strlen(string));

  printf("\n\n");
  return(0);

}
```

This is how the comparison happens visually

```
hereroheroero
   !
hero


hereroheroero
 !
 hero


hereroheroero
  !
  hero


hereroheroero
   !
   hero


hereroheroero
    !
```

```
    hero


hereroheroero
     !
     hero


hereroheroero
      |||| ----> Hash values match, so do the strings!
      hero


hereroheroero
       !
       hero


hereroheroero
        !
        hero


hereroheroero
         !
         hero
```

Method3

The Knuth-Morris-Pratt or the Morris-Pratt algorithms are extensions of the basic Brute Force algorithm. They use precomputed data to skip forward not by 1 character, but by as many as possible for the search to succeed.

Here is some code

```
void preComputeData(char *x, int m, int Next[])
{
   int i, j;
   i = 0;
   j = Next[0] = -1;

   while (i < m)
   {
      while (j > -1 && x[i] != x[j])
         j = Next[j];
      Next[++i] = ++j;

   }
}


void MorrisPrat(char *x, int m, char *y, int n)
{
   int i, j, Next[1000];

   /* Preprocessing */
   preComputeData(x, m, Next);

   /* Searching */
   i = j = 0;
   while (j < n)
   {
      while (i > -1 && x[i] != y[j])
         i = Next[i];
      i++;
      j++;
      if (i >= m)
      {
         printf("\nMatch found at : [%d]\n",j - i);
         i = Next[i];
      }
   }
}


int main()
{
```

```
  char *string="hereroheroero";
  char *pattern="hero";

  MorrisPrat(pattern,strlen(pattern),string,strlen(string));

  printf("\n\n");
  return(0);
}
```

This is how the comparison happens visually

```
hereroheroero
   !
hero


hereroheroero
   !
   hero


hereroheroero
    !
    hero


hereroheroero
     !
     hero



hereroheroero
      |||| ----> Match found!
      hero


hereroheroero
          !
          hero
```

Method4

The Boyer Moore algorithm is the fastest string searching algorithm. Most editors use this algorithm.

It compares the pattern with the actual string from right to left. Most other algorithms compare from left to right. If the character that is compared with the rightmost pattern symbol does not occur in the pattern at all, then the pattern can be shifted by m positions behind this text symbol.

The following example illustrates this situation.

Example:

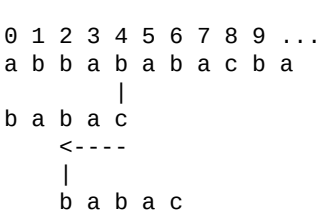
```
0 1 2 3 4 5 6 7 8 9 ...
a b b a d a b a c b a
        | |
b a b a c |
  <------ |
          |
          b a b a c
```

The comparison of "d" with "c" at position 4 does not match. "d" does not occur in the pattern. Therefore, the pattern cannot match at any of the positions 0,1,2,3,4, since all corresponding windows contain a "d". The pattern can be shifted to position 5. The best case for the Boyer-Moore algorithm happens if, at each search attempt the first compared character does not occur in the pattern. Then the algorithm requires only O(n/m) comparisons .

Bad character heuristics

This method is called bad character heuristics. It can also be applied if the bad character (the character that causes a mismatch), occurs somewhere else in the pattern. Then the pattern can be shifted so that it is aligned to this text symbol. The next example illustrates this situation.

Example:

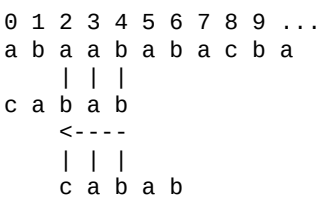


Comparison between "b" and "c" causes a mismatch. The character "b" occurs in the pattern at positions 0 and 2. The pattern can be shifted so that the rightmost "b" in the pattern is aligned to "b".

Good suffix heuristics

Sometimes the bad character heuristics fails. In the following situation the comparison between "a" and "b" causes a mismatch. An alignment of the rightmost occurence of the pattern symbol a with the text symbol a would produce a negative shift. Instead, a shift by 1 would be possible. However, in this case it is better to derive the maximum possible shift distance from the structure of the pattern. This method is called good suffix heuristics.

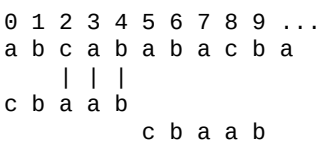
Example:



The suffix "ab" has matched. The pattern can be shifted until the next occurence of ab in the pattern is aligned to the text symbols ab, i.e. to position 2.

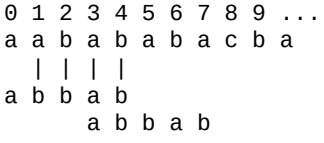
In the following situation the suffix "ab" has matched. There is no other occurence of "ab" in the pattern.Therefore, the pattern can be shifted behind "ab", i.e. to position 5.

Example:



In the following situation the suffix "bab" has matched. There is no other occurence of "bab" in the pattern. But in this case the pattern cannot be shifted to position 5 as before, but only to position 3, since a prefix of the pattern "ab" matches the end of "bab". We refer to this situation as case 2 of the good suffix heuristics.

Example:



The pattern is shifted by the longest of the two distances that are given by the bad character and the good suffix heuristics.

The Boyer-Moore algorithm uses two different heuristics for determining the maximum possible shift distance in case of a mismatch: the "bad character" and the "good suffix" heuristics. Both heuristics can lead to a shift distance of m. For the bad character heuristics this is the case, if the first comparison causes a mismatch and the corresponding text symbol does not occur in the pattern at all. For the good suffix heuristics this is the case, if only the first comparison was a match, but that symbol does not occur elsewhere in the pattern.

A lot of these algorithms have been explained here with good visualizations. Remember, again that its sufficient to know the basic Brute force algorithm and be aware of the other methods. No one expects you to know every possible algorithm on earth.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(23 / 301)

Write your own printf() function in C

This is again one of the most frequently asked interview questions. Here is a C program which implements a basic version of printf(). This is a really, really simplified version of printf(). Note carefully how floating point and other compilcated support has been left out. Also, note how we use low level puts() and putchar(). Dont make a fool of yourself by using printf() within the implementation of printf()!

```
#include<stdio.h>
#include<stdarg.h>

main()
{
    void myprintf(char *,...);
    char * convert(unsigned int, int);
    int i=65;
    char str[]="This is my string";
    myprintf("\nMessage = %s%d%x",str,i,i);
}

void myprintf(char * frmt,...)
{

    char *p;
    int i;
    unsigned u;
    char *s;
    va_list argp;


    va_start(argp, fmt);

    p=fmt;
    for(p=fmt; *p!='\0';p++)
    {
        if(*p=='%')
        {
            putchar(*p);continue;
        }

        p++;

        switch(*p)
        {
            case 'c' : i=va_arg(argp,int);putchar(i);break;
            case 'd' : i=va_arg(argp,int);
                           if(i<0){i=-i;putchar('-');}puts(convert(i,10));break;
            case 'o': i=va_arg(argp,unsigned int); puts(convert(i,8));break;
            case 's': s=va_arg(argp,char *); puts(s); break;
            case 'u': u=va_arg(argp,argp, unsigned int); puts(convert(u,10));break;
            case 'x': u=va_arg(argp,argp, unsigned int); puts(convert(u,16));break;
```

```
            case '%': putchar('%');break;
        }
    }

    va_end(argp);
}

char *convert(unsigned int, int)
{
    static char buf[33];
    char *ptr;

    ptr=&buf[sizeof(buff)-1];
    *ptr='\0';
    do
    {
        *--ptr="0123456789abcdef"[num%base];
        num/=base;
    }while(num!=0);
    return(ptr);
}
```

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(24 / 301)

Implement the strcpy() function.

Here are some C programs which implement the strcpy() function. This is one of the most frequently asked C interview questions.

Method1

```
char *mystrcpy(char *dst, const char *src)
{
  char *ptr;
  ptr = dst;
  while(*dst++=*src++);
  return(ptr);
}
```

The strcpy function copies src, including the terminating null character, to the location specified by dst. No overflow checking is performed when strings are copied or appended. The behavior of strcpy is undefined if the source and destination strings overlap. It returns the destination string. No return value is reserved to indicate an error.

Note that the prototype of strcpy as per the C standards is

```
char *strcpy(char *dst, const char *src);
```

Notice the const for the source, which signifies that the function must not change the source string in anyway!.

Method2

```
char *my_strcpy(char dest[], const char source[])
{
  int i = 0;
  while (source[i] != '\0')
  {
    dest[i] = source[i];
    i++;
  }
  dest[i] = '\0';
  return(dest);
}
```

Simple, isn't it?

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(25 / 301)

Implement the strcmp(str1, str2) function.

There are many ways one can implement the strcmp() function. Note that strcmp(str1,str2) returns a -ve number if str1 is alphabetically above str2, 0 if both are equal and +ve if str2 is alphabetically above str1.

Here are some C programs which implement the strcmp() function. This is also one of the most frequently asked interview questions. The prototype of strcmp() is

```
int strcmp( const char *string1, const char *string2 );
```

Here is some C code..

```
#include <stdio.h>

int mystrcmp(const char *s1, const char *s2);

int main()
{
  printf("\nstrcmp() = [%d]\n", mystrcmp("A","A"));
  printf("\nstrcmp() = [%d]\n", mystrcmp("A","B"));
  printf("\nstrcmp() = [%d]\n", mystrcmp("B","A"));
  return(0);
}

int mystrcmp(const char *s1, const char *s2)
{
    while (*s1==*s2)
    {
        if(*s1=='\0')
           return(0);
        s1++;
        s2++;
    }
    return(*s1-*s2);
}
```

And here is the output...

```
strcmp() = [0]
strcmp() = [-1]
strcmp() = [1]
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(26 / 301)

Implement the substr() function in C.

Here is a C program which implements the substr() function in C.

```
int main()
{
  char str1[] = "India";
  char str2[25];

  substr(str2, str1, 1, 3);
  printf("\nstr2 : [%s]", str2);
  return(0);
}

substr(char *dest, char *src, int position, int length)
{
    dest[0]='\0';
    strncat(dest, (src + position), length);
}
```

Here is another C program to do the same...

```
#include <stdio.h>
#include <conio.h>

void mySubstr(char *dest, char *src, int position, int length);

int main()
{
 char subStr[100];
 char str[]="My Name Is Sweet";

 mySubstr(subStr, str, 1, 5);
 printf("\nstr    = [%s]"
        "\nsubStr = [%s]\n\n",
        str, subStr);
 getch();
 return(0);
}

void mySubstr(char *dest, char *src, int position, int length)
{
  while(length > 0)
  {
    *dest = *(src+position);
    dest++;
```

```
    src++;
    length--;
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(27 / 301)

Write your own copy() function

Here is some C code that simulates a file copy action.

```
#include <stdio.h>              /* standard I/O routines. */
#define MAX_LINE_LEN 1000 /* maximum line length supported. */


void main(int argc, char* argv[])
{
    char* file_path_from;
    char* file_path_to;
    FILE* f_from;
    FILE* f_to;
    char buf[MAX_LINE_LEN+1];

    file_path_from = "<something>";
    file_path_to   = "<something_else>";

    f_from = fopen(file_path_from, "r");
    if (!f_from) {exit(1);}

    f_to = fopen(file_path_to, "w+");
    if (!f_to) {exit(1);}

    /* Copy source to target, line by line. */
    while (fgets(buf, MAX_LINE_LEN+1, f_from))
    {
        if (fputs(buf, f_to) == EOF){exit(1);}
    }

    if (!feof(f_from)){exit(1);}

    if (fclose(f_from) == EOF) {exit(1);}
    if (fclose(f_to) == EOF)   {exit(1);}

    return(0);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(28 / 301)

Write C programs to implement the toupper() and the isupper() functions

toUpper()

```
int toUpper(int ch)
{
    if(ch>='a' && c<='z')
        return('A' + ch - 'a');
    else
        return(ch);
}
```

isUpper()

```
int isUpper(int ch)
{
    if(ch>='A' && ch <='Z')
       return(1); //Yes, its upper!
    else
       return(0); // No, its lower!
}
```

Its important to know that the upper and lower case alphabets have corresponding integer values.

```
A-Z - 65-90
a-z - 97-122
```

Another way to do this conversion is to maintain a correspondance between the upper and lower case alphabets. The program below does that. This frees us from the fact that these alphabets have a corresponding integer values. I dont know what one should do for non-english alphabets. Do other languages have upper and lower case letters in the first place :) !

```
#include <string.h>

#define UPPER   "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
#define LOWER   "abcdefghijklmnopqrstuvwxyz"

int toUpper(int c)
{
    const char *upper;
```

```
    const char *const lower = LOWER;

    // Get the position of the lower case alphabet in the LOWER string using the strchr() function ..
    upper = ( ((CHAR_MAX >= c)&&(c > '\0')) ? strchr(lower, c) : NULL);


    // Now return the corresponding alphabet at that position in the UPPER string ..
    return((upper != NULL)?UPPER[upper - lower] : c);
}
```

Note that these routines dont have much error handling incorporated in them. Its really easy to add error handling to these routines or just leave it out (as I like it). This site consciously leaves out error handling for most of the programs to prevent unwanted clutter and present the core logic first.

* *

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(29 / 301)

Write a C program to implement your own strdup() function.

Here is a C program to implement the strdup() function.

```
char *mystrdup(char *s)
{
    char *result = (char*)malloc(strlen(s) + 1);
    if (result == (char*)0){return (char*)0;}
    strcpy(result, s);
    return result;
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own ....
(30 / 301)

Write a C program to implement the strlen() function

The prototype of the strlen() function is...

```
size_t strlen(const char *string);
```

Here is some C code which implements the strlen() function....

```
int my_strlen(char *string)
{
  int length;
  for (length = 0; *string != '\0', string++)
  {
    length++;
  }
  return(length);
}
```

Also, see another example

```
int my_strlen(char *s)
{
  char *p=s;

  while(*p!='\0')
    p++;

  return(p-s);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own strcat() function

Here is a C function which implements the strcat() function...

```
/* Function to concatenate string t to end of s; return s */
char *myStrcat(char *s, const char *t)
{
    char *p = s;

    if (s == NULL || t == NULL)
        return s;   /* we need not have to do anything */

    while (*s)
        s++;

    while (*s++ = *t++)
        ;

    return p;
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to swap two variables without using a temporary variable

This questions is asked almost always in every interview.

The best way to swap two variables is to use a temporary variable.

```
int a,b,t;

t = a;
a = b;
b = t;
```

There is no way better than this as you will find out soon. There are a few slick expressions that do swap variables without using temporary storage. But they come with their own set of problems.

Method1 (The XOR trick)

```
a ^= b ^= a ^= b;
```

Although the code above works fine for most of the cases, it tries to modify variable 'a' two times between sequence points, so the behavior is undefined. What this means is it wont work in all the cases. This will also not work for floating-point values. Also, think of a scenario where you have written your code like this

```
swap(int *a, int *b)
{
  *a ^= *b ^= *a ^= *b;
}
```

Now, if suppose, by mistake, your code passes the pointer to the same variable to this function. Guess what happens? Since Xor'ing an element with itself sets the variable to zero, this routine will end up setting the variable to zero (ideally it should have swapped the variable with itself). This scenario is quite possible in sorting algorithms which sometimes try to swap a variable with itself (maybe due to some small, but not so fatal coding error). One solution to this problem is to check if the numbers to be swapped are already equal to each other.

```
swap(int *a, int *b)
{
  if(*a!=*b)
  {
    *a ^= *b ^= *a ^= *b;
  }
}
```

Method2

This method is also quite popular

```
a=a+b;
b=a-b;
a=a-b;
```

But, note that here also, if a and b are big and their addition is bigger than the size of an int, even this might end up giving you wrong results.

Method3

One can also swap two variables using a macro. However, it would be required to pass the type of the variable to the macro. Also, there is an interesting problem using macros. Suppose you have a swap macro which looks something like this

```
#define swap(type,a,b) type temp;temp=a;a=b;b=temp;
```

Now, think what happens if you pass in something like this

```
swap(int,temp,a) //You have a variable called "temp" (which is quite possible).
```

This is how it gets replaced by the macro

```
int temp;
temp=temp;
temp=b;
b=temp;
```

Which means it sets the value of "b" to both the variables!. It never swapped them! Scary, isn't it?

So the moral of the story is, dont try to be smart when writing code to swap variables. Use a temporary variable. Its not only fool proof, but also easier to understand and maintain.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is the 8 queens problem? Write a C program to solve it.

The 8 queens problem is a classic problem using the chess board. This problem is to place 8 queens on the chess board so that they do not attack each other horizontally, vertically or diagonally. It turns out that there are 12 essentially distinct solutions to this problem.

Suppose we have an array t[8] which keeps track of which column is occupied in which row of the chess board. That is, if t[0]==5, then it means that the queen has been placed in the fifth column of the first row. We need to couple the backtracking algorithm with a procedure that checks whether the tuple is completable or not, i.e. to check that the next placed queen 'i' is not menaced by any of the already placed 'j' (j < i):

```
Two queens are in the same column         if t[i]=t[j]
Two queens are in the same major diagonal if (t[i]-t[j])=(i-j)
two queens are in the same minor diagonal if (t[j]-t[i])=(i-j)
```

Here is some working C code to solve this problem using backtracking

```
#include<stdio.h>
static int t[10]={-1};
void queens(int i);
int empty(int i);

void print_solution();

int main()
{
  queens(1);
  print_solution();
  return(0);
}

void queens(int i)
{
  for(t[i]=1;t[i]<=8;t[i]++)
  {
    if(empty(i))
    {
       if(i==8)
       {
          print_solution();
          /* If this exit is commented, it will show ALL possible combinations */
          exit(0);
       }
       else
       {
          // Recurse!
```

```
          queens(i+1);
        }

    }// if

  }// for
}


int empty(int i)
{
  int j;
  j=1;

  while(t[i]!=t[j] && abs(t[i]-t[j])!=(i-j) &&j<8)j++;

  return((i==j)?1:0);
}


void print_solution()
{
  int i;
  for(i=1;i<=8;i++)printf("\nt[%d] = [%d]",i,t[i]);
}
```

And here is one of the possible solutions

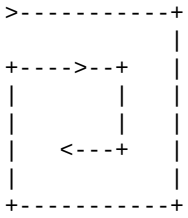
```
t[1] = [1] // This means the first square of the first row.
t[2] = [5] // This means the fifth square of the second row.
t[3] = [8] ..
t[4] = [6] ..
t[5] = [3] ..
t[6] = [7] ..
t[7] = [2] ..
t[8] = [4] // This means the fourth square of the last row.
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to print a square matrix helically.

Here is a C program to print a matrix helically. Printing a matrix helically means printing it in this spiral fashion



This is a simple program to print a matrix helically.

```
#include<stdio.h>

/* HELICAL MATRIX */

int main()
{
	int arr[][4] = { {1,2,3,4},
			 {5,6,7,8},
			 {9,10,11,12},
			 {13, 14, 15, 16}
			};

	int i, j, k,middle,size;
	printf("\n\n");
	size = 4;

	for(i=size-1, j=0; i>0; i--, j++)
	{
		for(k=j; k<i; k++) printf("%d ", arr[j][k]);
		for(k=j; k<i; k++) printf("%d ", arr[k][i]);
		for(k=i; k>j; k--) printf("%d ", arr[i][k]);
		for(k=i; k>j; k--) printf("%d ", arr[k][j]);
	}

	middle = (size-1)/2;
	if (size % 2 == 1) printf("%d", arr[middle][middle]);
	printf("\n\n");
	return 1;
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to reverse a string

There are a number of ways one can reverse strings. Here are a few of them. These should be enough to impress the interviewer! The methods span from recursive to non-recursive (iterative).

Also note that there is a similar question about reversing the words in a sentence, but still keeping the words in place. That is

```
I am a good boy
```

would become

```
boy good a am I
```

This is dealt with in another question. Here I only concentrate on reversing strings. That is

```
I am a good boy
```

would become

```
yob doog a ma I
```

Here are some sample C programs to do the same

Method1 (Recursive)

```
#include <stdio.h>

static char str[]="STRING TO REVERSE";

int main(int argc, char *argv)
{
   printf("\nOriginal string : [%s]", str);

   // Call the recursion function
   reverse(0);

   printf("\nReversed string : [%s]", str);
   return(0);
}

int reverse(int pos)
{
```

```
   // Here I am calculating strlen(str) everytime.
   // This can be avoided by doing this computation
   // earlier and storing it somewhere for later use.

   if(pos<(strlen(str)/2))
   {
      char ch;

      // Swap str[pos] and str[strlen(str)-pos-1]
      ch = str[pos];
      str[pos]=str[strlen(str)-pos-1];
      str[strlen(str)-pos-1]=ch;

      // Now recurse!
      reverse(pos+1);
   }
}
```

Method2

```
#include <stdio.h>
#include <malloc.h>
#include <string.h>

void ReverseStr ( char *buff, int start, int end )
{
   char tmp ;

   if ( start >= end )
   {
      printf ( "\n%s\n", buff );
      return;
   }

   tmp = *(buff + start);
   *(buff + start) = *(buff + end);
   *(buff + end) = tmp ;

   ReverseStr (buff, ++start, --end );
}


int main()
{
   char buffer[]="This is Test";
   ReverseStr(buffer,0,strlen(buffer)-1);
   return 0;
}
```

Method3

```
public static String reverse(String s)
{
    int N = s.length();
    if (N <= 1) return s;
    String left = s.substring(0, N/2);
    String right = s.substring(N/2, N);
    return reverse(right) + reverse(left);
}
```

Method4

```
for(int i = 0, j = reversed.Length - 1; i < j; i++, j--)
{
    char temp = reversed[i];
    reversed[i] = reversed[j];
    reversed[j] = temp;
}
```

```
return new String(reversed);
```

Method5

```
public static String reverse(String s)
{
   int N = s.length();
   String reverse = "";
   for (int i = 0; i < N; i++)
        reverse = s.charAt(i) + reverse;
   return reverse;
}
```

Method6

```
public static String reverse(String s)
{
   int N = s.length();
   char[] a = new char[N];
   for (int i = 0; i < N; i++)
       a[i] = s.charAt(N-i-1);
   String reverse = new String(a);
   return reverse;
}
```

Isn't that enough?

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to reverse the words in a sentence in place.

That is, given a sentence like this

```
I am a good boy
```

The in place reverse would be

```
boy good a am I
```

Method1

First reverse the whole string and then individually reverse the words

```
I am a good boy
<-------------->

yob doog  a   ma   I
<-> <--> <->  <-> <->

boy good a am I
```

Here is some C code to do the same ....

```
/*
  Algorithm..

  1. Reverse whole sentence first.
  2. Reverse each word individually.

  All the reversing happens in-place.
*/

#include <stdio.h>

void rev(char *l, char *r);


int main(int argc, char *argv[])
{
   char buf[] = "the world will go on forever";
   char *end, *x, *y;

   // Reverse the whole sentence first..
```

```
   for(end=buf; *end; end++);
   rev(buf,end-1);


   // Now swap each word within sentence...
   x = buf-1;
   y = buf;

   while(x++ < end)
   {
      if(*x == '\0' || *x == ' ')
      {
        rev(y,x-1);
        y = x+1;
      }
   }

   // Now print the final string....
   printf("%s\n",buf);

   return(0);
}


// Function to reverse a string in place...
void rev(char *l,char *r)
{
   char t;
   while(l<r)
   {
      t    = *l;
      *l++ = *r;
      *r-- = t;
   }
}
```

Method2

Another way to do it is, allocate as much memory as the input for the final output. Start from the right of the string and copy the words one by one to the output.

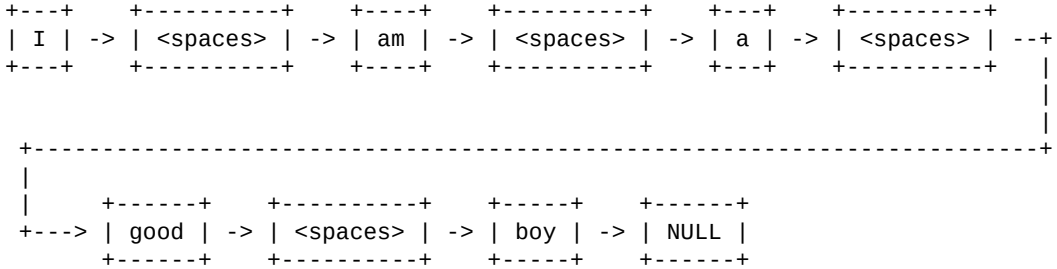
```
Input  : I am a good boy
                     <--
                 <-------
              <---------
           <------------
         <--------------


Output : boy
       : boy good
       : boy good a
       : boy good a am
       : boy good a am I
```

The only problem to this solution is the extra space required for the output and one has to write this code really well as we are traversing the string in the reverse direction and there is no null at the start of the string to know we have reached the start of the string!. One can use the strtok() function to breakup the string into multiple words and rearrange them in the reverse order later.

Method3

Create a linked list like



Now its a simple question of reversing the linked list!. There are plenty of algorithms to reverse a linked list easily. This also keeps track of the number of spaces between the words. Note that the linked list algorithm, though inefficient, handles multiple spaces between the words really well.

I really dont know what is the use of reversing a string or a sentence like this!, but its still asked. Can someone tell me a really practical application of this? Please!

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program generate permutations.

Iterative C program

```
#include <stdio.h>
#define SIZE 3
int main(char *argv[],int argc)
{
  char list[3]={'a','b','c'};
  int i,j,k;

  for(i=0;i<SIZE;i++)
    for(j=0;j<SIZE;j++)
      for(k=0;k<SIZE;k++)
        if(i!=j && j!=k && i!=k)
          printf("%c%c%c\n",list[i],list[j],list[k]);

  return(0);
}
```

Recursive C program

```
#include <stdio.h>
#define N  5


int main(char *argv[],int argc)
{
  char list[5]={'a','b','c','d','e'};
  permute(list,0,N);
  return(0);
}


void permute(char list[],int k, int m)
{
  int i;
  char temp;

  if(k==m)
  {
    /* PRINT A FROM k to m! */
    for(i=0;i<N;i++){printf("%c",list[i]);}
    printf("\n");
  }
  else
  {
```

```
    for(i=k;i<m;i++)
    {
       /* swap(a[i],a[m-1]); */
       temp=list[i];
       list[i]=list[m-1];
       list[m-1]=temp;

       permute(list,k,m-1);

       /* swap(a[m-1],a[i]); */

       temp=list[m-1];
       list[m-1]=list[i];
       list[i]=temp;
     }
  }
}
```

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program for calculating the factorial of a number

Here is a recursive C program

```
fact(int n)
{
    int fact;
    if(n==1)
        return(1);
    else
        fact = n * fact(n-1);
    return(fact);
}
```

Please note that there is no error handling added to this function (to check if n is negative or 0. Or if n is too large for the system to handle). This is true for most of the answers in this website. Too much error handling and standard compliance results in a lot of clutter making it difficult to concentrate on the crux of the solution. You must ofcourse add as much error handling and comply to the standards of your compiler when you actually write the code to implement these algorithms.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to calculate pow(x,n)?

There are again different methods to do this in C

Brute force C program

```
int pow(int x, int y)
{
  if(y == 1) return x ;
  return x * pow(x, y-1) ;
}
```

Divide and Conquer C program

```
#include <stdio.h>
int main(int argc, char*argv[])
{
  printf("\n[%d]\n",pow(5,4));
}

int pow(int x, int n)
{
  if(n==0)return(1);
  else if(n%2==0)
  {
    return(pow(x,n/2)*pow(x,(n/2)));
  }
  else
  {
    return(x*pow(x,n/2)*pow(x,(n/2)));
  }
}
```

Also, the code above can be optimized still by calculating pow(z, (n/2)) only one time (instead of twice) and using its value in the two return() expressions above.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program which does wildcard pattern matching algorithm

Here is an example C program...

```
#include<stdio.h>
#define TRUE 1
#define FALSE 0

int wildcard(char *string, char *pattern);

int main()
{
  char *string = "hereheroherr";
  char *pattern = "*hero*";

  if(wildcard(string, pattern)==TRUE)
  {
    printf("\nMatch Found!\n");
  }
  else
  {
    printf("\nMatch not found!\n");
  }
  return(0);
}

int wildcard(char *string, char *pattern)
{
  while(*string)
  {
     switch(*pattern)
     {
        case '*': do {++pattern;}while(*pattern == '*');
                  if(!*pattern) return(TRUE);
                  while(*string){if(wildcard(pattern,string++)==TRUE)return(TRUE);}
                  return(FALSE);
        default : if(*string!=*pattern)return(FALSE); break;
     }
     ++pattern;
     ++string;
   }

   while (*pattern == '*') ++pattern;
   return !*pattern;
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you calculate the maximum subarray of a list of numbers?

This is a very popular question

You are given a large array X of integers (both positive and negative) and you need to find the maximum sum found in any contiguous subarray of X.

```
Example X = [11, -12, 15, -3, 8, -9, 1, 8, 10, -2]
Answer is 30.
```

There are various methods to solve this problem, some are listed below

Brute force

```
maxSum = 0
for L = 1 to N
{
  for R = L to N
  {
     sum = 0
     for i = L to R
     {
        sum = sum + X[i]
     }
     maxSum = max(maxSum, sum)
   }
}
```

O(N^3)

Quadratic

Note that sum of [L..R] can be calculated from sum of [L..R-1] very easily.

```
maxSum = 0
for L = 1 to N
{
  sum = 0
  for R = L to N
  {
    sum = sum + X[R]
    maxSum = max(maxSum, sum)
  }
}
```

Using divide-and-conquer

O(N log(N))

```
maxSum(L, R)
{
  if L > R then
    return 0

  if L = R then
    return max(0, X[L])

  M = (L + R)/2

  sum = 0; maxToLeft = 0
  for i = M downto L do
  {
     sum = sum + X[i]
     maxToLeft = max(maxToLeft, sum)
  }

  sum = 0; maxToRight = 0
  for i = M to R do
  {
     sum = sum + X[i]
     maxToRight = max(maxToRight, sum)
  }


  maxCrossing = maxLeft + maxRight
  maxInA = maxSum(L,M)
  maxInB = maxSum(M+1,R)
  return max(maxCrossing, maxInA, maxInB)
}
```

Here is working C code for all the above cases

```
#include<stdio.h>
#define N 10
int maxSubSum(int left, int right);
int list[N] = {11, -12, 15, -3, 8, -9, 1, 8, 10, -2};

int main()
{
  int i,j,k;
  int maxSum, sum;

  /*---------------------------------------
  * CUBIC - O(n*n*n)
  *---------------------------------------*/

  maxSum = 0;
  for(i=0; i<N; i++)
  {
    for(j=i; j<N; j++)
    {
      sum = 0;
      for(k=i ; k<j; k++)
      {
        sum = sum + list[k];
      }
      maxSum = (maxSum>sum)?maxSum:sum;
    }
   }

   printf("\nmaxSum = [%d]\n", maxSum);


   /*-------------------------------------
    * Quadratic - O(n*n)
    * ------------------------------------ */
```

```
   maxSum = 0;
   for(i=0; i<N; i++)
   {
      sum=0;
      for(j=i; j<N ;j++)
      {
         sum = sum + list[j];
         maxSum = (maxSum>sum)?maxSum:sum;
      }
   }

   printf("\nmaxSum = [%d]\n", maxSum);


   /*--------------------------------------
    * Divide and Conquer - O(nlog(n))
    * -------------------------------------- */

   printf("\nmaxSum : [%d]\n", maxSubSum(0,9));

   return(0);
}


int maxSubSum(int left, int right)
{
  int mid, sum, maxToLeft, maxToRight, maxCrossing, maxInA, maxInB;
  int i;

  if(left>right){return 0;}
  if(left==right){return((0>list[left])?0:list[left]);}
  mid = (left + right)/2;

  sum=0;
  maxToLeft=0;
  for(i=mid; i>=left; i--)
  {
     sum = sum + list[i];
     maxToLeft = (maxToLeft>sum)?maxToLeft:sum;
  }


  sum=0;
  maxToRight=0;
  for(i=mid+1; i<=right; i++)
  {
      sum = sum + list[i];
      maxToRight = (maxToRight>sum)?maxToRight:sum;
  }

  maxCrossing = maxToLeft + maxToRight;
  maxInA = maxSubSum(left,mid);
  maxInB = maxSubSum(mid+1,right);
  return(((maxCrossing>maxInA)?maxCrossing:maxInA)>maxInB?((maxCrossing>maxInA)?maxCrossing:maxInA):
maxInB);
}
```

Note that, if the array has all negative numbers, then this code will return 0. This is wrong because it should return the maximum sum, which is the least negative integer in the array. This happens because we are setting maxSum to 0 initially. A small change in this code can be used to handle such cases.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Programs
(42 / 301)

How to generate fibonacci numbers? How to find out if a given number is a fibonacci number or not? Write C programs to do both.

Lets first refresh ourselves with the Fibonacci sequence

```
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, .....
```

Fibonacci numbers obey the following rule

```
F(n) = F(n-1) + F(n-2)
```

Here is an iterative way to generate fibonacci numbers and also return the nth number.

```
int fib(int n)
{
```

```
   int f[n+1];
   f[1] = f[2] = 1;

   printf("\nf[1] = %d", f[1]);
   printf("\nf[2] = %d", f[2]);

   for (int i = 3; i <= n; i++)
   {
       f[i] = f[i-1] + f[i-2];
       printf("\nf[%d] = [%d]",i,f[i]);
   }
   return f[n];
}
```

Here is a recursive way to generate fibonacci numbers.

```
int fib(int n)
{
  if (n <= 2) return 1
  else return fib(n-1) + fib(n-2)
}
```

Here is an iterative way to just compute and return the nth number (without storing the previous numbers).

```
int fib(int n)
{
  int a = 1, b = 1;
  for (int i = 3; i <= n; i++)
  {
     int c = a + b;
     a = b;
     b = c;
  }
  return a;
}
```

There are a few slick ways to generate fibonacci numbers, a few of them are listed below

Method1

If you know some basic math, its easy to see that

```
         n
 [ 1 1 ]      =   [ F(n+1) F(n)   ]
 [ 1 0 ]          [ F(n)   F(n-1) ]
```

or

```
(f(n) f(n+1)) [ 0 1 ] = (f(n+1) f(n+2))
              [ 1 1 ]
```

or

```
                    n
(f(0) f(1)) [ 0 1 ]  = (f(n) f(n+1))
            [ 1 1 ]
```

The n-th power of the 2 by 2 matrix can be computed efficiently in O(log n) time. This implies an O(log n) algorithm for computing the n-th Fibonacci number.

Here is the pseudocode for this

```
int Matrix[2][2] = {{1,0}{0,1}}

int fib(int n)
{
    matrixpower(n-1);
    return Matrix[0][0];
}

void matrixpower(int n)
{
   if (n > 1)
   {
      matrixpower(n/2);
      Matrix = Matrix * Matrix;
   }
   if (n is odd)
   {
      Matrix = Matrix * {{1,1}{1,0}}
   }
```

```
}
```

And here is a program in C which calculates fib(n)

```
#include<stdio.h>

int M[2][2]={{1,0},{0,1}};
int A[2][2]={{1,1},{1,0}};
int C[2][2]={{0,0},{0,0}};  // Temporary matrix used for multiplication.

void matMul(int n);
void mulM(int m);

int main()
{
   int n;
   n=6;

   matMul(n-1);

   // The nth fibonacci will be stored in M[0][0]
   printf("\n%dth Fibonaci number : [%d]\n\n", n, M[0][0]);
   return(0);

}


// Recursive function with divide and conquer strategy

void matMul(int n)
{
   if(n>1)
   {
     matMul(n/2);
     mulM(0);     // M * M
   }
   if(n%2!=0)
   {
    mulM(1);     //  M * {{1,1}{1,0}}
   }
}


// Function which does some basic matrix multiplication.

void mulM(int m)
{
```

```
   int i,j,k;

   if(m==0)
   {
     // C = M * M

     for(i=0;i<2;i++)
      for(j=0;j<2;j++)
      {
        C[i][j]=0;
        for(k=0;k<2;k++)
           C[i][j]+=M[i][k]*M[k][j];
      }
   }
   else
   {
     // C = M *  {{1,1}{1,0}}

     for(i=0;i<2;i++)
      for(j=0;j<2;j++)
      {
        C[i][j]=0;
        for(k=0;k<2;k++)
         C[i][j]+=A[i][k]*M[k][j];
      }
   }

   // Copy back the temporary matrix in the original matrix M

   for(i=0;i<2;i++)
     for(j=0;j<2;j++)
     {
       M[i][j]=C[i][j];
     }
}
```

Method2

```
f(n) = (1/sqrt(5)) * (((1+sqrt(5))/2) ^ n - ((1-sqrt(5))/2) ^ n)
```

So now, how does one find out if a number is a fibonacci or not?.

The cumbersome way is to generate fibonacci numbers till this number and see if this number is one of them. But there is another slick way to check if a number is a fibonacci number or not.

```
N is a Fibonacci number if and only if (5*N*N + 4) or (5*N*N - 4) is a perfect square!
```

Dont believe me?

```
3 is a Fibonacci number since (5*3*3 + 4) is 49 which is 7*7
5 is a Fibonacci number since (5*5*5 - 4) is 121 which is 11*11
4 is not a Fibonacci number since neither (5*4*4 + 4) = 84 nor (5*4*4 - 4) = 76 are perfect squares.
```

To check if a number is a perfect square or not, one can take the square root, round it to the nearest integer and then square the result. If this is the same as the original whole number then the original was a perfect square.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Solve the Rat In A Maze problem using backtracking.

This is one of the classical problems of computer science. There is a rat trapped in a maze. There are multiple paths in the maze from the starting point to the ending point. There is some cheese at the exit. The rat starts from the entrance of the maze and wants to get to the cheese.

This problem can be attacked as follows.

- Have a m*m matrix which represents the maze.
- For the sake of simplifying the implementation, have a boundary around your matrix and fill it up with all ones. This is so that you know when the rat is trying to go out of the boundary of the maze. In the real world, the rat would know not to go out of the maze, but hey! So, initially the matrix (I mean, the maze) would be something like (the ones represent the "exra" boundary we have added). The ones inside specify the obstacles.

```
11111111111111111111
10000000000000000001
10000001000000000001
10000001000000000001
10000000100001000001
10000100001000000001
10000000100000000001
10000000000000000001
11111111111111111111
```

- The rat can move in four directions at any point in time (well, right, left, up, down). Please note that the rat can't move diagonally. Imagine a real maze and not a matrix. In matrix language
    - Moving right means adding {0,1} to the current coordinates.
    - Moving left means adding {0,-1} to the current coordinates.
    - Moving up means adding {-1,0} to the current coordinates.
    - Moving right means adding {1,0} to the current coordinates.
- The rat can start off at the first row and the first column as the entrance point.
- From there, it tries to move to a cell which is currently free. A cell is free if it has a zero in it.
- It tries all the 4 options one-by-one, till it finds an empty cell. If it finds one, it moves to that cell and marks it with a 1 (saying it has visited it once). Then it continues to move ahead from that cell to other cells.
- If at a particular cell, it runs out of all the 4 options (that is it cant move either right, left, up or down), then it needs to backtrack. It backtracks till a point where it can move ahead and be closer to the exit.
- If it reaches the exit point, it gets the cheese, ofcourse.
- The complexity is O(m*m).

Here is some pseudocode to chew upon

```
findpath()
```

```
{
  Position offset[4];
  Offset[0].row=0; offset[0].col=1;//right;
  Offset[1].row=1; offset[1].col=0;//down;
  Offset[2].row=0; offset[2].col=-1;//left;
  Offset[3].row=-1; offset[3].col=0;//up;

  // Initialize wall of obstacles around the maze
  for(int i=0; i<m+1;i++)
    maze[0][i] = maze[m+1][i]=1; maze[i][0] = maze[i][m+1]=1;

  Position here;
  Here.row=1;
  Here.col=1;

  maze[1][1]=1;
  int option = 0;
  int lastoption = 3;

  while(here.row!=m || here.col!=m)
  {
     //Find a neighbor to move
     int r,c;

       while (option<=LastOption)
       {
          r=here.row + offset[position].row;
          c=here.col + offset[option].col;
          if(maze[r][c]==0)break;
          option++;
       }

       //Was a neighbor found?
       if(option<=LastOption)
       {
         path->Add(here);
         here.row=r;here.col=c;
         maze[r][c]=1;
         option=0;
       }
       else
       {
          if(path->Empty())return(False);
          Position  next;
          Path->Delete(next);
          If(new.row==here.row)
             Option=2+next.col - here.col;
          Else { option = 3 + next.row - here.col;}
             Here=next;
       }
       return(TRUE);
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What Little-Endian and Big-Endian? How can I determine whether a machine's byte order is big-endian or little endian? How can we convert from one to another?

First of all, Do you know what Little-Endian and Big-Endian mean?

Little Endian means that the lower order byte of the number is stored in memory at the lowest address, and the higher order byte is stored at the highest address. That is, the little end comes first.

For example, a 4 byte, 32-bit integer

```
Byte3 Byte2 Byte1 Byte0
```

will be arranged in memory as follows:

```
Base_Address+0   Byte0
Base_Address+1   Byte1
Base_Address+2   Byte2
Base_Address+3   Byte3
```

Intel processors use "Little Endian" byte order.

"Big Endian" means that the higher order byte of the number is stored in memory at the lowest address, and the lower order byte at the highest address. The big end comes first.

```
Base_Address+0   Byte3
Base_Address+1   Byte2
Base_Address+2   Byte1
Base_Address+3   Byte0
```

Motorola, Solaris processors use "Big Endian" byte order.

In "Little Endian" form, code which picks up a 1, 2, 4, or longer byte number proceed in the same way for all formats. They first pick up the lowest order byte at offset 0 and proceed from there. Also, because of the 1:1 relationship between address offset and byte number (offset 0 is byte 0), multiple precision mathematic routines are easy to code. In "Big Endian" form, since the high-order byte comes first, the code can test whether the number is positive or negative by looking at the byte at offset zero. Its not required to know how long the number is, nor does the code have to skip over any bytes to find the byte containing the sign information. The numbers are also stored in the order in which they are printed out, so binary to decimal routines are particularly efficient.

Here is some code to determine what is the type of your machine

```
int num = 1;
if(*(char *)&num == 1)
{
```

```
  printf("\nLittle-Endian\n");
}
else
{
  printf("Big-Endian\n");
}
```

And here is some code to convert from one Endian to another.

```
int myreversefunc(int num)
{
  int byte0, byte1, byte2, byte3;

  byte0 = (num & x000000FF) >>  0 ;
  byte1 = (num & x0000FF00) >>  8 ;
  byte2 = (num & x00FF0000) >> 16 ;
  byte3 = (num & xFF000000) >> 24 ;

  return((byte0 << 24) | (byte1 << 16) | (byte2 << 8) | (byte3 << 0));
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to solve the Tower of Hanoi problem.

Here is an example C program to solve the Tower Of Hanoi problem...

```
main()
{
    towers_of_hanio(n,'L','R','C');
}

towers_of_hanio(int n, char from, char to, char temp)
{
    if(n>0)
    {
        tower_of_hanio(n-1, from, temp, to);
        printf("\nMove disk %d from %c to %c\n", n, from, to);
        tower_of_hanio(n-1, temp, to, from);
    }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to return a string from a function

This is one of the most popular interview questions

This C program wont work!

```
char *myfunction(int n)
{
   char buffer[20];
   sprintf(buffer, "%d", n);
   return retbuf;
}
```

This wont work either!

```
char *myfunc1()
{
  char temp[] = "string";
  return temp;
}

char *myfunc2()
{
   char temp[] = {'s', 't', 'r', 'i', 'n', 'g', '\0'};
   return temp;
}

int main()
{
    puts(myfunc1());
    puts(myfunc2());
}
```

The returned pointer should be to a static buffer (like static char buffer[20];), or to a buffer passed in by the caller function, or to memory obtained using malloc(), but not to a local array.

This will work

```
char *myfunc()
{
   char *temp = "string";
   return temp;
```

```
}

int main()
{
   puts(someFun());
}
```

So will this

```
calling_function()
{
  char *string;
  return_string(&string);
  printf(?\n[%s]\n?, string);
}

boolean return_string(char **mode_string /*Pointer to a pointer! */)
{
   *string = (char *) malloc(100 * sizeof(char));
   DISCARD strcpy((char *)*string, (char *)?Something?);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program which produces its own source code as its output

This is one of the most famous interview questions

One of the famous C programs is...

```
char*s="char*s=%c%s%c;main(){printf(s,34,s,34);}";main(){printf(s,34,s,34);}
```

So how does it work?

It's not difficult to understand this program. In the following statement,

```
printf(f,34,f,34,10);
```

the parameter "f" not only acts as the format string, but also as a value for the %s specifier. The ASCII value of double quotes is 34, and that of new-line is 10. With these fact ready, the solution is just a matter of tracing the program.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C progam to convert from decimal to any base (binary, hex, oct etc...)

Here is some really cool C code

```
#include <stdio.h>

int main()
{
  decimal_to_anybase(10, 2);
  decimal_to_anybase(255, 16);
  getch();
}

decimal_to_anybase(int n, int base)
{
  int i, m, digits[1000], flag;
  i=0;

  printf("\n\n[%d] converted to base [%d] : ", n, base);

  while(n)
  {
     m=n%base;
     digits[i]="0123456789abcdefghijklmnopqrstuvwxyz"[m];
     n=n/base;
     i++;
   }

   //Eliminate any leading zeroes
   for(i--;i>=0;i--)
   {
     if(!flag && digits[i]!='0')flag=1;
     if(flag)printf("%c",digits[i]);
   }
}
```

Write C code to check if an integer is a power of 2 or not in a single line?

Even this is one of the most frequently asked interview questions. I really dont know whats so great in it. Nevertheless, here is a C program

Method1

```
if(!(num & (num - 1)) && num)
{
  // Power of 2!
}
```

Method2

```
if(((~i+1)&i)==i)
{
 //Power of 2!
}
```

I leave it up to you to find out how these statements work.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to find the GCD of two numbers.

Here is a C program ....

```
#include <stdio.h>

int gcd(int a, int b);
int gcd_recurse(int a, int b);


int main()
{
  printf("\nGCD(%2d,%2d) = [%d]", 6,4,  gcd(6,4));
  printf("\nGCD(%2d,%2d) = [%d]", 4,6,  gcd(4,6));
  printf("\nGCD(%2d,%2d) = [%d]", 3,17, gcd(3,17));
  printf("\nGCD(%2d,%2d) = [%d]", 17,3, gcd(17,3));
  printf("\nGCD(%2d,%2d) = [%d]", 1,6,  gcd(1,6));
  printf("\nGCD(%2d,%2d) = [%d]", 10,1, gcd(10,1));
  printf("\nGCD(%2d,%2d) = [%d]", 10,6, gcd(10,6));

  printf("\nGCD(%2d,%2d) = [%d]", 6,4,  gcd_recurse(6,4));
  printf("\nGCD(%2d,%2d) = [%d]", 4,6,  gcd_recurse(4,6));
  printf("\nGCD(%2d,%2d) = [%d]", 3,17, gcd_recurse(3,17));
  printf("\nGCD(%2d,%2d) = [%d]", 17,3, gcd_recurse(17,3));
  printf("\nGCD(%2d,%2d) = [%d]", 1,6,  gcd_recurse(1,6));
  printf("\nGCD(%2d,%2d) = [%d]", 10,1, gcd_recurse(10,1));
  printf("\nGCD(%2d,%2d) = [%d]", 10,6, gcd_recurse(10,6));


  getch();
  getch();
  return(0);
}

// Iterative algorithm
int gcd(int a, int b)
{
   int temp;

   while(b)
   {
      temp = a % b;
      a = b;
      b = temp;
   }

   return(a);
}
```

```
// Recursive algorithm
int gcd_recurse(int a, int b)
{
   int temp;

   temp = a % b;

   if (temp == 0)
   {
      return(b);
   }
   else
   {
      return(gcd_recurse(b, temp));
   }
}
```

And here is the output ...

```
Iterative
----------------
GCD( 6, 4) = [2]
GCD( 4, 6) = [2]
GCD( 3,17) = [1]
GCD(17, 3) = [1]
GCD( 1, 6) = [1]
GCD(10, 1) = [1]
GCD(10, 6) = [2]

Recursive
----------------
GCD( 6, 4) = [2]
GCD( 4, 6) = [2]
GCD( 3,17) = [1]
GCD(17, 3) = [1]
GCD( 1, 6) = [1]
GCD(10, 1) = [1]
GCD(10, 6) = [2]
```

Note that you should add error handling to check if someone has passed negative numbers and zero.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Finding a duplicated integer problem

Given an array of n integers from 1 to n with one integer repeated..

Here is the simplest of C programs (kind of dumb)

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>

int i,j=0,k,a1[10];

main()
{
     printf("Enter the array of numbers between 1 and 100(you can repeat the numbers):");
     for(i=0;i<=9;i++)
     {
         scanf("%d",&a1[i]);
     }

     while(j<10)
     {
        for(k=0;k<10;k++)
        {
            if(a1[j]==a1[k] && j!=k)
            {
                printf("Duplicate found!");
                printf("The duplicate is %d\n",a1[j]);
                getch();
            }
        }
        j=j+1;
    }

    getch();
    return(0);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write code to remove duplicates in a sorted array.

There are a number of ways to remove duplicates from a sorted array. Here are a few C programs...

Method1

In this simple C program, we change the original array and also send the new size of the array back to the caller.

```
#include <stdio.h>

int removeDuplicates(int a[], int array_size);

// The main function
int main()
{

  // Different test cases..
  int my_array1[]    = {1, 1, 2, 3, 5, 6, 6, 7, 10, 25, 100, 123, 123};
  int my_array1_size = 13;

  int my_array2[]    = {1, 2, 3, 5, 6};
  int my_array2_size = 5;

  int my_array3[]    = {1, 1, 1, 1, 1};
  int my_array3_size = 5;

  int my_array4[]    = {123, 123};
  int my_array4_size = 2;

  int my_array5[]    = {1, 123, 123};
  int my_array5_size = 3;

  int my_array6[]    = {123, 123, 166};
  int my_array6_size = 3;

  int my_array7[]    = {1, 2, 8, 8 , 24, 60, 60, 60, 60, 75, 100, 100, 123};
  int my_array7_size = 13;


  my_array1_size = removeDuplicates(my_array1, my_array1_size);
  my_array2_size = removeDuplicates(my_array2, my_array2_size);
  my_array3_size = removeDuplicates(my_array3, my_array3_size);
  my_array4_size = removeDuplicates(my_array4, my_array4_size);
  my_array5_size = removeDuplicates(my_array5, my_array5_size);
  my_array6_size = removeDuplicates(my_array6, my_array6_size);
  my_array7_size = removeDuplicates(my_array7, my_array7_size);

  return(0);
}
```

```
// Function to remove the duplicates
int removeDuplicates(int a[], int array_size)
{
  int i, j;

  j = 0;


  // Print old array...
  printf("\n\nOLD : ");
  for(i = 0; i < array_size; i++)
  {
     printf("[%d] ", a[i]);
  }


  // Remove the duplicates ...
  for (i = 1; i < array_size; i++)
  {
    if (a[i] != a[j])
    {
      j++;
      a[j] = a[i]; // Move it to the front
    }
  }

  // The new array size..
  array_size = (j + 1);


  // Print new array...
  printf("\n\nNEW : ");
  for(i = 0; i< array_size; i++)
  {
     printf("[%d] ", a[i]);
  }
  printf("\n\n");



  // Return the new size...
  return(j + 1);
}
```

And here is the output...

```
OLD : [1] [1] [2] [3] [5] [6] [6] [7] [10] [25] [100] [123] [123]
NEW : [1] [2] [3] [5] [6] [7] [10] [25] [100] [123]


OLD : [1] [2] [3] [5] [6]
NEW : [1] [2] [3] [5] [6]


OLD : [1] [1] [1] [1] [1]
NEW : [1]


OLD : [123] [123]
NEW : [123]


OLD : [1] [123] [123]
NEW : [1] [123]


OLD : [123] [123] [166]
NEW : [123] [166]


OLD : [1] [2] [8] [8] [24] [60] [60] [60] [60] [75] [100] [100] [123]
NEW : [1] [2] [8] [24] [60] [75] [100] [123]
```

Method2

If we dont want to change the input array and just want to print the array without any duplicates, the solution is very simple. Check out the removeDuplicatesNoModify() function in the program below. It keeps a track of the most recently seen number and does not print any duplicates of it when traversing the sorted array.

```
#include <stdio.h>

void removeDuplicatesNoModify(int my_array[], int my_array1_size);
void print_array(int array[], int array_size, int current_pos, int dup_start, int dup_end);


// The main function
int main()
{
  // Different inputs...
  int my_array1[]    = {1, 1, 2, 3, 5, 6, 6, 7, 10, 25, 100, 123, 123};
  int my_array1_size = 13;

  int my_array2[]    = {1, 2, 3, 5, 6};
  int my_array2_size = 5;

  int my_array3[]    = {1, 1, 1, 1, 1};
  int my_array3_size = 5;

  int my_array4[]    = {123, 123};
  int my_array4_size = 2;

  int my_array5[]    = {1, 123, 123};
  int my_array5_size = 3;

  int my_array6[]    = {123, 123, 166};
  int my_array6_size = 3;

  int my_array7[]    = {1, 2, 8, 8 , 24, 60, 60, 60, 60, 75, 100, 100, 123};
  int my_array7_size = 13;


  removeDuplicatesNoModify(my_array1, my_array1_size);
  removeDuplicatesNoModify(my_array2, my_array2_size);
  removeDuplicatesNoModify(my_array3, my_array3_size);
  removeDuplicatesNoModify(my_array4, my_array4_size);
  removeDuplicatesNoModify(my_array5, my_array5_size);
  removeDuplicatesNoModify(my_array6, my_array6_size);
  removeDuplicatesNoModify(my_array7, my_array7_size);

  return(0);
}


//  This function just prints the array without duplicates.
//  It does not modify the original array!

void removeDuplicatesNoModify(int array[], int array_size)
{
   int i, last_seen_unique;

   if(array_size <= 1){return;}

   last_seen_unique = array[0];

   printf("\n\nOld : ", array_size);

   for(i = 0; i < array_size; i++)
   {
     printf("[%2d] ", array[i]);
   }

   printf("\nNew : ", array_size);

   printf("[%2d] ", array[0]);
   for(i=1; i < array_size; i++)
   {
       if(array[i]!=last_seen_unique)
       {
          printf("[%2d] ", array[i]);
```

```
            last_seen_unique = array[i];
        }
    }

    printf("\n");
}
```

And here is the output..

```
Old : [ 1] [ 1] [ 2] [ 3] [ 5] [ 6] [ 6] [ 7] [10] [25] [100] [123] [123]
New : [ 1] [ 2] [ 3] [ 5] [ 6] [ 7] [10] [25] [100] [123]


Old : [ 1] [ 2] [ 3] [ 5] [ 6]
New : [ 1] [ 2] [ 3] [ 5] [ 6]


Old : [ 1] [ 1] [ 1] [ 1] [ 1]
New : [ 1]


Old : [123] [123]
New : [123]


Old : [ 1] [123] [123]
New : [ 1] [123]


Old : [123] [123] [166]
New : [123] [166]


Old : [ 1] [ 2] [ 8] [ 8] [24] [60] [60] [60] [60] [75] [100] [100] [123]
New : [ 1] [ 2] [ 8] [24] [60] [75] [100] [123]
```

Method3

Here is a slightly compilcated, but more visual version of the removeDuplicates() function. It shrinks the original array as and when it find duplicates. It is also optimized to identify continuous strings of duplicates and eliminate them at one shot.

```
#include <stdio.h>

void removeDuplicates(int array[], int *array_size) ;
void print_array(int array[], int array_size, int current_pos, int dup_start, int dup_end);


// The main function
int main()
{
  // Different inputs...
  int my_array1[]    = {1, 1, 2, 3, 5, 6, 6, 7, 10, 25, 100, 123, 123};
  int my_array1_size = 13;

  int my_array2[]    = {1, 2, 3, 5, 6};
  int my_array2_size = 5;

  int my_array3[]    = {1, 1, 1, 1, 1};
  int my_array3_size = 5;

  int my_array4[]    = {123, 123};
  int my_array4_size = 2;

  int my_array5[]    = {1, 123, 123};
  int my_array5_size = 3;

  int my_array6[]    = {123, 123, 166};
  int my_array6_size = 3;

  int my_array7[]    = {1, 2, 8, 8 , 24, 60, 60, 60, 60, 75, 100, 100, 123};
  int my_array7_size = 13;
```

```
  removeDuplicates(my_array1, &my_array1_size);
  removeDuplicates(my_array2, &my_array2_size);
  removeDuplicates(my_array3, &my_array3_size);
  removeDuplicates(my_array4, &my_array4_size);
  removeDuplicates(my_array5, &my_array5_size);
  removeDuplicates(my_array6, &my_array6_size);
  removeDuplicates(my_array7, &my_array7_size);

  return(0);
}



// Changes the original array and resets the size of the array if duplicates
// have been removed.

void removeDuplicates(int array[], int *array_size)
{
   int i, j, k, l;
   int current_pos;
   int dup_start;
   int dup_end;

   printf("\nInitial array (size : [%d])\n\n", *array_size);
   for(i = 0; i < *array_size; i++)
   {
     printf("[%2d] ", array[i]);
   }
   printf("\n\n\n-------------------------------------------------\n");


   if(*array_size == 1){return;}


   // Remove the dups...
   for (i = 0; (i < *array_size); i++)
   {
      //Start with the next element in the array and check if its a duplicate...
      for(j = i+1; (j < *array_size); j++)
      {
         if(array[i]!=array[j])
         {
            // No duplicate, just continue...
            break;
         }
         else
         {
            // The next element is a duplicate.
            // See if there are more duplicates, as we want to optimize here.
            //
            // That is, if we have something like
            //
            // Array : [1, 1, 1, 2]
            //
            // then, we want to copy 2 directly in the second position and reduce the
            // array to
            //
            // Array : [1, 2].
            //
            // in a single iteration.

            current_pos = i;
            dup_start = j;

            j++;

            while((array[i]==array[j]) && (j < *array_size))
            {
              j++;
            }

            dup_end = j-1;

            print_array(array, *array_size, current_pos, dup_start, dup_end);

            // Now remove elements of the array from "dup_start" to "dup_end"
            // and shrink the size of the array.
```

```
            for(k = (dup_end + 1), l = dup_start ; k < *array_size;)
            {
               array[l++]=array[k++];
            }

            // Reduce the array size by the number of elements removed.
            *array_size = *array_size - (dup_end - dup_start + 1);

         }
      }
   }

   printf("\n\n-------------------------------------------------");
   printf("\n\nFinal array (size : [%d])\n\n", *array_size);
   for(i = 0; i < *array_size; i++)
   {
     printf("[%2d] ", array[i]);
   }
   printf("\n\n");

}



// This function prints the array with some special pointers to the numbers that
// are duplicated.
//
// Dont bother too much about this function, it just helps in understanding
// how and where the duplicates are being removed from.

void print_array(int array[], int array_size, int current_pos, int dup_start, int dup_end)
{
  int i;

  printf("\n\n");

  for(i = 0; i < array_size; i++)
  {
    printf("[%2d] ", array[i]);
  }

  printf("\n");

  for(i = 0; i < array_size; i++)
  {
    if((i == current_pos) ||
       (i == dup_start && i == dup_end) ||
       ((i == dup_start || i == dup_end) && (dup_start != dup_end)))
    {
       printf("  ^  ");
    }
    else
    {
       printf("     ");
    }
  }

  printf("\n");

  for(i = 0; i < array_size; i++)
  {
    if((i == current_pos) ||
       (i == dup_start && i == dup_end) ||
       ((i == dup_start || i == dup_end) && (dup_start != dup_end)))
    {
       printf("  |  ");
    }
    else
    {
       printf("     ");
    }
  }

  printf("\n");
```

```
  for(i = 0; i < array_size; i++)
  {
    if(i == current_pos)
    {
       printf("  C  ");
    }
    else if(i == dup_start && i == dup_end)
    {
       printf(" S/E ");
    }
    else if((i == dup_start || i == dup_end) && (dup_start != dup_end))
    {
       if(i == dup_start)
       {
          printf("  S--");
       }
       else
       {
          printf("--E  ");
       }
    }
    else if(i>dup_start && i<dup_end)
    {
       printf("-----");
    }
    else
    {
       printf("     ");
    }
  }

}
```

And here is the output (for one of the inputs)...

```
C - Current position.
S - Start of duplicates.
E - End of duplicates.



Old : [ 1] [ 2] [ 8] [ 8] [24] [60] [60] [60] [60] [75] [100] [100] [123]

---------------------------------------------------------------------------
[ 1] [ 2] [ 8] [ 8] [24] [60] [60] [60] [60] [75] [100] [100] [123]
            ^    ^
            |    |
            C   S/E


[ 1] [ 2] [ 8] [24] [60] [60] [60] [60] [75] [100] [100] [123]
                      ^    ^         ^
                      |    |         |
                      C    S---------E


[ 1] [ 2] [ 8] [24] [60] [75] [100] [100] [123]
                               ^    ^
                               |    |
                               C   S/E

---------------------------------------------------------------------------
New : [ 1] [ 2] [ 8] [24] [60] [75] [100] [123]
```

If there are other elegant methods of removing duplicate numbers from an array, please let me know!.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Find the maximum of three integers using the ternary operator.

Here is how you do it

```
max = ((a>b)?((a>c)?a:c):((b>c)?b:c));
```

Here is another way

```
max = ((a>b)?a:b)>c?((a>b)?a:b):c;
```

Here is some code to find the max of 4 numbers...

Method1

```
#include <stdio.h>
#include <stdlib.h>

#define max2(x,y)  ((x)>(y)?(x):(y))
#define max4(a,b,c,d)  max2(max2((a),(b)),max2((c),(d)))

int main ( void )
{
    printf ( "Max: %d\n", max4(10,20,30,40));
    printf ( "Max: %d\n", max4(10,0,3,4));
    return EXIT_SUCCESS;
}
```

Method2

```
#include <stdio.h>
#include <stdlib.h>

int retMax(int i1, int i2, int i3, int i4)
{
   return(((i1>i2)?i1:i2) > ((i3>i4)?i3:i4)? ((i1>i2)?i1:i2):((i3>i4)?i3:i4));
}

int main()
{
   int val = 0 ;

   val = retMax(10, 200, 10, 530);
```

```
    val = retMax(9, 2, 5, 7);

    return 0;
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you initialize a pointer inside a function?

This is one of the very popular interview questions, so take a good look at it!.

```
myfunction(int *ptr)
{
  int myvar = 100;
  ptr = &myvar;
}

main()
{
 int *myptr;
 myfunction(myptr);

 //Use pointer myptr.

}
```

Do you think this works? It does not!.

Arguments in C are passed by value. The called function changed the passed copy of the pointer, and not the actual pointer.

There are two ways around this problem

Method1

Pass in the address of the pointer to the function (the function needs to accept a pointer-to-a-pointer).

```
calling_function()
{
  char *string;
  return_string(/* Pass the address of the pointer */&string);
  printf(?\n[%s]\n?, string);
}

boolean return_string(char **mode_string /*Pointer to a pointer! */)
{
  *string = (char *) malloc(100 * sizeof(char)); // Allocate memory to the pointer passed, not its copy.
  DISCARD strcpy((char *)*string, (char *)?Something?);
}
```

Method2

Make the function return the pointer.

```
char *myfunc()
{
  char *temp = "string";
  return temp;
}

int main()
{
  puts(myfunc());
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to dynamically allocate one, two and three dimensional arrays (using malloc())

Its pretty simple to do this in the C language if you know how to use C pointers. Here are some example C code snipptes....

One dimensional array

```
int *myarray = malloc(no_of_elements * sizeof(int));

//Access elements as myarray[i]
```

Two dimensional array

Method1

```
int **myarray = (int **)malloc(no_of_rows * sizeof(int *));
for(i = 0; i < no_of_rows; i++)
{
 myarray[i] = malloc(no_of_columns * sizeof(int));
}

// Access elements as myarray[i][j]
```

Method2 (keep the array's contents contiguous)

```
int **myarray = (int **)malloc(no_of_rows * sizeof(int *));
myarray[0] = malloc(no_of_rows * no_of_columns * sizeof(int));

for(i = 1; i < no_of_rows; i++)
  myarray[i] = myarray[0] + (i * no_of_columns);

// Access elements as myarray[i][j]
```

Method3

```
int *myarray = malloc(no_of_rows * no_of_columns * sizeof(int));

// Access elements using myarray[i * no_of_columns + j].
```

Three dimensional array

```
#define MAXX 3
#define MAXY 4
```

```
#define MAXZ 5

main()
{
    int ***p,i,j;
    p=(int ***) malloc(MAXX * sizeof(int ***));

    for(i=0;i<MAXX;i++)
    {
        p[i]=(int **)malloc(MAXY * sizeof(int *));
        for(j=0;j<MAXY;j++)
            p[i][j]=(int *)malloc(MAXZ * sizeof(int));
    }

    for(k=0;k<MAXZ;k++)
        for(i=0;i<MAXX;i++)
            for(j=0;j<MAXY;j++)
                p[i][j][k]=<something>;

}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How would you find the size of structure without using sizeof()?

Try using pointers

```
struct MyStruct
{
  int i;
  int j;
};

int main()
{
   struct MyStruct *p=0;
   int size = ((char*)(p+1))-((char*)p);
   printf("\nSIZE : [%d]\nSIZE : [%d]\n", size);
   return 0;
}
```

Write a C program to multiply two matrices.

Are you sure you know this? A lot of people think they already know this, but guess what? So take a good look at this C program. Its asked in most of the interviews as a warm up question.

```
// Matrix A (m*n)
// Matrix B (n*k)
// Matrix C (m*k)

for(i=0; i<m; i++)
{
   for(j=0;j<k;j++)
   {
      c[i][j]=0;
      for(l=0;l<n;l++)
           c[i][j] += a[i][l] * b[l][j];
   }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to check for palindromes.

An example of a palidrome is "avon sees nova"

There a number of ways in which we can find out if a string is a palidrome or not. Here are a few sample C programs...

Method1

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

void isPalindrome(char *string);

int main()
{
  isPalindrome("avon sees nova");
  isPalindrome("a");
  isPalindrome("avon sies nova");
  isPalindrome("aa");
  isPalindrome("abc");
  isPalindrome("aba");
  isPalindrome("3a2");
  exit(0);
}

void isPalindrome(char *string)
{
  char *start, *end;

  if(string)
  {
     start = string;
     end   = string + strlen(string) - 1;

     while((*start == *end) && (start!=end))
     {
       if(start<end)start++;
       if(end>start)end--;
     }

     if(*start!=*end)
     {
        printf("\n[%s] - This is not a palidrome!\n", string);
     }
     else
     {
        printf("\n[%s] - This is a palidrome!\n", string);
```

```
    }
  }
  printf("\n\n");
}
```

Method2

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int isPalindrome(char string[]);

int main()
{
  isPalindrome("avon sees nova");
  isPalindrome("a");
  isPalindrome("avon sies nova");
  isPalindrome("aa");
  isPalindrome("abc");
  isPalindrome("aba");
  isPalindrome("3a2");
  return(0);
}

int isPalindrome(char string[])
{
  int count, countback, end, N;

  N   = strlen(string);
  end = N-1;

  for((count=0, countback = end); count <= (end/2); ++count,--countback)
  {
    if(string[count]!=string[countback])
    {
      return(1);
    }
  }

  printf("\n[%s] is a palidrome!\n", string);
  return(0);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to convert a decimal number into a binary number.

99% of the people who attend interviews can't answer this question, believe me!. Here is a recursive C program which does this....

```
#include<stdio.h>
void generatebits(int num);

void generatebits(int num)
{
  int temp;
  if(num)
  {
    temp = num % 2;
    generatebits(num >>= 1);
    printf("%d",temp);
  }
}

int main()
{
  int num;
  printf("\nEnter a number\n");
  scanf("%d", &num);
  printf("\n\n");
  generatebits(num);
  getch();
  return(0);
}
```

The reason we have shown a recursive algorithm is that, because of the magic of recursion, we dont have to reverse the bits generated to produce the final output. One can always write an iterative algorithm to accomplish the same, but it would require you to first store the bits as they are generated and then reverse them before producing the final output.

Write C code to implement the Binary Search algorithm.

Here is a C function

```
int binarySearch(int arr[],int size, int item)
{
   int left, right, middle;
   left  = 0;
   right = size-1;

   while(left<=right)
   {
      middle = ((left + right)/2);

      if(item == arr[middle])
      {
        return(middle);
      }

      if(item > arr[middle])
      {
        left  = middle+1;
      }
      else
      {
        right = middle-1;
      }
   }

   return(-1);
}
```

Note that the Binary Search algorithm has a prerequisite that the array passed to it must be already sorted in ascending order. This will not work on an unsorted array. The complexity of this algorithm is O(log(n)).

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Wite code to evaluate a polynomial.

```
typedef struct node
{
  float cf;
  float px;
  float py;
  struct node *next;
}mynode;


float evaluate(mynode *head)
{
   float x,y,sum;
   sum = 0;
   mynode *poly;

   for(poly = head->next; poly != head; poly = poly->next)
   {
     sum = sum + poly->cf * pow(x, poly->px) * pow(y, poly->py);
   }

   return(sum);
}
```

Write code to add two polynomials

Here is some pseudocode

```
mynode *polynomial_add(mynode *h1, mynode *h2, mynode *h3)
{
   mynode *p1, *p2;
   int x1, x2, y1, y2, cf1, cf2, cf;

   p1 = h1->next;

   while(p1!=h1)
   {
      x1  = p1->px;
      y1  = p1->py;
      cf1 = p1->cf;

      // Search for this term in the second polynomial

      p2 = h2->next;

      while(p2 != h2)
      {
         x2  = p2->px;
         y2  = p2->py;
         cf2 = p2->cf;

         if(x1 == x2 && y1 == y2)break;

         p2 = p2->next;

      }


      if(p2 != h2)
      {
          // We found something in the second polynomial.

          cf = cf1 + cf2;
          p2->flag = 1;

          if(cf!=0){h3=addNode(cf,x1,y1,h3);}
      }
      else
      {
         h3=addNode(cf,x1,y1,h3);
      }

      p1 = p1->next;
```

```
   }//while


   // Add the remaining elements of the second polynomail to the result

   while(p2 != h2)
   {
      if(p2->flag==0)
      {
          h3=addNode(p2->cf, p2->px, p2->py, h3);
      }
      p2=p2->next;
   }

   return(h3);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a program to add two long positive numbers (each represented by linked lists).

Check out this simple implementation

```
mynode *long_add(mynode *h1, mynode *h2, mynode *h3)
{
  mynode *c, *c1, *c2;
  int sum, carry, digit;

  carry = 0;
  c1 = h1->next;
  c2 = h2->next;

  while(c1 != h1 && c2 != h2)
  {
     sum   = c1->value + c2->value + carry;
     digit = sum % 10;
     carry = sum / 10;

     h3 = insertNode(digit, h3);

     c1 = c1->next;
     c2 = c2->next;
  }

  if(c1 != h1)
  {
     c = c1;
     h = h1;
  }
  else
  {
     c = c2;
     h = h2;
  }

  while(c != h)
  {
    sum   = c->value + carry;
    digit = sum % 10;
    carry = sum / 10;
    h3 = insertNode(digit, h3);
    c = c->next;
  }

  if(carry==1)
  {
     h3 = insertNode(carry, h3);
  }
```

```
  return(h3);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you compare floating point numbers?

This is Wrong!.

```
double a, b;

if(a == b)
{
  ...
}
```

The above code might not work always. Thats because of the way floating point numbers are stored.

A good way of comparing two floating point numbers is to have a accuracy threshold which is relative to the magnitude of the two floating point numbers being compared.

```
#include <math.h>
if(fabs(a - b) <= accurary_threshold * fabs(a))
```

There is a lot of material on the net to know how floating point numbers can be compared. Got for it if you really want to understand.

Another way which might work is something like this. I have not tested it!

```
int compareFloats(float f1, float f2)
{
  char *b1, *b2;
  int i;

  b1 = (char *)&f1;
  b2 = (char *)&f2;

  /* Assuming sizeof(float) is 4 bytes) */

  for (i = 0; i<4; i++, b1++, b2++)
  {
    if (*b1 != *b2)
    {
      return(NOT_EQUAL); /* You must have defined this before */
    }
  }

  return(EQUAL);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What's a good way to implement complex numbers in C?

Use structures and some routines to manipulate them.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How can I display a percentage-done indication on the screen?

The character '\r' is a carriage return without the usual line feed, this helps to overwrite the current line. The character '\b' acts as a backspace, and will move the cursor one position to the left.

Write a program to check if a given year is a leap year or not?

Use this if() condition to check if a year is a leap year or not

```
if(year % 4 == 0 && (year % 100 != 0 || year % 400 == 0))
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Is there something we can do in C but not in C++?

I have a really funny answer

Declare variable names that are keywords in C++ but not C.

```
#include <stdio.h>
int main(void)
{
  int old, new=3;
  return 0;
}
```

This will compile in C, but not in C++!

How to swap the two nibbles in a byte ?

Try this

```
#include <stdio.h>

unsigned char swap_nibbles(unsigned char c)
{
  unsigned char temp1, temp2;
  temp1 = c & 0x0F;
  temp2 = c & 0xF0;
  temp1=temp1 << 4;
  temp2=temp2 >> 4;

  return(temp2|temp1); //adding the bits
}

int main(void)
{
  char ch=0x34;
  printf("\nThe exchanged value is %x",swap_nibbles(ch));
  return 0;
}
```

Programs
(70 / 301) *

How to scan a string till we hit a new line using scanf()?

Use

```
scanf("%[^\n]", address);
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write pseudocode to compare versions (like 115.10.1 vs 115.11.5).

This question is also quite popular, because it has real practical uses, specially during patching when version comparison is required

The pseudocode is something like

```
while(both version strings are not NULL)
{
  // Extract the next version segment from Version string1.
  // Extract the next version segment from Version string2.

  // Compare these segments, if they are equal, continue
  // to check the next version segments.

  // If they are not equal, return appropriate code depending
  // on which segment is greater.
}
```

And here is some code in PL-SQL

```
--------------------------------------------------------------------
compare_versions()

Function to compare releases. Very useful when comparing file versions!

This function compare two versions in pl-sql language. This function can compare
Versions like 115.10.1 vs. 115.10.2 (and say 115.10.2 is greater), 115.10.1 vs. 115.10 (and say
115.10.1 is greater), 115.10 vs. 115.10 (and say both are equal)
--------------------------------------------------------------------

function compare_releases(release_1 in varchar2, release_2 in varchar2)
return boolean is

  release_1_str varchar2(132);
  release_2_str varchar2(132);
  release_1_ver number;
  release_2_ver number;
  ret_status boolean := TRUE;

begin

  release_1_str := release_1 || '.';
  release_2_str := release_2 || '.';

  while release_1_str is not null or release_2_str is not null loop

    -- Parse out a current version segment from release_1
```

```
    if (release_1_str is null) then
       release_1_ver := 0;
    else
      release_1_ver := nvl(to_number(substr(release_1_str,1, instr(release_1_str,'.')-1)),-1);
      release_1_str := substr(release_1_str,instr(release_1_str,'.')+1);
    end if;

    -- Next parse out a version segment from release_2
    if (release_2_str is null) then
       release_2_ver := 0;
    else
       release_2_ver := nvl(to_number(substr(release_2_str,1, instr(release_2_str,'.')-1)),-1);
       release_2_str := substr(release_2_str,instr(release_2_str,'.')+1);
    end if;


     if (release_1_ver > release_2_ver) then
        ret_status := FALSE;
        exit;
     elsif (release_1_ver < release_2_ver) then
        exit;
     end if;

     -- Otherwise continue to loop.
   end loop;

   return(ret_status);

end compare_releases;
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you get the line numbers in C?

Use the following Macros

```
__FILE__ Source file name (string constant) format "patx.c"
__LINE__ Current source line number (integer)
__DATE__ Date compiled (string constant)format "Dec 14 1985"
__TIME__ Time compiled (string constant) format "15:24:26"
__TIMESTAMP__ Compile date/time (string constant)format "Tue Nov 19 11:39:12 1997"
```

Usage example

```
static char stamp[] =    "***\nmodule " __FILE__    "\ncompiled " __TIMESTAMP__     "\n***";

...

int main()
{
   ...

   if ( (fp = fopen(fl,"r")) == NULL )
   {
      printf( "open failed, line %d\n%s\n",__LINE__, stamp );
      exit( 4 );
   }

   ...
}
```

And the output is something like

```
*** open failed, line 67
******
module myfile.c
compiled Mon Jan 15 11:15:56 1999
***
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to fast multiply a number by 7?

Try

```
(num<<3 - num)
```

This is same as

```
num*8 - num = num * (8-1) = num * 7
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Programs
(74 / 301) *

Write a simple piece of code to split a string at equal intervals

Suppose you have a big string

```
This is a big string which I want to split at equal intervals, without caring about the words.
```

Now, to split this string say into smaller strings of 20 characters each, try this

```
#define maxLineSize 20

split(char *string)
{
  int i, length;
  char dest[maxLineSize + 1];

  i           = 0;
  length      = strlen(string);

  while((i+maxLineSize) <= length)
  {
     strncpy(dest, (string+i), maxLineSize);
     dest[maxLineSize - 1] = '\0';
     i = i + strlen(dest) - 1;
     printf("\nChunk : [%s]\n", dest);
  }

  strcpy(dest, (string + i));
  printf("\nChunk : [%s]\n", dest);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Is there a way to multiply matrices in lesser than o(n^3) time complexity?

Yes. Divide and conquer method suggests Strassen's matrix multiplication method to be used. If we follow this method, the time complexity is O(n^2.81) times rather O(n^3) times.

Here are some more details about this method.

Suppose we want to multiply two matrices of size N x N: for example A x B = C

```
[C11 C12]   [A11 A12] [B11 B12]
[C21 C22] = [A21 A22] [B21 B22]
```

Now, this guy called Strassen's somehow :) came up with a bunch of equations to calculate the 4 elements of the resultant matrix

```
C11 = a11*b11 + a12*b21
C12 = a11*b12 + a12*b22
C21 = a21*b11 + a22*b21
C22 = a21*b12 + a22*b22
```

If you are aware, the rudimentary matrix multiplication goes something like this

```
void matrix_mult()
{
  for (i = 1; i <= N; i++)
  {
     for (j = 1; j <= N; j++)
     {
        compute Ci,j;
     }
  }
}
```

So, essentially, a 2x2 matrix multiplication can be accomplished using 8 multiplications. And the complexity becomes

```
2^log 8 =2^3
```

Strassen showed that 2x2 matrix multiplication can be accomplished in 7 multiplications and 18 additions or subtractions. So now the complexity becomes

```
2^log7 =2^2.807
```

This is how he did it

```
P1 = (A11+ A22)(B11+B22)
P2 = (A21 + A22) * B11
P3 = A11 * (B12 - B22)
P4 = A22 * (B21 - B11)
P5 = (A11 + A12) * B22
P6 = (A21 - A11) * (B11 + B12)
P7 = (A12 - A22) * (B21 + B22)

C11 = P1 + P4 - P5 + P7
C12 = P3 + P5
C21 = P2 + P4
C22 = P1 + P3 - P2 + P6
```

Now, there is no need to memorize this stuff!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you find out if a machine is 32 bit or 64 bit?

One common answer is that all compilers keep the size of integer the same as the size of the register on a perticular architecture. Thus, to know whether the machine is 32 bit or 64 bit, just see the size of integer on it.

I am not sure how true this is!

* *

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a program to have the output go two places at once (to the screen and to a file also)

You can write a wrapper function for printf() which prints twice.

```
myprintf(...)
{
  // printf();        -> To screen.
  // write_to_file(); -> To file.
}
```

A command in shell, called tee does have this functionality!

Write code to round numbers

Use something like

```
(int)(num < 0 ? (num - 0.5) : (num + 0.5))
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How can we sum the digits of a given number in single statement?

Try something like this

```
# include<stdio.h>

void main()
{
  int num=123456;
  int sum=0;

  for(;num>0;sum+=num%10,num/=10);  // This is the "single line".

  printf("\nsum = [%d]\n", sum);
}
```

If there is a simpler way to do this, let me know!

Given two strings A and B, how would you find out if the characters in B were a subset of the characters in A?

Here is a simple, yet efficient C program to accomplish the same...

```
#include <stdio.h>
#include <conio.h>

int isSubset(char *a, char *b);

int main()
{
 char str1[]="defabc";
 char str2[]="abcfed";

 if(isSubset(str1, str2)==0)
 {
   printf("\nYes, characters in B=[%s] are a subset of characters in A=[%s]\n",str2,str1);
 }
 else
 {
   printf("\nNo, characters in B=[%s] are not a subset of characters in A=[%s]\n",str2,str1);
 }

 getch();
 return(0);
}


// Function to check if characters in "b" are a subset
// of the characters in "a"

int isSubset(char *a, char *b)
{
 int letterPresent[256];
 int i;

 for(i=0; i<256; i++)
    letterPresent[i]=0;

 for(i=0; a[i]!='\0'; i++)
    letterPresent[a[i]]++;

 for(i=0; b[i]!='\0'; i++)
    if(!letterPresent[b[i]])
       return(1);

 return(0);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Programs
(81 / 301)

Write a program to merge two arrays in sorted order, so that if an integer is in both the arrays, it gets added into the final array only once.

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a program to check if the stack grows up or down

Try noting down the address of a local variable. Call another function with a local variable declared in it and check the address of that local variable and compare!.

```
#include <stdio.h>
#include <stdlib.h>

void stack(int *local1);

int main()
{
  int local1;
  stack(&local1);
  exit(0);
}

void stack(int *local1)
{
   int local2;
   printf("\nAddress of first  local : [%u]", local1);
   printf("\nAddress of second local : [%u]", &local2);
   if(local1 < &local2)
   {
     printf("\nStack is growing downwards.\n");
   }
   else
   {
     printf("\nStack is growing upwards.\n");
   }
   printf("\n\n");
}
```

How to add two numbers without using the plus operator?

Actually,

```
SUM   = A XOR B
CARRY = A AND B
```

On a wicked note, you can add two numbers wihtout using the + operator as follows

```
a - (- b)
```

:)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to generate prime numbers? How to generate the next prime after a given prime?

This is a very vast subject. There are numerous methods to generate primes or to find out if a given number is a prime number or not. Here are a few of them. I strongly recommend you to search on the Internet for more elaborate information.

Brute Force

Test each number starting with 2 and continuing up to the number of primes we want to generate. We divide each numbr by all divisors upto the square root of that number. If no factors are found, its a prime.

Using only primes as divisors

Test each candidate only with numbers that have been already proven to be prime. To do so, keep a list of already found primes (probably using an array, a file or bit fields).

Test with odd candidates only

We need not test even candidates at all. We could make 2 a special case and just print it, not include it in the list of primes and start our candidate search with 3 and increment by 2 instead of one everytime.

Table method

Suppose we want to find all the primes between 1 and 64. We write out a table of these numbers, and proceed as follows. 2 is the first integer greater than 1, so its obviously prime. We now cross out all multiples of two. The next number we haven't crossed out is 3. We circle it and cross out all its multiples. The next non-crossed number is 5, sp we circle it and cross all its mutiples. We only have to do this for all numbers less than the square root of our upper limit, since any composite in the table must have atleast one factor less than the square root of the upper limit. Whats left after this process of elimination is all the prime numbers between 1 and 64.

Write a program to print numbers from 1 to 100 without using loops!

Another "Yeah, I am a jerk, kick me! kind of a question. I simply dont know why they ask these questions.

Nevertheless, for the benefit of mankind...

Method1 (Using recursion)

```
void printUp(int startNumber, int endNumber)
{
  if (startNumber > endNumber)
    return;

  printf("[%d]\n", startNumber++);
  printUp(startNumber, endNumber);
}
```

Method2 (Using goto)

```
void printUp(int startNumber, int endNumber)
{
start:

    if (startNumber > endNumber)
    {
       goto end;
    }
    else
    {
       printf("[%d]\n", startNumber++);
       goto start;
    }

end:
    return;
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own trim() or squeeze() function to remove the spaces from a string.

Here is one version...

```
#include <stdio.h>

char *trim(char *s);

int main(int argc, char *argv[])
{
  char str1[]=" Hello   I am Good ";
  printf("\n\nBefore trimming : [%s]", str1);
  printf("\n\nAfter trimming  : [%s]", trim(str1));

  getch();
}


// The trim() function...

char *trim(char *s)
{
    char *p, *ps;

    for (ps = p = s; *s != '\0'; s++)
    {
        if (!isspace(*s))
        {
            *p++ = *s;
        }
    }

    *p = '\0';

    return(ps);
}
```

And here is the output...

```
Before trimming : [ Hello   I am Good ]
After trimming  : [HelloIamGood]
```

Another version of this question requires one to reduce multiple spaces, tabs etc to single spaces...

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write your own random number generator function in C.

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Programs
(88 / 301) *

Write your own sqrt() function in C

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(89 / 301)

Write a C program to find the depth or height of a tree.

Here is some C code to get the height of the three

```
tree_height(mynode *p)
{
   if(p==NULL)return(0);
   if(p->left){h1=tree_height(p->left);}
   if(p=>right){h2=tree_height(p->right);}
   return(max(h1,h2)+1);
}
```

The degree of the leaf is zero. The degree of a tree is the max of its element degrees. A binary tree of height n, h > 0, has at least h and at most (2^h -1) elements in it. The height of a binary tree that contains n, n>0, elements is at most n and atleast log(n+1) to the base 2.

Log(n+1) to the base 2 = h

n = (2^h - 1)

Trees
(90 / 301)

Write a C program to determine the number of elements (or size) in a tree.

Try this C program

```
int tree_size(struct node* node)
{
  if (node==NULL)
  {
    return(0);
  }
  else
  {
    return(tree_size(node->left) + tree_size(node->right) + 1);
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to delete a tree (i.e, free up its nodes)

Free up the nodes using Postorder traversal!.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to determine if two trees are identical

Here is a C program using recursion

```
int identical(struct node* a, struct node* b)
{
  if (a==NULL && b==NULL){return(true);}
  else if (a!=NULL && b!=NULL)
  {
    return(a->data == b->data &&
           identical(a->left, b->left) &&
           identical(a->right, b->right));
  }
  else return(false);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(93 / 301)

Write a C program to find the mininum value in a binary search tree.

Here is some sample C code. The idea is to keep on moving till you hit the left most node in the tree

```
int minValue(struct node* node)
{
  struct node* current = node;

  while (current->left != NULL)
  {
    current = current->left;
  }

  return(current->data);
}
```

On similar lines, to find the maximum value, keep on moving till you hit the right most node of the tree.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to compute the maximum depth in a tree?

Here is some C code...

```
int maxDepth(struct node* node)
{
  if (node==NULL)
  {
    return(0);
  }
  else
  {
    int leftDepth  = maxDepth(node->left);
    int rightDepth = maxDepth(node->right);
    if (leftDepth > rightDepth) return(leftDepth+1);
    else return(rightDepth+1);
  }
}
```

Write a C program to create a mirror copy of a tree (left nodes become right and right nodes become left)!

This C code will create a new mirror copy tree.

```
mynode *copy(mynode *root)
{
  mynode *temp;

  if(root==NULL)return(NULL);

  temp = (mynode *) malloc(sizeof(mynode));
  temp->value = root->value;

  temp->left  = copy(root->right);
  temp->right = copy(root->left);

  return(temp);
}
```

This code will will only print the mirror of the tree

```
void tree_mirror(struct node* node)
{
   struct node *temp;

   if (node==NULL)
   {
     return;
   }
   else
   {
      tree_mirror(node->left);
      tree_mirror(node->right);

      // Swap the pointers in this node
      temp = node->left;
      node->left = node->right;
      node->right = temp;
   }
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to return a pointer to the nth node of an inorder traversal of a BST.

Here is a C program. Study it carefully, it has some Gotchas!

```
typedef struct node
{
  int value;
  struct node *left;
  struct node *right;
}mynode;

mynode *root;
static ctr;

void nthnode(mynode *root, int n, mynode **nthnode /* POINTER TO A POINTER! */);

int main()
{
  mynode *temp;
  root = NULL;

  // Construct the tree
  add(19);
  add(20);
  ...
  add(11);

  // Plain Old Inorder traversal
  // Just to see if the next function is really returning the nth node?
  inorder(root);

  // Get the pointer to the nth Inorder node
  nthinorder(root, 6, &temp);

  printf("\n[%d]\n, temp->value);
  return(0);
}

// Get the pointer to the nth inorder node in "nthnode"
void nthinorder(mynode *root, int n, mynode **nthnode)
{
  static whichnode;
  static found;

  if(!found)
  {
    if(root)
    {
       nthinorder(root->left, n , nthnode);
```

```
       if(++whichnode == n)
       {
          printf("\nFound %dth node\n", n);
          found = 1;
          *nthnode = root; // Store the pointer to the nth node.
        }
       nthinorder(root->right, n , nthnode);
    }
  }
}


inorder(mynode *root)
{
  // Plain old inorder traversal
}


// Function to add a new value to a Binary Search Tree
add(int value)
{
  mynode *temp, *prev, *cur;

  temp = malloc(sizeof(mynode));
  temp->value = value;
  temp->left  = NULL;
  temp->right = NULL;

  if(root == NULL)
  {
    root = temp;
  }
  else
  {
    prev = NULL;
    cur  = root;

    while(cur)
    {
      prev = cur;
      cur  = (value < cur->value)? cur->left : cur->right;
    }

    if(value > prev->value)
       prev->right = temp;
    else
       prev->left  = temp;
  }
}
```

There seems to be an easier way to do this, or so they say. Suppose each node also has a weight associated with it. This weight is the number of nodes below it and including itself. So, the root will have the highest weight (weight of its left subtree + weight of its right subtree + 1). Using this data, we can easily find the nth inorder node.

Note that for any node, the (weight of the leftsubtree of a node + 1) is its inorder rankin the tree!. Thats simply because of how the inorder traversal works (left->root->right). So calculate the rank of each node and you can get to the nth inorder node easily. But frankly speaking, I really dont know how this method is any simpler than the one I have presented above. I see more work to be done here (calculate thw weights, then calculate the ranks and then get to the nth node!).

Also, if (n > weight(root)), we can error out saying that this tree does not have the nth node you are looking for.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to implement the preorder(), inorder() and postorder() traversals. Whats their time complexities?

Here are the C program snippets to implement these traversals...

Preorder

```
preorder(mynode *root)
{
  if(root)
  {
    printf("Value : [%d]", root->value);
    preorder(root->left);
    preorder(root->right);
  }
}
```

Postorder

```
postorder(mynode *root)
{
  if(root)
  {
    postorder(root->left);
    postorder(root->right);
    printf("Value : [%d]", root->value);
  }
}
```

Inorder

```
inorder(mynode *root)
{
  if(root)
  {
    inorder(root->left);
    printf("Value : [%d]", root->value);
    inorder(root->right);
  }
}
```

Time complexity of traversals is O(n).

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to create a copy of a tree

Here is a C program which does that...

```
mynode *copy(mynode *root)
{
  mynode *temp;

  if(root==NULL)return(NULL);

  temp = (mynode *) malloc(sizeof(mynode));
  temp->value = root->value;

  temp->left  = copy(root->left);
  temp->right = copy(root->right);

  return(temp);
}
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to check if a given binary tree is a binary search tree or not?

Here is a C program which checks if a given tree is a Binary Search Tree or not...

```
int isThisABST(struct node* mynode)
{
    if (mynode==NULL) return(true);
    if (node->left!=NULL && maxValue(mynode->left) > mynode->data)
       return(false);
    if (node->right!=NULL && minValue(mynode->right) <= mynode->data)
       return(false);
    if (!isThisABST(node->left) || !isThisABST(node->right))
       return(false);

    return(true);
}
```

Trees
(100 / 301)

Write C code to implement level order traversal of a tree.

If this is the tree,

```
        1
   2          3
 5   6      7   8
```

its level order traversal would be

```
1 2 3 5 6 7 8
```

You need to use a queue to do this kind of a traversal

```
Let t be the tree root.
while (t != null)
{
  visit t and put its children on a FIFO queue;
  remove a node from the FIFO queue and
  call it t;
  // Remove returns null when queue is empty
}
```

Pseduocode

```
Level_order_traversal(p)
{
   while(p)
   {
     Visit(p);
     If(p->left)Q.Add(p->left);
     If(p->right)Q.Add(p->right);
     Delete(p);
   }
}
```

Here is some C code (working :))..

```
#include <stdio.h>

typedef struct node
{
  int value;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

add_node(int value);

void levelOrderTraversal(mynode *root);

int main(int argc, char* argv[])
{
  root = NULL;

  add_node(5);
  add_node(1);
  add_node(-20);
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);


  printf("\n\n\nLEVEL ORDER TRAVERSAL\n\n");
  levelOrderTraversal(root);

  getch();
}



// Function to add a new node...
add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;
}



// Level order traversal..
void levelOrderTraversal(mynode *root)
{
  mynode *queue[100] = {(mynode *)0}; // Important to initialize!
  int size = 0;
  int queue_pointer = 0;

  while(root)
```

```
  {
      printf("[%d] ", root->value);

      if(root->left)
      {
        queue[size++] = root->left;
      }

      if(root->right)
      {
        queue[size++] = root->right;
      }

      root = queue[queue_pointer++];
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to delete a node from a Binary Search Tree?

The node to be deleted might be in the following states

- The node does not exist in the tree - In this case you have nothing to delete.
- The node to be deleted has no children - The memory occupied by this node must be freed and either the left link or the right link of the parent of this node must be set to NULL.
- The node to be deleted has exactly one child - We have to adjust the pointer of the parent of the node to be deleted such that after deletion it points to the child of the node being deleted.
- The node to be deleted has two children - We need to find the inorder successor of the node to be deleted. The data of the inorder successor must be copied into the node to be deleted and a pointer should be setup to the inorder successor. This inorder successor would have one or zero children. This node should be deleted using the same procedure as for deleting a one child or a zero child node. Thus the whole logic of deleting a node with two children is to locate the inorder successor, copy its data and reduce the problem to a simple deletion of a node with one or zero children.

Here is some C code for these two situations

```
Situation 1

     100 (parent)
   50 (cur == psuc)
20    80 (suc)
         90
       85  95


Situation 2
           100 (parent)
        50 (cur)
     20    90
         80
       70 (suc)
          75
```

```
mynode *delete(int item, mynode *head)
{
     mynode *cur, *parent, *suc, *psuc, q;

     if(head->left==NULL){printf("\nEmpty tree!\n");return(head);}

     parent = head;
     cur    = head->left;

     while(cur!=NULL && item != cur->value)
     {
         parent  =  cur;
         cur     =  (item < cur->next)? cur->left:cur->right;
     }

     if(cur == NULL)
     {
        printf("\nItem to be deleted not found!\n");
        return(head);
     }


     // Item found, now delete it

     if(cur->left == NULL)
         q = cur->right;
     else if(cur->right == NULL)
         q = cur->left;
     else
     {
           // Obtain the inorder successor and its parent

           psuc = cur;
           cur  = cur->left;

           while(suc->left!=NULL)
           {
               psuc = suc;
               suc  = suc->left;
           }


           if(cur==psuc)
           {
                // Situation 1

                suc->left = cur->right;
           }
           else
           {
                // Situation 2

                suc->left  = cur->left;
                psuc->left = suc->right;
                suc->right = cur->right;
           }

           q = suc;

     }


     // Attach q to the parent node

     if(parent->left == cur)
        parent->left=q;
     else
        parent->rlink=q;

     freeNode(cur);

     return(head);
```

```
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(102 / 301)

Write C code to search for a value in a binary search tree (BST).

Here are a few C programs....

```
mynode *search(int value, mynode *root)
{
   while(root!=NULL && value!=root->value)
   {
     root = (value < root->value)?root->left:root->right;
   }

   return(root);
}
```

Here is another way to do the same

```
mynode *recursive_search(int item, mynode *root)
{
  if(root==NULL || item == root->value){return(root);}
  if(item<root->info)return{recursive_search(item, root->left);}
  return{recursive_search(item, root->right);}
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code to count the number of leaves in a tree

Here is a C program to do the same...

```
void count_leaf(mynode *root)
{
   if(root!=NULL)
   {
      count_leaf(root->left);

      if(root->left == NULL && root->right==NULL)
      {
        // This is a leaf!
        count++;
      }

      count_leaf(root->right);
   }
}
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write C code for iterative preorder, inorder and postorder tree traversals

Here is a complete C program which prints a BST using both recursion and iteration. The best way to understand these algorithms is to get a pen and a paper and trace out the traversals (with the stack or the queue) alongside. Dont even try to memorize these algorithms!

```
#include <stdio.h>

typedef struct node
{
  int value;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

add_node(int value);
void postorder(mynode *root);
void inorder(mynode *root);
void preorder(mynode *root);

void iterativePreorder(mynode *root);
void iterativeInorder (mynode *root);
void iterativePostorder(mynode *root);


int main(int argc, char* argv[])
{
  root = NULL;

  add_node(5);
  add_node(1);
  add_node(-20);
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);

  printf("\nPreorder (R)    : ");
  preorder(root);
  printf("\nPreorder (I)    : ");
  iterativePreorder(root);

  printf("\n\nPostorder (R)   : ");
  postorder(root);
  printf("\nPostorder (R)   : ");
  iterativePostorder(root);
```

```
  printf("\n\nInorder (R)     : ");
  inorder(root);
  printf("\nInorder (I)     : ");
  iterativeInorder(root);

}

// Function to add a new node to the BST
add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;
}


// Recursive Preorder
void preorder(mynode *root)
{
  if(root)
  {
    printf("[%d] ", root->value);
    preorder(root->left);
    preorder(root->right);
  }
}

// Iterative Preorder
void iterativePreorder(mynode *root)
{
  mynode *save[100];
  int top = 0;

  if (root == NULL)
  {
    return;
  }

  save[top++] = root;
  while (top != 0)
  {
    root = save[--top];

    printf("[%d] ", root->value);

    if (root->right != NULL)
      save[top++] = root->right;
    if (root->left != NULL)
      save[top++] = root->left;
  }
}

// Recursive Postorder
void postorder(mynode *root)
{
```

```
  if(root)
  {
    postorder(root->left);
    postorder(root->right);
    printf("[%d] ", root->value);
  }
}

// Iterative Postorder
void iterativePostorder(mynode *root)
{
  struct
  {
    mynode *node;
    unsigned vleft :1;   // Visited left?
    unsigned vright :1;  // Visited right?
  }save[100];

  int top = 0;

  save[top++].node = root;

  while ( top != 0 )
  {
      /* Move to the left subtree if present and not visited */
      if(root->left != NULL && !save[top].vleft)
      {
          save[top].vleft = 1;
          save[top++].node = root;
          root = root->left;
          continue;
      }

      /* Move to the right subtree if present and not visited */
      if(root->right != NULL && !save[top].vright )
      {
          save[top].vright = 1;
          save[top++].node = root;
          root = root->right;
          continue;
      }

      printf("[%d] ", root->value);

      /* Clean up the stack */
      save[top].vleft = 0;
      save[top].vright = 0;

      /* Move up */
      root = save[--top].node;
   }
}


// Recursive Inorder
void inorder(mynode *root)
{
  if(root)
  {
    inorder(root->left);
    printf("[%d] ", root->value);
    inorder(root->right);
  }
}


// Iterative Inorder..
void iterativeInorder (mynode *root)
{
  mynode *save[100];
  int top = 0;

  while(root != NULL)
  {
      while (root != NULL)
      {
           if (root->right != NULL)
           {
```

```
            save[top++] = root->right;
          }
          save[top++] = root;
          root = root->left;
      }

      root = save[--top];
      while(top != 0 && root->right == NULL)
      {
          printf("[%d] ", root->value);
          root = save[--top];
      }

      printf("[%d] ", root->value);
      root = (top != 0) ? save[--top] : (mynode *) NULL;
  }
}
```

And here is the output...

```
Creating the root..

Preorder (R)    : [5] [1] [-20] [100] [23] [13] [67]
Preorder (I)    : [5] [1] [-20] [100] [23] [13] [67]

Postorder (R)   : [-20] [1] [13] [67] [23] [100] [5]
Postorder (R)   : [-20] [1] [13] [67] [23] [100] [5]

Inorder (R)     : [-20] [1] [5] [13] [23] [67] [100]
Inorder (I)     : [-20] [1] [5] [13] [23] [67] [100]
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Can you construct a tree using postorder and preorder traversal?

No

Consider 2 trees below

```
Tree1

  a
b
```

```
Tree 2

a
  b
```

```
preorder  = ab
postorder = ba
```

Preorder and postorder do not uniquely define a binary tree. Nor do preorder and level order (same example). Nor do postorder and level order.

Trees
(106 / 301)

Construct a tree given its inorder and preorder traversal strings. Similarly construct a tree given its inorder and post order traversal strings.

For Inorder And Preorder traversals

```
inorder  = g d h b e i a f j c
preorder = a b d g h e i c f j
```

Scan the preorder left to right using the inorder sequence to separate left and right subtrees. For example, "a" is the root of the tree; "gdhbei" are in the left subtree; "fjc" are in the right subtree. "b" is the next root; "gdh" are in the left subtree; "ei" are in the right subtree. "d" is the next root; "g" is in the left subtree; "h" is in the right subtree.

For Inorder and Postorder traversals

Scan postorder from right to left using inorder to separate left and right subtrees.

```
inorder   = g d h b e i a f j c
postorder = g h d i e b j f c a
```

Tree root is "a"; "gdhbei" are in left subtree; "fjc" are in right subtree.

For Inorder and Levelorder traversals

Scan level order from left to right using inorder to separate left and right subtrees.

```
inorder     = g d h b e i a f j c
level order = a b c d e f g h i j
```

Tree root is "a"; "gdhbei" are in left subtree; "fjc" are in right subtree.

Here is some working code which creates a tree out of the Inorder and Postorder traversals. Note that here the tree has been represented as an array. This really simplifies the whole implementation.

Converting a tree to an array is very easy

Suppose we have a tree like this

```
      A
  B       C
```

```
 D E     F G
```

The array representation would be

```
a[1] a[2] a[3] a[4] a[5] a[6] a[7]
  A    B    C    D    E    F    G
```

That is, for every node at position `j` in the array, its left child will be stored at position `(2*j)` and right child at `(2*j + 1)`. The root starts at position `1`.

```
// CONSTRUCTING A TREE GIVEN THE INORDER AND PREORDER SEQUENCE

#include<stdio.h>
#include<string.h>
#include<ctype.h>

/*-------------------------------------------------------------
* Algorithm
*
* Inorder And Preorder
* inorder = g d h b e i a f j c
* preorder = a b d g h e i c f j
* Scan the preorder left to right using the inorder to separate left
* and right subtrees. a is the root of the tree; gdhbei are in the
* left subtree; fjc are in the right subtree.
*-------------------------------------------------------------*/

static char io[]="gdhbeiafjc";
static char po[]="abdgheicfj";
static char t[100][100]={'\0'};  //This is where the final tree will be stored
static int hpos=0;

void copy_str(char dest[], char src[], int pos, int start, int end);
void print_t();

int main(int argc, char* argv[])
{
  int i,j,k;
  char *pos;
  int posn;

  // Start the tree with the root and its
  // left and right elements to start off

  for(i=0;i<strlen(io);i++)
  {
     if(io[i]==po[0])
     {
        copy_str(t[1],io,1,i,i);             // We have the root here
        copy_str(t[2],io,2,0,i-1);           // Its left subtree
        copy_str(t[3],io,3,i+1,strlen(io));  // Its right subtree
        print_t();
     }
  }

  // Now construct the remaining tree
  for(i=1;i<strlen(po);i++)
  {
     for(j=1;j<=hpos;j++)
     {
        if((pos=strchr((const char *)t[j],po[i]))!=(char *)0 && strlen(t[j])!=1)
        {
            for(k=0;k<strlen(t[j]);k++)
            {
               if(t[j][k]==po[i]){posn=k;break;}
            }
            printf("\nSplitting [%s] for po[%d]=[%c] at %d..\n", t[j],i,po[i],posn);

            copy_str(t[2*j],t[j],2*j,0,posn-1);
            copy_str(t[2*j+1],t[j],2*j+1,posn+1,strlen(t[j]));
            copy_str(t[j],t[j],j,posn,posn);
            print_t();
        }
```

```
    }
  }
}

// This function is used to split a string into three seperate strings
// This is used to create a root, its left subtree and its right subtree

void copy_str(char dest[], char src[], int pos, int start, int end)
{
  char mysrc[100];
  strcpy(mysrc,src);
  dest[0]='\0';
  strncat(dest,mysrc+start,end-start+1);
  if(pos>hpos)hpos=pos;
}


void print_t()
{
  int i;
  for(i=1;i<=hpos;i++)
  {
    printf("\nt[%d] = [%s]", i, t[i]);
  }
  printf("\n");
}
```

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Find the closest ancestor of two nodes in a tree.

Here is some working C code...

```
#include <stdio.h>

typedef struct node
{
  int value;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

mynode *add_node(int value);
void levelOrderTraversal(mynode *root);
mynode *closestAncestor(mynode* root, mynode* p, mynode* q);


int main(int argc, char* argv[])
{
  mynode *node_pointers[7], *temp;
  root = NULL;

  // Create the BST.
  // Store the node pointers to use later...
  node_pointers[0] = add_node(5);
  node_pointers[1] = add_node(1);
  node_pointers[2] = add_node(-20);
  node_pointers[3] = add_node(100);
  node_pointers[4] = add_node(23);
  node_pointers[5] = add_node(67);
  node_pointers[6] = add_node(13);


  printf("\n\n\nLEVEL ORDER TRAVERSAL\n\n");
  levelOrderTraversal(root);


  // Calculate the closest ancestors of a few nodes..

  temp = closestAncestor(root, node_pointers[5], node_pointers[6]);
  printf("\n\nClosest ancestor of [%d] and [%d] is [%d]\n\n",
         node_pointers[5]->value,
         node_pointers[6]->value,
         temp->value);


  temp = closestAncestor(root, node_pointers[2], node_pointers[6]);
```

```
  printf("\n\nClosest ancestor of [%d] and [%d] is [%d]\n\n",
         node_pointers[2]->value,
         node_pointers[6]->value,
         temp->value);


  temp = closestAncestor(root, node_pointers[4], node_pointers[5]);
  printf("\n\nClosest ancestor of [%d] and [%d] is [%d]\n\n",
         node_pointers[4]->value,
         node_pointers[5]->value,
         temp->value);


  temp = closestAncestor(root, node_pointers[1], node_pointers[3]);
  printf("\n\nClosest ancestor of [%d] and [%d] is [%d]\n\n",
         node_pointers[1]->value,
         node_pointers[3]->value,
         temp->value);


  temp = closestAncestor(root, node_pointers[2], node_pointers[6]);
  printf("\n\nClosest ancestor of [%d] and [%d] is [%d]\n\n",
         node_pointers[2]->value,
         node_pointers[6]->value,
         temp->value);

}


// Function to add a new node to the tree..
mynode *add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;

   return(temp);

}



// Level order traversal..
void levelOrderTraversal(mynode *root)
{
  mynode *queue[100] = {(mynode *)0};
  int size = 0;
  int queue_pointer = 0;

  while(root)
  {
      printf("[%d] ", root->value);

      if(root->left)
      {
```

```
        queue[size++] = root->left;
      }

      if(root->right)
      {
        queue[size++] = root->right;
      }

      root = queue[queue_pointer++];
  }
}


// Function to find the closest ancestor...
mynode *closestAncestor(mynode* root, mynode* p, mynode* q)
{
   mynode *l, *r, *tmp;

   if(root == NULL)
   {
      return(NULL);
   }

   if(root->left==p || root->right==p || root->left==q || root->right==q)
   {
     return(root);
   }
   else
   {
      l = closestAncestor(root->left, p, q);
      r = closestAncestor(root->right, p, q);

      if(l!=NULL && r!=NULL)
      {
        return(root);
      }
      else
      {
         tmp = (l!=NULL) ? l : r;
         return(tmp);
      }
   }
}
```

Here is the tree for you to visualize...

```
                        5 (node=0)

       1 (node=1)                   100 (node=3)

-20 (node=2)                 23 (node=4)

                   13 (node=5)   67 (node=6)
```

Here is the output...

```
LEVEL ORDER TRAVERSAL

[5] [1] [100] [-20] [23] [13] [67]


Closest ancestor of [67] and [13] is [23]

Closest ancestor of [-20] and [13] is [5]

Closest ancestor of [23] and [67] is [100]

Closest ancestor of [1] and [100] is [5]

Closest ancestor of [-20] and [13] is [5]
```

* *

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(108 / 301)

Given an expression tree, evaluate the expression and obtain a paranthesized form of the expression.

The code below prints the paranthesized form of a tree.

```
infix_exp(p)
{
   if(p)
   {
      printf("(");
      infix_exp(p->left);
      printf(p->data);
      infix_exp(p->right);
      printf(")");
   }
}
```

Creating a binary tree for a postfix expression

```
mynode *create_tree(char postfix[])
{
   mynode *temp, *st[100];
   int i,k;
   char symbol;

   for(i=k=0; (symbol = postfix[i])!='\0'; i++)
   {
        temp = (mynode *) malloc(sizeof(struct node));
        temp->value = symbol;
        temp->left = temp->right = NULL;

        if(isalnum(symbol))
              st[k++] = temp;
        else
        {
              temp->right = st[--k];
              temp->left  = st[--k];
              st[k++]     = temp;
        }
   }
   return(st[--k]);
}
```

Evaluate a tree

```
float eval(mynode *root)
{
   float num;

   switch(root->value)
   {
        case '+' : return(eval(root->left) + eval(root->right)); break;
        case '-' : return(eval(root->left) - eval(root->right)); break;
        case '/' : return(eval(root->left) / eval(root->right)); break;
        case '*' : return(eval(root->left) * eval(root->right)); break;
        case '$' : return(eval(root->left) $ eval(root->right)); break;
        default  : if(isalpha(root->value))
                   {
                      printf("%c = ", root->value);
                      scanf("%f", &num);
                      return(num);
                   }
                   else
                   {
                      return(root->value - '0');
                   }

   }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How do you convert a tree into an array?

The conversion is based on these rules

```
If i > 1, i/2 is the parent
If 2*i > n, then there is no left child, else 2*i is the left child.
If (2*i + 1) > n, then there is no right child, else (2*i + 1) is the right child.
```

Converting a tree to an array is very easy

Suppose we have a tree like this

```
      A
  B       C
 D E     F G
```

The array representation would be

```
a[1] a[2] a[3] a[4] a[5] a[6] a[7]
  A    B    C    D    E    F    G
```

That is, for every node at position i in the array, its left child will be stored at position (2*i) and right child at (2*i + 1). The root starts at position 1.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is an AVL tree?

AVL trees are self-adjusting, height-balanced binary search trees and are named after the inventors: Adelson-Velskii and Landis. A balanced binary search tree has O(log n) height and hence O(log n) worst case search and insertion times. However, ordinary binary search trees have a bad worst case. When sorted data is inserted, the binary search tree is very unbalanced, essentially more of a linear list, with O(n) height and thus O(n) worst case insertion and lookup times. AVL trees overcome this problem.

An AVL tree is a binary search tree in which every node is height balanced, that is, the difference in the heights of its two subtrees is at most 1. The balance factor of a node is the height of its right subtree minus the height of its left subtree (right minus left!). An equivalent definition, then, for an AVL tree is that it is a binary search tree in which each node has a balance factor of -1, 0, or +1. Note that a balance factor of -1 means that the subtree is left-heavy, and a balance factor of +1 means that the subtree is right-heavy. Each node is associated with a Balancing factor.

```
Balance factor of each node = height of right subtree at that node - height of left subtree at that node.
```

Please be aware that we are talking about the height of the subtrees and not the weigths of the subtrees. This is a very important point. We are talking about the height!.

Here is some recursive, working! C code that sets the Balance factor for all nodes starting from the root....

```
#include <stdio.h>

typedef struct node
{
  int value;
  int visited;
  int bf;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

mynode *add_node(int value);
void levelOrderTraversal(mynode *root);
int setbf(mynode *p);


// The main function
int main(int argc, char* argv[])
{
  root = NULL;

  // Construct the tree..
  add_node(5);
  add_node(1);
  add_node(-20);
```

```
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);

  // Set the balance factors
  setbf(root);

  printf("\n\n\nLEVEL ORDER TRAVERSAL\n\n");
  levelOrderTraversal(root);
  getch();
}

// Function to add a new node to the tree...
mynode *add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->visited = 0;
   temp->bf = 0;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     //printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;

   return(temp);

}


// Recursive function to set the balancing factor
// of each node starting from the root!
int setbf(mynode *p)
{
   int templ, tempr;
   int count;
   count = 1;

   if(p == NULL)
   {
     return(0);
   }
   else
   {
       templ = setbf(p->left);
       tempr = setbf(p->right);

       if(templ < tempr)
         count = count + tempr;
       else
         count = count + templ;
   }

   // Set the nodes balancing factor.
   printf("\nNode = [%3d], Left sub-tree height = [%1d], Right sub-tree height = [%1d], BF = [%1d]\n",
          p->value, templ, tempr, (tempr - templ));
   p->bf = tempr - templ;
   return(count);
```

```
}


// Level order traversal..
void levelOrderTraversal(mynode *root)
{
  mynode *queue[100] = {(mynode *)0};
  int size = 0;
  int queue_pointer = 0;

  while(root)
  {
      printf("\n[%3d] (BF : %3d) ", root->value, root->bf);

      if(root->left)
      {
        queue[size++] = root->left;
      }

      if(root->right)
      {
        queue[size++] = root->right;
      }

      root = queue[queue_pointer++];
  }
}
```

And here is the output...

```
Node = [-20], Left sub-tree height = [0], Right sub-tree height = [0], BF = [0]
Node = [  1], Left sub-tree height = [1], Right sub-tree height = [0], BF = [-1]
Node = [ 13], Left sub-tree height = [0], Right sub-tree height = [0], BF = [0]
Node = [ 67], Left sub-tree height = [0], Right sub-tree height = [0], BF = [0]
Node = [ 23], Left sub-tree height = [1], Right sub-tree height = [1], BF = [0]
Node = [100], Left sub-tree height = [2], Right sub-tree height = [0], BF = [-2]
Node = [  5], Left sub-tree height = [2], Right sub-tree height = [3], BF = [1]


LEVEL ORDER TRAVERSAL

[  5] (BF :   1)
[  1] (BF :  -1)
[100] (BF :  -2)
[-20] (BF :   0)
[ 23] (BF :   0)
[ 13] (BF :   0)
[ 67] (BF :   0)
```

Here is the tree which we were dealing with above

```
           5
       1         100
-20           23
          13   67
```

After insertion, the tree might have to be readjusted as needed in order to maintain it as an AVL tree. A node with balance factor -2 or 2 is considered unbalanced and requires rebalancing the tree. The balance factor is either stored directly at each node or computed from the heights of the subtrees, possibly stored at nodes. If, due to an instertion or deletion, the tree becomes unbalanced, a corresponding left rotation or a right rotation is performed on that tree at a particular node. A balance factor > 1 requires a left rotation (i.e. the right subtree is heavier than the left subtree) and a balance factor < -1 requires a right rotation (i.e. the left subtree is heavier than the right subtree).

Here is some pseudo code to demonstrate the two types of rotations...

Left rotation

```
BEFORE

        0 (par)
```

```
   0          0 (p)

         0         0 (tmp)

       0   0     0   0
                (a) (b)
```

Here we left rotate the tree around node p

```
tmp         = p->right;
p->right    = tmp->left;
tmp->left   = p;

if(par)
{
   if(p is the left child of par)
   {
     par->left=tmp;
   }
   else
   {
     par->right=tmp;
   }
}
else
{
   root=tmp;
}

// Reclaculate the balance factors
setbf(root);
```

```
AFTER
        0 (par)

   0             0
                (tmp)

          0           0
         (p)         (b)

      0      0
            (a)

   0     0
```

Right rotation

```
BEFORE

        0 (par)

   0            0 (p)

         0 (tmp)      0

      0    0       0     0
     (a)  (b)
```

Here we right rotate the tree around node p

```
tmp         = p->left;
p->left     = tmp->right;
tmp->right  = p;

if(par)
```

```
{
   if(p is the left child of par)
   {
     par->left=tmp;
   }
   else
   {
     par->right=tmp;
   }
}
else
{
   root=tmp;
}

// Recalculate the balancing factors...
setbf(root);
```

AFTER

```
        0 (par)

   0            0 (tmp)

          0           0
         (a)         (p)

                  0         0
                 (b)

                         0     0
```

**

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(111 / 301)

How many different trees can be constructed using n nodes?

Good question!

Its

2^n - n

So, if there are 10 nodes, you will have (1024 - 10) = 1014 different trees!! Confirm it yourself with a small number if you dont believe the formula.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(112 / 301) *

A full N-ary tree has M non-leaf nodes, how many leaf nodes does it have?

Use Geometric progression.

```
M + (N ^ (n-1)) = (1 - (N ^ n)) / (1 - N)
```

Here (N ^ (n-1)) is the number of leaf-nodes.

Solving for this leads to the answer

```
Leaf nodes = M * (N - 1) + 1
```

Suppose you have a 3-ary tree

```
              A
     B        C        D
  E  F  G   H  I  J   K  L  M
```

So, here M=4 and N=3. So using the formula above,

```
Leaf nodes = M * (N - 1) + 1 = 4 * (3 - 1) + 1 = 9
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Implement Breadth First Search (BFS) and Depth First Search (DFS)

Depth first search (DFS)

Depth First Search (DFS) is a generalization of the preorder traversal. Starting at some arbitrarily chosen vertex v, we mark v so that we know we've visited it, process v, and then recursively traverse all unmarked vertices adjacent to v (v will be a different vertex with every new method call). When we visit a vertex in which all of its neighbors have been visited, we return to its calling vertex, and visit one of its unvisited neighbors, repeating the recursion in the same manner. We continue until we have visited all of the starting vertex's neighbors, which means that we're done. The recursion (stack) guides us through the graph.

```
public void depthFirstSearch(Vertex v)
{
        v.visited = true;

        // print the node

        for(each vertex w adjacent to v)
        {
           if(!w.visited)
           {
              depthFirstSearch(w);
           }
        }
}
```

Here is some working C code for a DFS on a BST..

```
#include <stdio.h>

typedef struct node
{
  int value;
  int visited;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

mynode *add_node(int value);
void treeDFS(mynode *root);
```

```
int main(int argc, char* argv[])
{
  root = NULL;

  // Construct the tree..
  add_node(5);
  add_node(1);
  add_node(-20);
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);

  // Do a DFS..
  printf("\n\nDFS : ");
  treeDFS(root);

  getch();
}

// Function to add a new node to the tree...
mynode *add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->visited = 0;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;

   return(temp);

}



// DFS..
void treeDFS(mynode *root)
{
    printf("[%d] ", root->value);
    root->visited = 1;

    if (root->left)
    {
      if(root->left->visited==0)
      {
        treeDFS(root->left);
      }
    }

    if (root->right)
    {
      if(root->right->visited==0)
      {
        treeDFS(root->right);
      }
```

```
    }
}
```

Breadth First Search

Breadth First Search (BFS) searches the graph one level (one edge away from the starting vertex) at a time. In this respect, it is very similar to the level order traversal that we discussed for trees. Starting at some arbitrarily chosen vertex v, we mark v so that we know we've visited it, process v, and then visit and process all of v's neighbors. Now that we've visited and processed all of v's neighbors, we need to visit and process all of v's neighbors neighbors. So we go to the first neighbor we visited and visit all of its neighbors, then the second neighbor we visited, and so on. We continue this process until we've visited all vertices in the graph. We don't use recursion in a BFS because we don't want to traverse recursively. We want to traverse one level at a time. So imagine that you visit a vertex v, and then you visit all of v's neighbors w. Now you need to visit each w's neighbors. How are you going to remember all of your w's so that you can go back and visit their neighbors? You're already marked and processed all of the w's. How are you going to find each w's neighbors if you don't remember where the w's are? After all, you're not using recursion, so there's no stack to keep track of them. To perform a BFS, we use a queue. Every time we visit vertex w's neighbors, we dequeue w and enqueue w's neighbors. In this way, we can keep track of which neighbors belong to which vertex. This is the same technique that we saw for the level-order traversal of a tree. The only new trick is that we need to makr the verticies, so we don't visit them more than once -- and this isn't even new, since this technique was used for the blobs problem during our discussion of recursion.

```
public void breadthFirstSearch(vertex v)
{
   Queue q = new Queue();

   v.visited = true;
   q.enQueue(v);

   while( !q.isEmpty() )
   {
       Vertex w = (Vertex)q.deQueue();

       // Print the node.

       for(each vertex x adjacent to w)
       {
           if( !x.visited )
           {
              x.visited = true;
              q.enQueue(x);
           }
        }
    }
}
```

BFS traversal can be used to produce a tree from a graph.

Here is some C code which does a BFS (level order traversal) on a BST...

```
#include <stdio.h>

typedef struct node
{
  int value;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

add_node(int value);

void levelOrderTraversal(mynode *root);

int main(int argc, char* argv[])
{
  root = NULL;

  add_node(5);
  add_node(1);
```

```
  add_node(-20);
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);


  printf("\n\n\nLEVEL ORDER TRAVERSAL\n\n");
  levelOrderTraversal(root);

  getch();
}



// Function to add a new node...
add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;
}



// Level order traversal..
void levelOrderTraversal(mynode *root)
{
  mynode *queue[100] = {(mynode *)0}; // Important to initialize!
  int size = 0;
  int queue_pointer = 0;

  while(root)
  {
      printf("[%d] ", root->value);

      if(root->left)
      {
        queue[size++] = root->left;
      }

      if(root->right)
      {
        queue[size++] = root->right;
      }

      root = queue[queue_pointer++];
  }
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write pseudocode to add a new node to a Binary Search Tree (BST)

Here is a C code to construct a BST right from scratch...

```
#include <stdio.h>

typedef struct node
{
  int value;
  struct node *right;
  struct node *left;
}mynode;

mynode *root;

add_node(int value);
void postorder(mynode *root);
void inorder(mynode *root);
void preorder(mynode *root);

int main(int argc, char* argv[])
{
  root = NULL;

  add_node(5);
  add_node(1);
  add_node(-20);
  add_node(100);
  add_node(23);
  add_node(67);
  add_node(13);

  printf("\nPreorder    : ");
  preorder(root);
  printf("\n\nPostorder : ");
  postorder(root);
  printf("\n\nInorder   : ");
  inorder(root);

  return(0);
}

// Function to add a new node...
add_node(int value)
{
   mynode *prev, *cur, *temp;

   temp        = (mynode *) malloc(sizeof(mynode));
   temp->value = value;
```

```
   temp->right = NULL;
   temp->left  = NULL;

   if(root==NULL)
   {
     printf("\nCreating the root..\n");
     root = temp;
     return;
   }

   prev=NULL;
   cur=root;

   while(cur!=NULL)
   {
      prev=cur;
      cur=(value<cur->value)?cur->left:cur->right;
   }

   if(value < prev->value)
     prev->left=temp;
   else
     prev->right=temp;
}



void preorder(mynode *root)
{
  if(root)
  {
    printf("[%d] ", root->value);
    preorder(root->left);
    preorder(root->right);
  }
}

void postorder(mynode *root)
{
  if(root)
  {
    postorder(root->left);
    postorder(root->right);
    printf("[%d] ", root->value);
  }
}



void inorder(mynode *root)
{
  if(root)
  {
    inorder(root->left);
    printf("[%d] ", root->value);
    inorder(root->right);
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Trees
(115 / 301)

What is a threaded binary tree?

Since traversing the three is the most frequent operation, a method must be devised to improve the speed. This is where Threaded tree comes into picture. If the right link of a node in a tree is NULL, it can be replaced by the address of its inorder successor. An extra field called the rthread is used. If rthread is equal to 1, then it means that the right link of the node points to the inorder success. If its equal to 0, then the right link represents an ordinary link connecting the right subtree.

```
struct node
{
  int value;
  struct node *left;
  struct node *right;
  int rthread;
}
```

Function to find the inorder successor

```
mynode *inorder_successor(mynode *x)
{
  mynode *temp;

  temp = x->right;

  if(x->rthread==1)return(temp);

  while(temp->left!=NULL)temp = temp->left;

  return(temp);
}
```

Function to traverse the threaded tree in inorder

```
void inorder(mynode *head)
{
   mynode *temp;

   if(head->left==head)
   {
      printf("\nTree is empty!\n");
      return;
   }

   temp = head;

   for(;;)
   {
```

```
       temp = inorder_successor(temp);
       if(temp==head)return;
       printf("%d ", temp->value);
   }

}
```

Inserting toward the left of a node in a threaded binary tree.

```
void insert(int item, mynode *x)
{
   mynode *temp;
   temp = getnode();
   temp->value = item;
   x->left = temp;
   temp->left=NULL;
   temp->right=x;
   temp->rthread=1;
}
```

Function to insert towards the right of a node in a threaded binary tree.

```
void insert_right(int item, mynode *x)
{
   mynode *temp, r;

   temp=getnode();
   temp->info=item;
   r=x->right;
   x->right=temp;
   x->rthread=0;
   temp->left=NULL;
   temp->right=r;
   temp->rthread=1;
}
```

Function to find the inorder predecessor (for a left threaded binary three)

```
mynode *inorder_predecessor(mynode *x)
{
     mynode *temp;

     temp = x->left;

     if(x->lthread==1)return(temp);

     while(temp->right!=NULL)
       temp=temp->right;

     return(temp);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Write a C program to count bits set in an integer?

This is one of the most frequently asked interview questions of all times...

There are a number of ways to count the number of bits set in an integer. Here are some C programs to do the same.

Method1

This is the most basic way of doing it.

```
#include<stdio.h>
int main()
{
  unsinged int num=10;
  int ctr=0;

  for(;num!=0;num>>=1)
  {
    if(num&1)
    {
      ctr++;
    }
  }

  printf("\n Number of bits set in [%d] = [%d]\n", num, ctr);
  return(0);

}
```

Method2

This is a faster way of doing the same thing. Here the control goes into the while loop only as many times as the number of bits set to 1 in the integer!.

```
#include<stdio.h>
int main()
{
  int num=10;
  int ctr=0;

  while(num)
  {
    ctr++;
    num = num & (num - 1); // This clears the least significant bit set.
  }
```

```
  printf("\n Number of bits set in [%d] = [%d]\n", num, ctr);
  return(0);

}
```

Method3

This method is very popular because it uses a lookup table. This speeds up the computation. What it does is it keeps a table which hardcodes the number of bits set in each integer from 0 to 256.

For example

```
0 - 0 Bit(s) set.
1 - 1 Bit(s) set.
2 - 1 Bit(s) set.
3 - 2 Bit(s) set.
...
```

So here is the code...

```
const unsigned char LookupTable[]     = {  0, 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4,
                                           1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
                                           1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
                                           1, 2, 2, 3, 2, 3, 3, 4, 2, 3, 3, 4, 3, 4, 4, 5,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
                                           2, 3, 3, 4, 3, 4, 4, 5, 3, 4, 4, 5, 4, 5, 5, 6,
                                           3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
                                           3, 4, 4, 5, 4, 5, 5, 6, 4, 5, 5, 6, 5, 6, 6, 7,
                                           4, 5, 5, 6, 5, 6, 6, 7, 5, 6, 6, 7, 6, 7, 7, 8};


unsigned int num;
unsigned int ctr;   // Number of bits set.

ctr = LookupTable[num & 0xff] +
      LookupTable[(num >> 8) & 0xff] +
      LookupTable[(num >> 16) & 0xff] +
      LoopupTable[num >> 24];
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Bit Fiddling
(117 / 301)

What purpose do the bitwise and, or, xor and the shift operators serve?

The AND operator

```
Truth Table
-----------
0 AND 0 = 0
0 AND 1 = 0
1 AND 0 = 0
1 AND 1 = 1

x AND 0 = 0
x AND 1 = x
```

We use bitwise "and" to test if certain bit(s) are one or not. And'ing a value against a pattern with ones only in the bit positions you are interested in will give zero if none of them are on, nonzero if one or more is on. We can also use bitwise "and" to turn off (set to zero) any desired bit(s). If you "and" a pattern against a variable, bit positions in the pattern that are ones will leave the target bit unchanged, and bit positions in the pattern that are zeros will set the target bit to zero.

The OR operator

```
Truth Table
-----------
0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1

x OR 0 = x
x OR 1 = 1
```

Use bitwise "or" to turn on (set to one) desired bit(s).

The XOR operator

```
0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0

x XOR 0 = x
x XOR 1 = ~x
```

Use bitwise "exclusive or" to flip or reverse the setting of desired bit(s) (make it a one if it was zero or make it zero if it was one).

The >>, <<, >>=, <<= operators

Operators >> and << can be used to shift the bits of an operand to the right or left a desired number of positions. The number of positions to be shifted can be specified as a constant, in a variable or as an expression. Bits shifted out are lost. For left shifts, bit positions vacated by shifting always filled with zeros. For right shifts, bit positions vacated by shifting filled with zeros for unsigned data type and with copy of the highest (sign) bit for signed data type. The right shift operator can be used to achieve quick multiplication by a power of 2. Similarly the right shift operator can be used to do a quick division by power of 2 (unsigned types only). The operators >> and <<, dont change the operand at all. However, the operators >>= and <<= also change the operand after doing the shift operations.

```
x << y  - Gives value x shifted left y bits (Bits positions vacated by shift are filled with zeros).
x <<= y - Shifts variable x left y bits (Bits positions vacated by shift are filled with zeros).

A left shift of n bits multiplies by 2 raise to n.

x >> y  - Gives value x shifted right y bits.
x >>= y - Shifts variable x right y bits.

For the right shift, All bits of operand participate in the shift. For unsigned data type,
bits positions vacated by shift are filled with zeros. For signed data type, bits positions
vacated by shift are filled with the original highest bit (sign bit). Right shifting n bits
divides by 2 raise to n. Shifting signed values may fail because for negative values the result never
gets past -1:

-5 >> 3 is -1 and not 0 like -5/8.
```

Good interview question

A simple C command line utility takes a series of command line options. The options are given to the utility like this : <utility_name> options= [no]option1,[no]options2,[no]option3?... Write C code using bitwise operators to use these flags in the code.

```
//Each option will have a bit reserved in the global_options_bits integer. The global_options_bits
// integer will have a bit set or not set depending on how the option was specified by the user.
// For example, if the user said nooption1, the bit for OPTION1 in global_options_bits
// will be 0. Likewise, if the user specified option3, the bit for OPTION3 in global_options_bits
// will be set to 1.

#define OPTION1 "option1" // For strcmp() with the passed arguments.
#define OPTION1_BITPOS (0x00000001) // Bit reserved for this option.
#define OPTION2 "option2"
#define OPTION2_BITPOS (0x00000002)
#define OPTION3 "option3"
#define OPTION3_BITPOS (0x00000004)

//Required to do the bit operations.
#define ALL_BITPOS (0x0001ffff)

// Assume you have already parsed the command line option and that
// parsed_argument_without_no has option1 or option2 or option3 (depending on what has
// been provided at the command line) and the variable negate_argument says if the
// option was negated or not (i.e, if it was option1 or nooption1)

if (strcmp((char *) parsed_argument_without_no, (char *) OPTION1) == 0)
{
    // Copy the global_options_bits to a temporary variable.
    tmp_action= global_options_bits;

    if (negate_argument)
    {
        // Setting the bit for this particular option to 0 as the option has
        // been negated.
        action_mask= ~(OPTION1_BITPOS) & ALL_BITPOS;
        tmp_action= tmp_action & action_mask;
    }
    else
    {
        //Setting the bit for this particular option to 1.
        action_mask= (OPTION1_BITPOS);
        tmp_action= tmp_action | action_mask;
    }
```

```
    // Copy back the final bits to the global_options_bits variable
    global_options_bits= tmp_action;
}
else if (strcmp((char *) parsed_argument_without_no, (char *) OPTION2) == 0)
{
    //Similar code for OPTION2
}
else if (strcmp((char *) parsed_argument_without_no, (char *) OPTION3) == 0)
{
    //Similar code for OPTION3
}


//Now someone who wishes to check if a particular option was set or not can use the
// following type of code anywhere else in the code.
if(((global_options_bits & OPTION1_BITPOS) == OPTION1_BITPOS)
{
    //Do processing for the case where OPTION1 was active.
}
else
{
    //Do processing for the case where OPTION1 was NOT active.
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to reverse the bits in an interger?

Here are some ways to reverse the bits in an integer.

Method1

```
unsigned int num;            // Reverse the bits in this number.
unsigned int temp = num;     // temp will have the reversed bits of num.

int i;

for (i = (sizeof(num)*8-1); i; i--)
{
 temp = temp | (num & 1);
 temp <<= 1;
 num  >>= 1;
}

temp = temp | (num & 1);
```

Method2

In this method, we use a lookup table.

```
const unsigned char ReverseLookupTable[] =
{
     0x00, 0x80, 0x40, 0xC0, 0x20, 0xA0, 0x60, 0xE0, 0x10, 0x90, 0x50, 0xD0, 0x30, 0xB0, 0x70, 0xF0,
     0x08, 0x88, 0x48, 0xC8, 0x28, 0xA8, 0x68, 0xE8, 0x18, 0x98, 0x58, 0xD8, 0x38, 0xB8, 0x78, 0xF8,
     0x04, 0x84, 0x44, 0xC4, 0x24, 0xA4, 0x64, 0xE4, 0x14, 0x94, 0x54, 0xD4, 0x34, 0xB4, 0x74, 0xF4,
     0x0C, 0x8C, 0x4C, 0xCC, 0x2C, 0xAC, 0x6C, 0xEC, 0x1C, 0x9C, 0x5C, 0xDC, 0x3C, 0xBC, 0x7C, 0xFC,
     0x02, 0x82, 0x42, 0xC2, 0x22, 0xA2, 0x62, 0xE2, 0x12, 0x92, 0x52, 0xD2, 0x32, 0xB2, 0x72, 0xF2,
     0x0A, 0x8A, 0x4A, 0xCA, 0x2A, 0xAA, 0x6A, 0xEA, 0x1A, 0x9A, 0x5A, 0xDA, 0x3A, 0xBA, 0x7A, 0xFA,
     0x06, 0x86, 0x46, 0xC6, 0x26, 0xA6, 0x66, 0xE6, 0x16, 0x96, 0x56, 0xD6, 0x36, 0xB6, 0x76, 0xF6,
     0x0E, 0x8E, 0x4E, 0xCE, 0x2E, 0xAE, 0x6E, 0xEE, 0x1E, 0x9E, 0x5E, 0xDE, 0x3E, 0xBE, 0x7E, 0xFE,
     0x01, 0x81, 0x41, 0xC1, 0x21, 0xA1, 0x61, 0xE1, 0x11, 0x91, 0x51, 0xD1, 0x31, 0xB1, 0x71, 0xF1,
     0x09, 0x89, 0x49, 0xC9, 0x29, 0xA9, 0x69, 0xE9, 0x19, 0x99, 0x59, 0xD9, 0x39, 0xB9, 0x79, 0xF9,
     0x05, 0x85, 0x45, 0xC5, 0x25, 0xA5, 0x65, 0xE5, 0x15, 0x95, 0x55, 0xD5, 0x35, 0xB5, 0x75, 0xF5,
     0x0D, 0x8D, 0x4D, 0xCD, 0x2D, 0xAD, 0x6D, 0xED, 0x1D, 0x9D, 0x5D, 0xDD, 0x3D, 0xBD, 0x7D, 0xFD,
     0x03, 0x83, 0x43, 0xC3, 0x23, 0xA3, 0x63, 0xE3, 0x13, 0x93, 0x53, 0xD3, 0x33, 0xB3, 0x73, 0xF3,
     0x0B, 0x8B, 0x4B, 0xCB, 0x2B, 0xAB, 0x6B, 0xEB, 0x1B, 0x9B, 0x5B, 0xDB, 0x3B, 0xBB, 0x7B, 0xFB,
     0x07, 0x87, 0x47, 0xC7, 0x27, 0xA7, 0x67, 0xE7, 0x17, 0x97, 0x57, 0xD7, 0x37, 0xB7, 0x77, 0xF7,
     0x0F, 0x8F, 0x4F, 0xCF, 0x2F, 0xAF, 0x6F, 0xEF, 0x1F, 0x9F, 0x5F, 0xDF, 0x3F, 0xBF, 0x7F, 0xFF
};

unsigned int num; // Reverse the bits in this number.
unsigned int temp; // Store the reversed result in this.

temp = (ReverseLookupTable[num & 0xff] << 24) |
```

```
    (ReverseLookupTable[(num >> 8) & 0xff] << 16) |
    (ReverseLookupTable[(num >> 16) & 0xff] << 8) |
    (ReverseLookupTable[(num >> 24) & 0xff]);
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Check if the 20th bit of a 32 bit integer is on or off?

AND it with x00001000 and check if its equal to x00001000

```
if((num & x00001000)==x00001000)
```

Note that the digits represented here are in hex.

```
    0      0      0      0      1      0      0      0
                               ^
                               |

x0000   0000   0000   0000   0001   0000   0000   0000 = 32 bits

 ^                             ^                    ^
 |                             |                    |

 0th bit                       20th bit             32nd bit
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Bit Fiddling
(120 / 301) *

How to reverse the odd bits of an integer?

XOR each of its 4 hex parts with 0101.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Bit Fiddling
(121 / 301)

How would you count the number of bits set in a floating point number?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is heap sort?

A Heap is an almost complete binary tree.In this tree, if the maximum level is i, then, upto the (i-1)th level should be complete. At level i, the number of nodes can be less than or equal to 2^i. If the number of nodes is less than 2^i, then the nodes in that level should be completely filled, only from left to right.

The property of an ascending heap is that, the root is the lowest and given any other node i, that node should be less than its left child and its right child. In a descending heap, the root should be the highest and given any other node i, that node should be greater than its left child and right child.

To sort the elements, one should create the heap first. Once the heap is created, the root has the highest value. Now we need to sort the elements in ascending order. The root can not be exchanged with the nth element so that the item in the nth position is sorted. Now, sort the remaining (n-1) elements. This can be achieved by reconstructing the heap for (n-1) elements.

A highly simplified pseudocode is given below

```
heapsort()
{
   n = array();   // Convert the tree into an array.
   makeheap(n);   // Construct the initial heap.

   for(i=n; i>=2; i--)
   {
      swap(s[1],s[i]);
      heapsize--;
      keepheap(i);
   }
}

makeheap(n)
{
   heapsize=n;
   for(i=n/2; i>=1; i--)
     keepheap(i);
}

keepheap(i)
{
   l = 2*i;
   r = 2*i + 1;

   p = s[l];
   q = s[r];
   t = s[i];

   if(l<=heapsize && p->value > t->value)
     largest = l;
   else
     largest = i;
```

```
   m = s[largest];

   if(r<=heapsize && q->value > m->value)
     largest = r;

   if(largest != i)
   {
      swap(s[i], s[largest]);
      keepheap(largest);
   }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is the difference between Merge Sort and Quick sort?

Both Merge-sort and Quick-sort have same time complexity i.e. O(nlogn). In merge sort the file a[1:n] was divided at its midpoint into sub-arrays which are independently sorted and later merged. Whereas, in quick sort the division into two sub-arrays is made so that the sorted sub-arrays do not need to be merged latter.

Sorting Techniques
(124 / 301)

Give pseudocode for the mergesort algorithm

```
Mergesort(a, left, right)
{
   if(left<right)
   {
     I=(left+right)/2;
     Mergesort(a,left, I);
     Mergesort(a,I+1,right);
     Merge(a,b,left,I,right);
     Copy(b,a,left,right);
   }
}
```

The merge would be something liks this

```
merge(int a[], int b[], int c[], int m, int n)
{
  int i, j, k;
  i = 0;
  j = 0;
  k = 0;

  while(i<=m && j<=n)
  {
    if(a[i] < a[j])
    {
        c[k]=a[i];
        i=i+1;
    }
    else
    {
        c[k]=a[j];
        j=j+1;
    }
    k=k+1;
  }

  while(i<=m)
    c[k++]=a[i++];

  while(j<=n)
    c[k++]=a[j++];
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Sorting Techniques
(125 / 301)

Implement the bubble sort algorithm. How can it be improved? Write the code for selection sort, quick sort, insertion sort.

Here is the Bubble sort algorithm

```
void bubble_sort(int a[], int n)
{
  int i, j, temp;

  for(j = 1; j < n; j++)
  {
     for(i = 0; i < (n - j); i++)
     {
        if(a[i] >= a[i + 1])
        {
           //Swap a[i], a[i+1]
        }
     }
  }
}
```

To improvise this basic algorithm, keep track of whether a particular pass results in any swap or not. If not, you can break out without wasting more cycles.

```
void bubble_sort(int a[], int n)
{
  int i, j, temp;
  int flag;

  for(j = 1; j < n; j++)
  {
     flag = 0;
     for(i = 0; i < (n - j); i++)
     {
        if(a[i] >= a[i + 1])
        {
           //Swap a[i], a[i+1]
           flag = 1;
        }
     }

     if(flag==0)break;
  }
}
```

This is the selection sort algorithm

```
void selection_sort(int a[], int n)
{
   int i, j, small, pos, temp;

   for(i = 0; i < (n - 1); i++)
   {
      small = a[i];
      pos   = i;

      for(j = i + 1; j < n; j++)
      {
         if(a[j] < small)
         {
            small = a[j];
            pos   = j;
         }
      }

      temp   = a[pos];
      a[pos] = a[i];
      a[i]   = temp;
   }
}
```

Here is the Quick sort algorithm

```
int partition(int a[], int low, int high)
{
   int i, j, temp, key;

   key = a[low];
   i   = low + 1;
   j   = high;

   while(1)
   {
       while(i < high && key >= a[i])i++;

       while(key < a[j])j--;

       if(i < j)
       {
           temp = a[i];
           a[i] = a[j];
           a[j] = temp;
       }
       else
       {
           temp   = a[low];
           a[low] = a[j];
           a[j]   = temp;
           return(j);
       }
   }
}

void quicksort(int a[], int low, int high)
{
   int j;

   if(low < high)
   {
       j = partition(a, low, high);
       quicksort(a, low, j - 1);
       quicksort(a, j + 1, high);
   }
}

int main()
{
```

```
  // Populate the array a
  quicksort(a, 0, n - 1);
}
```

Here is the Insertion sort algorithm

```
void insertion_sort(int a[], int n)
{
   int i, j, item;

   for(i = 0; i < n; i++)
   {
       item = a[i];
       j    = i - 1;

       while(j >=0 && item < a[j])
       {
          a[j + 1] = a[j];
          j--;
       }

       a[j + 1] = item;
   }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Sorting Techniques
(126 / 301) *

How can I sort things that are too large to bring into memory?

A sorting program that sorts items that are on secondary storage (disk or tape) rather than primary storage (memory) is called an external sort. Exactly how to sort large data depends on what is meant by ?too large to fit in memory.? If the items to be sorted are themselves too large to fit in memory (such as images), but there aren?t many items, you can keep in memory only the sort key and a value indicating the data?s location on disk. After the key/value pairs are sorted, the data is rearranged on disk into the correct order. If ?too large to fit in memory? means that there are too many items to fit into memory at one time, the data can be sorted in groups that will fit into memory, and then the resulting files can be merged. A sort such as a radix sort can also be used as an external sort, by making each bucket in the sort a file. Even the quick sort can be an external sort. The data can be partitioned by writing it to two smaller files. When the partitions are small enough to fit, they are sorted in memory and concatenated to form the sorted file.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(127 / 301)

What does *p++ do? Does it increment p or the value pointed by p?

The postfix "++" operator has higher precedence than prefix "*" operator. Thus, *p++ is same as *(p++); it increments the pointer p, and returns the value which p pointed to before p was incremented. If you want to increment the value pointed to by p, try (*p)++.

C Pointers
(128 / 301)

What is a NULL pointer? How is it different from an unitialized pointer? How is a NULL pointer defined?

A null pointer simply means "I am not allocated yet!" and "I am not pointing to anything yet!".

The C language definition states that for every available pointer type, there is a special value which is called the null pointer. It is guaranteed to compare unequal to a pointer to any object or function.

A null pointer is very different from an uninitialized pointer. A null pointer does not point to any object or function; but an uninitialized pointer can point anywhere.

There is usually a null pointer for each type of a pointer, and the internal values of these null pointers for different pointer types may be different, its up to the compiler. The & operator will never yield a null pointer, nor will a successful call to malloc() (malloc() does return a null pointer when it fails).

```
execl("/bin/ls", "ls", "-l", (char *)0);
```

In this call to execl(), the last argument has been explicitly casted to force the 0 to be treated as a pointer.

Also, if ptr is a pointer then

```
if(ptr){}
```

and

```
if(!ptr){}
```

are perfectly valid.

How is NULL defined?, you may ask.

ANSI C allows the following definition

```
#define NULL ((void *)0)
```

NULL and 0 are interchangeable in pointer contexts.

Make sure you are able to distinguish between the following : the null pointer, the internal representation of a null pointer, the null pointer constant (i.e, 0), the NULL macro, the ASCII null character (NUL), the null string ("").

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(129 / 301)

What is a null pointer assignment error?

This error means that the program has written, through a null (probably because its an uninitialized) pointer, to a location thats invalid.

More to come....

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Does an array always get converted to a pointer? What is the difference between arr and &arr? How does one declare a pointer to an entire array?

Well, not always.

In C, the array and pointer arithmetic is such that a pointer can be used to access an array or to simulate an array. What this means is whenever an array appears in an expression, the compiler automatically generates a pointer to the array's first element (i.e, `&a[0]`).

There are three exceptions to this rule

```
1. When the array is the operand of the sizeof() operator.
2. When using the & operator.
3. When the array is a string literal initializer for a character array.
```

Also, on a side note, the rule by which arrays decay into pointers is not applied recursively!. An array of arrays (i.e. a two-dimensional array in C) decays into a pointer to an array, not a pointer to a pointer.

If you are passing a two-dimensional array to a function:

```
int myarray[NO_OF_ROWS][NO_OF_COLUMNS];
myfunc(myarray);
```

then, the function's declaration must match:

```
void myfunc(int myarray[][NO_OF_COLUMNS])
```

or

```
void myfunc(int (*myarray)[NO_OF_COLUMNS])
```

Since the called function does not allocate space for the array, it does not need to know the overall size, so the number of rows, NO_OF_ROWS, can be omitted. The width of the array is still important, so the column dimension NO_OF_COLUMNS must be present.

An array is never passed to a function, but a pointer to the first element of the array is passed to the function. Arrays are automatically allocated memory. They can't be relocated or resized later. Pointers must be assigned to allocated memory (by using (say) malloc), but pointers can be reassigned and made to point to other memory chunks.

So, whats the difference between `func(arr)` and `func(&arr)`?

In C, `&arr` yields a pointer to the entire array. On the other hand, a simple reference to `arr` returns a pointer to the first element of the array `arr`. Pointers to arrays (as in `&arr`) when subscripted or incremented, step over entire arrays, and are useful only when operating on arrays of arrays. Declaring a pointer to an entire array can be done like `int (*arr)[N];`, where N is the size of the array.

Also, note that sizeof() will not report the size of an array when the array is a parameter to a function, simply because the compiler pretends that the array parameter was declared as a
pointer and sizeof reports the size of the pointer.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Is the cast to malloc() required at all?

Before ANSI C introduced the void * generic pointer, these casts were required because older compilers used to return a char pointer.

```
int *myarray;
myarray = (int *)malloc(no_of_elements * sizeof(int));
```

But, under ANSI Standard C, these casts are no longer necessary as a void pointer can be assigned to any pointer. These casts are still required with C++, however.

What does malloc() , calloc(), realloc(), free() do? What are the common problems with malloc()? Is there a way to find out how much memory a pointer was allocated?

malloc() is used to allocate memory. Its a memory manager.

calloc(m, n) is also used to allocate memory, just like malloc(). But in addition, it also zero fills the allocated memory area. The zero fill is all-bits-zero. calloc(m.n) is essentially equivalent to

```
p = malloc(m * n);
memset(p, 0, m * n);
```

The malloc() function allocates raw memory given a size in bytes. On the other hand, calloc() clears the requested memory to zeros before return a pointer to it. (It can also compute the request size given the size of the base data structure and the number of them desired.)

The most common source of problems with malloc() are

```
1. Writing more data to a malloc'ed region than it was allocated to hold.
2. malloc(strlen(string)) instead of (strlen(string) + 1).
3. Using pointers to memory that has been freed.
4. Freeing pointers twice.
5. Freeing pointers not obtained from malloc.
6. Trying to realloc a null pointer.
```

How does free() work?

Any memory allocated using malloc() realloc() must be freed using free(). In general, for every call to malloc(), there should be a corresponding call to free(). When you call free(), the memory pointed to by the passed pointer is freed. However, the value of the pointer in the caller remains unchanged. Its a good practice to set the pointer to NULL after freeing it to prevent accidental usage. The malloc()/free() implementation keeps track of the size of each block as it is allocated, so it is not required to remind it of the size when freeing it using free(). You can't use dynamically-allocated memory after you free it.

Is there a way to know how big an allocated block is?

Unfortunately there is no standard or portable way to know how big an allocated block is using the pointer to the block!. God knows why this was left out in C.

Is this a valid expression?

```
pointer = realloc(0, sizeof(int));
```

Yes, it is!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(133 / 301)

What's the difference between const char *p, char * const p and const char * const p?

```
const char *p     -   This is a pointer to a constant char. One cannot change the value
                      pointed at by p, but can change the pointer p itself.

                      *p = 'A' is illegal.
                      p  = "Hello" is legal.

                      Note that even char const *p is the same!

const * char p    -   This is a constant pointer to (non-const) char. One cannot change
                      the pointer p, but can change the value pointed at by p.

                      *p = 'A' is legal.
                      p  = "Hello" is illegal.


const char * const p  -   This is a constant pointer to constant char! One cannot
                          change the value pointed to by p nor the pointer.

                          *p = 'A' is illegal.
                          p  = "Hello" is also illegal.
```

To interpret these declarations, let us first consider the general form of declaration:

```
  [qualifier] [storage-class] type [*[*]..] [qualifier] ident ;

                                or

  [storage-class] [qualifier] type [*[*]..] [qualifier] ident ;
```

where,

```
qualifier:
            volatile
            const

storage-class:

            auto            extern
```

```
            static          register

type:
            void            char            short
            int             long            float
            double          signed          unsigned
            enum-specifier
            typedef-name
            struct-or-union-specifier
```

Both the forms are equivalent. Keywords in the brackets are optional. The simplest tip here is to notice the relative position of the `const' keyword with respect to the asterisk (*).

Note the following points:

- If the `const' keyword is to the left of the asterisk, and is the only such keyword in the declaration, then object pointed by the pointer is constant, however, the pointer itself is variable. For example:

  ```
  const char * pcc;
  char const * pcc;
  ```

- If the `const' keyword is to the right of the asterisk, and is the only such keyword in the declaration, then the object pointed by the pointer is variable, but the pointer is constant; i.e., the pointer, once initialized, will always point to the same object through out it's scope. For example:

  ```
  char * const cpc;
  ```

- If the `const' keyword is on both sides of the asterisk, the both the pointer and the pointed object are constant. For example:

  ```
  const char * const cpcc;
  char const * const cpcc2;
  ```

One can also follow the "nearness" principle; i.e.,

- If the `const' keyword is nearest to the `type', then the object is constant. For example:

  ```
  char const * pcc;
  ```

- If the `const' keyword is nearest to the identifier, then the pointer is constant. For example:

  ```
  char * const cpc;
  ```

- If the `const' keyword is nearest, both to the identifier and the type, then both the pointer and the object are constant. For example:

  ```
  const char * const cpcc;
  char const * const cpcc2;
  ```

However, the first method seems more reliable...

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(134 / 301)

What is a void pointer? Why can't we perform arithmetic on a void * pointer?

The void data type is used when no other data type is appropriate. A void pointer is a pointer that may point to any kind of object at all. It is used when a pointer must be specified but its type is unknown.

The compiler doesn't know the size of the pointed-to objects incase of a void * pointer. Before performing arithmetic, convert the pointer either to char * or to the pointer type you're trying to manipulate

What do Segmentation fault, access violation, core dump and Bus error mean?

The segmentation fault, core dump, bus error kind of errors usually mean that the program tried to access memory it shouldn't have.

Probable causes are overflow of local arrays; improper use of null pointers; corruption of the malloc() data structures; mismatched function arguments (specially variable argument functions like sprintf(), fprintf(), scanf(), printf()).

For example, the following code is a sure shot way of inviting a segmentation fault in your program:

```
sprintf(buffer,
        "%s %d",
        "Hello");
```

So whats the difference between a bus error and a segmentation fault?

A bus error is a fatal failure in the execution of a machine language instruction resulting from the processor detecting an anomalous condition on its bus.

Such conditions include:

```
- Invalid address alignment (accessing a multi-byte number at an odd address).
- Accessing a memory location outside its address space.
- Accessing a physical address that does not correspond to any device.
- Out-of-bounds array references.
- References through uninitialized or mangled pointers.
```

A bus error triggers a processor-level exception, which Unix translates into a "SIGBUS" signal,which if not caught, will terminate the current process. It looks like a SIGSEGV, but the difference between the two is that SIGSEGV indicates an invalid access to valid memory, while SIGBUS indicates an access to an invalid address.

Bus errors mean different thing on different machines. On systems such as Sparcs a bus error occurs when you access memory that is not positioned correctly.

Maybe an example will help to understand how a bus error occurs

```
#include < stdlib.h>
#include < stdio.h>

int main(void)
{
  char *c;
  long int *i;
  c = (char *) malloc(sizeof(char));
  c++;
  i = (long int *)c;
  printf("%ld", *i);
  return 0;
```

```
}
```

On Sparc machines long ints have to be at addresses that are multiples of four (because they are four bytes long), while chars do not (they are only one byte long so they can be put anywhere). The example code uses the char to create an invalid address, and assigns the long int to the invalid address. This causes a bus error when the long int is dereferenced.

A segfault occurs when a process tries to access memory that it is not allowed to, such as the memory at address 0 (where NULL usually points). It is easy to get a segfault, such as the following example, which dereferences NULL.

```
#include < stdio.h>

int main(void)
{
  char *p;
  p = NULL;
  putchar(*p);
  return 0;
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(136 / 301)

What is the difference between an array of pointers and a pointer to an array?

This is an array of pointers

```
int *p[10];
```

This is a pointer to a 10 element array

```
int (*p)[10];
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is a memory leak?

Its an scenario where the program has lost a reference to an area in the memory. Its a programming term describing the loss of memory. This happens when the program allocates some memory but fails to return it to the system

C Pointers
(138 / 301) *

What are brk() and sbrk() used for? How are they different from malloc()?

brk() and sbrk() are the only calls of memory management in UNIX. For one value of the address, beyond the last logical data page of the process, the MMU generates a segmentation violation interrupt and UNIX kills the process. This address is known as the break address of a process. Addition of a logical page to the data space implies raising of the break address (by a multiple of a page size). Removal of an entry from the page translation table automatically lowers the break address.

brk()and sbrk() are systems calls for this process

```
char *brk(char *new_break);
char *sbrk(displacement)
```

Both calls return the old break address to the process. In brk(), the new break address desired needs to be specified as the parameter. In sbrk(), the displacement (+ve or -ve) is the difference between the new and the old break address. sbrk() is very similar to malloc() when it allocates memory (+ve displacement).

malloc() is really a memory manager and not a memory allocator since, brk/sbrk only can do memory allocations under UNIX. malloc () keeps track of occupied and free peices of memory. Each malloc request is expected to give consecutive bytes and hence malloc selects the smallest free pieces that satisfy a request. When free is called, any consecutive free pieces are coalesced into a large free piece. These is done to avoid fragmentation.

realloc() can be used only with a preallocated/malloced/realloced memory. realloc() will automatically allocate new memory and transfer maximum possible contents if the new space is not available. Hence the returned value of realloc must always be stored back into the old pointer itself.

C Pointers
(139 / 301)

What is a dangling pointer? What are reference counters with respect to pointers?

A pointer which points to an object that no longer exists. Its a pointer referring to an area of memory that has been deallocated. Dereferencing such a pointer usually produces garbage.

Using reference counters which keep track of how many pointers are pointing to this memory location can prevent such issues. The reference counts are incremented when a new pointer starts to point to the memory location and decremented when they no longer need to point to that memory. When the reference count reaches zero, the memory can be safely freed. Also, once freed, the corresponding pointer must be set to NULL.

C Pointers
(140 / 301)

What do pointers contain?

Since addresses are always whole numbers, pointers always contain whole numbers.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(141 / 301)

Is *(*(p+i)+j) is equivalent to p[i][j]? Is num[i] == i[num] == *(num + i) == *(i + num)?

Yes

`*(*(p+i)+j) == p[i][j].`

So is

```
num[i] == i[num] == *(num + i) == *(i + num)
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Pointers
(142 / 301)

What operations are valid on pointers? When does one get the Illegal use of pointer in function error?

This is what is Valid

```
px<py
px>=py
px==py
px!=py
px==NULL
px=px+n
px=px-n
px-py
```

Everything else is invalid (multiplication, division, addition of two pointers)!

Something like

```
j = j * 2;
k = k / 2;
```

where j and k are pointers will give this error

```
Illegal use of pointer in function main
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What are near, far and huge pointers?

While working under DOS only 1 mb of memory is accessible. Any of these memory locations are accessed using CPU registers. Under DOS, the CPU registers are only 16 bits long. Therefore, the minimum value present in a CPU register could be 0, and maximum 65,535. Then how do we access memory locations beyond 65535th byte? By using two registers (segment and offset) in conjunction. For this the total memory (1 mb) is divided into a number of units each comprising 65,536 (64 kb) locations. Each such unit is called a segment. Each segment always begins at a location number which is exactly divisible by 16. The segment register contains the address where a segment begins, whereas the offset register contains the offset of the data/code from where the segment begins. For example, let us consider the first byte in B block of video memory. The segment address of video memory is B0000h (20-bit address), whereas the offset value of the first byte in the upper 32K block of this segment is 8000h. Since 8000h is a 16-bit address it can be easily placed in the offset register, but how do we store the 20-bit address B0000h in a 16-bit segment register? For this out of B0000h only first four hex digits (16 bits) are stored in segment register. We can afford to do this because a segment address is always a multiple of 16 and hence always contains a 0 as the last digit. Therefore, the first byte in the upper 32K chunk of B block of video memory is referred using segment:offset format as B000h:8000h. Thus, the offset register works relative to segment register. Using both these, we can point to a specific location anywhere in the 1 mb address space.

Suppose we want to write a character `A' at location B000:8000. We must convert this address into a form which C understands. This is done by simply writing the segment and offset addresses side by side to obtain a 32 bit address. In our example this address would be 0xB0008000. Now whether C would support this 32 bit address or not depends upon the memory model in use. For example, if we are using a large data model (compact, large, huge) the above address is acceptable. This is because in these models all pointers to data are 32 bits long. As against this, if we are using a small data model (tiny, small, medium) the above address won't work since in these models each pointer is 16 bits long.

What if we are working in small data model and still want to access the first byte of the upper 32K chunk of B block of video memory? In such cases both Microsoft C and Turbo C provide a keyword called far, which is used as shown below,

char far *s = 0XB0008000;

A far pointer is always treated as 32 bit pointer and contains both a segment address and an offset.

A huge pointer is also 32 bits long, again containing a segment address and an offset. However, there are a few differences between a far pointer and a huge pointer.

A near pointer is only 16 bits long, it uses the contents of CS register (if the pointer is pointing to code) or contents of DS register (if the pointer is pointing to data) for the segment part, whereas the offset part is stored in the 16-bit near pointer. Using near pointer limits your data/code to current 64 kb segment.

A far pointer (32 bit) contains the segment as well as the offset. By using far pointers we can have multiple code segments, which in turn allow you to have programs longer than 64 kb. Likewise, with far data pointers we can address more than 64 kb worth of data. However, while using far pointers some problems may crop up as is illustrated by the following program.

```
main( )
{
  char far *a = OX00000120;
  char far *b = OX00100020;
  char far *c = OX00120000;

  if ( a == b )
```

```
    printf ( "Hello" ) ;

  if ( a == c )
    printf ( "Hi" ) ;

  if ( b == c )
    printf ( "Hello Hi" ) ;

  if ( a > b && a > c && b > c )
    printf ( "Bye" ) ;

}
```

Note that all the 32 bit addresses stored in variables a, b, and c refer to the same memory location. This deduces from the method of obtaining the 20-bit physical address from the segment:offset pair. This is shown below.

```
00000  segment address left shifted by 4 bits
0120   offset address
--------
00120 resultant 20 bit address



00100  segment address left shifted by 4 bits
0020   offset address
--------
00120 resultant 20 bit address



00120  segment address left shifted by 4 bits
0000   offset address
--------
00120 resultant 20 bit address
```

Now if a, b and c refer to same location in memory we expect the first three ifs to be satisfied. However this doesn't happen. This is because while comparing the far pointers using == (and !=) the full 32-bit value is used and since the 32-bit values are different the ifs fail. The last if however gets satisfied, because while comparing using > (and >=, <, <=) only the offset value is used for comparison. And the offset values of a, b and c are such that the last condition is satisfied.

These limitations are overcome if we use huge pointer instead of far pointers. Unlike far pointers huge pointers are `normalized' to avoid these problems. What is a normalized pointer? It is a 32- bit pointer which has as much of its value in the segment address as possible. Since a segment can start every 16 bytes, this means that the offset will only have a value from 0 to F.

How do we normalize a pointer? Simple. Convert it to its 20-bit address then use the the left 16 bits for the segment address and the right 4 bits for the offset address. For example, given the pointer 500D:9407, we convert it to the absolute address 594D7, which we then normalize to 594D:0007.

Huge pointers are always kept normalized. As a result, for any given memory address there is only one possible huge address - segment: offset pair for it. Run the above program using huge instead of far and now you would find that the first three ifs are satisfied, whereas the fourth fails. This is more logical than the result obtained while using far. But then there is a price to be paid for using huge pointers. Huge pointer arithmetic is done with calls to special subroutines. Because of this, huge pointer arithmetic is significantly slower than that of far or near pointers. The information presented is specific to DOS operating system only.

What is the difference between malloc() and calloc()?

First lets look at the prototypes of these two popular functions..

```
#include <stdlib.h>
void *calloc(size_t n, size_t size);
void *malloc(size_t size);
```

The two functions malloc() and calloc() are functionally same in that they both allocate memory from a storage pool (generally called heap). Actually, the right thing to say is that these two functions are memory managers and not memory allocators. Memory allocation is done by OS specific routines (like brk() and sbrk()). But lets not get into that for now...

Here are some differences between these two functions..

- malloc() takes one argument, whereas calloc() takes two.
- calloc() initializes all the bits in the allocated space to zero (this is all-bits-zero!, where as malloc() does not do this.
- A call to calloc() is equivalent to a call to malloc() followed by one to memset().

  ```
  calloc(m, n)

  is essentially equivalent to

  p = malloc(m * n);
  memset(p, 0, m * n);
  ```

  Using calloc(), we can carry out the functionality in a faster way than a combination of malloc() and memset() probably would. You will agree that one libray call is faster than two calls. Additionally, if provided by the native CPU, calloc() could be implementated by the CPU's "allocate-and-initialize-to-zero" instruction.
- The reason for providing the "n" argument is that sometimes it is required to allocate a number ("n") of uniform objects of a particular size ("size"). Database application, for instance, will have such requirements. Proper planning for the values of "n" and "size" can lead to good memory utilization.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Why is sizeof() an operator and not a function?

sizeof() is a compile time operator. To calculate the size of an object, we need the type information. This type information is available only at compile time. At the end of the compilation phase, the resulting object code doesn't have (or not required to have) the type information. Of course, type information can be stored to access it at run-time, but this results in bigger object code and less performance. And most of the time, we don't need it. All the runtime environments that support run time type identification (RTTI) will retain type information even after compilation phase. But, if something can be done in compilation time itself, why do it at run time?

On a side note, something like this is illegal...

```
printf("%u\n", sizeof(main));
```

This asks for the size of the main function, which is actually illegal:

```
6.5.3.4 The sizeof operator
The sizeof operator shall not be applied to an expression that has function type....
```

What is an opaque pointer?

A pointer is said to be opaque if the definition of the type to which it points to is not included in the current translation unit. A translation unit is the result of merging an implementation file with all its headers and header files.

C Pointers
(147 / 301) *

What are the common causes of pointer bugs?

- Uninitialized pointers : One of the easiest ways to create a pointer bug is to try to reference the value of a pointer even though the pointer is uninitialized and does not yet point to a valid address. For example:

```
int *p;
*p = 12;
```

  The pointer p is uninitialized and points to a random location in memory when you declare it. It could be pointing into the system stack, or the global variables, or into the program's code space, or into the operating system. When you say *p=12;, the program will simply try to write a 12 to whatever random location p points to. The program may explode immediately, or may wait half an hour and then explode, or it may subtly corrupt data in another part of your program and you may never realize it. This can make this error very hard to track down. Make sure you initialize all pointers to a valid address before dereferencing them.

- Invalid Pointer References : An invalid pointer reference occurs when a pointer's value is referenced even though the pointer doesn't point to a valid block. One way to create this error is to say p=q;, when q is uninitialized. The pointer p will then become uninitialized as well, and any reference to *p is an invalid pointer reference. The only way to avoid this bug is to draw pictures of each step of the program and make sure that all pointers point somewhere. Invalid pointer references cause a program to crash inexplicably for the same reasons given in cause 1.

- Zero Pointer Reference : A zero pointer reference occurs whenever a pointer pointing to zero is used in a statement that attempts to reference a block. For example, if p is a pointer to an integer, the following code is invalid:

```
p = 0;
*p = 12;
```

  There is no block pointed to by p. Therefore, trying to read or write anything from or to that block is an invalid zero pointer reference. There are good, valid reasons to point a pointer to zero. Dereferencing such a pointer, however, is invalid.

How to declare a pointer to a function?

Use something like this

```
int myfunc(); // The function.
int (*fp)();  // Pointer to that function.

fp = myfunc;  // Assigning the address of the function to the pointer.

(*fp)();      // Calling the function.
fp();         // Another way to call the function.
```

So then, what are function pointers, huh?

In simple words, a function pointer is a pointer variable which holds the address of a function and can be used to invoke that function indirectly. Function pointers are useful when you want to invoke seperate functions based on different scenarios. In that case, you just pass in the pointer to the function you want to be invoked. Function pointers are used extensively when writing code with call back scenarios.

For example, when writing code involving a XML parser, you pass in the pointers to your event handling functions and whenever there is a specific event, the XML parser calls your functions through their pointers. This function whose pointer is passed is generally called as the call back function.

Here is an implementation of a Generic linked list which uses function pointers really well...

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct list {
    void *data;
    struct list *next;
} List;

struct check {
    int i;
    char c;
    double d;
} chk[] = { { 1, 'a', 1.1 },
          { 2, 'b', 2.2 },
          { 3, 'c', 3.3 } };
```

```
// See how the print() function takes in a pointer to a function!
void print(List *, void (*)(void *));




void insert(List **, void *, unsigned int);
void printstr(void *);
void printint(void *);
void printchar(void *);
void printcomp(void *);

List *list1, *list2, *list3, *list4;

int main(void)
{
    char c[]    = { 'a', 'b', 'c', 'd' };
    int i[]     = { 1, 2, 3, 4 };
    char *str[] = { "hello1", "hello2", "hello3", "hello4" };

    list1 = list2 = list3 = list4 = NULL;

    insert(&list1, &c[0], sizeof(char));
    insert(&list1, &c[1], sizeof(char));
    insert(&list1, &c[2], sizeof(char));
    insert(&list1, &c[3], sizeof(char));

    insert(&list2, &i[0], sizeof(int));
    insert(&list2, &i[1], sizeof(int));
    insert(&list2, &i[2], sizeof(int));
    insert(&list2, &i[3], sizeof(int));

    insert(&list3, str[0], strlen(str[0])+1);
    insert(&list3, str[1], strlen(str[0])+1);
    insert(&list3, str[2], strlen(str[0])+1);
    insert(&list3, str[3], strlen(str[0])+1);

    insert(&list4, &chk[0], sizeof chk[0]);
    insert(&list4, &chk[1], sizeof chk[1]);
    insert(&list4, &chk[2], sizeof chk[2]);

    printf("Printing characters:");
    print(list1, printchar);
    printf(" : done\n\n");

    printf("Printing integers:");
    print(list2, printint);
    printf(" : done\n\n");

    printf("Printing strings:");
    print(list3, printstr);
    printf(" : done\n\n");

    printf("Printing composite:");
    print(list4, printcomp);
    printf(" : done\n");

    return 0;
}

void insert(List **p, void *data, unsigned int n)
{
    List *temp;
    int i;

    /* Error check is ignored */
    temp = malloc(sizeof(List));
    temp->data = malloc(n);
    for (i = 0; i < n; i++)
        *(char *)(temp->data + i) = *(char *)(data + i);
    temp->next = *p;
    *p = temp;
}

void print(List *p, void (*f)(void *))
{
    while (p)
    {
```

```
        (*f)(p->data);
        p = p->next;
    }
}

void printstr(void *str)
{
    printf(" \"%s\"", (char *)str);
}

void printint(void *n)
{
    printf(" %d", *(int *)n);
}

void printchar(void *c)
{
    printf(" %c", *(char *)c);
}

void printcomp(void *comp)
{
    struct check temp = *(struct check *)comp;
    printf(" '%d:%c:%f", temp.i, temp.c, temp.d);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(149 / 301) *

Does extern in a function declaration mean anything?

The `extern` in a function's declaration is sometimes used to indicate that the function's definition is in some other source file, but there is no difference between

`extern` int function_name();

and

int function_name();

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(150 / 301)

How can I return multiple values from a function?

You can pass pointers to locations which the function being called can populate, or have the function return a structure containing the desired values, or use global variables.

C Functions
(151 / 301) *

Does C support function overloading?

Function overload is not present in C. In C, either use different names or pass a union of supported types (with additional identifier that give hints of the type to be used).

Most people think its supported because they might unknowingly be using a C++ compiler to compile their C code and C+= does have function overloading.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(152 / 301) *

What is the purpose of a function prototype?

A function prototype tells the compiler to expect a given function to be used in a given way. That is, it tells the compiler the nature of the parameters passed to the function (the quantity, type and order) and the nature of the value returned by the function.

What are inline functions?

Before reading the answer, please be aware that inline functions are non-standard C. They are provided as compiler extensions. But, nevertheless, one should know what they are used for.

The inline comment is a request to the compiler to copy the code into the object at every place the function is called. That is, the function is expanded at each point of call. Most of the advantage of inline functions comes from avoiding the overhead of calling an actual function. Such overhead includes saving registers, setting up stack frames, and so on. But with large functions the overhead becomes less important. Inlining tends to blow up the size of code, because the function is expanded at each point of call.

```
int myfunc(int a)
{
  ...
}
```

```
inline int myfunc(int a)
{
  ...
}
```

Inlined functions are not the fastest, but they are the kind of better than macros (which people use normally to write small functions).

```
#define myfunc(a)   \
{ \
   ...                          \
}
```

The problem with macros is that the code is literally copied into the location it was called from. So if the user passes a "double" instead of an "int" then problems could occur. However, if this senerio happens with an inline function the compiler will complain about incompatible types. This will save you debugging time stage.

Good time to use inline functions is

```
1. There is a time criticle function.
2. That is called often.
3. Its small. Remember that inline functions take more space than normal functions.
```

Some typical reasons why inlining is sometimes not done are:

```
1. The function calls itself, that is, is recursive.
2. The function contains loops such as for(;;) or while().
3. The function size is too large.
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(154 / 301) *

How to declare an array of N pointers to functions returning pointers to functions returning pointers to characters?

Declare it this way

```
char *(*(*a[N])())();
```

Even this seems to be correct

```
(char*(*fun_ptr)())(*func[n])();
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(155 / 301) *

Can we declare a function that can return a pointer to a function of the same type?

We cannot do it directly. Either have the function return a generic function pointer, with casts to adjust the types as the pointers are passed around; or have it return a structure containing only a pointer to a function returning that structure.

C Functions
(156 / 301)

How can I write a function that takes a variable number of arguments? What are the limitations with this? What is vprintf()?

The header stdarg.h provides this functionality. All functions like printf(), scanf() etc use this functionality.

The program below uses a var_arg type of function to count the overall length of strings passed to the function.

```
#include <stdarg.h>

int myfunction(char *first_argument,...)
{
   int length;
   va_list argp;
   va_start(argp, first);
   char *p;

   length = strlen(first_argument);
   while((p = va_arg(argp, char *)) != NULL)
   {
     length = length + strlen(p);
   }

   va_end(argp);
   return(length);
}

int main()
{
  int length;
  length = myfunction("Hello","Hi","Hey!",(char *)NULL);
  return(0);
}
```

How can I find how many arguments a function was passed?

Any function which takes a variable number of arguments must be able to determine from the arguments themselves, how many of them there have been passed. printf() and some similar functions achieve this by looking for the format string. This is also why these functions fail badly if the format string does not match the argument list. Another common technique, applicable when the arguments are all of the same type, is to use a sentinel value (often 0, -1, or an appropriately-cast null pointer) at the end of the list. Also, one can pass an explicit count of the number of variable arguments. Some older compilers did provided a nargs() function, but it was never portable.

Is this allowed?

```
int f(...)
{
  ...
}
```

No! Standard C requires at least one fixed argument, in part so that you can hand it to va_start().

So how do I get floating point numbers passed as arguments?

Arguments of type float are always promoted to type double, and types char and short int are promoted to int. Therefore, it is never correct to invoke

```
va_arg(argp, float);
```

instead you should always use

```
va_arg(argp, double)
```

Similarly, use

```
va_arg(argp, int)
```

to retrieve arguments which were originally char, short, or int.

How can I create a function which takes a variable number of arguments and passes them to some other function (which takes a variable number of arguments)?

You should provide a version of that other function which accepts a va_list type of pointer.

So how can I call a function with an argument list built up at run time?

There is no portable way to do this. Instead of an actual argument list, you might want to pass an array of generic (void *) pointers. The called function can then step through the array, much like main() steps through char *argv[].

What is the use of vprintf(), vfprintf() and vsprintf()?

Below, the myerror() function prints an error message, preceded by the string "error: " and terminated with a newline:

```
#include <stdio.h>
#include <stdarg.h>
void myerror(char *fmt, ...)
{
  va_list argp;
  fprintf(stderr, "error: ");
  va_start(argp, fmt);
  vfprintf(stderr, fmt, argp);
  va_end(argp);
  fprintf(stderr, "\n");
}
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

With respect to function parameter passing, what is the difference between call-by-value and call-by-reference? Which method does C use?

In the case of call-by-reference, a pointer reference to a variable is passed into a function instead of the actual value. The function's operations will effect the variable in a global as well as local sense. Call-by-value (C's method of parameter passing), by contrast, passes a copy of the variable's value into the function. Any changes to the variable made by function have only a local effect and do not alter the state of the variable passed into the function.

Does C really have pass by reference?

No.

C uses pass by value, but it can pretend doing pass by reference, by having functions that have pointer arguments and by using the & operator when calling the function. This way, the compiler will simulate this feature (like when you pass an array to a function, it actually passes a pointer instead). C does not have something like the formal pass by reference or C++ reference parameters.

How will you decide whether to use pass by reference, by pointer and by value?

The selection of the argument passing depends on the situation.

- A function uses passed data without modifying it:
    - If the data object is small, such as a built-in data type or a small structure then pass it by value.
    - If the data object is an array, use a pointer because that is the only choice. Make the pointer a pointer to const.
    - If the data object is a good-sized structure, use a const pointer or a const reference to increase program efficiency. You save the time and space needed to copy a structure or a class design, make the pointer or reference const.
    - If the data object is a class object, use a const reference. The semantics of class design often require using a reference. The standard way to pass class object arguments is by reference.
- A function modifies data in the calling function:
    - If the data object is a built-in data type, use a pointer. If you spot a code like fixit(&x), where x is an int, its clear that this function intends to modify x.
    - If the data object is an array, use the only choice, a pointer.
    - If the data object is a structure, use a reference or a pointer.
    - If the data object is a class object, use a reference.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(158 / 301) *

If I have the name of a function in the form of a string, how can I invoke that function?

Keep a table of names and their function pointers:

```
int myfunc1(), myfunc2();

struct
{
  char *name;
  int (*func_ptr)();
} func_table[] = {"myfunc1", myfunc1,
                  "myfunc2", myfunc2,};
```

Search the table for the name, and call via the associated function pointer.

C Functions
(159 / 301)

What does the error, invalid redeclaration of a function mean?

If there is no declaration in scope then it is assumed to be declared as returning an int and without any argument type information. This can lead to discrepancies if the function is later declared or defined. Such functions must be declared before they are called. Also check if there is another function in some header file with the same name.

C Functions
(160 / 301)

How can I pass the variable argument list passed to one function to another function.

Good question

Something like this wont work

```
#include <stdarg.h>
main()
{
  display("Hello", 4, 12, 13, 14, 44);
}

display(char *s,...)
{
  show(s,...);
}

show(char *t,...)
{
  va_list ptr;
  int a;
  va_start(ptr,t);
  a = va_arg(ptr, int);
  printf("%f", a);
}
```

This is the right way of doing it

```
#include <stdarg.h>
main()
{
  display("Hello", 4, 12, 13, 14, 44);
}

display(char *s,...)
{
  va_list ptr;
  va_start(ptr, s);
  show(s,ptr);
}

show(char *t, va_list ptr1)
{
  int a, n, i;
  a=va_arg(ptr1, int);

  for(i=0; i<a; i++)
```

```
  {
     n=va_arg(ptr1, int);
     printf("\n%d", n);
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(161 / 301)

How do I pass a variable number of function pointers to a variable argument (va_arg) function?

Glad that you thought about doing something like this!

Here is some code

```
#include <stdarg.h>

main()
{
  int (*p1)();
  int (*p2)();
  int fun1(), fun2();

  p1 = fun1;
  p2 = fun2;
  display("Bye", p1, p2);
}

display(char *s,...)
{
  int (*pp1)(), (*pp2)();
  va_list ptr;
  typedef int (*f)(); //This typedef is  very important.

  va_start(ptr,s);

  pp1 = va_arg(ptr, f); // va_arg(ptr, int (*)()); would NOT have worked!
  pp2 = va_arg(ptr, f);

  (*pp1)();
  (*pp2)();

}

fun1()
{
  printf("\nHello!\n");
}

fun2()
{
  printf("\nHi!\n");
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions
(162 / 301)

Will C allow passing more or less arguments than required to a function.

It wont if the prototpe is around. It will ideally scream out with an error like

```
Too many arguments
```

or

```
Too few arguments
```

But if the prototype is not around, the behavior is undefined.

Try this out

```
#include <stdio.h>

/*
int foo(int a);
int foo2(int a, int b);
*/

int main(int a)
{
  int (*fp)(int a);

  a = foo();
  a = foo2(1);

  exit(0);
}

int foo(int a)
{
  return(a);
}

int foo2(int a, int b)
{
  return(a+b);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Whats short-circuiting in C expressions?

What this means is that the right hand side of the expression is not evaluated if the left hand side determines the outcome. That is if the left hand side is true for || or false for &&, the right hand side is not evaluated.

Whats wrong with the expression a[i]=i++; ? Whats a sequence point?

Although its surprising that an expression like i=i+1; is completely valid, something like a[i]=i++; is not. This is because all accesses to an element must be to change the value of that variable. In the statement a[i]=i++; , the access to i is not for itself, but for a[i] and so its invalid. On similar lines, i=i++; or i=++i; are invalid. If you want to increment the value of i, use i=i+1; or i+=1; or i++; or ++i; and not some combination.

A sequence point is a state in time (just after the evaluation of a full expression, or at the ||, &&, ?:, or comma operators, or just before a call to a function) at which there are no side effects.

The ANSI/ISO C Standard states that

```
Between the previous and next sequence point an object shall have its stored
value modified at most once by the evaluation of an expression. Furthermore,
the prior value shall be accessed only to determine the value to be stored.
```

At each sequence point, the side effects of all previous expressions will be completed. This is why you cannot rely on expressions such as a[i] = i++;, because there is no sequence point specified for the assignment, increment or index operators, you don't know when the effect of the increment on i occurs.

The sequence points laid down in the Standard are the following:

- The point of calling a function, after evaluating its arguments.
- The end of the first operand of the && operator.
- The end of the first operand of the || operator.
- The end of the first operand of the ?: conditional operator.
- The end of the each operand of the comma operator.
- Completing the evaluation of a full expression. They are the following:
    - Evaluating the initializer of an auto object.
    - The expression in an ?ordinary? statement?an expression followed by semicolon.
    - The controlling expressions in do, while, if, switch or for statements.
    - The other two expressions in a for statement.
    - The expression in a return statement.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Does the ?: (ternary operator) return a lvalue? How can I assign a value to the output of the ternary operator?

No, it does not return an "lvalue"

Try doing something like this if you want to assign a value to the output of this operator

```
*((mycondition) ? &var1 : &var2) = myexpression;
```

Is 5[array] the same as array[5]?

Yes!

Since array subscripting is commutative in C. That is

```
array[n] == *((array)+(n)) == *((n)+(array)) == n[array]
```

But frankly, this feature is of no use to anyone. Anyone asking this question is more or less a fool trying to be cool.

What are #pragmas?

The directive provides a single, well-defined "escape hatch" which can be used for all sorts of (nonportable) implementation-specific controls and extensions: source listing control, structure packing, warning suppression (like lint's old /* NOTREACHED */ comments), etc.

For example

```
#pragma once
```

inside a header file is an extension implemented by some preprocessors to help make header files idempotent (to prevent a header file from included twice).

C Statements
(168 / 301) *

What is the difference between if(0 == x) and if(x == 0)?

Nothing!. But, it's a good trick to prevent the common error of writing

```
if(x = 0)
```

The error above is the source of a lot of serious bugs and is very difficult to catch. If you cultivate the habit of writing the constant before the ==, the compiler will complain if you accidentally type

```
if(0 = x)
```

Of course, the trick only helps when comparing to a constant.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Should we use goto or not?

You should use gotos wherever it suits your needs really well. There is nothing wrong in using them. Really.

There are cases where each function must have a single exit point. In these cases, it makes much sense to use gotos.

```
myfunction()
{
  if(error_condition1)
  {
    // Do some processing.
    goto failure;
  }

  if(error_condition2)
  {
    // Do some processing.
    goto failure;
  }

success:
  return(TRUE);

failure:

  // Do some cleanup.
  return(FALSE);
}
```

Also, a lot of coding problems lend very well to the use of gotos. The only argument against gotos is that it can make the code a little un-readable. But if its commented properly, it works quite fine.

C Statements
(170 / 301)

Is ++i really faster than i = i + 1?

Anyone asking this question does not really know what he is talking about.

Any good compiler will and should generate identical code for ++i, i += 1, and i = i + 1. Compilers are meant to optimize code. The programmer should not be bother about such things. Also, it depends on the processor and compiler you are using. One needs to check the compiler's assembly language output, to see which one of the different approcahes are better, if at all.

Note that speed comes Good, well written algorithms and not from such silly tricks.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What do lvalue and rvalue mean?

An lvalue is an expression that could appear on the left-hand sign of an assignment (An object that has a location). An rvalue is any expression that has a value (and that can appear on the right-hand sign of an assignment).

The lvalue refers to the left-hand side of an assignment expression. It must always evaluate to a memory location. The rvalue represents the right-hand side of an assignment expression; it may have any meaningful combination of variables and constants.

Is an array an expression to which we can assign a value?

An lvalue was defined as an expression to which a value can be assigned. The answer to this question is no, because an array is composed of several separate array elements that cannot be treated as a whole for assignment purposes.

The following statement is therefore illegal:

```
int x[5], y[5];
x = y;
```

Additionally, you might want to copy the whole array all at once. You can do so using a library function such as the memcpy() function, which is shown here:

```
memcpy(x, y, sizeof(y));
```

It should be noted here that unlike arrays, structures can be treated as lvalues. Thus, you can assign one structure variable to another structure variable of the same type, such as this:

```
typedef struct t_name
{
  char last_name[25];
  char first_name[15];
  char middle_init[2];
} NAME;
...
  NAME my_name, your_name;
...
  your_name = my_name;
...
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(172 / 301) *

What does the term cast refer to? Why is it used?

Casting is a mechanism built into C that allows the programmer to force the conversion of data types. This may be needed because most C functions are very particular about the data types they process. A programmer may wish to override the default way the C compiler promotes data types.

C Statements
(173 / 301) *

What is the difference between a statement and a block?

A statement is a single C expression terminated with a semicolon. A block is a series of statements, the group of which is enclosed in curly-braces.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(174 / 301)

Can comments be nested in C?

No

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(175 / 301) *

What is type checking?

The process by which the C compiler ensures that functions and operators use data of the appropriate type(s). This form of check helps ensure the semantic correctness of the program.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Why can't you nest structure definitions?

This is a trick question!

You can nest structure definitions.

```
struct salary
{
    char empname[20];
    struct
    {
        int dearness;
    }
    allowance;
}employee;
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is a forward reference?

It is a reference to a variable or function before it is defined to the compiler. The cardinal rule of structured languages is that everything must be defined before it can be used. There are rare occasions where this is not possible. It is possible (and sometimes necessary) to define two functions in terms of each other. One will obey the cardinal rule while the other will need a forward declaration of the former in order to know of the former's existence.

C Statements
(178 / 301) *

What is the difference between the & and && operators and the | and || operators?

& and | are bitwise AND and OR operators respectively. They are usually used to manipulate the contents of a variable on the bit level. && and || are logical AND and OR operators respectively. They are usually used in conditionals

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(179 / 301)

Is C case sensitive (ie: does C differentiate between upper and lower case letters)?

Yes, ofcourse!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(180 / 301) *

Can goto be used to jump across functions?

No!

This wont work

```
main()
{
  int i=1;
  while (i<=5)
  {
    printf("%d",i);
    if (i>2)
        goto here;
    i++;
  }
}

fun()
{
here:
  printf("PP");
}
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(181 / 301) *

Whats wrong with #define myptr int *?

```
#define myptr int *
myptr p, q;
```

Only p will be a pointer to an int, and q will just be an int!.

Use typdef for such a requirement.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What purpose do #if, #else, #elif, #endif, #ifdef, #ifndef serve?

The following preprocessor directives are used for conditional compilation. Conditional compilation allows statements to be included or omitted based on conditions at compile time.

```
#if
#else
#elif
#endif
#ifdef
#ifndef
```

In the following example, the printf statements are compiled when the symbol DEBUG is defined, but not compiled otherwise

```
/* remove to suppress debug printf's*/
#define DEBUG
...
x = ....

  #ifdef DEBUG
    printf( "x=%d\n" );
  #endif...

y = ....;

  #ifdef DEBUG
    printf( "y=%d\n" );
  #endif...
```

#if, #else, #elif statements

#if directive

- #if is followed by a intger constant expression.
- If the expression is not zero, the statement(s) following the #if are compiled, otherwise they are ignored.
- #if statements are bounded by a matching #endif, #else or #elif
- Macros, if any, are expanded, and any undefined tokens are replaced with 0 before the constant expression is evaluated
- Relational operators and integer operators may be used

Expression examples

```
#if 1
```

```
#if 0
#if ABE == 3
#if ZOO < 12
#if ZIP == 'g'
#if (ABE + 2 - 3 * ZIP) > (ZIP - 2)
```

In most uses, expression is simple relational, often equality test

```
#if SPARKY == '7'
```

#else directive

- #else marks the beginning of statement(s) to be compiled if the preceding #if or #elif expression is zero (false)
- Statements following #else are bounded by matching #endif

Examples

```
#if OS = 'A'
  system( "clear" );
#else
  system( "cls" );
#endif
```

#elif directive

- #elif adds an else-if branch to a previous #if
- A series of #elif's provides a case-select type of structure
- Statement(s) following the #elif are compiled if the expression is not zero, ignored otherwise
- Expression is evaluated just like for #if

Examples

```
#if TST == 1
  z = fn1( y );
#elif TST == 2
  z = fn2( y, x );
#elif TST == 3
  z = fn3( y, z, w );
#endif

...
#if ZIP == 'g'
  rc = gzip( fn );
#elif ZIP == 'q'
  rc = qzip( fn );
#else
  rc = zip( fn );
#endif
```

#ifdef and #ifndef directives

Testing for defined macros with #ifdef, #ifndef, and defined()

- #ifdef is used to include or omit statements from compilation depending of whether a macro name is defined or not.
- Often used to allow the same source module to be compiled in different environments (UNIX/ DOS/MVS), or with different options (development/production).
- #ifndef similar, but includes code when macro name is not defined.

Examples

```
#ifdef TESTENV
  printf( "%d ", i );
#endif
#ifndef DOS
  #define LOGFL "/tmp/loga.b";
#else
  #define LOGFL "c:\\tmp\\log.b";
#endif
```

defined() operator

- defined(mac), operator is used with #if and #elif and gives 1 (true) if macro name mac is defined, 0 (false) otherwise.
- Equivalent to using #ifdef and #ifndef, but many shops prefer #if with defined(mac) or !defined(mac)

Examples

```
#if defined(TESTENV)
  printf( "%d ", i );
#endif
#if !defined(DOS)
  #define LOGFL "/tmp/loga.b";
#else
  #define LOGFL "c:\\tmp\\log.b";
#endif
```

Nesting conditional statements

Conditional compilation structures may be nested:

```
#if defined(UNIX)
  #if LOGGING == 'y'
     #define LOGFL "/tmp/err.log"
  #else
     #define LOGFL "/dev/null"
  #endif
#elif defined( MVS )
  #if LOGGING == 'y'
     #define LOGFL "TAP.AVS.LOG"
  #else
     #define LOGFL "NULLFILE"
  #endif
#elif defined( DOS )
  #if LOGGING == 'y'
     #define LOGFL "C:\\tmp\\err.log"
  #else
     #define LOGFL "nul"
  #endif
#endif
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Statements
(183 / 301)

Can we use variables inside a switch statement? Can we use floating point numbers? Can we use expressions?

No

The only things that case be used inside a switch statement are constants or enums. Anything else will give you a

```
constant expression required
```

error. That is something like this is not valid

```
switch(i)
{
  case 1: // Something;
          break;
  case j: // Something;
          break;
}
```

So is this. You cannot switch() on strings

```
switch(i)
{
  case "string1" : // Something;
                   break;
  case "string2" : // Something;
                   break;
}
```

This is valid, however

```
switch(i)
{
  case 1:     // Something;
              break;
  case 1*2+4: // Something;
              break;
}
```

This is also valid, where t is an enum

```
switch(i)
{
  case 1: // Something;
          break;
  case t: // Something;
          break;
}
```

Also note that the default case does not require a break; if and only if its at the end of the switch() statement. Otherwise, even the default case requires a break;

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is more efficient? A switch() or an if() else()?

Both are equally efficient. Usually the compiler implements them using jump instructions. But each of them has their own unique advantages.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is the difference between a deep copy and a shallow copy?

Deep copy involves using the contents of one object to create another instance of the same class. In a deep copy, the two objects may contain ht same information but the target object will have its own buffers and resources. the destruction of either object will not affect the remaining object. The overloaded assignment operator would create a deep copy of objects.

Shallow copy involves copying the contents of one object into another instance of the same class thus creating a mirror image. Owing to straight copying of references and pointers, the two objects will share the same externally contained contents of the other object to be unpredictable.
Using a copy constructor we simply copy the data values member by member. This method of copying is called shallow copy. If the object is a simple class, comprised of built in types and no pointers this would be acceptable. This function would use the values and the objects and its behavior would not be altered with a shallow copy, only the addresses of pointers that are members are copied and not the value the address is pointing to. The data values of the object would then be inadvertently altered by the function. When the function goes out of scope, the copy of the object with all its data is popped off the stack. If the object has any pointers a deep copy needs to be executed. With the deep copy of an object, memory is allocated for the object in free store and the elements pointed to are copied. A deep copy is used for objects that are returned from a function.

C Statements
(186 / 301) *

What is operator precedence?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Arrays
(187 / 301) *

How to write functions which accept two-dimensional arrays when the width is not known before hand?

Try something like

```
myfunc(&myarray[0][0], NO_OF_ROWS, NO_OF_COLUMNS);
```

```
void myfunc(int *array_pointer, int no_of_rows, int no_of_columns)
{
  // myarray[i][j] is accessed as array_pointer[i * no_of_columns + j]
}
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Arrays
(188 / 301) *

Is char a[3] = "abc"; legal? What does it mean?

It declares an array of size three, initialized with the three characters 'a', 'b', and 'c', without the usual terminating '\0' character. The array is therefore not a true C string and cannot be used with strcpy, printf %s, etc. But its legal.

C Arrays
(189 / 301)

If a is an array, is a++ valid?

No

You will get an error like

```
Error message : Lvalue required in function main.
```

Doing a++ is asking the compiler to change the base address of the array. This the only thing that the compiler remembers about an array once its declared and it wont allow you to change the base address. If it allows, it would be unable to remember the beginning of the array.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Arrays
(190 / 301)

How can we find out the length of an array dynamically in C?

Here is a C program to do the same...

```
#include <stdio.h>
#include <conio.h>

int main()
{
  int arr[] = {3,4,65,78,1,2,4};
  int arr_length = sizeof(arr)/sizeof(arr[0]);

  printf("\nLength of the array is :[%d]\n\n", arr_length);

  getch();
  return(0);
}
```

C Variables
(191 / 301)

What is the difference between the declaration and the definition of a variable?

The definition is the one that actually allocates space, and provides an initialization value, if any.

There can be many declarations, but there must be exactly one definition. A definition tells the compiler to set aside storage for the variable. A declaration makes the variable known to parts of the program that may wish to use it. A variable might be defined and declared in the same statement.

C Variables
(192 / 301)

Do Global variables start out as zero?

Glad you asked!

Uninitialized variables declared with the "static" keyword are initialized to zero. Such variables are implicitly initialized to the null pointer if they are pointers, and to 0.0F if they are floating point numbers.

Local variables start out containing garbage, unless they are explicitly initialized.

Memory obtained with malloc() and realloc() is likely to contain junk, and must be initialized. Memory obtained with calloc() is all-bits-0, but this is not necessarily useful for pointer or floating-point values (This is in contrast to Global pointers and Global floating point numbers, which start as zeroes of the right type).

C Variables
(193 / 301)

Does C have boolean variable type?

No, C does not have a boolean variable type. One can use ints, chars, #defines or enums to achieve the same in C.

```
#define TRUE 1
#define FALSE 0

enum bool {false, true};
```

An enum may be good if the debugger shows the names of enum constants when examining variables.

C Variables
(194 / 301) *

Where may variables be defined in C?

Outside a function definition (global scope, from the point of definition downward in the source code). Inside a block before any statements other than variable declarations (local scope with respect to the block).

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Variables
(195 / 301) *

To what does the term storage class refer? What are auto, static, extern, volatile, const classes?

This is a part of a variable declaration that tells the compiler how to interpret the variable's symbol. It does not in itself allocate storage, but it usually tells the compiler how the variable should be stored. Storage class specifiers help you to specify the type of storage used for data objects. Only one storage class specifier is permitted in a declaration this makes sense, as there is only one way of storing things and if you omit the storage class specifier in a declaration, a default is chosen. The default depends on whether the declaration is made outside a function (external declarations) or inside a function (internal declarations). For external declarations the default storage class specifier will be extern and for internal declarations it will be auto. The only exception to this rule is the declaration of functions, whose default storage class specifier is always extern.

Here are C's storage classes and what they signify:

- auto - local variables.
- static - variables are defined in a nonvolatile region of memory such that they retain their contents though out the program's execution.
- register - asks the compiler to devote a processor register to this variable in order to speed the program's execution. The compiler may not comply and the variable looses it contents and identity when the function it which it is defined terminates.
- extern - tells the compiler that the variable is defined in another module.

In C, const and volatile are type qualifiers. The const and volatile type qualifiers are completely independent. A common misconception is to imagine that somehow const is the opposite of volatile and vice versa. This is wrong. The keywords const and volatile can be applied to any declaration, including those of structures, unions, enumerated types or typedef names. Applying them to a declaration is called qualifying the declaration?that's why const and volatile are called type qualifiers, rather than type specifiers.

- const means that something is not modifiable, so a data object that is declared with const as a part of its type specification must not be assigned to in any way during the run of a program. The main intention of introducing const objects was to allow them to be put into read-only store, and to permit compilers to do extra consistency checking in a program. Unless you defeat the intent by doing naughty things with pointers, a compiler is able to check that const objects are not modified explicitly by the user. It is very likely that the definition of the object will contain an initializer (otherwise, since you can't assign to it, how would it ever get a value?), but this is not always the case. For example, if you were accessing a hardware port at a fixed memory address and promised only to read from it, then it would be declared to be const but not initialized.
- volatile tells the compiler that other programs will be modifying this variable in addition to the program being compiled. For example, an I/O device might need write directly into a program or data space. Meanwhile, the program itself may never directly access the memory area in question. In such a case, we would not want the compiler to optimize-out this data area that never seems to be used by the program, yet must exist for the program to function correctly in a larger context. It tells the compiler that the object is subject to sudden change for reasons which cannot be predicted from a study of the program itself, and forces every reference to such an object to be a genuine reference.
- const volatile - Both constant and volatile.

The "volatile" modifier

The volatile modifier is a directive to the compiler?s optimizer that operations involving this variable should not be optimized in certain ways. There are two special cases in which use of the volatile modifier is desirable. The first case involves memory-mapped hardware (a device such as a graphics adaptor that appears to the computer?s hardware as if it were part of the computer?s memory), and the second involves shared memory (memory used by two or more programs running simultaneously). Most computers have a set of registers that can be accessed faster than the computer?s main memory. A good compiler will perform a kind of optimization called ?redundant load and store removal.? The compiler looks for places in the code where it can either remove an instruction to load data from memory because the value is already in a register, or remove an instruction to store data to memory because the value can stay in a register until it is changed again anyway.
If a variable is a pointer to something other than normal memory, such as memory-mapped ports on a peripheral, redundant load and store optimizations might be detrimental. For instance, here?s a piece of code that might be used to time some operation:

```
time_t time_addition(volatile const struct timer *t, int a)
{
  int n;
  int x;
  time_t then;
  x = 0;
  then = t->value;
  for (n = 0; n < 1000; n++)
  {
    x = x + a;
  }
  return t->value - then;
}
```

In this code, the variable t->value is actually a hardware counter that is being incremented as time passes. The function adds the value of a to x 1000 times, and it returns the amount the timer was incremented by while the 1000 additions were being performed. Without the volatile modifier, a clever optimizer might assume that the value of t does not change during the execution of the function, because there is no statement that explicitly changes it. In that case, there?s no need to read it from memory a second time and subtract it, because the answer will always be 0. The compiler might therefore ?optimize? the function by making it always return 0. If a variable points to data in shared memory, you also don?t want the compiler to perform redundant load and store optimizations. Shared memory is normally used to enable two programs to communicate with each other by having one program store data in the shared portion of memory and the other program read the same portion of memory. If the compiler optimizes away a load or store of shared memory, communication between the two programs will be affected.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Variables
(196 / 301)

What does the typedef keyword do?

This keyword provides a short-hand way to write variable declarations. It is not a true data typing mechanism, rather, it is syntactic "sugar coating".

For example

```
typedef struct node
{
  int value;
  struct node *next;
}mynode;
```

This can later be used to declare variables like this

```
mynode *ptr1;
```

```
and not by the lengthy expression
```

```
struct node *ptr1;
```

There are three main reasons for using typedefs:

- It makes the writing of complicated declarations a lot easier. This helps in eliminating a lot of clutter in the code.
- It helps in achieving portability in programs. That is, if we use typedefs for data types that are machine dependent, only the typedefs need to change when the program is ported to a new platform.
- It helps in providing better documentation for a program. For example, a node of a doubly linked list is better understood as ptrToList than just a pointer to a complicated structure.

C Variables
(197 / 301) *

What is the difference between constants defined through #define and the constant keyword?

A constant is similar to a variable in the sense that it represents a memory location (or simply, a value). It is different from a normal variable, in that it cannot change it's value in the proram - it must stay for ever stay constant. In general, constants are a useful because they can prevent program bugs and logical errors(errors are explained later). Unintended modifications are prevented from occurring. The compiler will catch attempts to reassign new values to constants.

Constants may be defined using the preprocessor directive #define. They may also be defined using the const keyword.

So whats the difference between these two?

```
#define ABC 5
```

and

```
const int abc = 5;
```

There are two main advantages of the second one over the first technique. First, the type of the constant is defined. "pi" is float. This allows for some type checking by the compiler. Second, these constants are variables with a definite scope. The scope of a variable relates to parts of your program in which it is defined.

There is also one good use of the important use of the const keyword. Suppose you want to make use of some structure data in some function. You will pass a pointer to that structure as argument to that function. But to make sure that your structure is readonly inside the function you can declare the structure argument as const in function prototype. This will prevent any accidental modification of the structure values inside the function.

C Variables
(198 / 301) *

What are Trigraph characters?

These are used when you keyboard does not support some special characters

```
??=    #
??(    [
??)    ]
??<    {
??>    }
??!    |
??/    \
??'    ^
??-    ~
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How are floating point numbers stored? Whats the IEEE format?

IEEE Standard 754 floating point is the most common representation today for real numbers on computers, including Intel-based PC's, Macintoshes, and most Unix platforms.

IEEE floating point numbers have three basic components: the sign, the exponent, and the mantissa. The mantissa is composed of the fraction and an implicit leading digit (explained below). The exponent base(2) is implicit and need not be stored.

The following figure shows the layout for single (32-bit) and double (64-bit) precision floating-point values. The number of bits for each field are shown (bit ranges are in square brackets):

```
                 Sign   Exponent   Fraction   Bias
-------------------------------------------------
Single Precision 1 [31] 8 [30-23]  23 [22-00] 127
Double Precision 1 [63] 11 [62-52] 52 [51-00] 1023
```

The sign bit is as simple as it gets. 0 denotes a positive number; 1 denotes a negative number. Flipping the value of this bit flips the sign of the number.

The exponent field needs to represent both positive and negative exponents. To do this, a bias is added to the actual exponent in order to get the stored exponent. For IEEE single-precision floats, this value is 127. Thus, an exponent of zero means that 127 is stored in the exponent field. A stored value of 200 indicates an exponent of (200-127), or 73. For reasons discussed later, exponents of -127 (all 0s) and +128 (all 1s) are reserved for special numbers. For double precision, the exponent field is 11 bits, and has a bias of 1023.

The mantissa, also known as the significand, represents the precision bits of the number. It is composed of an implicit leading bit and the fraction bits. To find out the value of the implicit leading bit, consider that any number can be expressed in scientific notation in many different ways. For example, the number five can be represented as any of these:

```
5.00 × 100
0.05 × 10 ^ 2
5000 × 10 ^ -3
```

In order to maximize the quantity of representable numbers, floating-point numbers are typically stored in normalized form. This basically puts the radix point after the first non-zero digit. In normalized form, five is represented as 5.0 × 100. A nice little optimization is available to us in base two, since the only possible non-zero digit is 1. Thus, we can just assume a leading digit of 1, and don't need to represent it explicitly. As a result, the mantissa has effectively 24 bits of resolution, by way of 23 fraction bits.

So, to sum up:

```
1. The sign bit is 0 for positive, 1 for negative.
2. The exponent's base is two.
3. The exponent field contains 127 plus the true exponent for single-precision,
   or 1023 plus the true exponent for double precision.
4. The first bit of the mantissa is typically assumed to be 1.f, where f is the
   field of fraction bits.
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Variables
(200 / 301) *

When should the register modifier be used?

The register modifier hints to the compiler that the variable will be heavily used and should be kept in the CPU?s registers, if possible, so that it can be accessed faster. There are several restrictions on the use of the register modifier.

First, the variable must be of a type that can be held in the CPU?s register. This usually means a single value of a size less than or equal to the size of an integer. Some machines have registers that can hold floating-point numbers as well. Second, because the variable might not be stored in memory, its address cannot be taken with the unary & operator. An attempt to do so is flagged as an error by the compiler. Some additional rules affect how useful the register modifier is. Because the number of registers is limited, and because some registers can hold only certain types of data (such as pointers or floating-point numbers), the number and types of register modifiers that will actually have any effect are dependent on what machine the program will run on. Any additional register modifiers are silently ignored by the compiler. Also, in some cases, it might actually be slower to keep a variable in a register because that register then becomes unavailable for other purposes or because the variable isn?t used enough to justify the overhead of loading and storing it. So when should the register modifier be used? The answer is never, with most modern compilers. Early C compilers did not keep any variables in registers unless directed to do so, and the register modifier was a valuable addition to the language. C compiler design has advanced to the point, however, where the compiler will usually make better decisions than the programmer about which variables should be stored in registers. In fact, many compilers actually ignore the register modifier, which is perfectly legal, because it is only a hint and not a directive.

C Variables
(201 / 301) *

When should a type cast be used?

There are two situations in which to use a type cast.

The first use is to change the type of an operand to an arithmetic operation so that the operation will be performed properly.

The second case is to cast pointer types to and from void * in order to interface with functions that expect or return void pointers. For example, the following line type casts the return value of the call to malloc() to be a pointer to a foo structure.

```
struct foo *p = (struct foo *) malloc(sizeof(struct foo));
```

A type cast should not be used to override a const or volatile declaration. Overriding these type modifiers can cause the program to fail to run correctly. A type cast should not be used to turn a pointer to one type of structure or data type into another. In the rare events in which this action is beneficial, using a union to hold the values makes the programmer?s intentions clearer.

C Structures
(202 / 301) *

Can structures be assigned to variables and passed to and from functions?

Yes, they can!

But note that when structures are passed, returned or assigned, the copying is done only at one level (The data pointed to by any pointer fields is not copied!.

C Structures
(203 / 301)

Can we directly compare two structures using the == operator?

No, you cannot!

The only way to compare two structures is to write your own function that compares the structures field by field. Also, the comparison should be only on fields that contain data (You would not want to compare the next fields of each structure!).

A byte by byte comparison (say using memcmp()) will also fail. This is because the comparison might fonder on random bits present in unused "holes" in the structure (padding used to keep the alignment of the later fields correct). So a memcmp() of the two structure will almost never work. Also, any strings inside the strucutres must be compared using strcmp() for similar reasons.

There is also a very good reason why structures can be compared directly - unions!. It is because of unions that structures cannot be compared for equality. The possibility that a structure might contain a union makes it hard to compare such structures; the compiler can't tell what the union currently contains and so wouldn't know how to compare the structures. This sounds a bit hard to swallow and isn't 100% true, most structures don't contain unions, but there is also a philosophical issue at stake about just what is meant by "equality" when applied to structures. Anyhow, the union business gives the Standard a good excuse to avoid the issue by not supporting structure comparison.

If your structures dont have stuff like floating point numbers, pointers, unions etc..., then you could possibly do a memset() before using the structure variables..

```
memset (&myStruct, 0, sizeof(myStruct));
```

This will set the whole structure (including the padding) to all-bits-zero. We can then do a memcmp() on two such structures.

```
memcmp (&s1,&s2,sizeof(s1));
```

But this is very risky and can end up being a major bug in your code!. So try not to do this kind of memcmp() operations on structures variables as far as possible!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Structures
(204 / 301) *

Can we pass constant values to functions which accept structure arguments?

If you are trying to do something like this

```
myfunction((struct mystruct){10,20});
```

then, it wont work!. Use a temporary structure variable.

C Structures
(205 / 301)

How does one use fread() and fwrite()? Can we read/write structures to/from files?

Its easy to write a structure into a file using fwrite()

```
fwrite(&somestruct, sizeof somestruct, 1, fp);
```

A similat fread() invocation can read it back in. But, data files so written will not be portable (specially if they contain floating point numbers). Also that if the structure has any pointers, only the pointer values will be written, and they are most unlikely to be valid when read back in. One must use the "rb/wb" flag when opening the files.

A more portable solution is to write code to write and read a structure, field-by-field, in a portable (perhaps ASCII) way!. This is simpler to port and maintain.

* *

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Why do structures get padded? Why does sizeof() return a larger size?

Padding enables the CPU to access the members faster. If they are not aligned (say to word boundaries), then accessing them might take up more time. So the padding results in faster access. This is also required to ensure that alignment properties are preserved when an array of contiguous structures is allocated. Even if the structure is not part of an array, the end padding remains, so that sizeof() can always return a consistent size.

Can we determine the offset of a field within a structure and directly access that element?

The offsetof() macro(<stddef.h>) does exactly this.

A probable implementation of the macro is

```
#define offsetof(type, mem) ((size_t)((char *)&((type *)0)->mem - (char *)(type *)0))
```

This can be used as follows. The offset of field a in struct mystruct is

```
offset_of_a = offsetof(struct mystruct, a)
```

If structpointer is a pointer to an instance of this structure, and field a is an int, its value can be set indirectly with

```
*(int *)((char *)structpointer + offset_of_a) = some_value;
```

C Structures
(208 / 301)

What are bit fields in structures?

To avoid wastage of memory in structures, a group of bits can be packed together into an integer and its called a bit field.

```
struct tag-name
{
    data-type name1:bit-length;
    data-type name2:bit-length;
    ...
    ...
    data-type nameN:bit-length;
}
```

A real example

```
struct student;
{
   char name[30];
   unsigned sex:1;
   unsigned age:5;
   unsigned rollno:7;
   unsigned branch:2;
};

struct student a[100];

scanf("%d", &sex);
a[i].sex=sex;
```

There are some limitations with respect to these bit fields, however:

```
1. Cannot scanf() directly into bit fields.
2. Pointers cannot be used on bit fields to access them.
3. Cannot have an array of bit fields.
```

The main use of bitfields is either to allow tight packing of data or to be able to specify the fields within some externally produced data files. C gives no guarantee of the ordering of fields within machine words, so if you do use them for the latter reason, you program will not only be non-portable, it will be compiler-dependent too. The Standard says that fields are packed into ?storage units?, which are typically machine words. The packing order, and whether or not a bitfield may cross a storage unit boundary, are implementation defined. To force alignment to a storage unit boundary, a zero width field is used before the one that you want to have aligned. Be careful using them. It can require a surprising amount of run-time code to manipulate these things and you can end up using more space than they save. Bit fields do not have addresses?you can't have pointers to them or arrays of them.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is a union? Where does one use unions? What are the limitations of unions?

A union is a variable type that can contain many different variables (like a structure), but only actually holds one of them at a time (not like a structure). This can save memory if you have a group of data where only one of the types is used at a time. The size of a union is equal to the size of it's largest data member. The C compiler allocates just enough space for the largest member. This is because only one member can be used at a time, so the size of the largest, is the most you will need. Here is an example:

```
union person
{
   int age;
   char name[100];
}person1;
```

The union above could be used to either store the age or it could be used to hold the name of the person. There are cases when you would want one or the other, but not both (This is a bad example, but you get the point). To access the fields of a union, use the dot operator (.) just as you would for a structure. When a value is assigned to one member, the other member(s) get whipped out since they share the same memory. Using the example above, the precise time can be accessed like this:

```
person1.age;
```

In larger programs it may be difficult to keep track of which field is the currently used field. This is usually handled by using another variable to keep track of that. For example, you might use an integer called field. When field equals one, the age is used. If field is two, then name is used. The C compiler does no more than work out what the biggest member in a union can be and allocates enough storage (appropriately aligned if neccessary). In particular, no checking is done to make sure that the right sort of use is made of the members. That is your task, and you'll soon find out if you get it wrong. The members of a union all start at the same address?there is guaranteed to be no padding in front of any of them.

ANSI Standard C allows an initializer for the first member of a union. There is no standard way of initializing any other member (nor, under a pre-ANSI compiler, is there generally any way of initializing a union at all).

It is because of unions that structures cannot be compared for equality. The possibility that a structure might contain a union makes it hard to compare such structures; the compiler can't tell what the union currently contains and so wouldn't know how to compare the structures. This sounds a bit hard to swallow and isn't 100% true, most structures don't contain unions, but there is also a philosophical issue at stake about just what is meant by "equality" when applied to structures. Anyhow, the union business gives the Standard a good excuse to avoid the issue by not supporting structure comparison.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Macros
(210 / 301)

How should we write a multi-statement macro?

This is the correct way to write a multi statement macro.

```
#define MYMACRO(argument1, argument2) do { \
                                  stament1; \
                                  stament2; \
                                 } while(1) /* no trailing semicolon */
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Macros
(211 / 301)

How can I write a macro which takes a variable number of arguments?

One can define the macro with a single, parenthesized "argument" which in the macro expansion becomes the entire argument list, parentheses and all, for a function such as printf():

```
#define DEBUG(args) (printf("DEBUG: "), printf args)

if(n != 0) DEBUG(("n is %d\n", n));
```

The caller must always remember to use the extra parentheses. Other possible solutions are to use different macros depending on the number of arguments. C9X will introduce formal support for function-like macros with variable-length argument lists. The notation ... will appear at the end of the macro "prototype" (just as it does for varargs functions).

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Macros
(212 / 301)

What is the token pasting operator and stringizing operator in C?

Token pasting operator

ANSI has introduced a well-defined token-pasting operator, ##, which can be used like this:

```
#define f(g,g2) g##g2

main()
{
  int var12=100;
  printf("%d",f(var,12));
}
```

```
O/P

100
```

Stringizing operator

```
#define sum(xy) printf(#xy " = %f\n",xy);

main()
{
    sum(a+b); // As good as printf("a+b = %f\n", a+b);
}
```

So what does the message "warning: macro replacement within a string literal" mean?

```
#define TRACE(var, fmt) printf("TRACE: var = fmt\n", var)
TRACE(i, %d);

gets expanded as

printf("TRACE: i = %d\n", i);
```

In other words, macro parameters were expanded even inside string literals and character constants. Macro expansion is *not* defined in this way by K&R or by Standard C. When you do want to turn macro arguments into
strings, you can use the new # preprocessing operator, along with string literal concatenation:

```
#define TRACE(var, fmt) printf("TRACE: " #var " = " #fmt "\n", var)
```

See and try to understand this special application of the strigizing operator!

```
#define Str(x) #x
#define Xstr(x) Str(x)
#define OP plus
```

char *opname = Xstr(OP); //This code sets opname to "plus" rather than "OP".

Here are some more examples

Example1

Define a macro DEBUG such that the following program

```
int main()
{
  int x=4;
  float a = 3.14;
  char ch = 'A';

  DEBUG(x, %d);
  DEBUG(a, %f);
  DEBUG(ch, %c);
}
```

outputs

```
DEBUG: x=4
DEBUG: y=3.140000
DEBUG: ch=A
```

The macro would be

```
#define DEBUG(var, fmt) printf("DEBUG:" #var "=" #fmt "\n", var);
```

Example2

Write a macro PRINT for the following program

```
int main()
{
 int x=4, y=4, z=5;
 int a=1, b=2, c=3;
 PRINT(x,y,z);
 PRINT(a,b,c);
}
```

such that it outputs

```
x=4 y=4 z=5
a=1 b=2 c=3
```

Here is a macro that will do this

```
#define PRINT(v1,v2,v3) printf("\n" #v1 "=%d" #v2 "=%d" #v3 "=%d", v1, v2, v3);
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Macros
(213 / 301) *

Define a macro called SQR which squares a number.

This is the wrong way of doing it.

```
#define SQR(x) x*x
```

Thats because, something like

```
#include stdio.h>
#define SQR(x) x*x

int main()
{
  int i;
  i = 64/SQR(4);
  printf("[%d]",i);
  return(0);
}
```

will produce an output of 64, because of how macros work (The macro call square(4) will substituted by 4*4 so the expression becomes i = 64/4*4 . Since / and * has equal priority the expression will be evaluated as (64/4)*4 i.e. 16*4 = 64).

This is the right way of doing it.

```
#define SQR(x) ((x)*(x))
```

But, if x is an expression with side effects, the macro evaluates them twice. The following is one way to do it while evaluating x only once.

```
#include <math.h>
#define SQR(x) pow((x),2.0)
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What should go in header files? How to prevent a header file being included twice? Whats wrong with including more headers?

Generally, a header (.h) file should have (A header file need not have a .h extension, its just a convention):

```
1. Macro definitions (preprocessor #defines).
2. Structure, union, and enumeration declarations.
3. Any typedef declarations.
4. External function declarations.
5. Global variable declarations.
```

Put declarations / definitions in header files if they will be shared between several other files. If some of the definitions / declarations should be kept private to some .c file, don;t add it to the header files.

How does one prevent a header file from included twice?. Including header files twice can lead to multiple-definition errors.

There are two methods to prevent a header file from included twice

Method1

```
#ifndef HEADER_FILE
  #define HEADER_FILE
  ...header file contents...
#endif
```

Method2

A line like

```
#pragma once
```

inside a header file is an extension implemented by some preprocessors to help make header files idempotent (to prevent a header file from included twice).

So, what's the difference between #include <> and #include "" ?

The <> syntax is typically used with Standard or system-supplied headers, while "" is typically used for a program's own header files. Headers named with <> syntax are searched for in one or more standard places. Header files named with font class=announcelink>"" syntax are first searched for in the "current directory," then (if not found) in the same standard places.

What happens if you include unwanted headers?

You will end up increasing the size of your executables!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Headers
(215 / 301)

Is there a limit on the number of characters in the name of a header file?

The limitation is only that identifiers be significant in the first six characters, not that they be restricted to six characters in length.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Headers
(216 / 301) *

Is it acceptable to declare/define a variable in a C header?

A global variable that must be accessed from more than one file can and should be declared in a header file. In addition, such a variable must be defined in one source file. Variables should not be defined in header files, because the header file can be included in multiple source files, which would cause multiple definitions of the variable. The ANSI C standard will allow multiple external definitions, provided that there is only one initialization. But because there?s really no advantage to using this feature, it?s probably best to avoid it and maintain a higher level of portability. "Global" variables that do not have to be accessed from more than one file should be declared static and should not appear in a header file.

How do stat(), fstat(), vstat() work? How to check whether a file exists?

The functions stat(), fstat(), lstat() are used to get the file status.

Here are their prototypes

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *file_name, struct stat *buf);
int fstat(int file_desc, struct stat *buf);
int lstat(const char *file_name, struct stat *buf);
```

These functions return information about the specified file. You do not need any access rights to the file to get this information but you need search rights to all directories named in the path leading to the file.

```
stat      -     stats the file pointed to by file_name and fills in buf.
lstat     -     identical to stat, except in the case of a symbolic link,
                where the link itself is stat-ed, not the file that it refers to.
fstat     -     identical to stat, only the open file pointed to by file_desc
                is stated in place of file_name.
```

They all return a stat structure, which contains the following fields:

```
struct stat {
    dev_t         st_dev;      /* device */
    ino_t         st_ino;      /* inode */
    mode_t        st_mode;     /* protection */
    nlink_t       st_nlink;    /* number of hard links */
    uid_t         st_uid;      /* user ID of owner */
    gid_t         st_gid;      /* group ID of owner */
    dev_t         st_rdev;     /* device type (if inode device) */
    off_t         st_size;     /* total size, in bytes */
    unsigned long st_blksize;  /* blocksize for filesystem I/O */
    unsigned long st_blocks;   /* number of blocks allocated */
    time_t        st_atime;    /* time of last access */
    time_t        st_mtime;    /* time of last modification */
    time_t        st_ctime;    /* time of last change */
};
```

On success, zero is returned. On error, -1 is returned, and errno is set appropriately.

Here is a small piece of code which returns the size of a file by accessing the st_size member of the stat structure.

```
boolean get_file_size(char *file_path,
                      int  *file_size)
{
  struct stat stat_buffer;

  if (stat((char *)file_path, &stat_buffer)!=0)
    return(FALSE);

  *file_size =  stat_buffer.st_size;
  return(TRUE);
}
```

So how do we check if a file exists or not?

Use functions like stat() as used above to find out if a file exits or not. Also, one can use fopen(). When using fopen(), just open for reading and close immediately. If the file does not exists, fopen() will given an error.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C File operations
(218 / 301) *

How can I insert or delete a line (or record) in the middle of a file?

The only way is to rewrite the file.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C File operations
(219 / 301) *

How can I recover the file name using its file descriptor?

Have a wrapper around `fopen()` to remember the names of files as you open them.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C File operations
(220 / 301) *

How can I delete a file? How do I copy files? How can I read a directory in a C program?

Deleting a file

The Standard C Library function is remove(). If thats not there, use unlink().

Copying a file

Directly use system() to invoke the operating system's copy() utility, or open the source and destination files, read characters or blocks of characters from the source file, and write them to the destination file.

How to read directories?

Use the opendir() and readdir() functions, which are part of the POSIX standard and are available on most Unix variants.

C File operations

(221 / 301) *

Whats the use of fopen(), fclose(), fprintf(), getc(), putc(), getw(), putw(), fscanf(), feof(), ftell(), fseek(), rewind(), fread(), fwrite(), fgets(), fputs(), freopen(), fflush(), ungetc()?

Whew!, thats a huge list.

fopen()

This function is used to open a stream.

```
FILE *fp;
fp = fopen("filename","mode");
fp = fopen("data","r");
fp = fopen("results","w");
```

```
Modes

    "r"     -> Open for reading.
    "w"     -> Open for writing.
    "a"     -> Open for appending.
    "r+"    -> Both reading and writing.
    "w+"    -> Both reading and writing, create new file if it exists,
    "a+"    -> Open for both reading and appending.
```

fopen()

fclose() is used to close a stream .

```
fclose(fp);
```

putc(), getc(), putw(), getw(), fgetc(), getchar(), putchar(), fputs()

These functions are used to read/write different types of data to the stream.

```
putc(ch,fp);
c=getc(fp);
putw(integer, fp);
integer=getw(fp);
```

fprintf(), fscanf()

Read/Write formatted data from/to the stream.

```
fprintf(fp,"control string",list);
fscanf(fp,"control string", list);
```

feof()

Check the status of a stream

```
if(feof(fp)!=0)
```

ftell(), fseek(), rewind(), fgetpos(), fsetpos()

Reposition the file pointer of a stream

```
n=ftell(fp); //Relative offset (in bytes) of the current position.
rewind(fp);

fseek(fp, offset, position);

Position can be

0->start of file
1->current position
2->end of file

fseek(fp,0L,0);  // Same as rewind.
fseek(fp,0L,1);  // Stay at current position.
fseek(fp,0L,2);  // Past end of file.
fseek(fp,m,0);   // Move to (m+1) byte.
fseek(fp,m,1)    // Go forward m bytes.
fseek(fp,-m,1);  // Go backward m bytes from current position.
fseek(fp,-m,2);  // Go backward from end of file.
```

fread(), fwrite()

Binary stream input/output.

```
fwrite(&customer, sizeof(record),1,fp);
fread(&customer,sizeof(record),1,fp);
```

Here is a simple piece of code which reads from a file

```
#include <stdio.h>
#include <conio.h>

int main()
{

 FILE *f;
 char buffer[1000];
 f=fopen("E:\\Misc\\__Temp\\FileDrag\\Main.cpp","r");
 if(f)
 {
```

```
  printf("\nOpenened the file!\n");
  while(fgets(buffer,1000,f))
  {
   printf("(%d)-> %s\n",strlen(buffer),buffer);
  }
 }

 fclose(f);
 getch();
 return(0);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to check if a file is a binary file or an ascii file?

Here is some sample C code. The idea is to check the bytes in the file to see if they are ASCII or not...

```
#include <stdio.h>

int main(int argc, char *argv[])
{
 unsigned char ch;
 FILE *file;
 int binaryFile = FALSE;

 file = fopen(<FILE_PATH>, "rb");            // Open in Binary mode for the first time.


 while((fread(&ch, 1, 1, file) == 1) && (binaryFile == FALSE))
 {
    if(ch < 9 || ch == 11 || (ch > 13 && ch < 32) || ch == 255)
    {
       binaryFile = 1;
    }
 }

 fclose(file);

 if(binaryFile)
    file = fopen(<FILE_PATH>, "rb");
 else
    file = fopen(<FILE_PATH>, "r");


 if(binaryFile)
 {
    while(fread(&ch, 1, 1, file) == 1)
    {
      // Do whatever you want here with the binary file byte...
    }
 }
 else
 {
    while(fread(&ch, 1, 1, file) == 1)
    {
      // This is ASCII data, can easily print it!
      putchar(ch);
    }
 }

 fclose(file);

 return(0);
```

```
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is the difference between char *a and char a[]?

There is a lot of difference!

```
char a[] = "string";
char *a = "string";
```

The declaration char a[] asks for space for 7 characters and see that its known by the name "a". In contrast, the declaration char *a, asks for a place that holds a pointer, to be known by the name "a". This pointer "a" can point anywhere. In this case its pointing to an anonymous array of 7 characters, which does have any name in particular. Its just present in memory with a pointer keeping track of its location.

```
char a[] = "string";

   +----+----+----+----+----+----+------+
a: | s  |  t |  r |  i |  n |  g | '\0' |
   +----+----+----+----+----+----+------+
   a[0] a[1] a[2] a[3] a[4] a[5] a[6]

char *a = "string";

+-----+          +---+---+---+---+---+---+------+
|  a: | *======> | s | t | r | i | n | g | '\0' |
+-----+          +---+---+---+---+---+---+------+
Pointer          Anonymous array
```

It is curcial to know that a[3] generates different code depending on whether a is an array or a pointer. When the compiler sees the expression a[3] and if a is an array, it starts at the location "a", goes three elements past it, and returns the character there. When it sees the expression a[3] and if a is a pointer, it starts at the location "a", gets the pointer value there, adds 3 to the pointer value, and gets the character pointed to by that value.

If a is an array, a[3] is three places past a. If a is a pointer, then a[3] is three places past the memory location pointed to by a. In the example above, both a[3] and a[3] return the same character, but the way they do it is different!

Doing something like this would be illegal.

```
char *p = "hello, world!";
p[0] = 'H';
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How can I declare an array with only one element and still access elements beyond the first element (in a valid fashion)?

This is a trick question :)

There is a way to do this. Using structures.

```
struct mystruct {
  int  value;
  int length;
  char string[1];
};
```

Now, when allocating memory to the structure using malloc(), allocate more memory than what the structure would normally require!. This way, you can access beyond string[0] (till the extra amount of memory you have allocated, ofcourse).

But remember, compilers which check for array bounds carefully might throw warnings. Also, you need to have a length field in the structure to keep a count of how big your one element array really is :).

A cleaner way of doing this is to have a pointer instead of the one element array and allocate memory for it seperately after allocating memory for the structure.

```
struct mystruct {
  int  value;
  char *string;  // Need to allocate memory using malloc() after allocating memory for the strucure.
};
```

C Declarations and Definitions
(225 / 301)

What is the difference between enumeration variables and the preprocessor #defines?

Functionality	Enumerations	#defines
Numeric values assigned automatically?	YES	NO
Can the debugger display the symbolic values?	YES	NO
Obey block scope?	YES	NO
Control over the size of the variables?	NO	NO

Whats the difference between gets() and fgets()? Whats the correct way to use fgets() when reading a file?

Unlike fgets(), gets() cannot be told the size of the buffer it's to read into, so it cannot be prevented from overflowing that buffer. As a general rule, always use fgets(). fgets() also takes in the size of the buffer, so that the end of the array cannot be overwritten.

```
fgets(buffer, sizeof(buffer), stdin);
if((p = strchr(buffer, '\n')) != NULL)
{
  *p = '\0';
}
```

Also, fgets() does not delete the trailing "\n", as gets().

So whats the right way to use fgets() when reading a file?

```
while(!feof(inp_file_ptr))
{
  fgets(buffer, MAX_LINE_SIZE, inp_file_ptr);
  fputs(buffer, out_file_ptr);
}
```

The code above will copy the last line twice! This is because, in C, end-of-file is only indicated after an input routine has tried to read, and failed. We should just check the return value of the input routine (in this case, fgets() will return NULL on end-of-file); often, you don't need to use feof() at all.

This is the right way to do it

```
while(fgets(buffer, MAX_LINE_SIZE, inp_file_ptr))
{
  fputs(buffer, out_file_ptr);
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(227 / 301) *

How can I have a variable field width with printf?

Use something like

```
printf("%*d", width, x);
```

Here is a C program...

```
#include <stdio.h>
#include <string.h>

#define WIDTH 5

int main ( void )
{
    char str1[] = "Good Boy";
    char str2[] = "The earth is round";

    int width = strlen ( str1 )  + WIDTH;
    int prec  = strlen ( str2 )  + WIDTH;

    printf ( "%*.*s\n", width, prec, str1 );
    return 0;
}
```

C Functions - built-in
(228 / 301) *

How can I specify a variable width in a scanf() format string?

You cant!.

An asterisk in a scanf() format string means to suppress assignment. You may be able to use ANSI stringizing and string concatenation to accomplish about the same thing, or you can construct the scanf format string at run time.

C Functions - built-in
(229 / 301)

How can I convert numbers to strings (the opposite of atoi)?

Use sprintf()

Note!, since sprintf() is also a variable argument function, it fails badly if passed with wrong arguments or if some of the arguments are missed causing segmentation faults. So be very careful when using sprintf() and pass the right number and type of arguments to it!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Why should one use strncpy() and not strcpy()? What are the problems with strncpy()?

strcpy() is the cause of many bugs. Thats because programmers almost always end up copying more data into a buffer than it can hold. To prevent this problem, strncpy() comes in handy as you can specify the exact number of bytes to be copied.

But there are problems with strncpy() also. strncpy() does not always place a '\0' terminator in the destination string. (Funnily, it pads short strings with multiple \0's, out to the specified length). We must often append a '\0' to the destination string by hand. You can get around the problem by using strncat() instead of strncpy(): if the destination string starts out empty, strncat() does what you probably wanted strncpy() to do. Another possibility is sprintf(dest, "%.*s", n, source). When arbitrary bytes (as opposed to strings) are being copied, memcpy() is usually a more appropriate function to use than strncpy().

How does the function strtok() work?

strtok() is the only standard function available for "tokenizing" strings.

The strtok() function can be used to parse a string into tokens. The first call to strtok() should have the string as its first argument. Subsequent calls should have the first argument set to NULL. Each call returns a pointer to the next token, or NULL when no more tokens are found. If a token ends with a delimiter, this delimiting character is overwritten with a "\0" and a pointer to the next character is saved for the next call to strtok(). The delimiter string delim may be different for each call.

The strtok_r() function works the same as the strtok() function, but instead of using a static buffer it uses a pointer to a user allocated char* pointer. This pointer must be the same while parsing the same string.

An example

```
main()
{
    char str[]="This is a test";
    char *ptr[10];
    char *p;
    int i=1;
    int j;

    p=strtok(str," ");

    if(p!=NULL)
    {
        ptr[0]=p;
        while(1)
        {
            p=strtok(NULL, " ");

            if(p==NULL)break;
            else
            {
               ptr[i]=p;i++;
            }
        }
    }

    for(j=0;j<i;j++)
    {
            printf("\n%s\n", ptr[j]);
    }
}
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(232 / 301)

Why do we get the floating point formats not linked error?

Some compilers leave out certain floating point support if it looks like it will not be needed. In particular, the non-floating-point versions of printf() and scanf() save space by not including code to handle %e, %f, and %g. It happens that Borland's heuristics for determining whether the program uses floating point are insufficient, and the programmer must sometimes insert a dummy call to a floating-point library function (such as sqrt(); any will do) to force loading of floating-point support.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(233 / 301)

Why do some people put void cast before each call to printf()?

printf() returns a value. Some compilers (and specially lint) will warn about return values not used by the program. An explicit cast to (void) is a way of telling lint that you have decided to ignore the return value from this specific function call. It's also common to use void casts on calls to strcpy() and strcat(), since the return value is never used for anything useful.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is assert() and when would I use it?

An assertion is a macro, defined in <assert.h>, for testing assumptions. An assertion is an assumption made by the programmer. If its violated, it would be caught by this macro.

For example

```
int i,j;

for(i=0;i<=10;i++)
{
  j+=5;
  assert(i<5);
}

Runtime error: Abnormal program termination.
assert failed (i<5), <file name>,<line number>
```

If anytime during the execution, i gets a value of 0, then the program would break into the assertion since the assumption that the programmer made was wrong.

C Functions - built-in
(235 / 301)

What do memcpy(), memchr(), memcmp(), memset(), strdup(), strncat(), strcmp(), strncmp(), strcpy(), strncpy(), strlen(), strchr(), strchr(), strpbrk(), strspn(), strcspn(), strtok() do?

Here are the prototypes...

```
void *memchr(const void *s, int c, size_t n);
int memcmp(const void *s1, const void *s2, size_t n);
void *memcpy(void * restrict s1, const void * restrict s2, size_t n);
void *memmove(void *s1, const void *s2, size_t n);
void *memset(void *s, int c, size_t n);
char *strcat(char *restrict s1, const char *restrict s2);
char *strchr(const char *s, int c);
int strcmp(const char *s1, const char *s2);
char *strcpy(char *restrict s1, const char *restrict s2);
size_t strcspn(const char *s1, const char *s2);
char *strdup(const char *s1);
size_t strlen(const char *s);
char *strncat(char *restrict s1, const char *restrict s2, size_t n);
int strncmp(const char *s1, const char *s2, size_t n);
char *strncpy(char *restrict s1, const char *restrict s2, size_t n);
char *strpbrk(const char *s1, const char *s2);
char *strrchr(const char *s, int c);
size_t strspn(const char *s1, const char *s2);
char *strstr(const char *s1, const char *s2);
char *strtok(char *restrict s1, const char *restrict s2);
```

The memchr() function shall locate the first occurrence of c (converted to an unsigned char) in the initial n bytes (each interpreted as unsigned char) of the object pointed to by s. The memchr() function shall return a pointer to the located byte, or a null pointer if the byte does not occur in the object.

The memcmp() function shall compare the first n bytes (each interpreted as unsigned char) of the object pointed to by s1 to the first n bytes of the object pointed to by s2. The sign of a non-zero return value shall be determined by the sign of the difference between the values of the first pair of bytes (both interpreted as type unsigned char) that differ in the objects being compared. The memcmp() function shall return an integer greater than, equal to, or less than 0, if the object pointed to by s1 is greater than, equal to, or less than the object pointed to by s2, respectively.

The memcpy() function shall copy n bytes from the object pointed to by s2 into the object pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined. The memcpy() function returns s1; no value is reserved to indicate an error. The memmove() function shall copy n bytes from the object pointed to by s2 into the object pointed to by s1. Copying takes place as if the n bytes from the object pointed to by s2 are first copied into a temporary of n bytes that does not overlap the objects pointed to by s1 and s2, and then the n bytes from the temporary array are copied into the object pointed to by s1. The memmove() function returns s1; no value is reserved to indicate an error.

The memset() function shall copy c (converted to an unsigned char) into each of the first n bytes of the object pointed to by s. The memset () function returns s; no value is reserved to indicate an error.

The strcat() function shall append a copy of the string pointed to by s2 (including the terminating null byte) to the end of the string pointed to

by s1. The initial byte of s2 overwrites the null byte at the end of s1. If copying takes place between objects that overlap, the behavior is undefined. The strcat() function shall return s1; no return value is reserved to indicate an error.

The strchr() function shall locate the first occurrence of c (converted to a char) in the string pointed to by s. The terminating null byte is considered to be part of the string. Upon completion, strchr() shall return a pointer to the byte, or a null pointer if the byte was not found.

The strcmp() function shall compare the string pointed to by s1 to the string pointed to by s2. The sign of a non-zero return value shall be determined by the sign of the difference between the values of the first pair of bytes (both interpreted as type unsigned char) that differ in the strings being compared. Upon completion, strcmp() shall return an integer greater than, equal to, or less than 0, if the string pointed to by s1 is greater than, equal to, or less than the string pointed to by s2, respectively.

The strcpy() function shall copy the string pointed to by s2 (including the terminating null byte) into the array pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined. The strcpy() function returns s1; no value is reserved to indicate an error.

The strcspn() function shall compute the length (in bytes) of the maximum initial segment of the string pointed to by s1 which consists entirely of bytes not from the string pointed to by s2. The strcspn() function shall return the length of the computed segment of the string pointed to by s1; no return value is reserved to indicate an error.

The strdup() function shall return a pointer to a new string, which is a duplicate of the string pointed to by s1, if memory can be successfully allocated for the new string. The returned pointer can be passed to free(). A null pointer is returned if the new string cannot be created. The function may set errno to ENOMEM if the allocation failed.

The strlen() function shall compute the number of bytes in the string to which s points, not including the terminating null byte. The strlen() function shall return the length of s; no return value shall be reserved to indicate an error.

The strncat() function shall append not more than n bytes (a null byte and bytes that follow it are not appended) from the array pointed to by s2 to the end of the string pointed to by s1. The initial byte of s2 overwrites the null byte at the end of s1. A terminating null byte is always appended to the result. If copying takes place between objects that overlap, the behavior is undefined. The strncat() function shall return s1; no return value shall be reserved to indicate an error.

The strncmp() function shall compare not more than n bytes (bytes that follow a null byte are not compared) from the array pointed to by s1 to the array pointed to by s2. The sign of a non-zero return value is determined by the sign of the difference between the values of the first pair of bytes (both interpreted as type unsigned char) that differ in the strings being compared. Upon successful completion, strncmp() shall return an integer greater than, equal to, or less than 0, if the possibly null-terminated array pointed to by s1 is greater than, equal to, or less than the possibly null-terminated array pointed to by s2 respectively.

The strncpy() function shall copy not more than n bytes (bytes that follow a null byte are not copied) from the array pointed to by s2 to the array pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined. If the array pointed to by s2 is a string that is shorter than n bytes, null bytes shall be appended to the copy in the array pointed to by s1, until n bytes in all are written. The strncpy () function shall return s1; no return value is reserved to indicate an error.

The strpbrk() function shall locate the first occurrence in the string pointed to by s1 of any byte from the string pointed to by s2. Upon successful completion, strpbrk() shall return a pointer to the byte or a null pointer if no byte from s2 occurs in s1.

The strrchr() function shall locate the last occurrence of c (converted to a char) in the string pointed to by s. The terminating null byte is considered to be part of the string. Upon successful completion, strrchr() shall return a pointer to the byte or a null pointer if c does not occur in the string.

The strspn() function shall compute the length (in bytes) of the maximum initial segment of the string pointed to by s1 which consists entirely of bytes from the string pointed to by s2. The strspn() function shall return the computed length; no return value is reserved to indicate an error.

The strstr() function shall locate the first occurrence in the string pointed to by s1 of the sequence of bytes (excluding the terminating null byte) in the string pointed to by s2. Upon successful completion, strstr() shall return a pointer to the located string or a null pointer if the string is not found. If s2 points to a string with zero length, the function shall return s1.

A sequence of calls to strtok() breaks the string pointed to by s1 into a sequence of tokens, each of which is delimited by a byte from the string pointed to by s2. The first call in the sequence has s1 as its first argument, and is followed by calls with a null pointer as their first argument. The separator string pointed to by s2 may be different from call to call. The first call in the sequence searches the string pointed to by s1 for the first byte that is not contained in the current separator string pointed to by s2. If no such byte is found, then there are no tokens in the string pointed to by s1 and strtok() shall return a null pointer. If such a byte is found, it is the start of the first token. The strtok() function then searches from there for a byte that is contained in the current separator string. If no such byte is found, the current token extends to the end of the string pointed to by s1, and subsequent searches for a token shall return a null pointer. If such a byte is found, it is overwritten by a null byte, which terminates the current token. The strtok() function saves a pointer to the following byte, from which the next search for a token shall start. Each subsequent call, with a null pointer as the value of the first argument, starts searching from the saved pointer and behaves as described above. Upon successful completion, strtok() shall return a pointer to the first byte of a token. Otherwise, if there is no token, strtok() shall return a null pointer.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(236 / 301)

What does alloca() do?

alloca() allocates memory which is automatically freed when the function which called alloca() returns. However, note that alloca() is not portable and its usage is not recommended.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(237 / 301)

Can you compare two strings like string1==string2? Why do we need strcmp()?

Do you think this will work?

```
if(string1 == string2)
{

}
```

No!, strings in C cannot be compared like that!.

The == operator will end up comparing two pointers (that is, if they have the same address). It wont compare the contents of those locations. In C, strings are represented as arrays of characters, and the language never manipulates (assigns, compares, etc.) arrays as a whole.

The correct way to compare strings is to use strcmp()

```
if(strcmp(string1, string2) == 0)
{

}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What does printf() return?

Upon a successful return, the printf() function returns the number of characters printed (not including the trailing '\0' used to end output to strings). If the output was truncated due to this limit then the return value is the number of characters (not including the trailing '\0') which would have been written to the final string if enough space had been available. Thus, a return value of size or more means that the output was truncated. If an output error is encountered, a negative value is returned.

There is a very funny yet interesting interview question around this....

Look at the code below.

```
void main()
{
   if(X)
  {
     printf("Hello");
  }
  else
  {
     printf(" World");
  }
}
```

What should X be replaced with inorder to get the output as "Hello World"?

And here comes the answer....

```
#include <stdio.h>
int main()
{
    if(!printf("Hello"))
    {
        printf("Hello");
    }
    else
    {
        printf(" World");
    }
}
```

Kind of stupid isn't it? But they do ask these type of questions. Believe me!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - built-in
(239 / 301)

What do setjmp() and longjump() functions do?

A goto statement implements a local jump of program execution, and the longjmp() and setjmp() functions implement a nonlocal, or far, jump of program execution. Generally, a jump in execution of any kind should be avoided because it is not considered good programming practice to use such statements as goto and longjmp in your program. A goto statement simply bypasses code in your program and jumps to a predefined position. To use the goto statement, you give it a labeled position to jump to. This predefined position must be within the same function. You cannot implement gotos between functions.

When your program calls setjmp(), the current state of your program is saved in a structure of type jmp_buf. Later, your program can call the longjmp() function to restore the program?s state as it was when you called setjmp().Unlike the goto statement, the longjmp() and setjmp() functions do not need to be implemented in the same function. However, there is a major drawback to using these functions: your program, when restored to its previously saved state, will lose its references to any dynamically allocated memory between the longjmp() and the setjmp(). This means you will waste memory for every malloc() or calloc() you have implemented between your longjmp() and setjmp(), and your program will be horribly inefficient. It is highly recommended that you avoid using functions such as longjmp() and setjmp() because they, like the goto statement, are quite often an indication of poor programming practice.

It is not possible to jump from one function to another by means of a goto and a label, since labels have only function scope. However, the macro setjmp and function longjmp provide an alternative, known as a non-local goto, or a non-local jump. The header file <setjmp.h> declares something called a jmp_buf, which is used by the cooperating macro and function to store the information necessary to make the jump. The declarations are as follows:

```
#include <setjmp.h>
int setjmp(jmp_buf env);
void longjmp(jmp_buf env, int val);
```

The setjmp macro is used to initialise the jmp_buf and returns zero on its initial call. Then, it returns again, later, with a non-zero value, when the corresponding longjmp call is made! The non-zero value is whatever value was supplied to the call of longjmp.

This is best explained by way of an example:

```
#include <stdio.h>
#include <stdlib.h>
#include <setjmp.h>

void func(void);
jmp_buf place;

main()
{
   int retval;

    /*
     * First call returns 0,
     * a later longjmp will return non-zero.
     */
```

```
     if(setjmp(place)) != 0)
     {
        printf("Returned using longjmp\n");
        exit(EXIT_SUCCESS);
     }

     /*
      * This call will never return - it
      * 'jumps' back above.
      */

     func();
     printf("What! func returned!\n");
}

void func(void)
{
     /*
      * Return to main.
      * Looks like a second return from setjmp,
      * returning 4!
      */

     longjmp(place, 4);
     printf("What! longjmp returned!\n");
}
```

The val argument to longjmp is the value seen in the second and subsequent ?returns? from setjmp. It should normally be something other than 0; if you attempt to return 0 via longjmp, it will be changed to 1. It is therefore possible to tell whether the setjmp was called directly, or whether it was reached by calling longjmp. If there has been no call to setjmp before calling longjmp, the effect of longjmp is undefined, almost certainly causing the program to crash. The longjmp function is never expected to return, in the normal sense, to the instructions immediately following the call. All accessible objects on ?return? from setjmp have the values that they had when longjmp was called, except for objects of automatic storage class that do not have volatile type; if they have been changed between the setjmp and longjmp calls, their values are indeterminate.

The longjmp function executes correctly in the contexts of interrupts, signals and any of their associated functions. If longjmp is invoked from a function called as a result of a signal arriving while handling another signal, the behaviour is undefined.

It's a serious error to longjmp to a function which is no longer active (i.e. it has already returned or another longjump call has transferred to a setjmp occurring earlier in a set of nested calls).

The Standard insists that, apart from appearing as the only expression in an expression statement, setjmp may only be used as the entire controlling expression in an if, switch, do, while, or for statement. A slight extension to that rule is that as long as it is the whole controlling expression (as above) the setjmp call may be the subject of the ! operator, or may be directly compared with an integral constant expression using one of the relational or equality operators. No more complex expressions may be employed.

Examples are:

```
setjmp(place);                    /* expression statement */
if(setjmp(place)) ...             /* whole controlling expression */
if(!setjmp(place)) ...            /* whole controlling expression */
if(setjmp(place) < 4) ...         /* whole controlling expression */
if(setjmp(place)<;4 && 1!=2) ...  /* forbidden */
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - The main function
(240 / 301) *

Whats the prototype of main()? Can main() return a structure?

The right declaration of main() is

```
int main(void)
```

or

```
int main(int argc, char *argv[])
```

or

```
int main(int argc, char *argv[], char *env[]) //Compiler dependent, non-standard C.
```

In C, main() cannot return anything other than an int.Something like

```
void main()
```

is illegal. There are only three valid return values from main() - 0, EXIT_SUCCESS, and EXIT_FAILURE, the latter two are defined in <stdlib.h>.

Something like this can cause unpredicted behavior

```
struct mystruct {
  int value;
  struct mystruct *next;
}

main(argc, argv)
{ ... }
```

Here the missing semicolon after the structure declaration causes main to be misdeclared.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - The main function
(241 / 301) *

Is exit(status) equivalent to returning the same status from main()?

No.

The two forms are not equivalent in a recursive call to main().

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - The main function
(242 / 301)

Can main() be called recursively?

Yes

```
main()
{
    main();
}
```

But this will go on till a point where you get a

```
Runtime error : Stack overflow.
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

C Functions - The main function
(243 / 301)

How to print the arguments recieved by main()?

Here is some code

```
main(int argc, char*argv[])
{
  int i;
  for(i=0; i<argc; i++)
  {
    printf("\n[%s]\n", argv[i]);
  }
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What do the system calls fork(), vfork(), exec(), wait(), waitpid() do? Whats a Zombie process? Whats the difference between fork() and vfork()?

The system call fork() is used to create new processes. It does not take any arguments and returns a process ID. The purpose of fork() is to create a new process, which becomes the child process of the caller (which is called the parent). After a new child process is created, both processes will execute the next instruction following the fork() system call. We can distinguish the parent from the child by testing the returned value of fork():

If fork() returns a negative value, the creation of a child process was unsuccessful. A call to fork() returns a zero to the newly created child process and the same call to fork() returns a positive value (the process ID of the child process) to the parent. The returned process ID is of type pid_t defined in sys/types.h. Normally, the process ID is an integer. Moreover, a process can use function getpid() to retrieve the process ID assigned to this process. Unix will make an exact copy of the parent's address space and give it to the child. Therefore, the parent and child processes will have separate address spaces. Both processes start their execution right after the system call fork(). Since both processes have identical but separate address spaces, those variables initialized before the fork() call have the same values in both address spaces. Since every process has its own address space, any modifications will be independent of the others. In other words, if the parent changes the value of its variable, the modification will only affect the variable in the parent process's address space. Other address spaces created by fork() calls will not be affected even though they have identical variable names.

Trick question!

How many processes are forked by the below program

```
main()
{
  fork();
  fork();
}
```

Answer is 2 raise to n, when n is the number of calls to fork (2 in this case).

Each process has certain information associated with it including:

```
The UID (numeric user identity)
The GID (numeric group identity)
A process ID number used to identify the process
A parent process ID
The execution status, e.g. active, runnable, waiting for input etc.
Environment variables and values.
The current directory.
Where the process currently resides in memory
The relative process priority see nice(1)
Where it gets standard input from
Where it sends standard output to
Any other files currently open
```

Certain process resources are unique to the individual process. A few among these are:

- Stack Space: which is where local variables, function calls, etc. are stored.
- Environment Space: which is used for storage of specific environment variables.
- Program Pointer (counter) : PC.
- File Descriptors
- Variables

Undex unix, each sub directory under /proc corresponds to a running process (PID #). A ps provide detailed information about processes running

A typical output of ps looks as follows:

```
PID               TTY            STAT           TIME         COMMAND
-------------------------------------------------------------------
8811              3              SW             0:00         (login)
3466              3              SW             0:00         (bash)
8777              3              SW             0:00         (startx)
   .              .               .               .          .
1262              p7              R             0:00         ps
```

The columns refer to the following:

```
PID      - The process id's (PID).
TTY      - The terminal the process was started from.
STAT     - The current status of all the processes. Info about the process status
           can be broken into more than 1 field. The first of these fields can
           contain the following entries:

            R        - Runnable.
            S        - Sleeping.
            D        - Un-interuptable sleep.
            T        - Stopped or Traced.
            Z        - Zombie Process.

The second field can contain the following entry:

            W       - If the process has no residual pages.

And the third field:

            N       - If the process has a positive nice value.
            TIME    - The CPU time used by the process so far.
            COMMAND - The actual command.
```

The init process is the very first process run upon startup. It starts additional processes. When it runs it reads a file called /etc/inittab which specifies init how to set up the system, what processes it should start with respect to specific runlevels. One crucial process which it starts is the getty program. A getty process is usually started for each terminal upon which a user can log into the system. The getty program produces the login: prompt on each terminal and then waits for activity. Once a getty process detects activity (at a user attempts to log in to the system), the getty program passes control over to the login program.

There are two command to set a process's priority nice and renice.

One can start a process using the system() function call

```
#include <stdio.h>
#include <stdlib.h>

int main()
{

  printf("Running ls.....\n");
  system("ls -lrt");
  printf("Done.\n");
  exit(0);
}
```

The exec() system call

The exec() functions replace a current process with another created according to the arguments given.

The syntax of these functions is as follows:

```
#include <unistd.h>

char *env[];

int  execl(const char *path, const char *arg0, ..., (char *)0);
int  execv(const char *path, const char *argv[]);

int execlp(const char *path, const char *arg0, ..., (char *)0);
int execvp(const char *path, const char *argv[]);

int execle(const char *path, const char *arg0, ... , (char *)0, const char *env[]);
int execve(const char *path, const char *argv[], const char *env[]);
```

The program given by the path argument is used as the program to execute in place of what is currently running. In the case of the execl() the new program is passed arguments arg0, arg1, arg2,... up to a null pointer. By convention, the first argument supplied (i.e. arg0) should point to the file name of the file being executed. In the case of the execv() programs the arguments can be given in the form of a pointer to an array of strings, i.e. the argv array. The new program starts with the given arguments appearing in the argv array passed to main. Again, by convention, the first argument listed should point to the file name of the file being executed. The function name suffixed with a p (execlp() and execvp())differ in that they will search the PATH environment variable to find the new program executable file. If the executable is not on the path, and absolute file name, including directories, will need to be passed to the function as a parameter. The global variable environ is available to pass a value for the new program environment. In addition, an additional argument to the exec() functions execle() and execve() is available for passing an array of strings to be used as the new program environment.

Examples to run the ls command using exec are:

```
const char *argv[] = ("ls", "-lrt", 0);
const char *env[] = {"PATH=/bin:/usr/bin", "TERM=console", 0};

execl("/bin/ls", "ls", "-lrt", 0);
execv("/bin/ls", argv);

execlp("ls", "ls", "-lrt", 0);
execle("/bin/ls", "ls", "-lrt", 0, env);

execvp("ls", argv);
execve("/bin/ls", argv, env);
```

A simple call to fork() would be something like this

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main()
{
  pid_t pid;
  pid=fork();
  switch(pid)
  {
     case -1: exit(1); // fork() error.
     case  0: // Child process, can call exec here.
              break;
     default: // Parent.
              break;
  }
  exit(0);
}
```

The call wait() can be used to determine when a child process has completed it's job and finished. We can arrange for the parent process to wait untill the child finishes before continuing by calling wait(). wait() causes a parent process to pause untill one of the child processes dies or is stopped. The call returns the PID of the child process for which status information is available. This will usually be a child process which has terminated. The status information allows the parent process to determine the exit status of the child process, the value returned from main or passed to exit. If it is not a null pointer the status information will be written to the location pointed to by stat_loc. We can interrogate the status information using macros defined in sys/wait.h.

```
Macro                  Definition
-------------------------------------------------------------------------------
```

```
WIFEXITED(stat_val);    Nonzero if the child is terminated normally
WEXITSTATUS(stat_val);  If WIFEXITED is nonzero, this returns child exit code.
WIFSIGNALLED(stat_val); Nonzero if the child is terminated on an uncaught signal.
WTERMSIG(stat_val);     If WIFSIGNALLED is nonzero, this returns a signal number.
WIFSTOPPED(stat_val);   Nonzero if the child stopped on a signal.
WSTOPSIG(stat_val);     If WIFSTOPPED is nonzero, this returns a signal number.
```

An example code which used wait() is shown below

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void){
    pid_t child_pid;
    int *status=NULL;
    if(fork()){
        /* wait for child, getting  PID */
        child_pid=wait(status);
        printf("I'm the parent.\n");
        printf("My child's PID was: %d\n",child_pid);
    } else {
        printf("I'm the child.\n");
    }
    return 0;
}
```

Or a more detailed program

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

int main()
{

  pid_t pid;
  int exit_code;

  pid = fork();
  switch(pid)
  {
    case -1: exit(1);
    case 0:  exit_code = 11; //Set the child exit process
             break;
    default: exit_code = 0;
             break;
  }

  if (pid)
  {
     // This is the parent process
     int status;
     pid_t child_pid;

     child_pid = wait(&status);

     printf("Child process finished with PID [%d]\n", child_pid);
     if (WIFEXITED(status))
     {
       printf("Child exited with code [%d]\n", WEXITSTATUS(status));
     }
     else
     {
       printf("Child terminated abnormally.\n");
     }
   }

   exit(exit_code);
}
```

So now, whats a Zombie process?

When using fork() to create child processes it is important to keep track of these processes. For instance, when a child process terminates, an association with the parent survives untill the parent either terminates normally or calls wait(). The child process entry in the process table is not freed up immediately. Although it is no longer active, the child process is still in the system because it's exit code needs to be stored in the even the parent process calls wait(). The child process is at that point referred to as a zombie process. Note, if the parent terminates abnormally then the child process gets the process with PID 1, (init) as parent. (such a child process is often referred to as an orphan). The child process is now a zombie. It is no longer running, it's origional parent process is gone, and it has been inherited by init. It will remain in the process table as a zombie untill the next time the table is processed. If the process table is long this may take a while. till init cleans them up. As a general rule, program wisely and try to avoid zombie processes. When zobbies accumulate they eat up valuable resources.

The waitpid() system call is another call that can be used to wait for child processes. This system call however can be used to wait for a specific process to terminate.

```
#include <sys/types.h>
#include <sys/wait.h>
pit_t waitpid(pid_t pid, int *status, int options);
```

The pid argument specifies the PID of the particular child process to wait for. If it is a -1 then waitpid() will return information to the child process. Status information will be written to the location pointed to by status. The options argument enables us to change the behavior of waitpid(). A very usefull oprion is WNOHANG which prevents the call to waitpid() from suspending the execution of the caller. We can it to find out whether any child process has terminated and, if not, to continue.

Synchronous and asynchronous process execution

In some cases, for example if the child process is a server or "daemon" (a process expected to run all the time in the background to deliver services such as mail forwarding) the parent process would not wait for the child to finish. In other cases, e.g. running an interactive command where it is not good design for the parent's and child's output to be mixed up into the same output stream, the parent process, e.g. a shell program, would normally wait for the child to exit before continuing. If you run a shell command with an ampersand as it's last argument, e.g. sleep 60 & the parent shell doesn't wait for this child process to finish.

So whats the difference between fork() and vfork()?

The system call vfork(), is a low overhead version of fork(), as fork() involves copying the entire address space of the process and is therefore quite expensive. The basic difference between the two is that when a new process is created with vfork(), the parent process is temporarily suspended, and the child process might borrow the parent's address space. This strange state of affairs continues until the child process either exits, or calls execve(), at which point the parent process continues. This means that the child process of a vfork() must be careful to avoid unexpectedly modifying variables of the parent process. In particular, the child process must not return from the function containing the vfork() call, and it must not call exit() (if it needs to exit, it should use _exit(); actually, this is also true for the child of a normal fork()).

However, since vfork() was created, the implementation of fork() has improved , most notably with the introduction of `copy-on-write', where the copying of the process address space is transparently faked by allowing both processes to refer to the same physical memory until either of them modify it. This largely removes the justification for vfork(); indeed, a large proportion of systems now lack the original functionality of vfork() completely. For compatibility, though, there may still be a vfork() call present, that simply calls fork() without attempting to emulate all of the vfork() semantics.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(245 / 301)

How does freopen() work?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(246 / 301)

What are threads? What is a lightweight process? What is a heavyweight process? How different is a thread from a process?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(247 / 301)

How are signals handled?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(248 / 301)

What is a deadlock?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(249 / 301)

What are semaphores?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(250 / 301) *

What is meant by context switching in an OS?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(251 / 301)

What is Belady's anomaly?

Usually, on increasing the number of frames allocated to a process' virtual memory, the process execution is faster, because fewer page faults occur. Sometimes, the reverse happens, i.e., the execution time increases even when more frames are allocated to the process. This is known as the Belady's Anomaly. This is true for certain page reference patterns. This is also called the FIFO anomaly.

OS Concepts
(252 / 301)

What is thrashing?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What are short-, long- and medium-term scheduling?

Long term scheduler determines which programs are admitted to the system for processing. It controls the degree of multiprogramming. Once admitted, a job becomes a process.

Medium term scheduling is part of the swapping function. This relates to processes that are in a blocked or suspended state. They are swapped out of main-memory until they are ready to execute. The swapping-in decision is based on memory-management criteria.

Short term scheduler, also know as a dispatcher executes most frequently, and makes the finest-grained decision of which process should execute next. This scheduler is invoked whenever an event occurs. It may lead to interruption of one process by preemption.

OS Concepts
(254 / 301)

What are turnaround time and response time?

Turnaround time is the interval between the submission of a job and its completion. Response time is the interval between submission of a request, and the first response to that request.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is the Translation Lookaside Buffer (TLB)?

In a cached system, the base addresses of the last few referenced pages is maintained in registers called the TLB that aids in faster lookup. TLB contains those page-table entries that have been most recently used. Normally, each virtual memory reference causes 2 physical memory accesses-- one to fetch appropriate page-table entry, and one to fetch the desired data. Using TLB in-between, this is reduced to just one physical memory access in cases of TLB-hit.

What is cycle stealing?

We encounter cycle stealing in the context of Direct Memory Access (DMA). Either the DMA controller can use the data bus when the CPU does not need it, or it may force the CPU to temporarily suspend operation. The latter technique is called cycle stealing. Note that cycle stealing can be done only at specific break points in an instruction cycle.

What is a reentrant program?

Re-entrancy is a useful, memory-saving technique for multiprogrammed timesharing systems. A Reentrant Procedure is one in which multiple users can share a single copy of a program during the same period.

Reentrancy has 2 key aspects:

- The program code cannot modify itself.
- The local data for each user process must be stored separately.

Thus, the permanent part is the code, and the temporary part is the pointer back to the calling program and local variables used by that program. Each execution instance is called activation. It executes the code in the permanent part, but has its own copy of local variables/ parameters. The temporary part associated with each activation is the activation record. Generally, the activation record is kept on the stack. Note: A reentrant procedure can be interrupted and called by an interrupting program, and still execute correctly on returning to the procedure.

When is a system in safe state?

The set of dispatchable processes is in a safe state if there exist at least one temporal order in which all processes can be run to completion without resulting in a deadlock.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(259 / 301)

What is busy waiting?

The repeated execution of a loop of code while waiting for an event to occur is called busy-waiting. The CPU is not engaged in any real productive activity during this period, and the process does not progress toward completion

OS Concepts
(260 / 301)

What is pages replacement? What are local and global page replacements?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(261 / 301)

What is meant by latency, transfer and seek time with respect to disk I/O?

Seek time is the time required to move the disk arm to the required track. Rotational delay or latency is the time to move the required sector to the disk head. Sums of seek time (if any) and the latency is the access time, for accessing a particular track in a particular sector. Time taken to actually transfer a span of data is transfer time.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(262 / 301)

What are monitors? How are they different from semaphores?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(263 / 301)

In the context of memory management, what are placement and replacement algorithms?

Placement algorithms determine where in the available main-memory to load the incoming process. Common methods are first-fit, next-fit, and best-fit.

Replacement algorithms are used when memory is full, and one process (or part of a process) needs to be swapped out to accommodate the new incoming process. The replacement algorithm determines which are the partitions (memory portions occupied by the processes) to be swapped out.

OS Concepts
(264 / 301)

What is paging? What are demand- and pre-paging?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is mounting?

Mounting is the mechanism by which two different file systems can be combined together. This is one of the services provided by the operating system, which allows the user to work with two different file systems, and some of the secondary devices.

What do you mean by dispatch latency?

The time taken by the dispatcher to stop one process and start running another process is known as the dispatch latency.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OS Concepts
(267 / 301)

What is multi-processing? What is multi-tasking? What is multi-threading? What is multi-programming?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is compaction?

Compaction refers to the mechanism of shuffling the memory portions such that all the free portions of the memory can be aligned (or merged) together in a single large block. OS to overcome the problem of fragmentation, either internal or external, performs this mechanism, frequently. Compaction is possible only if relocation is dynamic and done at run-time, and if relocation is static and done at assembly or load-time compaction is not possible.

What is memory-mapped I/O? How is it different frim I/O mapped I/O?

Memory-mapped I/O, meaning that the communication between the I/O devices and the processor is done through physical memory locations in the address space. Each I/O device will occupy some locations in the I/O address space. I.e., it will respond when those addresses are placed on the bus. The processor can write those locations to send commands and information to the I/O device and read those locations to get information and status from the I/O device. Memory-mapped I/O makes it easy to write device drivers in a high-level language as long as the high-level language can load and store from arbitrary addresses.

OS Concepts
(270 / 301)

List out some reasons for process termination.

- Normal completion
- Time limit exceeded
- Memory unavailable
- Bounds violation
- Protection error
- Arithmetic error
- Time overrun
- I/O failure
- Invalid instruction
- Privileged instruction
- Data misuse
- Operator or OS intervention
- Parent termination.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

General Concepts
(271 / 301) *

What is the difference between statically linked libraries and dynamically linked libraries (dll)?

Static linking means all the referenced code is included in the binary produced. When linking statically, the linker finds the bits that the program modules need, and physically copies them into the executable output file that it generates.

Incase of dynamic linking, the binary simply contains a pointer to the needed routine, which will be loaded by the OS as needed. An important benefit is that libraries can be updated to fix bugs or security problems and the binaries that use them are immediately using the fixed code. It also saves disk space, saves memory usage, saves launch time, improve multi-tasking efficiency. But, deployment of dynamically-linked executable generally requires coordination across hosts of shared-object libraries, installation locations, installation environment settings, settings for compile-, link-, and use-time environment variables Dynamic linking often precludes relocations backward across OS versions.

On Linux, static libraries have names like libname.a, while shared libraries are called libname.so.x.y.z where x.y.z is some form of version number. Shared libraries often also have links pointing to them, which are important, and (on a.out configurations) associated .sa files. The standard libraries come in both shared and static formats. You can find out what shared libraries a program requires by using ldd (List Dynamic Dependencies)

Dynamic linking allows an exe or dll to use required information at run time to call a DLL function. In static linking, the linker gets all the referenced functions from the static link library and places it with your code into your executable. Using DLLs instead of static link libraries makes the size of the executable file smaller. Dynamic linking is faster than static linking.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What are the most common causes of bugs in C?

Murphy's law states that "If something can go wrong, it will. So is the case with programming. Here are a few things than can go wrong when coding in C.

- Typing if(x=0) instead of if(x==0). Use if(0==x) instead.
- Using strcpy() instead of strncpy() and copying more memory than the buffer can hold.
- Dereferencing null pointers.
- Improper malloc(), free() usage. Assuming malloc'ed memory contains 0, assuming freed storage persists even after freeing it, freeing something twice, corrupting the malloc() data structures.
- Uninitialized local variables.
- Integer overflow.
- Mismatch between printf() format and arguments, especially trying to print long ints using %d. Problems with wrong arguments to sprintf() leading to core dumps.
- Array out of bounds problems, especially of small, temporary buffers.
- Leaving files opened.
- Undefined evaluation order, undefined statements.
- Omitted declaration of external functions, especially those which return something other than int, or have "narrow" or variable arguments.
- Floating point problems.
- Missing function prototypes.

Ofcourse, there is the mother of all things that can go wrong - Your logic!.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is hashing?

Hashing is the process of mapping strings to integers, in a relatively small range.

A hash function maps a string (or some other data structure) to a bounded number (called the hash bucket) which can more easily be used as an index in an array, or for performing repeated comparisons.

A mapping from a potentially huge set of strings to a small set of integers will not be unique. Any algorithm using hashing therefore has to deal with the possibility of collisions.

Here are some common methods used to create hashing functions

```
Direct method
Subtraction method
Modulo-Division method
Digit-Extraction method
Mid-Square method
Folding method
Pseudo-random method
```

Here is a simple implementation of a hashing scheme in C

```
#include <stdio.h>
#define HASHSIZE 197

typedef struct node
{
  int value;
  struct node *next;
}mynode;


typedef struct hashtable
{
  mynode *llist;
}HT;


HT myHashTable[HASHSIZE];


int main()
{
  initialize();
  add(2);
  add(2500);
  add(199);
```

```
  display_hash();
  getch();
  return(0);
}

int initialize()
{
  int i;
  for(i=0;i<HASHSIZE;i++)
     myHashTable[i].llist=NULL;
  return(1);
}

int add(int value)
{
  int hashkey;
  int i;
  mynode *tempnode1, *tempnode2, *tempnode3;

  hashkey=value%HASHSIZE;
  printf("\nHashkey : [%d]\n", hashkey);

  if(myHashTable[hashkey].llist==NULL)
   {
      //This hash bucket is empty, add the first element!
      tempnode1 = malloc(sizeof(mynode));
      tempnode1->value=value;
      tempnode1->next=NULL;
      myHashTable[hashkey].llist=tempnode1;
   }
   else
   {
      //This hash bucket already has some items. Add to it at the end.
      //Check if this element is already there?

      for(tempnode1=myHashTable[hashkey].llist;
          tempnode1!=NULL;
          tempnode3=tmpnode1,tempnode1=tempnode1->next)
      {
         if(tempnode1->value==value)
         {
            printf("\nThis value [%d] already exists in the Hash!\n", value);
            return(1);
         }
    }

    tempnode2 = malloc(sizeof(mynode));
    tempnode2->value = value;
    tempnode2->next=NULL;
    tempnode3->next=tempnode2;
  }
  return(1);
}


int display_hash()
{
  int i;
  mynode *tempnode;

  for(i=0;i<HASHSIZE;i++)
  {
     if(myHashTable[i].llist==NULL)
     {
        printf("\nmyHashTable[%d].llist -> (empty)\n",i);
     }
     else
     {
        printf("\nmyHashTable[%d].llist -> ",i);
        for(tempnode=myHashTable[i].llist;tempnode!=NULL;tempnode=tempnode->next)
        {
           printf("[%d] -> ",tempnode->value);
        }
        printf("(end)\n");
      }
      if(i%20==0)getch();
  }
  return(0);
```

```
}
```

And here is the output

```
Hashkey : [2]
Hashkey : [136]
Hashkey : [2]
Hashkey : [2]
This value [2] already exists in the Hash!
myHashTable[0].llist -> (empty)
myHashTable[1].llist -> (empty)
myHashTable[2].llist -> [2] -> [199] -> (end)
myHashTable[3].llist -> (empty)
myHashTable[4].llist -> (empty)
myHashTable[5].llist -> (empty)
myHashTable[6].llist -> (empty)
myHashTable[7].llist -> (empty)
myHashTable[8].llist -> (empty)
myHashTable[9].llist -> (empty)
myHashTable[10].llist -> (empty)
myHashTable[11].llist -> (empty)
myHashTable[12].llist -> (empty)
myHashTable[13].llist -> (empty)
myHashTable[14].llist -> (empty)
myHashTable[15].llist -> (empty)
myHashTable[16].llist -> (empty)
myHashTable[17].llist -> (empty)
myHashTable[18].llist -> (empty)
myHashTable[19].llist -> (empty)
myHashTable[20].llist -> (empty)
...
myHashTable[133].llist -> (empty)
myHashTable[134].llist -> (empty)
myHashTable[135].llist -> (empty)
myHashTable[136].llist -> [2500] -> (end)
myHashTable[137].llist -> (empty)
...
myHashTable[196].llist -> (empty)
```

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What do you mean by Predefined?

It means already written, compiled and linked together with our program at the time of linking.

General Concepts
(275 / 301) *

What is data structure?

A data structure is a way of organizing data that considers not only the items stored, but also their relationship to each other. Advance knowledge about the relationship between data items allows designing of efficient algorithms for the manipulation of data.

General Concepts
(276 / 301) *

What are the time complexities of some famous algorithms?

Here you are

```
Binary search                                :    O(logn)
Finding max & min for a given set of numbers :    O(3n/2-2)
Merge Sort                                   :    O(nlogn)
Insertion Sort                               :    O(n*n)
Quick Sort                                   :    O(nlogn)
Selection Sort                               :    O(n*n)
```

General Concepts
(277 / 301)

What is row major and column major form of storage in matrices?

This is row major representation

If this is your matrix

```
a b c d
e f g h
i j k l
```

Convert into 1D array by collecting elements by rows. Within a row elements are collected from left to right.
Rows are collected from top to bottom. So, `x[i][j]` is mapped to position `i*no_of_columns + j`.

This is column major representation

On similar lines, in column major representation, we store elements column wise. Here, `x[i][j]` is mapped to position `i + j * no_of_rows`.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

General Concepts
(278 / 301) *

Explain the BigOh notation.

Time complexity

The Big Oh for the function f(n) states that

```
f(n)=O(g(n)); iff there exist positive constants c and d such that: f(n) <=c*g(n) for all n,n>=d.
```

Here are some examples...

```
1       ->Constant
logn    ->Logarithmic
n       ->Linear
nlogn   ->nlogn
n*n     ->quadratic
n*n*n   ->cubic
2^n     ->exponential
n!      ->Factorial
```

We also have a few other notations...

Omega notation

```
f(n)=Omega(g(n)), iff positive constants c and d exist such as f(n)>=cg(n) for all n, n>=d.
```

Theta notation

```
f(n)=Theta(g(n)), iff positive constants c1 and c2 and an d exists such that c1g(n)<=f(n)<=c2g
(n) for all n, n>=d.
```

Little Oh(o)

```
f(n)=og(n), iff f(n)=Og(n) and f(n)!=Omega(g(n)).
```

Then there is also something called Space complexity...

Fixed part : This is the independent of the instance characteristics. This typically includes the instruction space (i.e. space for the code), space for simple variables and fixed-size component variables, space for constants, and so on.

Variable part: A variable part that consists of the space needed by the component variables whose size depends on the particular problem instance being solved, dynamically allocated space (to the extent that this space depends on the instance characteristics); and the recursion stack space (in so far as this space depends on the instance characteristics).

A complete description of these concepts is out of the scope of this website. There are plenty of books and millions of pages on the Internet. Help yourself!

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Give the most important types of algorithms.

There are five important basic designs for algorithms.

They are:

- Divide and conquer : For a function to compute on n inputs the divide and conquer strategy suggests the inputs into a k distinct subsets, 1<k<=n, yielding k sub-problems. These sub-problems must be solved and then a method must be found to combine the sub-solutions into a solution of the whole. An example for this approach is "binary search" algorithm. The time complexity of binary search algorithm is O(log n).
- The greedy method : The greedy method suggests that one can devise an algorithm that works in stages, considering one input at a time. At each stage, a decision is made regarding whether a particular input is an optimal solution. An example for solution using greedy method is "knapsack problem". Greedy algorithms attempt not only to find a solution, but to find the ideal solution to any given problem.
- Dynamic programming : Dynamic Programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions. An example for algorithm using dynamic programming is "multistage graphs". This class remembers older results and attempts to use this to speed the process of finding new results.
- Back-tracking : Here if we reach a dead-end, we use the stack to pop-off the dead end and try something else we had not tried before. The famous 8-queens problem uses back tracking. Backtracking algorithms test for a solution, if one is found the algorithm has solved, if not it recurs once and tests again, continuing until a solution is found.
- Branch and bound : Branch and bound algorithms form a tree of subproblems to the primary problem, following each branch until it is either solved or lumped in with another branch.

What is marshalling and demarshalling?

When objects in memory are to be passed across a network to another host or persisted to storage, their in-memory representation must be converted to a suitable out-of-memory format. This process is called marshalling, and converting back to an in memory representation is called demarshalling. During marshalling, the objects must be respresented with enough information that the destination host can understand the type of object being created. The objects? state data must be converted to the appropriate format. Complex object trees that refer to each other via object references (or pointers) need to refer to each other via some form of ID that is independent of any memory model. During demarshalling, the destination host must reverse all that and must also validate that the objects it receives are consistent with the expected object type (i.e. it validate that it doesn?t get a string where it expects a number).

General Concepts
(281 / 301)

What is the difference between the stack and the heap? Where are the different types of variables of a program stored in memory?

When a program is loaded into memory, it is organized into three areas of memory, called segments: the text segment, stack segment, and the heap segment. The text segment (sometimes also called the code segment) is where the compiled code of the program itself resides. This is the machine language representation of the program steps to be carried out, including all functions making up the program, both user defined and system.

The remaining two areas of system memory is where storage may be allocated by the compiler for data storage. The stack is where memory is allocated for automatic variables within functions. A stack is a Last In First Out (LIFO) storage device where new storage is allocated and deallocated at only one "end", called the Top of the stack. When a program begins executing in the function main(), space is allocated on the stack for all variables declared within main(). If main() calls a function, func(), additional storage is allocated for the variables in func() at the top of the stack. Notice that the parameters passed by main() to func() are also stored on the stack. If func() were to call any additional functions, storage would be allocated at the new Top of stack. When func() returns, storage for its local variables is deallocated, and the Top of the stack returns to its old position. If main() were to call another function, storage would be allocated for that function at the Top. The memory allocated in the stack area is used and reused during program execution. It should be clear that memory allocated in this area will contain garbage values left over from previous usage.

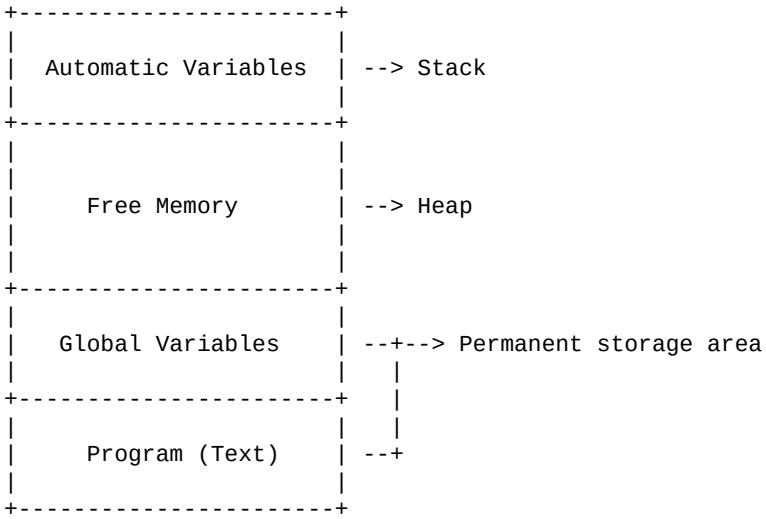
The heap segment provides more stable storage of data for a program; memory allocated in the heap remains in existence for the duration of a program. Therefore, global variables (storage class external), and static variables are allocated on the heap. The memory allocated in the heap area, if initialized to zero at program start, remains zero until the program makes use of it. Thus, the heap area need not contain garbage.

Describe the memory map of a C program.

This is a quite popular question. But I have never understood what exactly is the purpose of asking this question. The memory map of a C program depends heavily on the compiler used.

Nevertheless, here is a simple explanation..

During the execution of a C program, the program is loaded into main memory. This area is called permanent storage area. The memory for Global variables and static variables is also allocated in the permanent storage area. All the automatic variables are stored in another area called the stack. The free memory area available between these two is called the heap. This is the memory region available for dynamic memory allocation during the execution of a program.



Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

What is infix, prefix, postfix? How can you convert from one representation to another? How do you evaluate these expressions?

There are three different ways in which an expression like a+b can be represented.

Prefix (Polish)

```
+ab
```

Postfix (Suffix or reverse polish)

```
ab+
```

Infix

```
a+b
```

Note than an infix expression can have parathesis, but postfix and prefix expressions are paranthesis free expressions.

Conversion from infix to postfix

Suppose this is the infix expresssion

```
((A + (B - C) * D) ^ E + F)
```

To convert it to postfix, we add an extra special value] at the end of the infix string and push [onto the stack.

```
((A + (B - C) * D) ^ E + F)]
---------->
```

We move from left to right in the infix expression. We keep on pushing elements onto the stack till we reach a operand. Once we reach an operand, we add it to the output. If we reach a) symbol, we pop elements off the stack till we reach a corresponding { symbol. If we reach an operator, we pop off any operators on the stack which are of higher precedence than this one and push this operator onto the stack.

As an example

```
Expresssion                          Stack                    Output
-------------------------------------------------------------------------
((A + (B - C) * D) ^ E + F)]      [
^
```

```
((A + (B - C) * D) ^ E + F)]      [((                        A
  ^

((A + (B - C) * D) ^ E + F)]      [((+                       A
    ^

((A + (B - C) * D) ^ E + F)]      [((+(-                     ABC
           ^

((A + (B - C) * D) ^ E + F)]      [(                         ABC-D*+
                ^

((A + (B - C) * D) ^ E + F)]      [(                         ABC-D*+E^F+
                         ^

((A + (B - C) * D) ^ E + F)]      [                          ABC-D*+E^F+
                          ^
```

Is there a way to find out if the converted postfix expression is valid or not

Yes. We need to associate a rank for each symbol of the expression. The rank of an operator is -1 and the rank of an operand is +1. The total rank of an expression can be determined as follows:

```
- If an operand is placed in the post fix expression, increment the rank by 1.
- If an operator is placed in the post fix expression, decrement the rank by 1.
```

At any point of time, while converting an infix expression to a postfix expression, the rank of the expression can be greater than or equal to one. If the rank is anytime less than one, the expression is invalid. Once the entire expression is converted, the rank must be equal to 1. Else the expression is invalid.

Conversion from infix to prefix

This is very similar to the method mentioned above, except the fact that we add the special value [at the start of the expression and] to the stack and we move through the infix expression from right to left. Also at the end, reverse the output expression got to get the prefix expression.

Evaluation of a postfix expression

Here is the pseudocode. As we scan from left to right

- If we encounter an operand, push it onto the stack.
- If we encounter an operator, pop 2 operand from the stack. The first one popped is called operand2 and the second one is called operand1.
- Perform Result=operand1 operator operand2.
- Push the result onto the stack.
- Repeat the above steps till the end of the input.

Convert from postfix expression to infix

This is the same as postfix evaluation, the only difference being that we wont evaluate the expression, but just present the answer. The scanning is done from left to right.

```
ABC-D*+

A(B-C)D*+

A((B-C)*D)+

A+((B-C)*D)
```

Convert from postfix expression to infix

The scanning is done from right to left and is similar to what we did above.

```
+A*-BCD

Reverse it
```

```
DCB-*A+

D(B-C)*A+

((B-C)*D)A+

A+((B-C)*D)
```

PREV COMMENTS INDEX PRINT NEXT

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

General Concepts
(284 / 301)

How can we detect and prevent integer overflow and underflow?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to list all the predefined identifiers?

gcc provides a `-dM` option which works with `-E`.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Compiling and Linking
(286 / 301) *

How the compiler make difference between C and C++?

Most compilers recognized the file type by looking at the file extension.

You might also be able to force the compiler to ignore the file type by supplying compiler switch. In MS VC++ 6, for example, the MSCRT defines a macro, __cplusplus. If you undefine that macro, then the compiler will treat your code as C code. You don't define the __cplusplus macro. It is defined by the compiler when compiling a C++ source. In MSVC6 there's a switch for the compiler, /Tc, that forces a C compilation instead of C++.

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Compiling and Linking
(287 / 301) *

What are the general steps in compilation?

```
1. Lexical analysis.
2. Syntactic analysis.
3. Sematic analysis.
4. Pre-optimization of internal representation.
5. Code generation.
6. Post optimization.
```


www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Compiling and Linking
(288 / 301)

What are the different types of linkages?

Linkage is used to determine what makes the same name declared in different scopes refer to the same thing. An object only ever has one name, but in many cases we would like to be able to refer to the same object from different scopes. A typical example is the wish to be able to call printf() from several different places in a program, even if those places are not all in the same source file.

The Standard warns that declarations which refer to the same thing must all have compatible type, or the behaviour of the program will be undefined. Except for the use of the storage class specifier, the declarations must be identical.

The three different types of linkage are:

- external linkage
- internal linkage
- no linkage

In an entire program, built up perhaps from a number of source files and libraries, if a name has external linkage, then every instance of a that name refers to the same object throughout the program. For something which has internal linkage, it is only within a given source code file that instances of the same name will refer to the same thing. Finally, names with no linkage refer to separate things.

Linkage and definitions

Every data object or function that is actually used in a program (except as the operand of a sizeof operator) must have one and only one corresponding definition. This "exactly one" rule means that for objects with external linkage there must be exactly one definition in the whole program; for things with internal linkage (confined to one source code file) there must be exactly one definition in the file where it is declared; for things with no linkage, whose declaration is always a definition, there is exactly one definition as well.

The three types of accessibility that you will want of data objects or functions are:

- Throughout the entire program,
- Restricted to one source file,
- Restricted to one function (or perhaps a single compound statement).

For the three cases above, you will want external linkage, internal linkage, and no linkage respectively. The external linkage declarations would be prefixed with extern, the internal linkage declarations with static.

```
#include <stdio.h>

// External linkage.
extern int var1;
```

```
// Definitions with external linkage.
extern int var2 = 0;

// Internal linkage:
static int var3;

// Function with external linkage
void f1(int a){}

// Function can only be invoked by name from within this file.
static int f2(int a1, int a2)
{
    return(a1 * a2);
}
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Compiling and Linking
(289 / 301)

What do you mean by scope and duration?

The duration of an object describes whether its storage is allocated once only, at program start-up, or is more transient in its nature, being allocated and freed as necessary.

There are only two types of duration of objects: static duration and automatic duration. Static duration means that the object has its storage allocated permanently, automatic means that the storage is allocated and freed as necessary.

It's easy to tell which is which: you only get automatic duration if

- The declaration is inside a function.
- And the declaration does not contain the static or extern keywords.
- And the declaration is not the declaration of a function.

The scope of the names of objects defines when and where a given name has a particular meaning. The different types of scope are the following:

- function scope
- file scope
- block scope
- function prototype scope

The easiest is function scope. This only applies to labels, whose names are visible throughout the function where they are declared, irrespective of the block structure. No two labels in the same function may have the same name, but because the name only has function scope, the same name can be used for labels in every function. Labels are not objects?they have no storage associated with them and the concepts of linkage and duration have no meaning for them. Any name declared outside a function has file scope, which means that the name is usable at any point from the declaration on to the end of the source code file containing the declaration. Of course it is possible for these names to be temporarily hidden by declarations within compound statements. As we know, function definitions must be outside other functions, so the name introduced by any function definition will always have file scope. A name declared inside a compound statement, or as a formal parameter to a function, has block scope and is usable up to the end of the associated } which closes the compound statement. Any declaration of a name within a compound statement hides any outer declaration of the same name until the end of the compound statement. A special and rather trivial example of scope is function prototype scope where a declaration of a name extends only to the end of the function prototype. The scope of a name is completely independent of any storage class specifier that may be used in its declaration.

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Compiling and Linking
(290 / 301) *

What are makefiles? Why are they used?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

HTTP

(291 / 301) *

What are the basics of http? What is the difference be http and https?

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

How to work in ProC?

One of the best ProC documents can be found here.

Here is some simple code in ProC.

```
#ifndef SQLCA
  #include <sqlca.h>
#endif

#define SQLERRM         sqlca.sqlerrm.sqlerrmc

/* VTOS */
#define VARCHAR_TO_STRING(src)  src.arr[src.len]='\0'

/* VTOV */
#define VARCHAR_TO_VARCHAR(dest, src) dest.len=src.len ; \
        strncpy((char *)(dest.arr),(char *)(src.arr), src.len)

/* STOV */
#define STRING_TO_VARCHAR(dest,src)  (dest.len = strlen(strcpy((char *)(dest.arr), src))); \
                                     ((dest).arr[(dest).len] = (unsigned char)'\0')

/* Uses the .len part to null-terminate the .arr part */
#define NULL_TERM(src)   ((src).arr[(src).len] = '\0')

#define SET_LENGTH(src)  ((src).len = (strlen(((src).arr))))


/* ------------------------------------------------------------------------ */
/* TYPE USAGE IN ProC                                                       */
/* ------------------------------------------------------------------------ */

/* THIS IS TO FOOL THE ProC COMPILER AND TELL IT NOT TO SCREAM WHEN IT SEES "sb4" */

EXEC SQL BEGIN DECLARE SECTION;
EXEC SQL TYPE myint IS INTEGER (4);
EXEC SQL END DECLARE SECTION;

/* THIS IS ACTUAL THING THAT C COMPILER USES LATER FOR "sb4", AND
 * NOT THE INTEGER(4). NOTE THAT THESE TYPEDEFS MUST ONLY USE THE
 * DATA TYPES THAT ProC ALLOWS AS HOST VARIABLES!
 * */

typedef int myint;

/* ANOTHER EXAMPLE:
 *
 *     typedef unsigned char *my_raw;
```

```
 *     EXEC SQL TYPE my_raw IS VARRAW(4000) REFERENCE;
 *     my_raw buffer;
 *     ...
 *     buffer = malloc(4004);
 */

/* ---------------------------------------------------------------------------- */



int main()
{

  /* DECLARATIONS */
  EXEC SQL BEGIN DECLARE SECTION;
    VARCHAR connectString[10];
    VARCHAR exec_stmt[1000];
    VARCHAR l_row2[1000];
    myint   l_row1;
    VARCHAR l_owner[30];
    VARCHAR l_table_name[30];
  EXEC SQL END DECLARE SECTION;

  /* TO DO BULK INSERTS INTO A TABLE  (USING ARRAYS)
   *
   * LATER ALSO USED TO OPEN CURSOR ON THE TABLE AND
   * READ THE VALUES, 5 ROWS AT A TIME
   */

  EXEC SQL BEGIN DECLARE SECTION;
    myint   l_row1_arr[5];
    VARCHAR l_row2_arr[5][100];
    myint   bulk_insert_ctr;
  EXEC SQL END DECLARE SECTION;


  /* INITIALIZATIONS */
  char errmsgbuf[512];
  int  msglength;
  int  msgbufsize;
  int  i,j;

  long  numtab = 0;
  long  rowsfetched = 0;
  long totalrows_fetched = 0;


  /* ------------------------------------------------------------*/

  /* START : ERROR HANDLING */

    /* EXEC SQL WHENEVER NOT FOUND CONTINUE; */
    EXEC SQL WHENEVER SQLERROR goto main_fail;

  /* END  : ERROR HANDLING */


  /* ------------------------------------------------------------*/


  /* START : CONNECTING TO THE DATABASE */

    STRING_TO_VARCHAR(connectString, "apps/apps");

    printf("\nConnecting using [%s]....\n",
           connectString.arr);

    EXEC SQL CONNECT :connectString;

    printf("\nConnected.\n\n");

  /* END  : CONNECTING TO THE DATABASE */


  /* ------------------------------------------------------------*/

  /* START : DYNAMIC SQL */

    STRING_TO_VARCHAR(exec_stmt,
                      "DROP TABLE A");
```

```
    EXEC SQL PREPARE S1 FROM :exec_stmt;

    printf("\nPrepared       : [%s].",
           exec_stmt.arr);

    EXEC SQL EXECUTE S1;

    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);

continue1:

    STRING_TO_VARCHAR(exec_stmt,
                      "CREATE TABLE A(ROW1 NUMBER, ROW2 VARCHAR2(100))");

    EXEC SQL PREPARE S1 FROM :exec_stmt;

    printf("\nPrepared       : [%s].",
         exec_stmt.arr);

    EXEC SQL EXECUTE S1;

    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);

    STRING_TO_VARCHAR(exec_stmt,
                      "INSERT INTO A VALUES(1,'ABC')");

    EXEC SQL PREPARE S1 FROM :exec_stmt;

    printf("\nPrepared       : [%s].",
         exec_stmt.arr);

    EXEC SQL EXECUTE S1;

    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);


  /* END : DYNAMIC SQL */

  /* ------------------------------------------------------------------------ */

  /* START :  STATIC SQL */

    strcpy(l_row2.arr, "XXXXXXXXXXXXX");

    printf("\nSELECT ROW2 FROM A WHERE ROW1 = 1;");

    l_row1 = 1;

    EXEC SQL
      SELECT ROW2 INTO :l_row2
      FROM A
      where ROW1 = :l_row1;

    printf("\nROW2 (BEFORE NULLTERM) : [%s]", l_row2.arr);

    NULL_TERM(l_row2); /* DOES l_row2[l_row2.len]='\0'; */

    printf("\nROW2 (AFTER NULLTERM)  : [%s]\n", l_row2.arr);


    printf("\nINSERT INTO A(ROW1, ROW2) SELECT :l_row1, :l_row2 FROM DUAL;");

    EXEC SQL
         INSERT INTO A(ROW1, ROW2)
         select :l_row1, :l_row2 from dual;
```

```
 printf("\nRows processed : [%d]."
        "\nReturn Code    : [%d].\n\n",
        sqlca.sqlerrd[2],
        sqlca.sqlcode);


/* BULK INSERTS INTO TABLE USING ARRAYS */


l_row1_arr[0] = 3;
STRING_TO_VARCHAR(l_row2_arr[0], "CDE");
l_row1_arr[1] = 4;
STRING_TO_VARCHAR(l_row2_arr[1], "DEF");
l_row1_arr[2] = 5;
STRING_TO_VARCHAR(l_row2_arr[2], "EFG");
l_row1_arr[3] = 6;
STRING_TO_VARCHAR(l_row2_arr[3], "FGH");
l_row1_arr[4] = 7;
STRING_TO_VARCHAR(l_row2_arr[4], "GHI");


/* THIS WILL INSERT ALL 5 ELEMENTS OF THE ARRAY */

EXEC SQL
     INSERT INTO A(ROW1, ROW2)
     VALUES (:l_row1_arr, :l_row2_arr);


printf("\nRows processed : [%d]."
        "\nReturn Code    : [%d].\n\n",
        sqlca.sqlerrd[2],
        sqlca.sqlcode);

EXEC SQL COMMIT;



l_row1_arr[0] = 8;
STRING_TO_VARCHAR(l_row2_arr[0], "HIJ");
l_row1_arr[1] = 9;
STRING_TO_VARCHAR(l_row2_arr[1], "IJK");
l_row1_arr[2] = 10;



/* THIS WILL ONLY INSERT 3 ROWS INTO THE TABLE, ALTHOUGH THE
 * ARRAYS CAN ACCOMODATE MAX OF 5 ELEMENTS */

bulk_insert_ctr = 3;

EXEC SQL FOR :bulk_insert_ctr
     INSERT INTO A(ROW1, ROW2)
     VALUES (:l_row1_arr, :l_row2_arr);


printf("\nRows processed : [%d]."
        "\nReturn Code    : [%d].\n\n",
        sqlca.sqlerrd[2],
        sqlca.sqlcode);


/* The USUAL idea with these bulk inserts is to do something like this
 *
 *   bulk_insert_ctr = 0;
 *
 *   while(<say_reading_something_from_a_file>)
 *   {
 *      if(bulk_insert_ctr >= MAX_ARRAY_SIZE_WHICH_WE_HAVE)
 *      {
 *         EXEC SQL
 *           INSERT INTO <SOMETHING> (COL1, COL2, ....)
 *           VALUES                  (:l_col1_array, :l_col2_array, .....);
 *
 *         EXEC SQL COMMIT;
 *
 *         bulk_insert_ctr = 0;
 *
 *      }
 *
```

```
  *      >> KEEP FILLING THE ARRAYS TILL WE REACH THE LIMIT OF
  *      >> MAX_ARRAY_SIZE_WHICH_WE_HAVE
  *
  *      l_col1_array[bulk_insert_ctr] = <something>;
  *      STRING_TO_VARCHAR(l_col2_arr[bulk_insert_ctr], "<something>");
  *
  *      bulk_insert_ctr++;
  *
  *   }//while loop
  *
  *   >> HERE DO INSERTS FOR ANY RESIDUAL DATA THAT MIGHT HAVE
  *   >> ACCUMULATED INTO OUR LOCAL ARRAYS.
  *
  *   >> NOTICE THE USE OF "FOR :bulk_insert_ctr" HERE
  *
  *   EXEC SQL FOR :bulk_insert_ctr
  *     INSERT INTO <SOMETHING> (COL1, COL2, ....)
  *     VALUES                  (:l_col1_array, :l_col2_array, .....);
  *
  *   EXEC SQL COMMIT;
  *
  *   bulk_insert_ctr = 0;
  *
  *
  *
  *
  *
  *  ALSO NOTE THAT IF WE WISH TO INSERT A CONSTANT VALUE (say APPL_TOP_ID,
  *  which is determined at run-time), WE CANNOT USE A SINGLE VARIABLE LIKE
  *  l_appL_top_id AND USE IT IN THE BULK INSERT!
  *
  *  EVEN THIS NEEDS TO BE AN ARRAY l_appl_top_id[MAX_ARRAY_SIZE], EVENTHOUGH
  *  THIS WOULD MEANS DUPLICATION OF APPL_TOP_ID IN ALL THE ARRAY ELEMENTS!
  *
  *  NOTE THAT YOU CAN ALWAYS USE SOMETHING LIKE '1' OR SYSDATE THOUGH, THE
  *  ABOVE RESTRICTION IS ONLY FOR A RUN_TIME VARIABLE THAT REMAINS SAME
  *  FOR ALL THE INSERTS....
  *
  */


/* END  : STATIC SQL */


/* ------------------------------------------------------------------------ */


/* START : CALLING A PL/SQL PROCEDURE */

  STRING_TO_VARCHAR(l_owner, "APPS");
  STRING_TO_VARCHAR(l_table_name, "A");
  STRING_TO_VARCHAR(exec_stmt,"");

  /* BUILD UP THE SQL STATEMENT */

  strcat(exec_stmt.arr, "\nBEGIN");
  strcat(exec_stmt.arr, "\nFND_STATS.GATHER_TABLE_STATS");
  strcat(exec_stmt.arr, "\n(");
  strcat(exec_stmt.arr, "\n:owner, :tab_name");
  strcat(exec_stmt.arr, "\n);");
  strcat(exec_stmt.arr, "\nEND;");

  SET_LENGTH(exec_stmt);

  EXEC SQL PREPARE S1 FROM :exec_stmt;

  printf("\nPrepared       : [%s].",
         exec_stmt.arr);

  EXEC SQL EXECUTE S1 USING :l_owner, :l_table_name;

  printf("\nStatement      : [%s]."
         "\nRows processed : [%d]."
         "\nReturn Code    : [%d].\n\n",
         exec_stmt.arr,
         sqlca.sqlerrd[2],
         sqlca.sqlcode);
```

```
/* END : CALLING A PL/SQL PROCEDURE */


/* ---------------------------------------------------------------------- */

/* START : CURSORS */

printf("\n SPOOLING THE DATA IN THE TABLE\n");
printf("\n-----------------------------------------------\n");
printf("ROW1   ROW2");
printf("\n-----------------------------------------------\n");


STRING_TO_VARCHAR(exec_stmt,"");
sprintf(exec_stmt.arr, "SELECT ROW1, ROW2 FROM A");
SET_LENGTH(exec_stmt);


EXEC SQL PREPARE S1 FROM :exec_stmt;
EXEC SQL DECLARE mycursor CURSOR FOR S1;

EXEC SQL OPEN mycursor;

while(1)
{
  /* Fetch 5 rows at a time */
  EXEC SQL FETCH mycursor
    INTO :l_row1_arr, :l_row2_arr;

  rowsfetched = sqlca.sqlerrd[2] - numtab;
  totalrows_fetched = sqlca.sqlerrd[2];
  numtab = sqlca.sqlerrd[2];

  for(i=0; i<rowsfetched; i++)
  {
    NULL_TERM(l_row2_arr[i]);
    printf("\n[%d] [%s]\n", l_row1_arr[i], l_row2_arr[i].arr);
  }

  if(sqlca.sqlcode == 1403)break;

}

EXEC SQL CLOSE mycursor;

printf("\n-----------------------------------------------\n");

/* END   : CURSORS */

/* ---------------------------------------------------------------------- */


/* START : USING INDICATORS */

/*
 *
 *   EXEC SQL BEGIN DECLARE SECTION;
 *       varchar l_ret_cd[4];
 *       short l_ret_cd_ind = 0;
 *   EXEC SQL END DECLARE SECTION;
 *
 *   EXEC SQL EXECUTE S1 USING :l_rlse_id, :l_at_id, :l_ret_cd:l_ret_cd_ind;
 *
 *   if (l_ret_cd_ind == -1)
 *        STOV(l_ret_cd, "");
 *
 *   nullterm(l_ret_cd);
 *
 *   if (!strcmp((char *)l_ret_cd.arr, "AI"))
 *
 *
 *
 *   THE SAME CAN BE USED WHILE INSERTING VALUES INTO TABLES
 */



 /*
  *   EXEC SQL BEGIN DECLARE SECTION;
```

```
  *    VARCHAR sql_statemt[400];
  *    VARCHAR v_mode[200];
  *    short   v_ind_mode;
  *   EXEC SQL END   DECLARE SECTION;
  *
  *   STRING_TO_VARCHAR(sql_statemt, "");
  *
  *  strcat((char *)sql_statemt.arr, "\nBegin");
  *  strcat((char *)sql_statemt.arr,"\n:p_no_of_rep_ctr:=");
  *  strcat((char *)sql_statemt.arr,"\nUTIL.");
  *  strcat((char *)sql_statemt.arr,"GET_CURRENT_MODE;");
  *  strcat((char *)sql_statemt.arr, "\nEnd;");
  *  setlen(sql_statemt);
  *
  *  EXEC SQL PREPARE S1 FROM
  *              :sql_statemt;
  *
  *  EXEC SQL EXECUTE S1 USING
  *           :v_mode:v_ind_mode;
  *
  *  nullterm(v_mode);
  *
  */


 /* END : USING INDICATORS */

/* ------------------------------------------------------------------------ */


/*  Commiting, disconencting, rolbacking ...
 *
 *   EXEC SQL COMMIT WORK;
 *
 *   EXEC SQL COMMIT WORK RELEASE;
 *
 *   EXEC SQL ROLLBACK WORK;
 *
 *   EXEC SQL ROLLBACK WORK RELEASE;
 *
 */

 EXEC SQL COMMIT WORK RELEASE;


 /* ------------------------------------------------------------------- */

 /* START : IMPORTANT OPTIONS OR CODE SNIPPETS */

 /*
  * EXEC ORACLE OPTION (HOLD_CURSOR=YES);
  *
  *
  *
  * EXEC SQL
  * SELECT 'X'
  * INTO :l_dummy
  * FROM ACTIONS
  * WHERE BUG_ID = :l_bug_id
  * AND   FILE_ID = :l_file_id
  * AND   ACTION_ID = :l_act_id;
  *
  * if (sqlca.sqlcode == 0)
  * {
  *    *is_a_dup = TRUE;
  *    return TRUE;
  * }
  *
  *
  *
  *
  *
  * EXEC SQL LOCK TABLE  IN EXCLUSIVE MODE NOWAIT;
  *
  *
  *
  *
  *
```

```
    *
    *
    *  EXEC SQL DECLARE C_DRVR CURSOR FOR
    *  SELECT
    *  APPLIED_PATCH_ID, PATCH_DRIVER_ID
    *  FROM
    *  PATCH_DRIVERS
    *  WHERE DRIVER_FILE_NAME = :v_driverfile_name
    *  AND NVL(DRIVER_TYPE_C_FLAG, 'N') = NVL(:v_driver_type_c, 'N')
    *  AND NVL(DRIVER_TYPE_D_FLAG, 'N') = NVL(:v_driver_type_d, 'N')
    *  AND NVL(DRIVER_TYPE_G_FLAG, 'N') = NVL(:v_driver_type_g, 'N')
    *  AND PLATFORM = :v_platform
    *  AND PLATFORM_VERSION = :v_platform_version
    *  AND FILE_SIZE = :v_file_size
    *  AND FILE_CONTENTS_CHECKSUM = :v_check_sum;
    *
    *
    *
    */

    /* END : IMPORTANT OPTIONS OR CODE SNIPPETS */

    /* ------------------------------------------------------------------- */




  return(0);

main_fail:

  if(sqlca.sqlcode == -942)
  {
    goto continue1;
  }
  else if (sqlca.sqlcode != 0 )
  {
    msgbufsize = 512;
    sqlglm(errmsgbuf, &msgbufsize, &msglength);
    errmsgbuf[(msglength < msgbufsize)? msglength : msgbufsize] = '\0';
    printf("\nAn error occurred\n\n"
           "Code        : [%d]\n"
           "Description : [%s]\n\n",
           sqlca.sqlcode,
           errmsgbuf);
  }
  return(1);

}
```

Here is a sample output for this file

```
sh-2.05$ /home/sgadag/tmp/practice/ProC/myexe

Connecting using [apps/apps]....

Connected.


Prepared       : [DROP TABLE A].
Statement      : [DROP TABLE A].
Rows processed : [0].
Return Code    : [0].


Prepared       : [CREATE TABLE A(ROW1 NUMBER, ROW2 VARCHAR2(100))].
Statement      : [CREATE TABLE A(ROW1 NUMBER, ROW2 VARCHAR2(100))].
Rows processed : [0].
Return Code    : [0].


Prepared       : [INSERT INTO A VALUES(1,'ABC')].
Statement      : [INSERT INTO A VALUES(1,'ABC')].
Rows processed : [1].
Return Code    : [0].
```

```
SELECT ROW2 FROM A WHERE ROW1 = 1;
ROW2 (BEFORE NULLTERM) : [ABCXXXXXXXXX]
ROW2 (AFTER NULLTERM)  : [ABC]

INSERT INTO A(ROW1, ROW2) SELECT :l_row1, :l_row2 FROM DUAL;
Rows processed : [1].
Return Code    : [0].


Rows processed : [5].
Return Code    : [0].


Rows processed : [3].
Return Code    : [0].


Prepared       : [
BEGIN
FND_STATS.GATHER_TABLE_STATS
(
:owner, :tab_name
);
END;].
Statement      : [
BEGIN
FND_STATS.GATHER_TABLE_STATS
(
:owner, :tab_name
);
END;].
Rows processed : [1].
Return Code    : [0].


 SPOOLING THE DATA IN THE TABLE

-----------------------------------------------
ROW1   ROW2
-----------------------------------------------

[1] [ABC]

[1] [ABC]

[3] [CDE]

[4] [DEF]

[5] [EFG]

[6] [FGH]

[7] [GHI]

[8] [HIJ]

[9] [IJK]

[10] [EFG]

-----------------------------------------------
sh-2.05$
```

This code gets preprocessed into something like this (using the ProC preprocessor)

```
/* Result Sets Interface */
#ifndef SQL_CRSR
#  define SQL_CRSR
  struct sql_cursor
  {
    unsigned int curocn;
    void *ptr1;
    void *ptr2;
    unsigned int magic;
  };
```

```
  typedef struct sql_cursor sql_cursor;
  typedef struct sql_cursor SQL_CURSOR;
#endif /* SQL_CRSR */

/* Thread Safety */
typedef void * sql_context;
typedef void * SQL_CONTEXT;

/* Object support */
struct sqltvn
{
  unsigned char *tvnvsn;
  unsigned short tvnvsnl;
  unsigned char *tvnnm;
  unsigned short tvnnml;
  unsigned char *tvnsnm;
  unsigned short tvnsnml;
};
typedef struct sqltvn sqltvn;

struct sqladts
{
  unsigned int adtvsn;
  unsigned short adtmode;
  unsigned short adtnum;
  sqltvn adttvn[1];
};
typedef struct sqladts sqladts;

static struct sqladts sqladt = {
  1,1,0,
};

/* Binding to PL/SQL Records */
struct sqltdss
{
  unsigned int tdsvsn;
  unsigned short tdsnum;
  unsigned char *tdsval[1];
};
typedef struct sqltdss sqltdss;
static struct sqltdss sqltds =
{
  1,
  0,
};

/* File name & Package Name */
struct sqlcxp
{
  unsigned short fillen;
           char  filnam[9];
};
static struct sqlcxp sqlfpn =
{
    8,
    "ProC.opc"
};


static const unsigned int sqlctx = 1102759285;


static struct sqlexd {
   unsigned int   sqlvsn;
   unsigned int   arrsiz;
   unsigned int   iters;
   unsigned int   offset;
   unsigned short selerr;
   unsigned short sqlety;
   unsigned int   unused;
            short *cud;
   unsigned char  *sqlest;
            char  *stmt;
   sqladts *sqladtp;
   sqltdss *sqltdsp;
            void  **sqphsv;
   unsigned int   *sqphsl;
```

```
            int   *sqphss;
            void  **sqpind;
            int   *sqpins;
   unsigned int   *sqparm;
   unsigned int   **sqparc;
   unsigned short  *sqpadto;
   unsigned short  *sqptdso;
            void  *sqhstv[4];
   unsigned int   sqhstl[4];
            int   sqhsts[4];
            void  *sqindv[4];
            int   sqinds[4];
   unsigned int   sqharm[4];
   unsigned int   *sqharc[4];
   unsigned short  sqadto[4];
   unsigned short  sqtdso[4];
} sqlstm = {10,4};

/* SQLLIB Prototypes */
extern sqlcxt (/*_ void **, unsigned int *,
                   struct sqlexd *, struct sqlcxp * _*/);
extern sqlcx2t(/*_ void **, unsigned int ,
                   struct sqlexd *, struct sqlcxp * _*/);
extern sqlbuft(/*_ void **, char * _*/);
extern sqlgs2t(/*_ void **, char * _*/);
extern sqlorat(/*_ void **, unsigned int *, void * _*/);

/* Forms Interface */
static int IAPSUCC = 0;
static int IAPFAIL = 1403;
static int IAPFTL  = 535;
extern void sqliem(/*_ char *, int * _*/);

typedef struct { unsigned short len; unsigned char arr[1]; } VARCHAR;
typedef struct { unsigned short len; unsigned char arr[1]; } varchar;

/* CUD (Compilation Unit Data) Array */
static short sqlcud0[] =
{10,4274,0,0,0,
5,0,0,1,0,0,283,111,0,0,4,4,0,1,0,1,9,0,0,1,10,0,0,1,10,0,0,1,10,0,0,
36,0,0,2,0,0,273,125,0,0,1,1,0,1,0,1,9,0,0,
55,0,0,2,0,0,277,130,0,0,0,0,0,1,0,
70,0,0,2,0,0,273,144,0,0,1,1,0,1,0,1,9,0,0,
89,0,0,2,0,0,277,149,0,0,0,0,0,1,0,
104,0,0,2,0,0,273,161,0,0,1,1,0,1,0,1,9,0,0,
123,0,0,2,0,0,277,166,0,0,0,0,0,1,0,
138,0,0,3,43,0,260,189,0,0,2,1,0,1,0,2,9,0,0,1,3,0,0,
161,0,0,4,51,0,259,203,0,0,2,2,0,1,0,1,3,0,0,1,9,0,0,
184,0,0,5,41,0,259,230,0,0,2,2,0,1,0,1,3,0,0,1,9,0,0,
207,0,0,6,0,0,285,240,0,0,0,0,0,1,0,
222,0,0,7,41,0,259,257,0,0,2,2,0,1,0,1,3,0,0,1,9,0,0,
245,0,0,2,0,0,273,351,0,0,1,1,0,1,0,1,9,0,0,
264,0,0,2,0,0,277,356,0,0,2,2,0,1,0,1,9,0,0,1,9,0,0,
287,0,0,2,0,0,273,384,0,0,1,1,0,1,0,1,9,0,0,
306,0,0,2,0,0,301,387,0,0,0,0,0,1,0,
321,0,0,2,0,0,269,392,0,0,2,0,0,1,0,2,3,0,0,2,9,0,0,
344,0,0,2,0,0,271,409,0,0,0,0,0,1,0,
359,0,0,8,0,0,286,487,0,0,0,0,0,1,0,
};


#ifndef SQLCA
  #include <sqlca.h>
#endif

#define SQLERRM         sqlca.sqlerrm.sqlerrmc

/* VTOS */
#define VARCHAR_TO_STRING(src)  src.arr[src.len]='\0'

/* VTOV */
#define VARCHAR_TO_VARCHAR(dest, src) dest.len=src.len ; \
        strncpy((char *)(dest.arr),(char *)(src.arr), src.len)

/* STOV */
#define STRING_TO_VARCHAR(dest,src)  (dest.len = strlen(strcpy((char *)(dest.arr), src))); \
                                     ((dest).arr[(dest).len] = (unsigned char)'\0')
```

```
/* Uses the .len part to null-terminate the .arr part */
#define NULL_TERM(src)   ((src).arr[(src).len] = '\0')

#define SET_LENGTH(src)  ((src).len = (strlen(((src).arr))))


/* ------------------------------------------------------------------------ */
/* TYPE USAGE IN ProC                                                       */
/* ------------------------------------------------------------------------ */

/* THIS IS TO FOOL THE ProC COMPILER AND TELL IT NOT TO SCREAM WHEN IT SEES "sb4" */

/* EXEC SQL BEGIN DECLARE SECTION; */

/* EXEC SQL TYPE myint IS INTEGER (4); */

/* EXEC SQL END DECLARE SECTION; */


/* THIS IS ACTUAL THING THAT C COMPILER USES LATER FOR "sb4", AND
 * NOT THE INTEGER(4). NOTE THAT THESE TYPEDEFS MUST ONLY USE THE
 * DATA TYPES THAT ProC ALLOWS AS HOST VARIABLES!
 * */

typedef int myint;

/* ANOTHER EXAMPLE:
 *
 *     typedef unsigned char *my_raw;
 *     EXEC SQL TYPE my_raw IS VARRAW(4000) REFERENCE;
 *     my_raw buffer;
 *     ...
 *     buffer = malloc(4004);
 */

/* ------------------------------------------------------------------------ */



int main()
{

  /* DECLARATIONS */
  /* EXEC SQL BEGIN DECLARE SECTION; */

    /* VARCHAR connectString[10]; */
struct { unsigned short len; unsigned char arr[10]; } connectString;

    /* VARCHAR exec_stmt[1000]; */
struct { unsigned short len; unsigned char arr[1000]; } exec_stmt;

    /* VARCHAR l_row2[1000]; */
struct { unsigned short len; unsigned char arr[1000]; } l_row2;

    myint   l_row1;
    /* VARCHAR l_owner[30]; */
struct { unsigned short len; unsigned char arr[30]; } l_owner;

    /* VARCHAR l_table_name[30]; */
struct { unsigned short len; unsigned char arr[30]; } l_table_name;

  /* EXEC SQL END DECLARE SECTION; */


  /* TO DO BULK INSERTS INTO A TABLE  (USING ARRAYS)
   *
   * LATER ALSO USED TO OPEN CURSOR ON THE TABLE AND
   * READ THE VALUES, 5 ROWS AT A TIME
   */

  /* EXEC SQL BEGIN DECLARE SECTION; */

    myint   l_row1_arr[5];
    /* VARCHAR l_row2_arr[5][100]; */
struct { unsigned short len; unsigned char arr[102]; } l_row2_arr[5];

    myint   bulk_insert_ctr;
  /* EXEC SQL END DECLARE SECTION; */



  /* INITIALIZATIONS */
```

```
  char errmsgbuf[512];
  int  msglength;
  int  msgbufsize;
  int  i,j;

  long  numtab = 0;
  long  rowsfetched = 0;
  long totalrows_fetched = 0;


  /* ------------------------------------------------------------*/

  /* START : ERROR HANDLING */

    /* EXEC SQL WHENEVER NOT FOUND CONTINUE; */
    /* EXEC SQL WHENEVER SQLERROR goto main_fail; */


  /* END  : ERROR HANDLING */


  /* ------------------------------------------------------------*/


  /* START : CONNECTING TO THE DATABASE */

    STRING_TO_VARCHAR(connectString, "apps/apps");

    printf("\nConnecting using [%s]....\n",
           connectString.arr);

    /* EXEC SQL CONNECT :connectString; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.iters = (unsigned int  )100;
    sqlstm.offset = (unsigned int  )5;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&connectString;
    sqlstm.sqhstl[0] = (unsigned int  )12;
    sqlstm.sqhsts[0] = (         int  )12;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nConnected.\n\n");

  /* END  : CONNECTING TO THE DATABASE */


  /* ------------------------------------------------------------*/

  /* START : DYNAMIC SQL */

    STRING_TO_VARCHAR(exec_stmt,
                      "DROP TABLE A");

    /* EXEC SQL PREPARE S1 FROM :exec_stmt; */
```

```
{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )36;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nPrepared       : [%s].",
           exec_stmt.arr);

    /* EXEC SQL EXECUTE S1; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )55;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);

continue1:

    STRING_TO_VARCHAR(exec_stmt,
                      "CREATE TABLE A(ROW1 NUMBER, ROW2 VARCHAR2(100))");

    /* EXEC SQL PREPARE S1 FROM :exec_stmt; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
```

```
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )70;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nPrepared       : [%s].",
         exec_stmt.arr);

    /* EXEC SQL EXECUTE S1; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )89;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);

    STRING_TO_VARCHAR(exec_stmt,
                      "INSERT INTO A VALUES(1,'ABC')");

    /* EXEC SQL PREPARE S1 FROM :exec_stmt; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )104;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
```

```
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nPrepared       : [%s].",
         exec_stmt.arr);

    /* EXEC SQL EXECUTE S1; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )123;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);



  /* END : DYNAMIC SQL */

  /* ------------------------------------------------------------------------ */

  /* START :  STATIC SQL */

    strcpy(l_row2.arr, "XXXXXXXXXXXXX");

    printf("\nSELECT ROW2 FROM A WHERE ROW1 = 1;");

    l_row1 = 1;

    /* EXEC SQL
      SELECT ROW2 INTO :l_row2
      FROM A
      where ROW1 = :l_row1; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "select ROW2 into :b0  from A where ROW1=:b1";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )138;
    sqlstm.selerr = (unsigned short)1;
```

```
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&l_row2;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqhstv[1] = (         void  *)&l_row1;
    sqlstm.sqhstl[1] = (unsigned int  )4;
    sqlstm.sqhsts[1] = (         int  )0;
    sqlstm.sqindv[1] = (         void  *)0;
    sqlstm.sqinds[1] = (         int  )0;
    sqlstm.sqharm[1] = (unsigned int  )0;
    sqlstm.sqadto[1] = (unsigned short )0;
    sqlstm.sqtdso[1] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nROW2 (BEFORE NULLTERM) : [%s]", l_row2.arr);

    NULL_TERM(l_row2); /* DOES l_row2[l_row2.len]='\0'; */

    printf("\nROW2 (AFTER NULLTERM)  : [%s]\n", l_row2.arr);


    printf("\nINSERT INTO A(ROW1, ROW2) SELECT :l_row1, :l_row2 FROM DUAL;");

    /* EXEC SQL
         INSERT INTO A(ROW1, ROW2)
         select :l_row1, :l_row2 from dual; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "insert into A(ROW1,ROW2)select :b0 ,:b1  from dual ";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )161;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&l_row1;
    sqlstm.sqhstl[0] = (unsigned int  )4;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqhstv[1] = (         void  *)&l_row2;
    sqlstm.sqhstl[1] = (unsigned int  )1002;
    sqlstm.sqhsts[1] = (         int  )0;
    sqlstm.sqindv[1] = (         void  *)0;
    sqlstm.sqinds[1] = (         int  )0;
    sqlstm.sqharm[1] = (unsigned int  )0;
    sqlstm.sqadto[1] = (unsigned short )0;
    sqlstm.sqtdso[1] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
```

```
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           sqlca.sqlerrd[2],
           sqlca.sqlcode);


   /* BULK INSERTS INTO TABLE USING ARRAYS */


   l_row1_arr[0] = 3;
   STRING_TO_VARCHAR(l_row2_arr[0], "CDE");
   l_row1_arr[1] = 4;
   STRING_TO_VARCHAR(l_row2_arr[1], "DEF");
   l_row1_arr[2] = 5;
   STRING_TO_VARCHAR(l_row2_arr[2], "EFG");
   l_row1_arr[3] = 6;
   STRING_TO_VARCHAR(l_row2_arr[3], "FGH");
   l_row1_arr[4] = 7;
   STRING_TO_VARCHAR(l_row2_arr[4], "GHI");


   /* THIS WILL INSERT ALL 5 ELEMENTS OF THE ARRAY */

   /* EXEC SQL
        INSERT INTO A(ROW1, ROW2)
        VALUES (:l_row1_arr, :l_row2_arr); */

{
   struct sqlexd sqlstm;
   sqlstm.sqlvsn = 10;
   sqlstm.arrsiz = 4;
   sqlstm.sqladtp = &sqladt;
   sqlstm.sqltdsp = &sqltds;
   sqlstm.stmt = "insert into A(ROW1,ROW2) values (:b0,:b1)";
   sqlstm.iters = (unsigned int  )5;
   sqlstm.offset = (unsigned int  )184;
   sqlstm.cud = sqlcud0;
   sqlstm.sqlest = (unsigned char  *)&sqlca;
   sqlstm.sqlety = (unsigned short)0;
   sqlstm.sqhstv[0] = (         void  *)l_row1_arr;
   sqlstm.sqhstl[0] = (unsigned int  )4;
   sqlstm.sqhsts[0] = (         int  )4;
   sqlstm.sqindv[0] = (         void  *)0;
   sqlstm.sqinds[0] = (         int  )0;
   sqlstm.sqharm[0] = (unsigned int  )0;
   sqlstm.sqharc[0] = (unsigned int   *)0;
   sqlstm.sqadto[0] = (unsigned short )0;
   sqlstm.sqtdso[0] = (unsigned short )0;
   sqlstm.sqhstv[1] = (         void  *)l_row2_arr;
   sqlstm.sqhstl[1] = (unsigned int  )102;
   sqlstm.sqhsts[1] = (         int  )104;
   sqlstm.sqindv[1] = (         void  *)0;
   sqlstm.sqinds[1] = (         int  )0;
   sqlstm.sqharm[1] = (unsigned int  )0;
   sqlstm.sqharc[1] = (unsigned int   *)0;
   sqlstm.sqadto[1] = (unsigned short )0;
   sqlstm.sqtdso[1] = (unsigned short )0;
   sqlstm.sqphsv = sqlstm.sqhstv;
   sqlstm.sqphsl = sqlstm.sqhstl;
   sqlstm.sqphss = sqlstm.sqhsts;
   sqlstm.sqpind = sqlstm.sqindv;
   sqlstm.sqpins = sqlstm.sqinds;
   sqlstm.sqparm = sqlstm.sqharm;
   sqlstm.sqparc = sqlstm.sqharc;
   sqlstm.sqpadto = sqlstm.sqadto;
   sqlstm.sqptdso = sqlstm.sqtdso;
```

```
   sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
   if (sqlca.sqlcode < 0) goto main_fail;
}



   printf("\nRows processed : [%d]."
          "\nReturn Code    : [%d].\n\n",
          sqlca.sqlerrd[2],
          sqlca.sqlcode);

   /* EXEC SQL COMMIT; */

{
   struct sqlexd sqlstm;
   sqlstm.sqlvsn = 10;
   sqlstm.arrsiz = 4;
   sqlstm.sqladtp = &sqladt;
   sqlstm.sqltdsp = &sqltds;
   sqlstm.iters = (unsigned int  )1;
   sqlstm.offset = (unsigned int  )207;
   sqlstm.cud = sqlcud0;
   sqlstm.sqlest = (unsigned char  *)&sqlca;
   sqlstm.sqlety = (unsigned short)0;
   sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
   if (sqlca.sqlcode < 0) goto main_fail;
}




   l_row1_arr[0] = 8;
   STRING_TO_VARCHAR(l_row2_arr[0], "HIJ");
   l_row1_arr[1] = 9;
   STRING_TO_VARCHAR(l_row2_arr[1], "IJK");
   l_row1_arr[2] = 10;



   /* THIS WILL ONLY INSERT 3 ROWS INTO THE TABLE, ALTHOUGH THE
    * ARRAYS CAN ACCOMODATE MAX OF 5 ELEMENTS */

   bulk_insert_ctr = 3;

   /* EXEC SQL FOR :bulk_insert_ctr
        INSERT INTO A(ROW1, ROW2)
        VALUES (:l_row1_arr, :l_row2_arr); */

{
   struct sqlexd sqlstm;
   sqlstm.sqlvsn = 10;
   sqlstm.arrsiz = 4;
   sqlstm.sqladtp = &sqladt;
   sqlstm.sqltdsp = &sqltds;
   sqlstm.stmt = "insert into A(ROW1,ROW2) values (:b1,:b2)";
   sqlstm.iters = (unsigned int  )bulk_insert_ctr;
   sqlstm.offset = (unsigned int  )222;
   sqlstm.cud = sqlcud0;
   sqlstm.sqlest = (unsigned char  *)&sqlca;
   sqlstm.sqlety = (unsigned short)0;
   sqlstm.sqhstv[0] = (         void  *)l_row1_arr;
   sqlstm.sqhstl[0] = (unsigned int  )4;
   sqlstm.sqhsts[0] = (         int  )4;
   sqlstm.sqindv[0] = (         void  *)0;
   sqlstm.sqinds[0] = (         int  )0;
   sqlstm.sqharm[0] = (unsigned int  )0;
   sqlstm.sqharc[0] = (unsigned int   *)0;
   sqlstm.sqadto[0] = (unsigned short )0;
   sqlstm.sqtdso[0] = (unsigned short )0;
   sqlstm.sqhstv[1] = (         void  *)l_row2_arr;
   sqlstm.sqhstl[1] = (unsigned int  )102;
   sqlstm.sqhsts[1] = (         int  )104;
   sqlstm.sqindv[1] = (         void  *)0;
   sqlstm.sqinds[1] = (         int  )0;
   sqlstm.sqharm[1] = (unsigned int  )0;
   sqlstm.sqharc[1] = (unsigned int   *)0;
   sqlstm.sqadto[1] = (unsigned short )0;
```

```
   sqlstm.sqtdso[1] = (unsigned short )0;
   sqlstm.sqphsv = sqlstm.sqhstv;
   sqlstm.sqphsl = sqlstm.sqhstl;
   sqlstm.sqphss = sqlstm.sqhsts;
   sqlstm.sqpind = sqlstm.sqindv;
   sqlstm.sqpins = sqlstm.sqinds;
   sqlstm.sqparm = sqlstm.sqharm;
   sqlstm.sqparc = sqlstm.sqharc;
   sqlstm.sqpadto = sqlstm.sqadto;
   sqlstm.sqptdso = sqlstm.sqtdso;
   sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
   if (sqlca.sqlcode < 0) goto main_fail;
}



   printf("\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           sqlca.sqlerrd[2],
           sqlca.sqlcode);


   /* The USUAL idea with these bulk inserts is to do something like this
    *
    *   bulk_insert_ctr = 0;
    *
    *   while(<say_reading_something_from_a_file>)
    *   {
    *      if(bulk_insert_ctr >= MAX_ARRAY_SIZE_WHICH_WE_HAVE)
    *      {
    *         EXEC SQL
    *           INSERT INTO <SOMETHING> (COL1, COL2, ....)
    *           VALUES                  (:l_col1_array, :l_col2_array, .....);
    *
    *         EXEC SQL COMMIT;
    *
    *         bulk_insert_ctr = 0;
    *
    *      }
    *
    *      >> KEEP FILLING THE ARRAYS TILL WE REACH THE LIMIT OF
    *      >> MAX_ARRAY_SIZE_WHICH_WE_HAVE
    *
    *      l_col1_array[bulk_insert_ctr] = <something>;
    *      STRING_TO_VARCHAR(l_col2_arr[bulk_insert_ctr], "<something>");
    *
    *      bulk_insert_ctr++;
    *
    *   }//while loop
    *
    *   >> HERE DO INSERTS FOR ANY RESIDUAL DATA THAT MIGHT HAVE
    *   >> ACCUMULATED INTO OUR LOCAL ARRAYS.
    *
    *   >> NOTICE THE USE OF "FOR :bulk_insert_ctr" HERE
    *
    *   EXEC SQL FOR :bulk_insert_ctr
    *     INSERT INTO <SOMETHING> (COL1, COL2, ....)
    *     VALUES                  (:l_col1_array, :l_col2_array, .....);
    *
    *   EXEC SQL COMMIT;
    *
    *   bulk_insert_ctr = 0;
    *
    *
    *
    *
    *
    *  ALSO NOTE THAT IF WE WISH TO INSERT A CONSTANT VALUE (say APPL_TOP_ID,
    *  which is determined at run-time), WE CANNOT USE A SINGLE VARIABLE LIKE
    *  l_appL_top_id AND USE IT IN THE BULK INSERT!
    *
    *  EVEN THIS NEEDS TO BE AN ARRAY l_appl_top_id[MAX_ARRAY_SIZE], EVENTHOUGH
    *  THIS WOULD MEANS DUPLICATION OF APPL_TOP_ID IN ALL THE ARRAY ELEMENTS!
    *
    *  NOTE THAT YOU CAN ALWAYS USE SOMETHING LIKE '1' OR SYSDATE THOUGH, THE
    *  ABOVE RESTRICTION IS ONLY FOR A RUN_TIME VARIABLE THAT REMAINS SAME
    *  FOR ALL THE INSERTS....
```

```
    *
    */

  /* END  : STATIC SQL */


  /* ------------------------------------------------------------------------ */


  /* START : CALLING A PL/SQL PROCEDURE */

    STRING_TO_VARCHAR(l_owner, "APPS");
    STRING_TO_VARCHAR(l_table_name, "A");
    STRING_TO_VARCHAR(exec_stmt,"");

    /* BUILD UP THE SQL STATEMENT */

    strcat(exec_stmt.arr, "\nBEGIN");
    strcat(exec_stmt.arr, "\nFND_STATS.GATHER_TABLE_STATS");
    strcat(exec_stmt.arr, "\n(");
    strcat(exec_stmt.arr, "\n:owner, :tab_name");
    strcat(exec_stmt.arr, "\n);");
    strcat(exec_stmt.arr, "\nEND;");

    SET_LENGTH(exec_stmt);

    /* EXEC SQL PREPARE S1 FROM :exec_stmt; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )245;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
    sqlstm.sqhstl[0] = (unsigned int  )1002;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nPrepared       : [%s].",
           exec_stmt.arr);

    /* EXEC SQL EXECUTE S1 USING :l_owner, :l_table_name; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.stmt = "";
    sqlstm.iters = (unsigned int  )1;
    sqlstm.offset = (unsigned int  )264;
```

```
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)&l_owner;
    sqlstm.sqhstl[0] = (unsigned int  )32;
    sqlstm.sqhsts[0] = (         int  )0;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqhstv[1] = (         void  *)&l_table_name;
    sqlstm.sqhstl[1] = (unsigned int  )32;
    sqlstm.sqhsts[1] = (         int  )0;
    sqlstm.sqindv[1] = (         void  *)0;
    sqlstm.sqinds[1] = (         int  )0;
    sqlstm.sqharm[1] = (unsigned int  )0;
    sqlstm.sqadto[1] = (unsigned short )0;
    sqlstm.sqtdso[1] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    printf("\nStatement      : [%s]."
           "\nRows processed : [%d]."
           "\nReturn Code    : [%d].\n\n",
           exec_stmt.arr,
           sqlca.sqlerrd[2],
           sqlca.sqlcode);


  /* END : CALLING A PL/SQL PROCEDURE */


  /* ------------------------------------------------------------------------ */

  /* START : CURSORS */

  printf("\n SPOOLING THE DATA IN THE TABLE\n");
  printf("\n---------------------------------------------\n");
  printf("ROW1   ROW2");
  printf("\n---------------------------------------------\n");


  STRING_TO_VARCHAR(exec_stmt,"");
  sprintf(exec_stmt.arr, "SELECT ROW1, ROW2 FROM A");
  SET_LENGTH(exec_stmt);


  /* EXEC SQL PREPARE S1 FROM :exec_stmt; */

{
  struct sqlexd sqlstm;
  sqlstm.sqlvsn = 10;
  sqlstm.arrsiz = 4;
  sqlstm.sqladtp = &sqladt;
  sqlstm.sqltdsp = &sqltds;
  sqlstm.stmt = "";
  sqlstm.iters = (unsigned int  )1;
  sqlstm.offset = (unsigned int  )287;
  sqlstm.cud = sqlcud0;
  sqlstm.sqlest = (unsigned char  *)&sqlca;
  sqlstm.sqlety = (unsigned short)0;
  sqlstm.sqhstv[0] = (         void  *)&exec_stmt;
  sqlstm.sqhstl[0] = (unsigned int  )1002;
  sqlstm.sqhsts[0] = (         int  )0;
  sqlstm.sqindv[0] = (         void  *)0;
  sqlstm.sqinds[0] = (         int  )0;
```

```
  sqlstm.sqharm[0] = (unsigned int  )0;
  sqlstm.sqadto[0] = (unsigned short )0;
  sqlstm.sqtdso[0] = (unsigned short )0;
  sqlstm.sqphsv = sqlstm.sqhstv;
  sqlstm.sqphsl = sqlstm.sqhstl;
  sqlstm.sqphss = sqlstm.sqhsts;
  sqlstm.sqpind = sqlstm.sqindv;
  sqlstm.sqpins = sqlstm.sqinds;
  sqlstm.sqparm = sqlstm.sqharm;
  sqlstm.sqparc = sqlstm.sqharc;
  sqlstm.sqpadto = sqlstm.sqadto;
  sqlstm.sqptdso = sqlstm.sqtdso;
  sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
  if (sqlca.sqlcode < 0) goto main_fail;
}


  /* EXEC SQL DECLARE mycursor CURSOR FOR S1; */


  /* EXEC SQL OPEN mycursor; */

{
  struct sqlexd sqlstm;
  sqlstm.sqlvsn = 10;
  sqlstm.arrsiz = 4;
  sqlstm.sqladtp = &sqladt;
  sqlstm.sqltdsp = &sqltds;
  sqlstm.stmt = "";
  sqlstm.iters = (unsigned int  )1;
  sqlstm.offset = (unsigned int  )306;
  sqlstm.cud = sqlcud0;
  sqlstm.sqlest = (unsigned char  *)&sqlca;
  sqlstm.sqlety = (unsigned short)0;
  sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
  if (sqlca.sqlcode < 0) goto main_fail;
}



  while(1)
  {
    /* Fetch 5 rows at a time */
    /* EXEC SQL FETCH mycursor
      INTO :l_row1_arr, :l_row2_arr; */

{
    struct sqlexd sqlstm;
    sqlstm.sqlvsn = 10;
    sqlstm.arrsiz = 4;
    sqlstm.sqladtp = &sqladt;
    sqlstm.sqltdsp = &sqltds;
    sqlstm.iters = (unsigned int  )5;
    sqlstm.offset = (unsigned int  )321;
    sqlstm.cud = sqlcud0;
    sqlstm.sqlest = (unsigned char  *)&sqlca;
    sqlstm.sqlety = (unsigned short)0;
    sqlstm.sqhstv[0] = (         void  *)l_row1_arr;
    sqlstm.sqhstl[0] = (unsigned int  )4;
    sqlstm.sqhsts[0] = (         int  )4;
    sqlstm.sqindv[0] = (         void  *)0;
    sqlstm.sqinds[0] = (         int  )0;
    sqlstm.sqharm[0] = (unsigned int  )0;
    sqlstm.sqharc[0] = (unsigned int   *)0;
    sqlstm.sqadto[0] = (unsigned short )0;
    sqlstm.sqtdso[0] = (unsigned short )0;
    sqlstm.sqhstv[1] = (         void  *)l_row2_arr;
    sqlstm.sqhstl[1] = (unsigned int  )102;
    sqlstm.sqhsts[1] = (         int  )104;
    sqlstm.sqindv[1] = (         void  *)0;
    sqlstm.sqinds[1] = (         int  )0;
    sqlstm.sqharm[1] = (unsigned int  )0;
    sqlstm.sqharc[1] = (unsigned int   *)0;
    sqlstm.sqadto[1] = (unsigned short )0;
    sqlstm.sqtdso[1] = (unsigned short )0;
    sqlstm.sqphsv = sqlstm.sqhstv;
    sqlstm.sqphsl = sqlstm.sqhstl;
    sqlstm.sqphss = sqlstm.sqhsts;
```

```
    sqlstm.sqpind = sqlstm.sqindv;
    sqlstm.sqpins = sqlstm.sqinds;
    sqlstm.sqparm = sqlstm.sqharm;
    sqlstm.sqparc = sqlstm.sqharc;
    sqlstm.sqpadto = sqlstm.sqadto;
    sqlstm.sqptdso = sqlstm.sqtdso;
    sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
    if (sqlca.sqlcode < 0) goto main_fail;
}



    rowsfetched = sqlca.sqlerrd[2] - numtab;
    totalrows_fetched = sqlca.sqlerrd[2];
    numtab = sqlca.sqlerrd[2];

    for(i=0; i<rowsfetched; i++)
    {
      NULL_TERM(l_row2_arr[i]);
      printf("\n[%d] [%s]\n", l_row1_arr[i], l_row2_arr[i].arr);
    }

    if(sqlca.sqlcode == 1403)break;

  }

  /* EXEC SQL CLOSE mycursor; */

{
  struct sqlexd sqlstm;
  sqlstm.sqlvsn = 10;
  sqlstm.arrsiz = 4;
  sqlstm.sqladtp = &sqladt;
  sqlstm.sqltdsp = &sqltds;
  sqlstm.iters = (unsigned int  )1;
  sqlstm.offset = (unsigned int  )344;
  sqlstm.cud = sqlcud0;
  sqlstm.sqlest = (unsigned char  *)&sqlca;
  sqlstm.sqlety = (unsigned short)0;
  sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
  if (sqlca.sqlcode < 0) goto main_fail;
}



  printf("\n---------------------------------------------\n");

  /* END   : CURSORS */

  /* ------------------------------------------------------------------------ */


  /* START : USING INDICATORS */

  /*
   *
   *   EXEC SQL BEGIN DECLARE SECTION;
   *       varchar l_ret_cd[4];
   *       short l_ret_cd_ind = 0;
   *   EXEC SQL END DECLARE SECTION;
   *
   *   EXEC SQL EXECUTE S1 USING :l_rlse_id, :l_at_id, :l_ret_cd:l_ret_cd_ind;
   *
   *   if (l_ret_cd_ind == -1)
   *        STOV(l_ret_cd, "");
   *
   *   nullterm(l_ret_cd);
   *
   *   if (!strcmp((char *)l_ret_cd.arr, "AI"))
   *
   *
   *
   *   THE SAME CAN BE USED WHILE INSERTING VALUES INTO TABLES
   */



   /*
    *   EXEC SQL BEGIN DECLARE SECTION;
```

```
    *    VARCHAR sql_statemt[400];
    *    VARCHAR v_mode[200];
    *    short   v_ind_mode;
    *   EXEC SQL END   DECLARE SECTION;
    *
    *   STRING_TO_VARCHAR(sql_statemt, "");
    *
    *  strcat((char *)sql_statemt.arr, "\nBegin");
    *  strcat((char *)sql_statemt.arr,"\n:p_no_of_rep_ctr:=");
    *  strcat((char *)sql_statemt.arr,"\nFND_APPS_MODE_UTIL.");
    *  strcat((char *)sql_statemt.arr,"GET_CURRENT_MODE;");
    *  strcat((char *)sql_statemt.arr, "\nEnd;");
    *  setlen(sql_statemt);
    *
    *  EXEC SQL PREPARE S1 FROM
    *              :sql_statemt;
    *
    *  EXEC SQL EXECUTE S1 USING
    *           :v_mode:v_ind_mode;
    *
    *  nullterm(v_mode);
    *
    */


   /* END : USING INDICATORS */

  /* ------------------------------------------------------------------------ */


  /*  Commiting, disconencting, rolbacking ...
   *
   *   EXEC SQL COMMIT WORK;
   *
   *   EXEC SQL COMMIT WORK RELEASE;
   *
   *   EXEC SQL ROLLBACK WORK;
   *
   *   EXEC SQL ROLLBACK WORK RELEASE;
   *
   */

   /* EXEC SQL COMMIT WORK RELEASE; */

{
   struct sqlexd sqlstm;
   sqlstm.sqlvsn = 10;
   sqlstm.arrsiz = 4;
   sqlstm.sqladtp = &sqladt;
   sqlstm.sqltdsp = &sqltds;
   sqlstm.iters = (unsigned int  )1;
   sqlstm.offset = (unsigned int  )359;
   sqlstm.cud = sqlcud0;
   sqlstm.sqlest = (unsigned char  *)&sqlca;
   sqlstm.sqlety = (unsigned short)0;
   sqlcxt((void **)0, &sqlctx, &sqlstm, &sqlfpn);
   if (sqlca.sqlcode < 0) goto main_fail;
}



   /* ------------------------------------------------------------------- */

   /* START : IMPORTANT OPTIONS OR CODE SNIPPETS */

   /*
    * EXEC ORACLE OPTION (HOLD_CURSOR=YES);
    *
    *
    *
    * EXEC SQL
    * SELECT 'X'
    * INTO :l_dummy
    * FROM AD_PATCH_RUN_BUG_ACTIONS
    * WHERE PATCH_RUN_BUG_ID = :l_patch_run_bug_id
    * AND   FILE_ID = :l_file_id
    * AND   COMMON_ACTION_ID = :l_com_act_id;
    *
```

```
    * if (sqlca.sqlcode == 0)
    * {
    *    *is_a_dup = TRUE;
    *    return TRUE;
    * }
    *
    *
    *
    *
    *
    * EXEC SQL LOCK TABLE  IN EXCLUSIVE MODE NOWAIT;
    *
    *
    *
    *
    *
    *
    *
    *
    *  EXEC SQL DECLARE C_PTCH_DRVR CURSOR FOR
    *  SELECT
    *  APPLIED_PATCH_ID, PATCH_DRIVER_ID
    *  FROM
    *  AD_PATCH_DRIVERS
    *  WHERE DRIVER_FILE_NAME = :v_driverfile_name
    *  AND NVL(DRIVER_TYPE_C_FLAG, 'N') = NVL(:v_driver_type_c, 'N')
    *  AND NVL(DRIVER_TYPE_D_FLAG, 'N') = NVL(:v_driver_type_d, 'N')
    *  AND NVL(DRIVER_TYPE_G_FLAG, 'N') = NVL(:v_driver_type_g, 'N')
    *  AND PLATFORM = :v_platform
    *  AND PLATFORM_VERSION = :v_platform_version
    *  AND FILE_SIZE = :v_file_size
    *  AND FILE_CONTENTS_CHECKSUM = :v_check_sum;
    *
    *
    *
    */

    /* END : IMPORTANT OPTIONS OR CODE SNIPPETS */

    /* ------------------------------------------------------------------- */




  return(0);

main_fail:

  if(sqlca.sqlcode == -942)
  {
    goto continue1;
  }
  else if (sqlca.sqlcode != 0 )
  {
    msgbufsize = 512;
    sqlglm(errmsgbuf, &msgbufsize, &msglength);
    errmsgbuf[(msglength < msgbufsize)? msglength : msgbufsize] = '\0';
    printf("\nAn error occurred\n\n"
           "Code        : [%d]\n"
           "Description : [%s]\n\n",
           sqlca.sqlcode,
           errmsgbuf);
  }
  return(1);

}
```

This is a sample on how to compile and link a ProC file

```
comp_it: Starting on Fri Dec 10 07:43:57 PST 2004

     Compiling Product C files ...
================================================================================
++++ Processing file ProC.opc...
proc include=. include=/local/db/8.0.6/precomp/public include=/addev/ad/11.5/include
include=/fnddev/fnd/11.5/include include=/appldev/tools/6.0.8/include include=/appldev/oracle/8.0.6/
include
```

```
include=/appldev/oracle/8.0.6/oracore3/public include=/appldev/oracle/8.0.6/oracore3/include
include=/appldev/oracle/8.0.6/nlsrtl3/include include=/appldev/oracle/8.0.6/rdbms/public
include=/appldev/oracle/8.0.6/rdbms/include include=/appldev/oracle/8.0.6/network/include
include=/appldev/oracle/8.0.6/otrace/
public define=PROC_KLUDGE ireclen=161 dbms=v8 unsafe_null=yes mode=oracle
release_cursor=no maxopencursors=100 hold_cursor=yes char_map=varchar2 parse=partial sqlcheck=syntax
iname=ProC.opc

Pro*C/C++: Release 8.0.6.3.0 - Production on Fri Dec 10 07:43:57 2004

(c) Copyright 1999 Oracle Corporation.  All rights reserved.

System default option values taken from: /local/db/8.0.6/precomp/admin/pcscfg.cfg

cc -g -I. -I/addev/ad/11.5/include -I/fnddev/fnd/11.5/include -I/appldev/tools/6.0.8/forms60/pub
-I/appldev/tools/6.0.8/include/forms/60/inc/ic -I/appldev/tools/6.0.8/include/forms/60/inc/if
-I/appldev/tools/6.0.8/include/forms/60/inc/sosd -I/appldev/tools/6.0.8/include/forms/60/inc/tk
-I/appldev/tools/6.0.8/include/forms/60/inc/tk/sosd -I/appldev/tools/6.0.8/include/forms/60/inc/ut
-I/appldev/tools/6.0.8/include/forms/60/inc/uat -I/appldev/tools/6.0.8/include/forms/60/inc/mm
-I/appldev/tools/6.0.8/reports60/pub -I/appldev/tools/6.0.8/include/forms/60/inc/d2f
-I/local/db/8.0.6/precomp/public -I/appldev/oracle/8.0.6/oracore3/public -I/appldev/oracle/8.0.6/
oracore3/include
-I/appldev/oracle/8.0.6/nlsrtl3/include -I/appldev/oracle/8.0.6/rdbms/public -I/appldev/oracle/8.0.6/
rdbms/include
-I/appldev/oracle/8.0.6/network/include -I/appldev/oracle/8.0.6/otrace/public
-I/appldev/oracle/8.0.6/precomp/include -I/appldev/oracle/8.0.6/network/public   -DDEBUG -DLINUX -
Dlinux
-DNLS_ASIA -D_GNU_SOURCE  -c ProC.c
touch ProC.t
rm -f ProC.lis
================================================================================
Done compiling Product C files

+++ Not moving product object modules into product lib directory.

comp_it: Done on     Fri Dec 10 07:43:58 PST 2004

+ gcc -g -L/local/db/8.0.6/lib -L/local/db/8.0.6/lib/stubs9iR2 -ldl -o myexe ProC.o
/addev/ad/11.5/lib/libad.a /fnddev/fnd/11.5/lib/libfnd.a /local/db/8.0.6/lib/libsql.a
/local/db/8.0.6/lib/nautab.o /local/db/8.0.6/lib/naeet.o /local/db/8.0.6/lib/naect.o
/local/db/8.0.6/lib/naedhs.o /local/db/8.0.6/lib/libnetv2.a /local/db/8.0.6/lib/libnttcp.a
/local/db/8.0.6/lib/libnetwork.a /local/db/8.0.6/lib/libncr.a /local/db/8.0.6/lib/libclient.a
/local/db/8.0.6/lib/libvsn.a /local/db/8.0.6/lib/libcommon.a /local/db/8.0.6/lib/libgeneric.a
/local/db/8.0.6/lib/libmm.a /local/db/8.0.6/lib/libnlsrtl3.a /local/db/8.0.6/lib/libcore4.a
/local/db/8.0.6/lib/libnlsrtl3.a /local/db/8.0.6/lib/libcore4.a /local/db/8.0.6/lib/libnlsrtl3.a
/local/db/8.0.6/lib/libnetv2.a /local/db/8.0.6/lib/libnttcp.a /local/db/8.0.6/lib/libnetwork.a
/local/db/8.0.6/lib/libncr.a /local/db/8.0.6/lib/libclient.a /local/db/8.0.6/lib/libvsn.a
/local/db/8.0.6/lib/libcommon.a /local/db/8.0.6/lib/libgeneric.a /local/db/8.0.6/lib/libepc.a
/local/db/8.0.6/lib/libnlsrtl3.a /local/db/8.0.6/lib/libcore4.a /local/db/8.0.6/lib/libnlsrtl3.a
/local/db/8.0.6/lib/libcore4.a /local/db/8.0.6/lib/libnlsrtl3.a /local/db/8.0.6/lib/libclient.a
/local/db/8.0.6/lib/libvsn.a /local/db/8.0.6/lib/libcommon.a /local/db/8.0.6/lib/libgeneric.a
/local/db/8.0.6/lib/libnlsrtl3.a /local/db/8.0.6/lib/libcore4.a /local/db/8.0.6/lib/libnlsrtl3.a
/local/db/8.0.6/lib/libcore4.a /local/db/8.0.6/lib/libnlsrtl3.a -ldl -lpthread -lm
/local/db/8.0.6/rdbms/lib/defopt.o /local/db/8.0.6/rdbms/lib/ssbbded.o
+ chmod 755 myexe
+ set +x
$
```


Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Gotcha! programs

Here are a few Gotcha! programs worth looking at....

- ```
  int main()
  {
    int const * p=5;
    printf("%d",++(*p));
  }
  ```

  Answer: "Cannot modify a constant value". p is a pointer to a "constant integer". But we tried to change the value of the "constant integer".

- ```
  int main()
  {
    char s[ ]="man";
    int i;
    for(i=0;s[ i ];i++)
    {
      printf("\n%c%c%c%c",s[ i ],*(s+i),*(i+s),i[s]);
    }
  }
  ```

  Answer:
  mmmm
  aaaa
  nnnn

  Thanks because, s[i], *(i+s), *(s+i), i[s] are all different ways of expressing the same idea. Generally array name is the base address for that array. Here s is the base address. i is the index number/displacement from the base address. So, indirecting it with * is same as s[i]. i[s] may be surprising. But in the case of C it is same as s[i].

- ```
  int main()
  {
    static int var = 5;
    printf("%d ",var--);
    if(var)
     main();
    return(0);
  }
  ```

  Answer: "5 4 3 2 1". When static storage class is given, it is initialized once. The change in the value of a static variable is retained even between the function calls. Main is also treated like any other ordinary function, which can be called recursively.
  ```

-

```
int main()
{
  extern int i;
  i=20;
  printf("%d",i);
  return(0);
}
```

Answer: "Linker Error : Undefined symbol '_i'". The extern storage class in the following declaration,
extern int i; specifies to the compiler that the memory for i is allocated in some other program and that address will be given to the current program at the time of linking. But linker finds that no other variable of name i is available in any other program with memory space allocated for it. Hence a linker error has occurred .

-

```
int main()
{
  char string[]="Hello World";
  display(string);
  return(0);
}

void display(char *string)
{
  printf("%s",string);
}
```

Answer: "Compiler Error : Type mismatch in redeclaration of function display". In third line, when the function display is encountered, the compiler doesn't know anything about the function display. It assumes the arguments and return types to be integers, (which is the default type). When it sees the actual function display, the arguments and type contradicts with what it has assumed previously. Hence a compile time error occurs.

-

```
int main()
{
  struct xx
  {
   char name[]="hello";
  };
  struct xx *s;
  printf("%s",s->name);
  return(0);
}
```

Answer: "Compiler Error". You should not initialize variables in structure declaration.

-

```
int main()
{
  printf("%p",main);
}
```

Answer: Some address will be printed. Function names are just addresses (just like array names are addresses).main() is also a function. So the address of function main will be printed. %p in printf specifies that the argument is an address. They are printed as hexadecimal numbers.

-

```
enum colors {BLACK,BLUE,GREEN};
int main()
{
  printf("%d..%d..%d",BLACK,BLUE,GREEN);
  return(0);
}
```

Answer: "0..1..2". The enum tag assigns numbers starting from 0, if not explicitly defined.

-

```
#include <stdio.h>
void fun();

int main()
```

```
{
  int i=1;
  while (i<=5)
  {
     printf("%d",i);
     if (i>2)
     {
       goto here;
     }
     i++;
  }
}

void fun()
{
here:
  printf("PP");
}
```

Answer: "Compiler error: Undefined label 'here' in function main". Labels have functions scope, in other words The scope of the labels is limited to functions . The label 'here' is available in function fun() Hence it is not visible in function main.

-

```
#include <stdio.h>
int main()
{
  static char names[5][20]={"pascal","ada","cobol","fortran","perl"};
  int i;
  char *t;

  t        = names[3];
  names[3] = names[4];
  names[4] = t;
  for (i=0;i<=4;i++)
  {
    printf("%s",names[i]);
  }
}
```

Answer: "Compiler error: Lvalue required in function main". Array names are pointer constants. So it cannot be modified.

-

```
#include <stdio.h>

int main()
{
  int i=1,j=2;
  switch(i)
  {
     case 1: printf("GOOD");
             break;
     case j: printf("BAD");
             break;
  }
}
```

Answer: "Compiler Error: Constant expression required in function main." The case statement can have only constant expressions (this implies that we cannot use variable names directly so an error). Note that enumerated types can be used in case statements.

-

```
int main()
{
  int i;
  printf("%d",scanf("%d",&i)); // value 10 is given as input here
  return(0);
}
```

Answer: "1". Scanf returns number of items successfully read.Here 10 is given as input which should have been scanned successfully. So number of items read is 1.

-

```
#define f(g,g2) g##g2

int main()
```

```
{
  int var12=100;
  printf("%d",f(var,12));
  return(0);
}
```

Answer: "100".

-

```
int main()
{
  extern int i;
  i=20;
  printf("%d",sizeof(i));
  return(0);
}
```

Answer: "Linker error: undefined symbol '_i'.". The extern declaration specifies that the variable i is defined somewhere else. The compiler passes the external variable to be resolved by the linker. So compiler doesn't find an error. During linking the linker searches for the definition of i. Since it is not found the linker flags an error.

-

```
#include <stdio.h>
int main()
{
  extern out;
  printf("%d", out);
  return(0);
}
int out=100;
```

Answer: "100". This is the correct way of writing the previous program.

-

```
int i,j;
for(i=0;i<=10;i++)
{
  j+=5;
  assert(i<5);
}
```

Answer: "Runtime error: Abnormal program termination. assert failed (i<5), <file name>,<line number>". The assert() statements are used during debugging to make sure that certain conditions are satisfied. If assertion fails, the program will terminate reporting the same. After debugging, use #undef NDEBUG and this will disable all the assertions from the source code. Assertion is a good debugging tool to make use of.

-

```
int main()
{
  main();
  return(0);
}
```

Answer: "Runtime error : Stack overflow.". The main function calls itself again and again. Each time the function is called its return address is stored in the call stack. Since there is no condition to terminate the function call, the call stack overflows at runtime. So it terminates the program and results in an error.

-

```
int main()
{
  char *cptr,c;
  void *vptr,v;
  c=10;
  v=0;
  cptr=&c;
  vptr=&v;
  printf("%c%v",c,v);
  return(0);
}
```

Answer: "Compiler error (at line number 4): size of v is Unknown." You can create a variable of type void * but not of type void, since void is an empty type. In the second line you are creating variable vptr of type void * and v of type void hence an error.

-

```
#include <stdio.h>
int i=10;
int main()
{
  extern int i;
  {
     int i=20;
     {
          const volatile unsigned i=30;
          printf("%d",i);
     }
     printf("%d",i);
  }
  printf("%d",i);
  return(0);
}
```

Answer: "30,20,10". The '{' introduces new block and thus new scope. In the innermost block i is declared as, const volatile unsigned which is a valid declaration. i is assumed of type int. So printf prints 30. In the next block, i has value 20 and so printf prints 20. In the outermost block, i is declared as extern, so no storage space is allocated for it. After compilation is over the linker resolves it to global variable i (since it is the only variable visible there). So it prints i's value as 10.

-

```
int main()
{
  const int i=4;
  float j;
  j = ++i;
  printf("%d %f", i,++j);
}
```

Answer: "Compiler error". Here, i is a constant. you cannot change the value of constant

-

```
void aaa()
{
  printf("hi");
}

void bbb()
{
  printf("hello");
}

void ccc()
{
  printf("bye");
}

int main()
{
  int (*ptr[3])();
  ptr[0]=aaa;
  ptr[1]=bbb;
  ptr[2]=ccc;
  ptr[2]();
}
```

Answer: "bye". Here, ptr is array of pointers to functions of return type int.ptr[0] is assigned to address of the function aaa. Similarly ptr[1] and ptr[2] for bbb and ccc respectively. ptr[2]() is in effect of writing ccc(), since ptr[2] points to ccc.

-

```
int main()
{
  int * j;
  void fun(int **);
  fun(&j);
  return(0);
}

void fun(int **k)
```

```
{
  int a =0;
  /* add a stmt here*/
}
```

Answer : "*k = &a;". The argument of the function is a pointer to a pointer.In the following pgm add a stmt in the function fun such that the address of 'a' gets stored in 'j'.

-

```
int abc(int a,float b)
{
  /* some code */
}
```

```
int abc(a,b)
int a; float b;
{
  /* some code*/
}
```

Answer: The first one is called the "ANSI C notation" and the second one the "Kernighan & Ritche notation".

-

```
int main()
{
  static int i=5;
  if(--i)
  {
    main();
    printf("%d ",i);
  }
}
```

Answer: "0 0 0 0". The variable "I" is declared as static, hence memory for I will be allocated for only once, as it encounters the statement. The function main() will be called recursively unless I becomes equal to 0, and since main() is recursively called, so the value of static I ie., 0 will be printed every time the control is returned.

-

```
int main()
{
  void *v;
  int integer=2;
  int *i=&integer;
  v=i;
  printf("%d",(int*)*v);
  return(0);
}
```

Answer: "Compiler Error. We cannot apply indirection on type void*.". Void pointer is a generic pointer type. No pointer arithmetic can be done on it. Void pointers are normally used for, 1. Passing generic pointers to functions and returning such pointers. 2. As a intermediate pointer type. 3. Used when the exact pointer type will be known at a later point of time.

-

```
int main()
{
  float f=5,g=10;
  enum{i=10,j=20,k=50};
  printf("%d\n",++k);
  printf("%f\n",f<<2);
  printf("%lf\n",f%g);
  printf("%lf\n",fmod(f,g));
  return(0);
}
```

Answer: "Line no 5: Error: Lvalue required, Line no 6: Cannot apply leftshift to float, Line no 7: Cannot apply mod to float". Enumeration constants cannot be modified, so you cannot apply ++.Bit-wise operators and % operators cannot be applied on float values.fmod() is to find the modulus values for floats as % operator is for ints.

-

```
int main()
```

```
{
   while (strcmp(?some?,?some\0?))
   {
     printf(?Strings are not equal\n?);
   }
}
```

Answer: "No output". Ending the string constant with \0 explicitly makes no difference. So ?some? and ?some\0? are equivalent. So, strcmp returns 0 (false) hence breaking out of the while loop.

-

```
int main()
{
  char str1[] = {?s?,?o?,?m?,?e?};
  char str2[] = {?s?,?o?,?m?,?e?,?\0?};
  while (strcmp(str1,str2))
  {
    printf(?Strings are not equal\n?);
  }
  return(0);
}
```

Answer: ?Strings are not equal?, ?Strings are not equal?, ?. If a string constant is initialized explicitly with characters, ?\0? is not appended automatically to the string. Since str1 doesn?t have null termination, it treats whatever the values that are in the following positions as part of the string until it randomly reaches a ?\0?. So str1 and str2 are not the same, hence the result.

-

```
int main()
{
  float i=1.5;
  switch(i)
  {
     case 1: printf("1");
     case 2: printf("2");
     default : printf("0");
  }
  return(0);
}
```

Answer: "Compiler Error: switch expression not integral". Switch statements can be applied only to integral types.

-

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

SQL/PL-SQL/Database
(294 / 301) *

Most frequently asked SQL/PL-SQL/Database questions

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Unix/Linux
(295 / 301) *

Most frequently asked unix questions

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Quantitative Aptitude
(296 / 301) *

Most frequently asked Quantitative aptitude questions

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Graphs
(297 / 301) *

Most frequently asked questions on Graphs

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Introduction to perl

PERL stands for Practical Extraction and Report Language. Perl is a full-fledged Programming language, providing APIs for numerous Functionalities, such as Database Access, Web Access (SOAP, HTML, WDDX, XML ..etc), CGI Programming, GUI Programming ..etc. Perl is also optimized for scanning arbitrary text files, extracting information from those text files, and printing reports based on that information.

This tutorial introduces the basic concepts of Perl. At the end of the tutorial, it also provides a lot of code snippets required to do the trivial tasks (cheat sheet).

```
#####################################################################
# The "use" directive
#####################################################################


use warnings;       # This is used to detect errors like
                    #
                    #  > Redeclaration of variables.
                    #  > Mis-spelt variable names.
                    #  > Syntactic warnings.
                    #  > Obsolete ways of programming.
                    #  > Problems with regexp.
                    #  > Issues with input and output.


use strict;         # Among other things, checks if we have declared
                    # all variables (i.e, strict on variables, strict on references
                    # and Strict on subroutines).


use diagnostics;    # Used to show explanatory text from the "perldiag" documentation.



# Adding any perl modules that we will be using for this script using the "use" command..


use Cwd 'chdir';                    # To use the command : chdir $incdir;
use File::Copy;                     # To use the command : copy "1.c", "2.c"
use Win32::File 'SetAttributes';    # To use the command : SetAttributes( "1.c",  "-READONLY" );
```

```
#########################################################################
# Important perl switches
#########################################################################



-e : To quickly write small perl programs
     Example : perl -e 'print "Hello world\n";'

-c : Just compile the program, dont run.

-I : Include directories apart from those pointed to by @INC
     Example : perl -I/private/perl/ <program_name>







#########################################################################
# The "require" command.
#########################################################################


# Set the required perl version for this script to run..

require 5.003;


# Add any perl pl files that you will be using...

require Exporter;



BEGIN
{
  require 5.004;
  push(@INC, $ENV{NEWBIN});  # Adding to @INC (include path)
}







#########################################################################
# Perl variables and their declarations
#
# > Explicit declaration is not needed.
# > Variables are case Sensitive.
# > The type is given by the first character ($ , @ or %).

#########################################################################



# Scalar Variables     $name
# Lists (Arrays)       @records
# Hashes               %employee


# Scalar Variables



> Scalar Variables will start with $ symbol.
> The name can begin with an alphabet or _
> The variable name may contain letters, _ and digits
```

```
> A variable can store the following values
  > An Integer.
  > A Floating point value.
  > A string.
  > A Boolean.
  > A reference.

Examples:

$x    = 7;
$str  = ?hello World!\n?;
$str  = ?hello World!\n?;
$no   = 345.65;
$null = ??;
$null = undef();




our $myglobalvar = "Perl";          # This is a global variable.
                                    # To access global variables from other packages, use
                                    # $Package::myvar

my  $myname      = "King Kong";     # This is a local variable.

my  ($x, $y, $z) = (1,2,3);         # Multiple assignements, () required.







#########################################################################
# Printing the name of the perl file
#########################################################################


my $prgname=$0;
print "$prgname";                    # This will print "perl.pl" (i.e, the name of the file).







#########################################################################
# Print statements
#########################################################################


print "Hello world.\n";
printf "Hello world.\n";
print 25_000_000, ",", -4, "\n";

print 255, "\n";
print 0377, "\n";
print 0b11111111, "\n";
print 0xFF, "\n";

print hex("0x30"), "\n";
print oct("030"), "\n";
print oct("0b11010"), "\n";


# Output


Hello world.
Hello world.
25000000,-4
255
255
255
255
48
```

```
# These three statements below will throw warnings (because you are trying
# to add strings to numbers)


print "12 Monkeys" + 0 + "\n";   # Prints 12 without "\n".
print "Monkeys" + 0 + "\n";      # Prints 0  without "\n".
print "0X30" + 0 + "\n";         # Prints 0 wihout "\n". Use hex();




#########################################################################
# Single and double quoted strings
#########################################################################


print "Hello : $myname\n";
print "$myglobalvar is great!\n";
print 'Hello : $myname', "\n";      # Notice how the "\n" has been seperated.
                                   # This will NOT evaluate the variable, because
                                   # of the single quote.


# The statements above would print


Hello : King Kong
Perl is great!
Hello : $myname




#########################################################################
# Alternative delimiters
#########################################################################


# print "Hello " World\n";    # This will fail!
print qq@Hello " World\n@;    # This WONT fail, it will print  Hello " World

my $mytimevar=8;
#print "This is the $mytimevarth time.";    # This WONT work.
print "This is the ${mytimevar}th time.";   # This is the right way of doing it and will
                                            # print "This is the 8th time."




#########################################################################
# The "Here" document
#########################################################################


print<<EOF;
This is a here-document. It starts here
and ends there. Write whatever you want to
write inbetween...
EOF
```

```
# The above "Here" document will print the following text as-it-is!


This is a here-document. It starts here
and ends there. Write whatever you want to
write inbetween...



# The "here" document is very useful in writing "usages" for files.
# Here is an example...



sub Usage
{
    print <<EOB;

Usage: utility.exe -option1=<value> -option2=<value> -option3=<value>
    <blah>
    <blah>
    <blah>

EOB
    exit 1;
}









################################################################################
# Perl Operators
################################################################################



print "The modulo 15%6  = ", 15%6, "\n";
print "51 ANDed with 85 = ", 51&85, "\n";
print "51 XORed with 85 = ", 51^85, "\n";
print "51  ORed with 85 = ", 51|85, "\n";
print "51 NEGated       = ", ~51, "\n";


# Output


The modulo 15%6  = 3
51 ANDed with 85 = 17
51 XORed with 85 = 102
51  ORed with 85 = 119
51 NEGated       = 4294967244







# <=> operator
# Returns 0, if both sides are equal.
# -1, if right hand side is bigger.
# 1, if left hand side is bigger.



print "6<=>6 : [" , 6<=>6 , "]\n5<=>6 : [", 5<=>6, "]\n6<=>5 : [", 6<=>5, "]\n";


# Output

6<=>6 : [0]
5<=>6 : [-1]
6<=>5 : [1]




# Concatenation operator is .
```

```
print "Hello"." World!\n";
```

Output

```
Hello World!
```

Repetition operator is x

```
print "Hello!"x3, "\n";
print "\n"x5;
```

Output

```
Hello!Hello!Hello!
<New Line>
<New Line>
<New Line>
<New Line>
<New Line>
```

The ord() function

```
print "Order of A,B = [" , ord('A'), "," , ord("a"), "]\n";
```

Output

```
Order of A,B = [65,97]
```

String comparison operators

```
print "\"ABC\" cmp \"ZYX\" : [", "ABC" cmp "ZYX", "]\n"; # O/P : -1
print "\"ZYX\" cmp \"ABC\" : [", "ZYX" cmp "ABC", "]\n"; # O/P : 1
print "\"ABC\" cmp \"ABC\" : [", "ABC" cmp "ABC", "]\n"; # O/P : 0
print "\"ABC\" gt \"ZYX\" : [", "ABC" gt "ZYX", "]\n";   # O/P : NULL
print "\"ABC\" lt \"ZYX\" : [", "ABC" lt "ZYX", "]\n";   # O/P : 1
print "\"ABC\" eq \"ABC\" : [", "ABC" eq "ABC", "]\n";   # O/P : 1
print "\"ABC\" eq \"ZYX\" : [", "ABC" eq "ZYX", "]\n";   # O/P : NULL
print "\"ABC\" ne \"ZYX\" : [", "ABC" ne "ZYX", "]\n";   # O/P : 1
```

Output

```
"ABC" cmp "ZYX" : [-1]
"ZYX" cmp "ABC" : [1]
"ABC" cmp "ABC" : [0]
"ABC" gt "ZYX"  : []
"ABC" lt "ZYX"  : [1]
"ABC" eq "ABC"  : [1]
"ABC" eq "ZYX"  : []
"ABC" ne "ZYX"  : [1]
```

The autoincrement operator.

```
my $myvar="A9";
print '++$myvar : [', ++$myvar, "]\n";
```

Output

```
++$myvar : [B0]
```

The "chomp" operator

```
my $mychompvar = "Hello\n";
print 'Before chomping : [' , $mychompvar, "]\n";
chomp($mychompvar);
print 'After chomping : [' , $mychompvar, "]\n";
```

Output

```
Before chomping : [Hello
]
After chomping : [Hello]
```

```
################################################################################
# Logical operators
################################################################################


if((1==1) && (2==2))
{
  print "AND condition is satisfied!\n";
}

if((1==1) and (2==2))
{
  print "AND condition is satisfied!\n";
}
```

```
################################################################################
# The "die" operator
################################################################################


my $abcdefg;

die "I am dying" if not defined $abcdefg;   # This will die with line number & filename.

die "I am dying\n" if not defined $abcdefg; # This will die without anything.

defined $abcdefg or die "I am dying" ;      # This will also die without anything.
```

```
################################################################################
# The "split" and the "join" operators...
################################################################################


$_ = 'king kong people hello world oracle sense monkey';

foreach my $myword (split)
{
  print "[$myword]\n";
}
```

Output

```
[king]
[kong]
[people]
[hello]
[world]
[oracle]
[sense]
[monkey]
```

Another example

```
@parts = split /\|/, $lastline;  # split on pipe symbol
```

Another example for the "split" operator

```
my $mysplitarray = "kake:x:10018:10020::/home/kake:/bin/bash";
print '$mysplitarray : [', $mysplitarray, "]\n";
my @fields = split /:/, $mysplitarray;

for (@fields)
{
  print '$_ -> [', $_, "]\n";
}
```

Output

```
$myfilepath : [/home/sgadag/adadmin/]
$mysplitarray : [kake:x:10018:10020::/home/kake:/bin/bash]
$_ -> [kake]
$_ -> [x]
$_ -> [10018]
$_ -> [10020]
$_ -> []
$_ -> [/home/kake]
$_ -> [/bin/bash]
```

The "join" operator.

```
print "After joining -> ", join "#", @fields;
```

Output

```
After joining -> kake#x#10018#10020##/home/kake#/bin/bash
```

Another example for the "join" operator...

```
@rec = ($login,$passwd,$uid,$gid,$gcos,$home,$shell);
$str = join(':', @rec);
```

```
##############################################################################
# Special variables in Perl
##############################################################################
```

The $_ variable.
This is the default input and pattern-searching space.

```
for (@fields)
{
  print '$_ -> [', $_, "]\n";
}
```

```
$_ = 'king kong people hello world oracle sense monkey';

foreach my $myword (split)
{
  print "[$myword]\n";
}
```

```
# The $. variable. This holds the current input line number of the last file handle that was read.
```

```
# The $" variable.
# The separator that joins elements of arrays interpolated in strings.

print '@arr';  $? = '\n';  print '@arr';
```

```
# The $! variable.
# If used in a numeric context, yields the current value of errno.
# If used in a string context, yields the corresponding error string.

$err = "Error value: $!\n";
```

```
# The $/ variable.
# This variable holds the delimiting character when reading lines from a file..
```

```
# The $| variable.
# This variable is used for I/O buffering...
```

```
# The $@ variable.

eval {mkpath($NEWBIN, 0, 0755)};
if ($@)
{
 print "Failed to create the directory:\n$NEWBIN";
 exit 1;
}
```

```
# The $? variable.
# The status returned by the last `...` command, pipe close or system operator.
```

```
copy ("$copyfrom_dir/readme.txt", "$NEWBIN/readme.txt");
if($? != 0)
{
  print "Copying failed!";
  exit 1;
}
```

Some more special variables..

```
$@      The Perl error message from the last eval or do EXPR command.
$0      The name of the file containing the Perl script being executed.
$$      The process ID of the currently executing Perl program.
$<      The real user ID of this process.
$>      The effective user ID of this process.
$(      The real group ID of this process.
$)      The effective group ID of this process.
```

```
##############################################################################
# Input from STDIN
##############################################################################


my $mystdinvar;
print "Enter a number please : ";
$mystdinvar=<STDIN>;
print "You entered : $mystdinvar";
```

Output

```
Enter a number please : 1
You entered : 1
```

```
##############################################################################
# Perl Lists
#
# Name of a list starts with '@'. A list may contain 0 or more scalars.
##############################################################################


print qw("one", "two", "three");  # Gives warnings for comma use.
print qw(one two three);          # No warnings. No need to use comma.
print (("A","B","C","D")[2]);     #
print (("A","B","C","D")[-1]);    #
```

Output

```
"one","two","three"
onetwothree
C
D
```

List slices.

```
print (("A","B","C","D")[1,2]);
print (("A","B","C","D")[0..3]);
print (("A","B","C","D")[0,2..3]);
```

Output

```
BC
ABCD
ACD
```

Swapping

```
my ($myswapvar1, $myswapvar2) = (1,10);
print "(var1, var2) : [",  $myswapvar1, ",", $myswapvar2, "]\n";
($myswapvar1, $myswapvar2) = ($myswapvar2, $myswapvar1);
print "(var1, var2) : [",  $myswapvar1, ",", $myswapvar2, "]\n";
```

Output

```
(var1, var2) : [1,10]
(var1, var2) : [10,1]
```

Range operator

```
print "Counting up   : ", (1..6), "\n";
print "Counting down : ", (6..1), "\n";          # No O/P. Thats because, RHS should be > LHS.
print "Counting down : ", reverse(1..6), "\n";
print "Counting up   : ", ('a'..'k'), "\n";
```

Output

```
Counting up   : 123456
Counting down :
Counting down : 654321
Counting up   : abcdefghijk
```

Assigning arrays

```
my @myarray1=qw(one two three);
my $myarray1=1;       # Valid!
print "My array : [", @myarray1, "]\n";
print "My array (scalar) : [", $myarray1, "]\n";
$myarray1=@myarray1;
print '$myarray1=@myarray1 : [', $myarray1, "]\n";
```

Output

```
My array : [onetwothree]
My array (scalar) : [1]
$myarray1=@myarray1 : [3]
```

Handling arrays

```
my @myarray2 = (1, @myarray1, 2);
print '@myarray2       : [', @myarray2, "]\n";
```

```
print '$myarray2[1]    : [', $myarray2[1] , "]\n";
print '@myarray2[2..4] : [', @myarray2[2..4], "]\n";
@myarray2[1..3]=("three", "two", "one");
print '@myarray2       : [', @myarray2, "]\n";
print 'Index of highest element in @myarray2 is $#myarray2 : [', $#myarray2, "]\n\n";
```

Output

```
@myarray2       : [1onetwothree2]
$myarray2[1]    : [one]
@myarray2[2..4] : [twothree2]
@myarray2       : [1threetwoone2]
Index of highest element in @myarray2 is $#myarray2 : [4]
```

Looping through lists

```
for my $element (@myarray2)
{
  print '$element[]: [', $element, "]\n";
}
```

Output

```
$element[]       : [1]
$element[]       : [three]
$element[]       : [two]
$element[]       : [one]
$element[]       : [2]
```

```
for (@myarray2)
{
  print '$_ : [', $_, "]\n";
}
```

Output

```
$_               : [1]
$_               : [three]
$_               : [two]
$_               : [one]
$_               : [2]
```

```
for (0..$#myarray2)  # Cannot use $myarray2 directly.
{
   print '$myarray2[', $_, ']    : [', $myarray2[$_], "]\n";
}
```

Output

```
$myarray2[0]    : [1]
$myarray2[1]    : [three]
$myarray2[2]    : [two]
$myarray2[3]    : [one]
$myarray2[4]    : [2]
```

Pop and Push operators. (TOP END).

```
print '@myarray2     : [' , @myarray2, "]\n";
my $mypopvar = pop @myarray2;
print 'After POP     : [' , @myarray2, "]\n";
push @myarray2, 2;
```

```
print 'After PUSH    : [' , @myarray2, "]\n\n";
```

Output

```
@myarray2     : [1threetwoone2]
After POP     : [1threetwoone]
After PUSH    : [1threetwoone2]
```

Shift and Unshift operators. (BOTTOM END).

```
print '@myarray2         : [' , @myarray2, "]\n";
unshift @myarray2, 3;
print 'After UNSHIFT     : [' , @myarray2, "]\n";
print 'After SHIFT       : [' , shift @myarray2, "]\n\n";
```

Output

```
@myarray2         : [1threetwoone2]
After UNSHIFT     : [31threetwoone2]
After SHIFT       : [3]
```

The "splice" operator

```
# splice ARRAY,OFFSET,LENGTH,LIST
splice(@days, 1, 4, @rep);
```

Sort operator

```
my @mysortedarray = qw(AZ AB AG AA AC);
print "Before sorting : [", "@mysortedarray", "]\n";
print "After sorting  : [", sort @mysortedarray, "]\n";
```

Output

```
Before sorting : [AZ AB AG AA AC]
After sorting  : [AAABACAGAZ]
```

Sort strings and numbers

```
my @myunsortedarray = (1,2,11,24,3,26,40,4);
print "Before sorting      : [", "@myunsortedarray", "]\n";
@mysortedarray = sort @myunsortedarray;
print "After sorting (sort): [", "@mysortedarray" , "]\n";
@mysortedarray = sort {$a cmp $b} @myunsortedarray;
print "After sorting (cmp) : [", "@mysortedarray" , "]\n";
@mysortedarray = sort {$a <=> $b} @myunsortedarray;
print "After sorting (<=>) : [", "@mysortedarray" , "]\n";
```

Output

```
Before sorting      : [1 2 11 24 3 26 40 4]
After sorting (sort): [1 11 2 24 26 3 4 40]
```

```
After sorting (cmp) : [1 11 2 24 26 3 4 40]
After sorting (<=>) : [1 2 3 4 11 24 26 40]
```

The "grep" operator

```
# The code below will return all lines which are not comments (i.e, starting with #)..
@foo = grep(!/^#/, @bar);
```

```
################################################################################
# Perl Hashes
################################################################################


my %myhash = ("1","ONE",
              "2","TWO",
              "3","THREE");


# Hash keys must be unique, else old one will be overwritten.
# Hash values can be non-unque.



print 'Hash %myhash : [', %myhash, "]\n";
print 'Hash %myhash : [', "%myhash", "]\n";


# Output

Hash %myhash : [1ONE2TWO3THREE]
Hash %myhash : [%myhash]



# Converting from hash to array and vice versa.
# Note that during such conversions, the order need not be maintained.


my @myhasharray = %myhash;
print '@myhasharray = %myhash : [' , "@myhasharray", "]\n";
%myhash = @myhasharray;
print '%myhash = @myhasharray : [' , %myhash, "]\n";


# Output

@myhasharray = %myhash : [1 ONE 2 TWO 3 THREE]
%myhash = @myhasharray : [1ONE2TWO3THREE]



# Accessing and changing hash values

print '$myhash{1} : [', $myhash{1}, "]\n";
$myhash{1} = "ONE MY FOOT";
print '$myhash{1} = "ONE MY FOOT" : [', $myhash{1}, "]\n";


# Output

$myhash{1} : [ONE]
$myhash{1} = "ONE MY FOOT" : [ONE MY FOOT]
```

Deleting elements from the Hash

```
delete $myhash{1};
print 'Deleted $myhash{1}', "\n";
```

print 'After deleting $myhash{1} : [' , $myhash{1}, "]\n"; # This will error!.

Using the "exists" keyword

```
if(exists($myhash{1}))
{
  print '$myhash{1} still exists!',"\n";
}
else
{
  # We can use if(not exists()) here.
  print '$myhash{1} has been deleted!',"\n";
}
```

Looping through a hash

for() loop

```
for (keys %myhash)
{
  print '$myhash{', $_, '} : [' , $myhash{$_} , "]\n";
}
```

Output

```
$myhash{2} : [TWO]
$myhash{3} : [THREE]
```

foreach() is same as for!

```
foreach (keys %myhash)
{
  print '$myhash{', $_, '} : [' , $myhash{$_} , "]\n";
}
```

Output

```
$myhash{2} : [TWO]
$myhash{3} : [THREE]
```

Other Hash and List Functions

```
delete $HASH{KEY}

each %HASH              while( ($key, $val) = each %hash1)     {    }

exists EXPR             exists $hash{$key};    exists ⊂

keys %HASH              foreach $key ( keys %hash1)      {    }

reverse LIST            @rl = reverse @list1;
```

```
shift [ @ARRAY ]        $x = shift @arr1;
```

While loop for hashes...

```
while (($key, $value) = each(%hash))
{
  # Code here...
}
```

```
##############################################################################
# For and While Loops, IF Conditions...
##############################################################################
```

Breaking through loops

```
for (keys %myhash)
{
  if($myhash{$_} eq "TWO")
  {
    print "Breaking out of loop using last;, saw $myhash{$_}\n";
    last;
  }
}
```

#Continuing through loops

```
for (keys %myhash)
{
  if($myhash{$_} eq "TWO")
  {
    print "Continuing the for loop using next;, saw $myhash{$_}\n";
    next;
  }
}
```

#Redoing through loops

```
for (keys %myhash)
{
  if($myhash{$_} eq "TWO")
  {
    print "Redoing the for loop using redo;, saw $myhash{$_}\n";
    #redo;      #Commented as this will result in infinite loop!
  }
}
```

While loops

```
while (condition) { }

until (condition) { }

do/while
```

Control Statements

```
goto LABEL  Continue execution at the specified label.

last [LABEL] Immediately exits the loop in question. Skips
                continue block.
```

```
next [LABEL] Starts the next iteration of the loop.

redo[LABEL]  Restarts the loop block without evaluating
                the conditional again.
```

IF Statements

```
if (condition)
{
  # True Block
}
else
{
  # False Block
}

# elsif is supported (multiple ifs)
```

```
################################################################################
# Getting the command line arguments.
################################################################################


print "Number of args : [", @ARGV+0, "]\n";
foreach (@ARGV)
{
  print '$_ : [', $_, "]\n";
}
```

#Another method to get the command line arguments

```
my $myargvvar1 = shift @ARGV;
my $myargvvar2 = shift @ARGV;
```

Number of ARGS = $#ARGV

```
################################################################################
# while <STDIN> loop
################################################################################


while(<STDIN>)
{
  print "$_";
}
```

```
################################################################################
# Regular expressions in Perl
#
# A regular expression is a concise representation of one or more
# strings, using normal characters (representing themselves) and
# special ones:
# > One of PERL's strongest features
# > Can be used to match, substitute and split
```

```
#
# Examples
#
# /Th?om(as)? (Ch|K)rist(ia|e)ns{1,2}[eo]n/
# ($h, $m, $s) = ($date = /(\d\d):(\d\d):(\d\d)/);
# @words = split(/\W+/, $text);
#
#
# [ EXPR =~ ] [ m ] /PATTERN/ [ g ] [ i ] [ m ] [ o ] [ s ] [ x ]
# Searches EXPR (default: $_) for a pattern. If you prepend an "m" you
# can use almost any pair of delimiters instead of the slashes. If used
# in array context, an array is returned consisting of the subexpressions
# matched by the parentheses in the pattern, i.e., ($1,$2,$3,...).
#
# Optional modifiers:
# g matches as many times as possible;
# i searches in a case-insensitive manner;
# o interpolates variables only once.
# m treats the string as multiple lines;
# s treats the string as a single line;
# x allows for regular expression extensions.
#
# \W, \w, \S, \s, \d, \D, \n, \r
#
# / (Expression) / igom
#
# '^' Beginning of a String
# '$' End of a string
# '.' Any character
# '|' Logical or
# '*' Zero or more
# '+' One or more
# \d Any digit
# \D Any non digit character
# \w Matches alphanumeric, including _
# \W Non-alphanumeric.
# {N,M} The minimum N and maximum M match count.
# {N} N times
# {N,} At least N times.
#
#
#
# [ $VAR =~ ] s/PATTERN/REPLACEMENT/ [ e ] [ g ] [ i ] [ m ] [ o ]
#
# Searches a string for a pattern, and if found, replaces that pattern with
# the replacement text. It returns the number of substitutions made, if any;
# if no substitutions are made, it returns false.
#
# Optional modifiers:
#
# g replaces all occurrences of the pattern;
#
# e evaluates the replacement string as a Perl expression; for any other modifiers
#
# s squeezes all sequences of characters that are translated into the same target
# character into one occurrence of this character.
#
#
# Single-character patterns
#
# A character class is represented by a list of characters enclosed between [ ]
# For the pattern to match, one and only one of these characters must be present
#
# [abcde]
# [0-9]
# [a-zA-Z0-9]
#
# Negated character class
#
# [^0-9] Matches any character except a digit
# [^aeiouAEIOU] Matches any single non-vowel
#
#
##########################################################################
```

```
my $myregexparray = "hello world plus cap kingkong";

print "$myregexparray\n",'$myregexparray =~ /world/',"\n";
if($myregexparray =~ /world/)
{
  print "Match found!\n";
}

print '$myregexparray !~ /worldplus/',"\n";
if($myregexparray !~ /worldplus/)
{
  print "Match not found!\n";
}

print '$myregexparray =~ /PLUS/i',"\n";
if($myregexparray =~ /PLUS/i)
{
  print "Match found!\n";
}

print '$myregexparray =~ /kingkong$/',"\n";
if($myregexparray =~ /kingkong$/)
{
  print "Match found!\n";
}

print '$myregexparray !~ /^kingkong$/',"\n";
if($myregexparray !~ /^kingkong$/)
{
  print "Match not found!\n";
}
```

Output

```
hello world plus cap kingkong
$myregexparray =~ /world/
Match found!
$myregexparray !~ /worldplus/
Match not found!
$myregexparray =~ /PLUS/i
Match found!
$myregexparray =~ /kingkong$/
Match found!
$myregexparray !~ /^kingkong$/
Match not found!
```

Special regular expression characters

```
\d          -> [0-9]
\w          -> [0-9A-Za-z]
\s          -> [ \t\n\r]
\D          -> negation of \d
\W          -> negation of \w
\S          -> negation of \s
[[:alpha:]] -> [a-zA-Z]
[[:alnum:]] -> [0-9A-Za-z]
[[:digit:]] -> \d
[[:lower:]] -> [a-z]
[[:upper:]] -> [A-Z]
[[:punct:]] -> [!"#$%&'()*+,-./:;<=>?@\[\\\]^_`{|}~]
```

Some examples

```
/bea?t/ Matches 'beat' or 'bet'.
/bea+t/ Matches 'beat', 'beaat', 'beaaat',...
/bea*t/ Matches 'bet', 'beat', 'beaat',...
/bea{2}t/
/bea{2,3}t/
/beat{2,}t/
/beat{,3}t/
[abc]  Any one of a b and c.
[^abc] Anything other than a,b,c.
```

```
(abc) The phrase 'abc' as a group.
\b The boundary between a \w character and a \W character.
```

#Special characters.

```
. * ? + [ ] ( ) { } ^ $ | \
```

Turn off special meanings using \Q, turn on again \E

Use ? to stop the greedy search.
Example, stop greedy search : ([a-z]+)(.*?)([a-z]+)

Searching and replacing.

```
my $myreplacearray = "hello world king good jack";

print 'Before replacing $myreplacearray : [', "$myreplacearray", "]\n";
$myreplacearray =~ s/world/myworld/g;
print 'After replacing  $myreplacearray : [', "$myreplacearray", "]\n";
$myreplacearray =~ s/(\w+) myworld (\w+)/$2/g;
print 'After replacing  $myreplacearray : [', "$myreplacearray", "]\n";
```

Output

```
Before replacing $myreplacearray : [hello world king good jack]
After replacing  $myreplacearray : [hello myworld king good jack]
After replacing  $myreplacearray : [king good jack]
```

Changing the delimiters

```
my $myfilepath = "/home/user/patch/";
print "\n",'$myfilepath : [', "$myfilepath]\n";
$myfilepath =~ s#/home/user/patch/#/home/user/admin/#g;
print "\n",'$myfilepath : [', "$myfilepath]\n";
```

Output

```
$myfilepath : [/home/user/patch]
$myfilepath : [/home/user/admin/]
```

Modifiers

```
/i - CASE insensitive matching.

/m - Treat the string as multiple lines. Normally, ^ and $ match
     the very start and very end of the string. If the /m modifier is
     in play, then they will match the starts and the ends of individual
     lines (seperated by \n). For example, given the string: "one\ntwo",
     the pattern /^two$/ will not match, but /^two$/m will.

/s - Treat the string as a single line. Normally, . does not match a
     new line character; when /s is given, then it will.

/g - Global replacer.

/x - allow use of whitespace and comments inside a match.
```

The Transliteration operator

```
$_ = "aieou";
print "\n\nTransliteration operator\n\n",'$mytransarray : [', $_, "]\n";
print "No of vowels : ", tr/aeiouu//, "\n";
my $mytransarray = "hello bhushan jack upper lack";
$mytransarray =~ tr/ //d;
print "After removing all spaces : ", $mytransarray;
```

Output

```
$mytransarray : [aieou]
No of vowels : 5
After removing all spaces : hellobhushanjackupperlack
```

The lookaheads and lookbehind operators
Look ahead and lookbehind operators : ?=, ?!, ?<=, ?<!

```
my $mylookarray = "fish cake and fish pie";
print 'Original array     : [', $mylookarray, "]\n\n";
$mylookarray =~ s/fish(?= cake)/cream/g;
print 'Positive lookahead : [', $mylookarray, "]\n\n";
$mylookarray = "fish cake and fish pie";
print 'Original array     : [', $mylookarray, "]\n\n";
$mylookarray =~ s/fish(?! cake)/cream/g;
print 'Negative lookahead : [', $mylookarray, "]\n\n";

$mylookarray = "fish cake and fish pie";
print 'Original array     : [', $mylookarray, "]\n\n";
$mylookarray =~ s/(?<=fish )cake/cream/g;
print 'Positive lookback  : [', $mylookarray, "]\n\n";
$mylookarray = "fish cake and fish pie";
print 'Original array     : [', $mylookarray, "]\n\n";
$mylookarray =~ s/(?<!fish )cake/cream/g;
print 'Negative lookback  : [', $mylookarray, "]\n\n";
```

Output

```
Original array     : [fish cake and fish pie]
Positive lookahead : [cream cake and fish pie]
Original array     : [fish cake and fish pie]
Negative lookahead : [fish cake and cream pie]
Original array     : [fish cake and fish pie]
Positive lookback  : [fish cream and fish pie]
Original array     : [fish cake and fish pie]
Negative lookback  : [fish cake and fish pie]
```

Some example Regular expressions...

```
my $rel = "11.5.10";
($release_without_dots = $rel) =~ s/\.//g;
```

```
$param =~ /utility_name=(.*)/i;
if ($1 ne "")
{
}
```

```
if ($preinstall_flag =~ /^Y(ES)?$/)
{
}

if ($preinstall_flag =~ /^NO?$/)
```

```
{
}



$curr_line =~ /^output_file=(.*)/)
$output_file = $1;




# Get the number of lines in a string..

@numLs     = ( $file =~ /(\n)/sg );


# Swapping variables

($VAR1, $VAR2) = ($VAR2, $VAR1);


# Processing a String One Character at a Time

while ($str =~ /(.)/g)
{
  # process $1
}


# Trim spaces

$string =~ s/^\s+//;
$string =~ s/\s+$//;
$string =~ s/\s+/ /;


# Save match

$line =~ /(dbdrv: (.*?) (.*?) (.*?))/i;
print $1;
print $2.$3.$4;






##############################################################################
# Reading and Writing to Files...
#
# I/O Functions
# > open
# > close
# > append, copy, read modes
#
# Example
#
# open(FH, "< content.log") or die "Unable to open file - $!";
# while ($line = <FH>) # FH is a file handler
# {
# print $line;
# }
# close(FH);
#
# Points:
#
# '<' read mode, '>>' append mode, '>' write mode
#
##############################################################################


# File handles

print STDERR "Hello world into STDERR\n";

open FH, "readme.txt.txt" or die $!;
```

```
while(<FH>)
{
  print "$_";
}

close FH;
```

Assigning the file contents to an array (memory intensive).

```
my @myfilearray = <FH>;
print "Array contents : [@myfilearray]";
```

The $/ character is the record seperator character.

```
print '$/ : [', $/, "]\n";
```

Output

```
$/ : [
]
```

Reads paragraphs at a time with the new assignment below.

```
$/ = ""
```

Read entire file into a scalar.

```
$/ = undef;
my $file = <FH>;
```

Writing to a file

```
open FH, "readme.txt" or die $!;
open FW, ">readmecopy.txt" or die $!;

while(<FH>)
{
  print FW "$_";
}

close FH;
close FW;
```

Sorting the contents of a file.

```
my @file = <INPUT>;
@file = sort @file;
print OUTPUT @file;
```

Assigning file handles

```
*INPUT = *STDIN;
```

#Binary mode

```
binmode FILEHANDLE;
```

Setting the default filehandle (So that you can just use print instead of print <FILEHANDLE>)

```
select FILEHANDLE;
select(STDERR);
```

To turn on I/O bufferring.

```
$| = 0;
```

To turn off I/O bufferring.

```
$| = 1;
```

Setting file permissions (files after issuing this will have these permissions).

```
umask(0666);
```

File tests

```
-e Exists?
-f Is this a file?
-d Is this a directory?
-z Is the size zero?
-s Is the file of non-zero size, if yes return the size of the file.
-r Readable?
-w Writeable?
-x Executable?
-o Owner?
```

Example

```
If (-e ?access.log?)
{
  # "access.log" exists, do something...
}
```

Renaming a file

```
rename($filename1,$filename2);
```

Getting Current Working Directory.."

```
use Cwd;   # Required!
my $dir = getcwd();
print $dir;
```

Getting files of some type in a directory.

```
my @filelisting = glob("readme*");
print "\n",'Any files of type readme* in the curdir using glob()\n\n", "@filelisting","\n\n";
```

Output

```
readme.txt
```

Reading directories.

```
opendir DH, "." or die "Could not get handle to CWD!";
while ($_ = readdir(DH))
{
  print '$_ -> ', "$_\n";
}
```

Change directory..

```
chdir $tmp_dir
```

Deleting a file..

```
unlink $snapshot_manifest_file
```

Creating directories...

```
eval {mkpath("$ENV{NEWLOG}", 0, 0755)};
```

Changing permissions...

```
chmod 0755, $utility_name;
```

Copying files....

```
copy ("$copyfrom_dir/readme.txt", "$NEWBIN/readme.txt");
if($? != 0)
{
  print "Copying failed!";
  exit 1;
}
```

Creating a directory...

```
mkdir($nondb_admin_lock,0755));
```

Deleting a directory tree..

```
eval{rmtree($nondb_admin_lock, 0, 1)};
```

Recursively process all files in a directory use File::Find;

```
find(\&d, $dir1);
sub d
{
  my $file = $File::Find::name;
  # Process the file
}
```

Get complete file into a string..

```
Open F1, "abc.txt";
@lines = <F1>;
Close (F1);
$file = join(??,@lines);
```

The "glob()" function...

```
my @old_files = glob "$my_dir/*.txt";
```

Some useful File APIs

```
# use File::Spec;
canonpath($path)
catdir($directory1, $directory2);
catfile($directory, $file);
tmpdir();
splitpath($path);
splitdir($path);
path();
```

```
##############################################################################
# References in Perl
#
# The arrow -> notation
#
# When you want to access an element from a referenced array or hash, or
# to call a subroutine, Perl offers additional syntax which often looks
# cleaner. If $ref is a reference to an array or hash or subroutine, then
#
# $ref->[n] accesses the nth element of an array,
# $ref->{'key'} accesses the value of 'key' in a hash, and
# $ref->('Hello') calls a subroutine with the argument 'Hello'.
#
#
# So each of the expressions on the following lines are also equivalent:
#
# $marray[0] ${$ref2marray}[0] $ref2marray->[0]
# $mhash{'ky'} ${$ref2mhash}{'ky'} $ref2mhash->{'ky'}
# &msub(1,2) &{$ref2msub}(1,2) $ref2msub->(1,2)
#
##############################################################################
```

```
my $scalar = 3;
my $scalar_r = \$scalar;   # Creating a reference (similar to a pointer)
```

Converting two lists into a hash..

```
my @myarr1 = qw(one two three four five six);
my @myarr2 = qw(1 2 3 4 5 6);
%myhash = (myarr1 => \@myarr1, myarr2 => \@myarr2);
my $myhash_r = \%myhash;

for (keys %{$myhash_r})
{
  print "$_\n"; #Prints myarr1 and myarr2;
}
```

Arrays inside arrays

```
my @array1 = (10, 20, 30, 40, 50);
my @array2 = (1, 2, \@array ,3 ,4);
my @array3 = (2, 4, \@array2, 6, 8);
my @array4 = (100, 200, \@array3, 300, 400);
```

Anonymous references
For referencing an array anonymously use brackets []
For referencing an array anonymously use brackets {}

```
my @array = (1, 2, 3, 4);
my $array_r = \@array;
OR
my $array_r = [1, 2, 3, 4, 5];
```

Dereferencing values
RULE : TO DEREFERENCE DATA, PUT THE REFERENCE IN CURLY BRACES
WHENEVER YOU WOULD NORMAlIY USE A VARIABLE'S NAME.

```
my @array = (1, 2, 3, 4, 5);
my $array_r = \@array;
my @array2 = @{$array_r};
```

Linked lists

```
friends => {"Baba Oreilly" => $addressbook("Baba Orielly"),
            "Mick ZYXZ"    => $addressbook("Mick XYC")}
```

```
##############################################################################
# Subroutines in Perl
#
# > Subroutine definitions can be anywhere in program text.
# > Subroutine definitions are global (For two subroutines
# with same name, the latter one overrides the former one).
# > By default, any variable reference inside a subroutine is global.
# Variables can be localized using my
#
# Example
#
# sub hello
# {
# my ($arg1, $arg2) = @_; # Receiving arguments
# print ?$arg1 \t $arg2 \n?;
# }
#
##############################################################################
```

#Prototypes

```
sub mysubroutine($$$$);             # Takes three args, and one reference
```

```
my @myvar = (10000, 20000, 30000);



# Pass arrays and hashes as references to prevent flattening!

my ($retval1, $retval2, $retval3, $retval4);
print "\n\nSUBROUTINES\n\nCalling mysubroutine()...\n";
($retval1, $retval2, $retval3) = mysubroutine(1,2,3,\@myvar);

print "Return values : [", $retval1, ", ", $retval2, ", ",$retval3, "]\n";


sub mysubroutine
{
  # Try not to use @_ directly

  my @myargsarray = @_;
  my $myrefvar = $myargsarray[3];

  print "\nEntered mysubroutine()";
  print "\n",'@_ : [', @myargsarray, "]";

  if(wantarray)
  {
    print "\nCaller wants an array back";
  }

   print "\n Reference (4th paramater) : [", "@{$myrefvar}", "]\n";
   print '@{$myrefvar}[2] : [', @{$myrefvar}[2], "]\n";
   print "\nExiting mysubroutine()";
   return(1,2,3);
}



# Passing file handles

sub say_hello
{
  my $fh = shift;
  print $fh "Hi there!\n";
}

sub get_line
{
  my $fh = shift;
  my $response = <$fh>;
  chomp $response;
  $response =~ s/^\s+//;
  return $response;
}

say_hello(*STDOUT);
get_line(*STDIN);



# Giving default values to parameters..

sub log
{
  my $message = shift || "somethings wrong!";
  my $time = shift || localtime;
}



# Recursion example (finding permutations).
# Check out the beautiful use of permutations in this example...



my @mypermutearray = ("a", "b", "c", "d");
```

```
sub permute(@$$);
permute(\@mypermutearray,0,4);

sub permute
{

  my $myarray_r = shift;
  my @myarray = @{$myarray_r};
  my $k = shift;
  my $m = shift;


  if($k==$m)
  {
    print "[@myarray]\n";
  }
  else
  {
    my $i;
    for($i=$k; $i<$m;$i++)
    {
       ($myarray[$i],$myarray[$m-1]) = ($myarray[$m-1],$myarray[$i]);
       permute(\@myarray,$k,($m-1));
       ($myarray[$i],$myarray[$m-1]) = ($myarray[$m-1],$myarray[$i]);
    }
  }
}




################################################################################
# Spawning commands
################################################################################


# Spawning something..

my $exit_status = system($cmd);

if ($exit_status != 0)
{
  die "$prgname error: Error while running \"${cmd}\"\n\n";
}



# Another example of spawning a command..

$cmdline = "move " . $ARGV[0] . " " . $ARGV[0] . ".org";
`$cmdline`;
$cmdline = "move " . $ARGV[0] . ".tmp " . $ARGV[0];
`$cmdline`;



# The "eval" statement...

eval 'exec perl -S $0 "$@"'



# Exit with some error code...

exit $BASE_ERROR + 2;




################################################################################
# Playing with environment variables
################################################################################
```

```
# Getting environment variables..

my $appltop = $ENV{PRODUCT_TOP} || die "$progname error: APPL_TOP is not set.\n\n\n";



# Setting environment variables...

$ENV{'PRODUCT_TOP'} = "C:\\MyFolder\";



# Get the hostname of the machine..

my $node_name = hostname();




################################################################################
# Date functions
################################################################################


# Date

my $date_val = strftime "%a %b %e %H:%M:%S %Y", localtime;






################################################################################
# Parse arguments...
################################################################################


# Simple program to process arguments passed to a function...


$bad_args=0;
for my $param(@ARGV)
{
  if($param =~ /restart=(\w+)/i)
  {
    # The program was passed with an argument "restart=<something>"
    $restart=$1;
    $restart=uc($restart);
    if($restart eq "")
    {
      $bad_args=1;
    }
  }
  elsif($param =~ /locator=(.*)/)
  {
    # The program was also passed with an argument "locator=<something>"
    $locator=$1;
    if($locator eq "")
    {
      $bad_args=1;
    }
  }

  # Process for other possible arguments in similar fashion...

  if($bad_args != 0)
  {
     print "\nError: Invalid argument: $param\n";
     exit 1;
   }
}
```

```
##############################################################################
# String functions
##############################################################################


chomp LIST                          Removes line endings from all elements of the list;
                                    returns the (total) number of characters removed.

chop LIST                          Chops off the last character on all elements of the
                                    list; returns the last chopped character.

eval EXPR                          EXPR is parsed and executed as if it were a Perl
                                    program. The value returned is the value of the last
                                    expression evaluated. If there is a syntax error or
                                    runtime error, an undefined string is returned by eval,
                                    and $@ is set to the error message.

index STR, SUBSTR [ , OFFSET ]     Returns the position of SUBSTR in STR at or after
                                    OFFSET. If the substring is not found, returns -1 (but
                                    see $[ in section Special Variables).

length EXPR                         Returns the length in characters of the value of
                                    EXPR.

lc EXPR                           Returns a lowercase version of EXPR.

lcfirst EXPR                 Returns EXPR with the first character lowercase.

rindex STR, SUBSTR [ , OFFSET ]  Returns the position of the last SUBSTR in STR
                                    at or before OFFSET.

substr EXPR, OFFSET [ , LEN ]    Extracts a substring of length LEN out of EXPR
                                    and returns it. If OFFSET is negative, counts
                                    from the end of the string. May be assigned to.
uc EXPR                     Returns an uppercased version of EXPR.

ucfirst EXPR                     Returns EXPR with the first character
                                    uppercased



# Convert to lowercase...

$utility = lc($1);



# Substring...

substr($CONTEXT,0,"$actual_buf_size - 2");



# Length of a string...

my $start_dir_len  = length($START_DIR);
my $content_length = length( $content );



##############################################################################
# Perl packages
##############################################################################


# Access variables
# $Package_name::variable_name
#
# To set the default package,
# package Package_name;
#
```

```
#############################################################################
# Perl modules
#
# Definition of Module:
#
# The unit of software reuse in PERL is the module, a file that has a
# collection of related functions designed to be used by other programs
# and library modules.
#
# Every module has a
#
# > Public interface
# > Set of variables
# > Functions
#
#
# All Perl module files will have the extension ".pm".
# Perl will search for modules in each of the directories listed in the @INC array.
#
#
# Creating a Perl Module
#
# Sample Module
# package Poker;
#
# use Exporter;
# @ISA = (?Exporter?);
# @EXPORT = qw(&shuffle @card_deck);
#
# @card_deck = ( );
# sub shuffle { }
# 1;
#
#
# Using a Perl Module
#
#
# use Poker;
# or
# require Poker;
# or
# use ?Poker.pm?;
#
# Modules included with use are processed at compile time, but require
# processing happens at run time. Use performs an implicit import on the
# included module?s package.
#
#
# Perl Classes
#
# package Person;
#
# sub new
# {
# my $class = shift;
# my $self = { };
# return bless $self, $class;
# }
#
# sub name
# {
# my $self = shift;
# $self->{NAME} = shift if @_;
# return $self->{NAME};
# }
#
#
# use Person;
#
# my $dude = Person->new();
#
# $dude->name("Jason");
#
# printf ?Name: %s \n", $dude->name;
#############################################################################
```

```
# A module is a package within a file
#
# Including other files
#
# do:
# do "printit.pl";
# Wont be able to see lexical vars from the main program once we are
# the additional code.
#
#
# require:
# require Monty::Python;
# Looks for Python.pm under @INC under directory called Monty.
# require will "do" only once.
#
# use:
#
# use MyProgram::Graphical;
# Like require, except that perl apples it before anything else in the program
# starts. "use" takes place at compile time and not run time.
#
# Add things to @INC
#
# sub BEGIN
# {
# push @INC, "my/module/directory";
# }
# use Wibble;
#
# OR
#
# use lib "my/module/directory";
# use Wibble;
#
#
#


#Example

use File::Spec::Functions;
foreach (path())
{
  my $test = catfile($_,"dir");
  print "Yes, it exists!";
  exit;
}
print "dir was not found!";



################################################################################
# Socket programming
################################################################################


use IO::Socket;

my $server = "my-server";
my $port   = 5000;

while (1)
{
  for ($i=2000; $i<320000; $i += int (rand(1000)))
  {

     print "Opening $server:$port -> ";
     if (($socks = IO::Socket::INET->new(Protocol => "tcp",
                                         PeerAddr => $server,
                                         PeerPort => $port,
                                         TimeOut => 5)))
     {
        printf "The local port is [%s], sending [%d] bytes\r", $socks->sockport(), $i;

$socks->send("crap" x $i);
```

```
     }
     else
     {
print "\nCan't open port\n";
last;
     }

  }# For loop
} # While loop
```

Important links

- www.perldoc.com
- www.cpan.org
- www.perl.org

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

OOPs/OOAD
(298 / 301) *

Most frequently asked OOPs and OOAD questions

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Computer Networks
(299 / 301) *

Most frequently asked questions on Computer Networks

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!

Client Server Technologies
(300 / 301) *

Most frequently asked Client - Server questions

This questions will be answered soon :)

Last updated: November 3, 2005

www.cracktheinterview.com - Your destination for the most common IT interview questions, answers, frequently asked interview questions (FAQ), C Programs, C Datastructures for technical interviews conducted by the top IT companies around the world!